信息约束下微网购电与储能协同调度

摘要

针对光伏和负载变化、日内预报更新及实时电价波动下的微网购电问题，建立以十分钟能量平衡为基础、以可得信息约束为边界的分层调度模型。模型将原计划、日内承诺变更和实际紧急补电分别记账，并将真实储电量跨日传递，避免用未来实现数据制定当前计划。

问题一构造含充放电互斥约束的混合整数线性规划。最优购电量为 59,482.699 kWh，购电费为 35,126.949 元，相对无储能基准节省 26.898%；连续松弛下界与整数最优值一致。问题二以近期和同星期组合预测为基础，用历史净负荷残差分位数形成上尾裕度，每月利用此前 21 天的执行费用选择策略。2 月至 12 月 334 天总费用为 14,758,543.05 元，紧急购电量为 146,312.14 kWh。

问题三在 0、6、12、18 时使用已发布的光伏预报，仅更新尚未执行的购电承诺，并比较保留原承诺与调整方案的预测经济价值。主结算口径下总费用为 14,392,539.81 元，较问题二减少 366,003.24 元；保持调整权限并屏蔽新光伏预报的对照表明，新预报对应的费用改善为 70,110.23 元。紧急电量减少 45.23%，但紧急时段数增加，说明费用与缺口频率应分别评价。

问题四区分未来预测价格与真实结算价格，引入历史联合残差、价格加权分位数及有限策略 CVaR 评价。两种交易权限下总费用分别为 15,399,318.84 元和 15,008,011.51 元。与冻结策略动态重计价相比，费用分别增加 19,772.05 元和减少 6,838.51 元，说明价格预测的经济价值依赖交易权限和执行方式，不能由预测精度单独判定。

通过物理与信息反事实测试、逐时能量核算、逐笔交易账本和工作簿全量回读验证模型。对效率解释、调减退款和价格发布时间分别报告替代口径。本文得到可执行、可复核的购电方案，同时保留有限策略优化及单年度样本的适用边界。

关键词：微网调度；储能；因果预测；滚动优化；交易账本；条件风险价值

1 问题分析与建模假设

微网的经济调控同时面对“何时购电”和“何时使用储能”两个问题。低价充电有助于减少高价购电，但充放电损耗会消耗一部分套利收益；光伏预测偏高或负载预测偏低还会触发五倍电价紧急购电。因此，预测平均误差最小不必对应实际购电费用最低。本文依据赛题及附件[1]，把可执行的时序决策作为四问共同主线。

表 1 四问的信息与交易权限

问题	普通购电决策	可用新增信息	实际执行
一	0 时确定全天	单日负荷及光伏预测	确定性调度
二	每日 0 时固定全天	此前负载、光伏及 SOC	当前供需反馈
三	0 时计划，6/12/18 时调整	已发布 24 小时光伏预报	供需反馈，跨日 SOC
四之二	继承问题二	历史实现电价	允许固定承诺储能优化
四之三	继承问题三	历史电价与已发布光伏预报	只调整未来承诺

1.1 统一假设

（1）十分钟数据代表对应区间的平均功率。根据 0:10 至次日 0:00 的标签序列，采用区间结束点解释，输出使用 0:00—0:10 至 23:50—24:00 的自然日区间，输入数值顺序保持一致。此解释属于建模选择，并非官方勘误。

（2）充、放电效率分别为 0.9；“往返效率为 90%”作为替代敏感性。储电量范围为 1,200 至 10,800 kWh，母线侧充放电功率均不超过 5,000 kW，初始储电量为 6,000 kWh。

（3）不向外网售电，不削减负载，不额外引入电池退化费、并网容量、辅助服务收益或其他设备。无法利用的多余能量允许弃用，不获得收入。

（4）问题一日初日末储电量相等；问题二至四真实 SOC 跨日连续，不每天重置。1 月用于历史积累和预运行，按照模板要求提交 2 月 1 日至 12 月 31 日的 334 天。1 月 1 日缺少历史时取零计划、储能保持初值，缺口紧急补足；其费用不计入正式区间。

（5）实际执行层把当前十分钟供需聚合量视为当期可反馈信息。这是准实时离散近似，未假定当前计划能获得后续时段真实数据；在更细时间尺度实施时，需要实时计量与控制系统。

（6）日内调减采用原计划费不退、另收 50%违约费的主口径 A，并给出允许退款的 B 与净偏差 B_net 对照。波动价格采用未来未知、交付时实现价结算的主解释，全天价格已知仅作信息条件敏感性。

2 数据处理与统一物理模型

2.1 日期和单位对齐

按日期和十分钟标签对齐附件 2 与附件 4，而非按未经验证的行号拼接。全年共 365×144＝52,560 个区间；正式区间共 334×144＝48,096 个区间。附件 3 含 365×4＝1,460 个发布版本，每版 24 个整点预报，总计 35,040 个预报值。功率统一乘以 Δt＝1/6 小时转为母线侧电量；电价保持元/kWh，不再次乘时间步长。

表 2 主要符号

符号	含义	单位
t，d，k	十分钟时段、日期、日内发布阶段	—
L_t，P_t	负载、光伏时段电量	kWh
G_t^0，G_t^k	0 时原计划、第 k 阶段有效普通购电承诺	kWh
H_t，W_t	紧急补电、弃用的多余电量	kWh
C_t，D_t，E_t	母线侧充电、放电、时段初储电量	kWh
U_{kt}，R_{kt}	相对上一有效承诺的调增、调减量	kWh
λ_t，$\hat{\lambda}_t$	交付时段真实电价、决策时预测电价	元/kWh
q，v，β，α	风险分位、终端电量价值、风险权重、置信水平	v 为元/kWh，其余无量纲

2.2 能量平衡和设备约束

参考储能日前调度模型的基本能量约束[2]，但仅保留本题给定的负荷、光伏、储能和外网购电。令 G_t 为当前已生效承诺，统一模型为

$$G_t + P_t + D_t + H_t = L_t + C_t + W_t \tag{1}$$

$$E_{t+1} = E_t + \eta_c C_t - D_t/\eta_d, \quad 1200 \le E_t \le 10800 \tag{2}$$

$$0 \le C_t \le \frac{5000}{6} z_t, \quad 0 \le D_t \le \frac{5000}{6}(1 - z_t), \quad z_t \in \{0, 1\} \tag{3}$$

此外 G、H、W 均非负。W 表示母线侧全部弃用能量，可能同时包含未利用的计划购电和光伏，不能未经来源分摊就把 W 全部称为弃光。实际充放电按真实供需与 SOC 计算；优化中的预测储能轨迹只用于形成计划。式（1）至（3）由全部输出逐点核验，代码与输入输出合同见附录 B。

3 问题一 确定性最优购电

3.1 混合整数模型与最优性

在已知单日曲线条件下，问题一无需紧急购电，令 $H_t=0$，在式（1）至（3）之外增加 $E_0=E_{144}=6{,}000$，以全天普通购电费最小为目标：

$$\min F_1=\sum_{t=0}^{143}\lambda_t G_t \tag{4}$$

调用 SciPy 封装的 HiGHS 求解器[5]求解混合整数线性规划，同时求连续松弛作为下界。整数目标为 35,126.948589 元，连续下界为 35,126.948589 元，MIP 相对间隙为 0；在所述离散模型下获得全局最优。

3.2 线性松弛为何可用于后续计划

若连续解同一时段同时充放电，取 $\delta=\min(C_t,D_t/(\eta_c\eta_d))$，将充电减 δ、放电减 $\eta_c\eta_d\delta$、弃用增加 $(1-\eta_c\eta_d)\delta$。储电状态变化及能量平衡均不变，普通购电不变，且至少一项充放电归零。因此，在允许无成本弃用且目标不奖励无效循环的本模型中，可将连续解恢复为互斥解而不增加目标值。该论证不适用于额外限制弃用或奖励循环的其他模型。

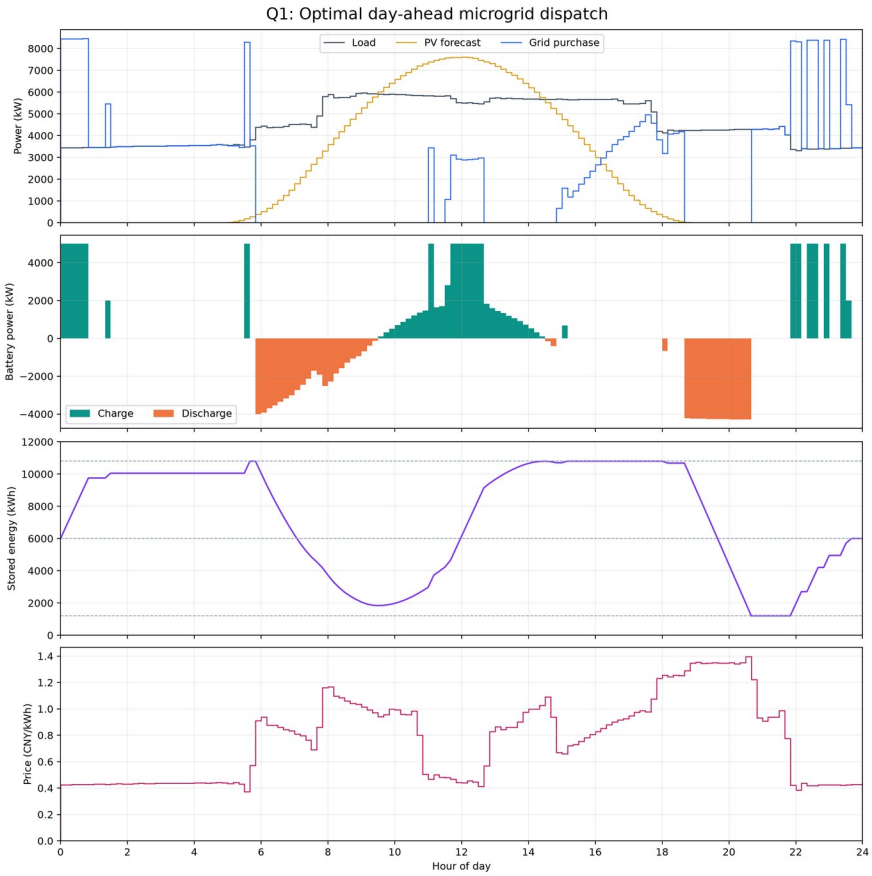


图 1 单日购电 储能充放电与储电量轨迹

3.3 问题一指定结果

表 3 问题一指定时段购电（对应题面表 1）

时间段	购电量	时间段	购电量	时间段或指标	数值
10:00-10:10	0.000	12:00-12:10	480.412	14:00-14:10	0.000
16:00-16:10	445.432	18:00-18:10	531.894	20:00-20:10	0.000
全天购电量	59,482.699	全天购电费	35,126.949	单位	kWh／元

表 4 问题一储能结果（对应题面表 2）

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4,500.000	0.000	4:00-8:00	833.333	6,365.841
8:00-12:00	4,787.964	1,702.997	12:00-16:00	5,286.035	91.101
16:00-20:00	0.000	5,780.132	20:00-24:00	5,333.333	2,859.868
0:00 储电量	6,000.000		24:00 储电量	6,000.000	

图 1 反映逐时动作，四小时汇总中充、放电均为正不代表同一时段同时充放电。上述表中电量单位均为 kWh，费用为元。完整 144 时段普通购电与四小时储能汇总写入 result1.xlsx；表内保留三位小数，工作簿与核算使用未舍入数值。

表 5 问题一经济性与能量核对

指标	数值
无储能购电费／元	48,052.047
储能节约费用／元	12,925.098
节省比例	26.8981%
全天负载／kWh	111,024.808
全天光伏／kWh	55,482.836
充放电损耗／kWh	3,940.727
弃用能量／kWh	0.000

由首末 SOC 相同，全天充电的 90%等于全天放电除以 90%。购电量加光伏电量等于负载加储能损耗，59,482.699＋55,482.836＝111,024.808＋3,940.727 kWh，在舍入误差内相等。这一独立能量核对同时检验了效率方向和功率转电量的单位。

若将题面的 90%解释为往返效率，采用 $\eta_c=\eta_d=\sqrt{0.9}$，购电费用为 33,801.496 元，较主口径下降 3.77%。差异主要来自循环损耗降低，因此论文与提交表必须采用同一效率解释。

4 问题二 因果预测与日前策略选择

4.1 历史信息和上尾裕度

每日 0 时只能利用此前已完成的负载、光伏、当前 SOC 和已知固定电价。比较昨日同刻、近 7 日均值、近期与同星期组合三种预测。组合预测将近 7 日均值与最近至多 4 个同星期日的同刻均值等权平均；样本不足时使用现有已完成日，不用未来日期填补。

$$\hat{y}_{d,t} = 0.5\bar{y}_{\text{recent},t} + 0.5\bar{y}_{\text{weekday},t} \tag{5}$$

分别预测负载和光伏，再以“真实净负荷减当时发布预测”的历史残差构造风险裕度。历史残差必须由原发布版本和已经实现的目标日期计算，不能用事后重拟合误差替代。最多取近 30 个可用起点，按预测时距分别校准。

$$\widetilde{N}_{d,t} = \hat{L}_{d,t} - \hat{P}_{d,t} + \max\{0, Q_q(\varepsilon_{\text{history},t})\} \tag{6}$$

将 $\widetilde{N}$ 代入问题一的连续计划模型。考虑 24、48、72 小时窗口；跨日窗口只执行当天普通购电，次日辅助计划不构成已提交承诺。终端条件候选包括周期约束和终端电量价值。终端单位价值 v 取已知日型中最低 24 个十分钟电价的均值除以充电效率，用作剩余电量的规划代理，不计为实际收入。

4.2 按过去经济效果选择有限策略

预先定义 97 个候选：3 种预测×8 个分位（0、0.65、0.70、0.75、0.80、0.85、0.90、0.95）×4 种窗口终端组合，加 1 个历史完整日场景候选。每月首日，各候选从过去 21 天共同起点出发、独立连续传递 SOC，比较历史实际费用并做终端价值校正；月内固定选中配置。

$$\theta_m = \arg\min_{\theta}\left\{\sum_{d\in\mathcal{H}_{21}} F_d(\theta) - v[E_{\text{end}}(\theta) - E_{\text{start}}]\right\} \tag{7}$$

式（7）只评价已经完成的日期。月度选择不读取当月未来结果；全年固定候选排名仅用于事后诊断。最终 11 个月均选中组合预测、24 小时窗口和终端价值，分位随月份变化，场景候选没有成为正式月度方案。

4.3 实时执行与结算

令 $r_t = G_t + P_t - L_t$。$r_t \ge 0$ 时，充电取剩余量、功率上限和 SOC 可充空间三者最小值，其余弃用；$r_t < 0$ 时，先用可放电量补缺，再紧急购电。

$$D_t = \min\{(-r_t)^+, 5000/6, 0.9(E_t - 1200)\}, \quad H_t = (-r_t - D_t)^+ \tag{8}$$

$$F_2 = \sum_{d,t}(\lambda_t G_{d,t} + 5\lambda_t H_{d,t}) \tag{9}$$

4.4 问题二全年结果与策略解释

表 6 问题二正式区间指标

指标	结果
计划购电费／元	13,847,451.37
紧急购电费／元	911,091.67
总费用／元	14,758,543.05
紧急电量／kWh	146,312.14
紧急时段／日期	923／149
期初／期末 SOC（kWh）	10,800.00／3,453.75
终端价值调整后费用／元	14,761,978.64

正式区间期初 SOC 来自 1 月预运行，为 10,800 kWh，而非重新设成 6,000 kWh。终端价值调整后费用用于比较库存变化，不是向电网收取或支付的额外现金。全年计划费占 93.83%，紧急费占 6.17%。

表 7 问题二月首选择的风险分位

月份	2	3	4	5	6	7	8	9	10	11	12
q	0.70	0.70	0.80	0.80	0.85	0.80	0.75	0.85	0.90	0.90	0.75

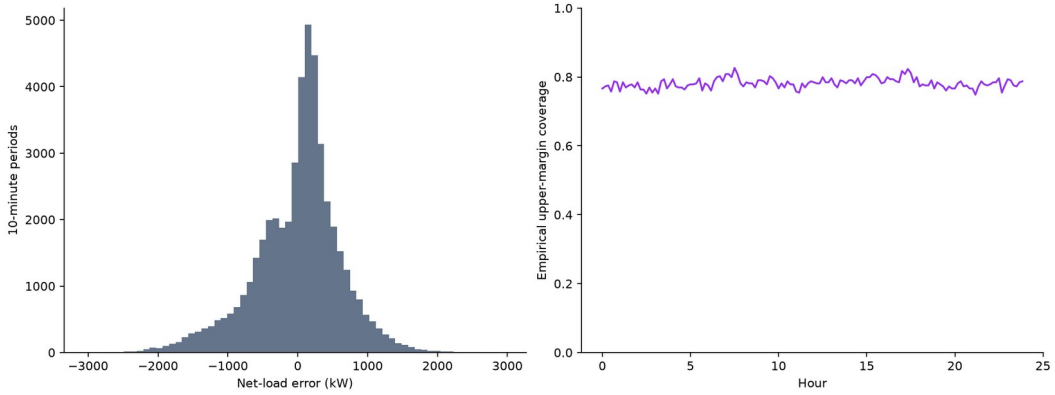


图 2 问题二预测误差与风险裕度经验覆盖

图 2 中的误差按预测版本与实际值配对计算。风险裕度的覆盖率不等于“无紧急购电概率”，因为电池能在时段间吸收偏差。按光伏优先利用的分摊规则，光伏消纳率为 90.57%；由于总弃用存在来源归属不唯一，该数值依赖分摊约定，不作为唯一物理可识别指标。

4.5 问题二指定日期购电结果

下表分别按题面表 1 列出六个指定区间的 0 时计划购电、全天计划量、计划费及包含紧急补电的总费用。电量单位为 kWh，费用为元；“计划费”和“全天总费”明确分列以便复算式（9）。

表 8 2025-03-20 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	535.96	14:00-14:10	0.00
16:00-16:10	486.30	18:00-18:10	694.86	20:00-20:10	0.00
全天计划量	65,378.45	计划费	39,399.28	全天总费	46,467.52

表 9 2025-06-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	646.96	14:00-14:10	0.00
16:00-16:10	430.22	18:00-18:10	596.87	20:00-20:10	0.00
全天计划量	55,766.28	计划费	34,009.68	全天总费	34,009.68

表 10 2025-09-23 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	456.88	14:00-14:10	0.00
16:00-16:10	516.36	18:00-18:10	785.21	20:00-20:10	0.00
全天计划量	67,016.55	计划费	41,524.09	全天总费	45,562.03

表 11 2025-12-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	990.88	14:00-14:10	0.00
16:00-16:10	871.48	18:00-18:10	714.43	20:00-20:10	0.00
全天计划量	92,944.95	计划费	60,534.68	全天总费	60,534.68

四个日期覆盖春分、夏至、秋分和冬至附近的不同供需形态。6 月 21 日没有紧急购电，但不能据此推断全天预测准确：过度采购或可用 SOC 同样可能避免缺口。完整策略及每个日期的汇总见 result2.xlsx。

4.6 问题二指定日期储能结果

以下按题面表 2 报告实际执行的充放电，不使用日前参考储能轨迹替代。全部电量单位为 kWh。每张表右下方列示当日 24 时 SOC，次日模型以该值继续执行。

表 12 2025-03-20 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,658.03	218.72	4:00-8:00	1,375.61	5,974.42
8:00-12:00	5,946.77	2,606.52	12:00-16:00	4,544.99	532.75
16:00-20:00	179.52	6,081.77	20:00-24:00	2,783.06	2,126.87
0:00 储电量	3,747.13		24:00 储电量	3,596.25	

表 13 2025-06-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	3,306.30	0.00	4:00-8:00	344.45	1,419.45
8:00-12:00	1,407.96	0.00	12:00-16:00	0.00	0.00
16:00-20:00	0.00	4,169.91	20:00-24:00	5,286.05	2,485.90
0:00 储电量	7,824.33		24:00 储电量	8,162.11	

表 14 2025-09-23 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,805.16	66.89	4:00-8:00	1,567.17	6,591.34
8:00-12:00	5,175.22	1,896.62	12:00-16:00	5,578.96	549.24
16:00-20:00	58.07	5,901.16	20:00-24:00	2,791.96	2,571.16
0:00 储电量	3,429.88		24:00 储电量	3,679.42	

表 15 2025-12-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	3,315.42	171.36	4:00-8:00	1,408.53	1,499.50
8:00-12:00	5,765.38	7,145.72	12:00-16:00	6,832.24	2,241.53
16:00-20:00	15.43	5,809.02	20:00-24:00	2,786.27	2,878.00
0:00 储电量	7,497.39		24:00 储电量	3,669.29	

4.7 问题二紧急购电结果

即使普通购电最终未全部利用，其计划费用也不退还；紧急购电只用于补足缺口，不额外购买紧急电量给储能充电。实际充放电不强行追随日前参考轨迹。

表 16 指定日期紧急购电区间与电量（对应题面表 3）

03/20	购电量	06/21	购电量	09/23	购电量	12/21	购电量
9:20-10:00	170.86	无	0.00	20:20-20:40	497.66	无	0.00
20:20-20:40	901.00	—	—	20:50-21:10	105.98	—	—
—	—	—	—	21:20-21:30	11.48	—	—
—	—	—	—	21:50-22:00	11.08	—	—

注：所有日期均为 2025 年，电量单位为 kWh；相邻的紧急购电十分钟区间合并为连续事件，数值为区间电量之和。“无”表示当日未发生紧急购电，“—”表示该列没有更多事件。

3 月 20 日紧急购电 1,071.86 kWh，费用 7,068.24 元；9 月 23 日紧急购电 626.19 kWh，费用 4,037.93 元。紧急区间合并只改变展示方式，各十分钟电价仍分别参与计费，不能用区间简单均价乘总电量替代逐时结算。

模型的因果性通过反事实检查：保持历史不变而修改当天及之后真实负载或光伏，0 时计划保持不变；实际动作仅在被修改的信息进入当期之后变化。预报模型的全年误差统计只供评价，不反向修改已经执行的日期。

4.8 不确定性处理的取舍

采用经济选参而非仅比较 MAE，有两个原因：第一，预测偏低触发高价紧急购电，偏高的计划购电又不可退费，两种误差的损失不对称；第二，电池状态使某一时段的偏差可能在后来体现费用。历史闭环重放将误差、SOC 和交易成本共同纳入评价，但仍属于有限候选策略中的选择，不是一般随机控制问题的全局最优解。

较长预测窗口可能改善终端处理，也会增加远期误差。正式选择最终采用 24 小时终端价值方案，说明在本数据与候选集合内，额外时域没有稳定超过简单终端处理的历史经济效果。对复杂模型的取舍以样本外实际费用为依据。

5 问题三 光伏预报更新与承诺调整

5.1 预报版本和十分钟插值

附件 3 每 6 小时提供未来 24 小时整点光伏预报。以发布时已经观测的最后一个光伏功率作为锚点，与未来 24 个整点值线性插值，在十分钟结束点采样并除以 6 转换为电量；整点处严格通过原预报值。常值保持及 PCHIP 作为插值敏感性。负载仍采用问题二组合预测。

阶段特征仅包含当前及以前发布的版本、已完成负载与光伏记录、当前 SOC 和已实现的历史残差。Q3 残差要求对应的完整 48 小时目标窗口全部实现，即使当前只计划至 24 时，也不借用未实现的历史窗口尾部。正式候选采用自然日剩余时域，跨日滚动 24 或 48 小时作为对照。

5.2 非追溯调整与交易账本

$$G_t^k = G_t^{k-1} + U_{k,t} - R_{k,t}, \quad U_{k,t}, R_{k,t} \geq 0, \quad t \geq 36k \tag{10}$$

k＝1、2、3 分别对应 6、12、18 时；t<36k 时保持之前承诺及执行结果不变。原计划 G^0 始终单独保存；输出“调整购电量”给出最终有效承诺，各阶段差额在账本逐笔保留。

$$F_3^A = \sum_t \lambda_t G_t^0 + \sum_{k,t} (1.5\lambda_t U_{k,t} + 0.5\lambda_t R_{k,t}) + \sum_t 5\lambda_t H_t \tag{11}$$

式（11）是原计划费不退款的 A 口径。由于多余能量可以免费弃用，保留一个偏高承诺至少不会比支付额外费用调减更差；因此 A 口径存在不调减的占优选择。正式结果 R＝0 是结算规则与弃用权限共同导致，不能解释成算法主动获得了退款收益。

5.3 调整方案的经济门控

每次更新均保留 KEEP 方案，并用当前预报生成 UPDATE 方案。采用相同当前场景、当前 SOC、预测电价及终端代理价值评价两者，只有预测费用改善超过 10^{-6} 元才调整：

$$\widehat{\mathrm{VOI}} = J(\mathrm{KEEP} \mid \mathcal{I}_k) - J(\mathrm{UPDATE} \mid \mathcal{I}_k) \tag{12}$$

16 个预注册候选覆盖分位、插值、日内偏差校正、规划时域、场景和门控选项。每月仍只利用前 21 个完成日选择。正式选择全部为线性插值、无额外偏差校正、计划至当日 24 时、门控开启；q＝0.65 用于 242 天，q＝0.70 用于 92 天。实际执行继续使用式（8）的反馈控制。

5.4 问题三经济性及结算敏感性

表 17 固定电价下问题二与问题三比较

指标	问题二	问题三
总费用／元	14,758,543.05	14,392,539.81
计划费／元	13,847,451.37	13,298,933.37
调增费／元	0.00	638,870.02
紧急费／元	911,091.67	454,736.42
紧急购电量／kWh	146,312.14	80,128.24
紧急时段数	923	1,303
紧急日期数	149	200
期初 SOC／kWh	10,800.00	8,170.60
期末 SOC／kWh	3,453.75	3,390.75

问题三总费用减少 366,003.24 元（2.48%），紧急购电量减少 66,183.91 kWh（45.23%）。但是紧急时段和日期数均增加，表现为更多小缺口，不能把减少电量写成全面提高可靠性。两种政策的 1 月预运行使正式期初 SOC 不同，因此跨问题现金差包含状态路径和交易权限变化，不是纯光伏信息收益。

表 18 问题三调减结算解释的敏感性

规则	总费用／元	累计调增／kWh	累计调减／kWh
A 原计划不退	14,392,539.81	618,526.19	0.00
B 逐事件净退款	14,367,594.10	798,436.70	941,638.41
B_net 相对 0 时净偏差	14,203,594.94	814,261.13	957,621.42

B 口径对每笔调减先冲销原价再收 50%违约费，故净调整项为 $1.5\lambda U-0.5\lambda R$。B_net 对相对原计划的净偏差函数 $\varphi(x)=1.5\lambda x^{+}-0.5\lambda(-x)^{+}$做阶段差分，避免反复调整的逐事件累计效果。三类模型分别求解，不能仅把同一策略更换账单后称为替代规则最优解。

正式 result3.xlsx 采用 A 口径。问题三主方案共发生 659 次非零阶段调整，调增 618,526.19 kWh；0 时原计划总量与最终有效普通购电总量分别为 21,537,531.51 与 22,156,057.70 kWh。

5.5 问题三指定日期原计划购电

按题面表 1 给出 0 时原计划，各阶段调整之后的完整有效承诺另见 result3.xlsx“调整购电量”。下表全天总费包含原计划费、全部日内调整费及紧急费，电量单位为 kWh，费用为元。

表 19 2025-03-20 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	525.62	14:00-14:10	0.00
16:00-16:10	636.76	18:00-18:10	691.35	20:00-20:10	0.00
全天计划量	66,234.93	计划费	40,172.20	全天总费	44,873.81

表 20 2025-06-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	399.23	18:00-18:10	583.22	20:00-20:10	0.00
全天计划量	43,823.84	计划费	27,647.61	全天总费	27,892.32

表 21 2025-09-23 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	356.46	14:00-14:10	0.00
16:00-16:10	569.79	18:00-18:10	759.11	20:00-20:10	0.00
全天计划量	65,727.49	计划费	40,509.15	全天总费	47,676.09

表 22 2025-12-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	940.86	14:00-14:10	0.00
16:00-16:10	847.42	18:00-18:10	711.63	20:00-20:10	0.00
全天计划量	89,896.85	计划费	58,211.69	全天总费	60,009.88

同一日的原计划费用不能用最终有效普通购电量直接乘原电价代替，后者会漏记调增溢价或历史调减违约费。指定日期表与完整账本按日期、时段逐项对应。

5.6 问题三指定日期储能结果

按题面表 2 给出最终实际储能动作。日内重优化使用当时真实 SOC，已发生的充放电与首末状态不回写。电量单位为 kWh。

表 23 2025-03-20 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,640.05	290.71	4:00-8:00	1,313.52	6,139.38
8:00-12:00	4,917.76	2,226.71	12:00-16:00	6,063.38	254.72
16:00-20:00	18.04	5,859.90	20:00-24:00	2,756.82	2,906.95
0:00 储电量	3,660.45		24:00 储电量	3,556.43	

表 24 2025-06-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	1,790.95	0.00	4:00-8:00	1,069.41	2,341.81
8:00-12:00	4,124.92	0.00	12:00-16:00	0.00	0.00
16:00-20:00	0.00	4,355.01	20:00-24:00	4,707.81	2,485.90
0:00 储电量	7,115.26		24:00 储电量	7,436.02	

表 25 2025-09-23 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,636.27	134.24	4:00-8:00	1,658.04	7,053.82
8:00-12:00	5,206.61	1,476.44	12:00-16:00	5,962.54	411.36
16:00-20:00	58.14	5,953.49	20:00-24:00	2,702.22	2,766.08
0:00 储电量	3,358.59		24:00 储电量	3,587.32	

表 26 2025-12-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	3,408.16	254.21	4:00-8:00	1,721.66	1,598.12
8:00-12:00	5,603.17	7,502.93	12:00-16:00	7,533.73	2,282.65
16:00-20:00	17.09	5,907.49	20:00-24:00	2,709.53	2,820.87
0:00 储电量	7,290.96		24:00 储电量	3,555.78	

5.7 问题三紧急购电和新预报价值

表 27 指定日期紧急购电区间与电量（对应题面表 3）

03/20	购电量	06/21	购电量	09/23	购电量	12/21	购电量
9:00-10:00	550.67	无	0.00	8:40-9:50	420.17	7:50-8:00	5.51
20:30-20:40	135.51	—	—	20:20-22:00	765.49	20:30-20:40	96.18
21:50-22:00	2.44	—	—	—	—	—	—

注：所有日期均为 2025 年，电量单位为 kWh；相邻的紧急购电十分钟区间合并为连续事件，数值为区间电量之和。“无”表示当日未发生紧急购电，“—”表示该列没有更多事件。

5.8 区分更新机会与新光伏信息

为回答是否需要其他时刻预报，设置三层对照。第一层保持同一正式配置日历，只删除调整时点；其变化同时移除了重新优化和更新机会。第二层允许仅 0 时策略独立用历史选参，避免借用全更新方案的配置。第三层保留全部调整时点和 SOC 反馈，仅以旧版本光伏替换新版本，尽量隔离新 PV 预报价值。

表 28 预报与调整信息对照

对照条件	总费用／元	相对全更新多付／元
仅 0 时，共享全更新配置	15,732,654.14	1,340,114.33
仅 0 时，独立历史选参	14,943,794.61	551,254.80
可全时点调整，但沿用旧 PV	14,462,650.04	70,110.23
全部时点与新 PV	14,392,539.81	0.00

因此，本数据支持保留日内更新机制；在相同调整权限下，新 PV 信息也有正的整体费用改善。上述改善仍依赖固定候选及其历史选择规则，不是跨市场普遍收益保证。各时点收益具有交互性，不能将余日重叠的局部事后价值相加为年度收入。

5.9 分时点消融与跨日信息解释

表 29 保持调整权限而单独屏蔽新 PV 版本

屏蔽时点	总费用／元	屏蔽后费用增加／元
6 时	14,391,691.66	−848.15
12 时	14,394,891.64	2,351.83
18 时	14,392,725.89	186.08

6 时新 PV 版本在这一固定对照中没有正的边际费用贡献，而 12 时和 18 时为正。这个结果与“整套日内机制有效”并不矛盾：额外信息可能包含噪声，模型又是有限策略和有限样本选择。它提示无需将每次新预报都无条件视为优于旧预报。

18 时发布的预报包含次日部分信息。正式策略只计划到当日 24 时，因而并未把全部次日预报直接变成次日购电承诺。跨日 48 小时候选与仅替换次日新 PV 信息的实验用于判断前瞻价值；在现有实验中未观察到额外的次日新 PV 收益，不能把 18 时更新的余日收益全部归因于跨日信息。

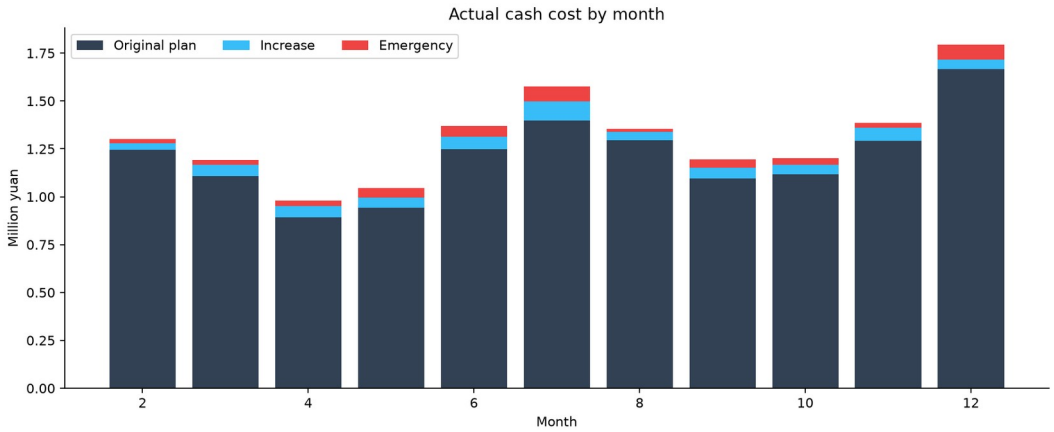


图 3 问题三计划 调整与紧急购电月度费用

图 3 说明费用改善来自原计划、调整与紧急补电之间的组合变化。调增虽然按 1.5 倍价格支付，但若能替代随后 5 倍紧急购电，仍可能降低总费用；门控同时考虑调整费、紧急费及储电量变化，而不是仅按光伏预测增减购电。

6 问题四 波动电价下的购电与执行

6.1 决策价格与结算价格分层

附件 4 包含实际发生的全年电价，但没有明确说明每日 0 时是否已公布全天报价。主模型假定未来未知，仅已完成的价格用于预测；当前执行报价可观察。普通购电及调整项按交付时段真实价格结算，所有未来实现价格只在环境结算和事后诊断中使用。全天已知价格以及锁定决策时报价另列敏感性。

电价预测比较昨日同刻、近 7 日均值、近期与同星期组合、工作日类型滚动中位数、30 日 Ridge 和扩展窗 Ridge。Ridge 使用滞后 1 日、7 日、近 7 日均值、日内正余弦和周末标记，正则系数 10，截距仅作极小正则；训练目标严格早于发布日。日内更新用最近 1 小时已实现价格偏差作修正，并按 2 小时尺度指数衰减。该设计借鉴电价预测基准强调的简单模型对照与时序评价原则[3]。

6.2 历史联合残差与风险评价

将负载、光伏和价格残差按同一历史发布日及目标时间对齐，保留整条路径，避免逐点独立抽样破坏日内相关性。每条历史路径都要完整实现后才可使用，截断后光伏与负载非负、价格至少 0.001。联合场景候选仅在 4 个风险分位产生的可执行购电策略中比较，不是完整多阶段随机规划的精确求解。

$$J = \frac{1}{S}\sum_{s}[\mathrm{Cash}_s - v(E_{s,\mathrm{end}} - E_{\mathrm{start}})] + \beta \mathrm{CVaR}_{\alpha}(\mathrm{Cash}) \tag{13}$$

$$\mathrm{CVaR}_{\alpha}(\mathrm{Cash}) = \min_{\zeta}\left\{\zeta + \frac{\sum_{s}(\mathrm{Cash}_s - \zeta)^{+}}{S(1-\alpha)}\right\} \tag{14}$$

式（14）采用 Rockafellar 与 Uryasev 的尾部损失表示[4]。实现对经验分布的所有候选 ζ 逐一求值，保留分位点处的部分概率质量；风险项作用于现金损失，均值项才进行终端价值调整，避免把论文写成与代码不同的目标。价格加权分位模型则用场景电价作净负荷残差权重，使高价缺口获得更高权重。

6.3 固定购电承诺的储能执行优化

参考滚动能量管理的思路[6]，设置 2 小时或 6 小时储能前瞻候选。普通承诺 G 固定，在当前供需和未来预测上优化 5χH−vE 末，每次只执行第一个动作。充电不超过预测剩余，放电不超过预测缺口；当前状态和当前负荷用真实值，其余使用预测值。该执行器不允许买入超出负载缺口的紧急电量用于套利充电。

问题四之二整天 G 不变；问题四之三仅 0、6、12、18 时允许更改未执行承诺。26 个配置按前 21 天历史费用逐月选择；两类权限分别选择、分别推进 SOC，不能共用某一方案的未来状态。

6.4 波动电价正式结果与信息对照

表 30 问题四全年费用及风险指标

指标	对应问题二	对应问题三
计划费／元	14,277,225.37	13,816,206.74
调增费／元	0.00	674,659.40
紧急费／元	1,122,093.47	517,145.37
总费用／元	15,399,318.84	15,008,011.51
紧急电量／kWh	175,821.72	90,025.48
紧急时段数	1,199	1,609
最差 5%日期平均费／元	87,090.02	79,541.48
高价时段紧急量／kWh	118,104.58	40,987.20

最差 5%日均费取 334 天中费用最大的 17 天均值；高价时段按正式区间实现电价上 10%分位事后划分，仅用于风险评价。两方案总费用差 391,307.33 元（2.54%）同时包含光伏信息、调整权限和 SOC 路径变化。

表 31 价格信息及旧策略对照

条件	问题四之二／元	问题四之三／元
正式因果预测	15,399,318.84	15,008,011.51
当天价格已知 Oracle-P	15,421,944.50	14,895,486.21
完全信息同首末 SOC 松弛	12,782,264.30	12,783,366.61
冻结旧策略动态重计价	15,379,546.79	15,014,850.02

Oracle-P 只揭示价格，保留相同配置日历及其余预测误差，不保证优于原策略；已揭示价格的场景残差设为零。完全信息松弛匹配各自正式期初、期末 SOC，并允许在已知完整供需和价格下规划整段普通购电，可作为该主结算边界的理想化下界，但不是可执行在线计划。

动态重计价固定旧 Q2/Q3 动作，再用附件 4 结算。正式方案相对它分别多付 19,772.05 元和少付 6,838.51 元；因此不能声称增加价格预测在两类权限下都创造净收益。Q4 之三在正式承诺固定时改用原反馈执行，费用为 15,009,663.44 元，比正式执行多 1,651.93 元，这一对照更接近固定承诺条件下的执行机制价值。

6.5 问题四指定日期计算结果

表 32 波动电价对应问题 2 的指定日期费用

日期	计划费／元	调增费／元	紧急费／元	总费／元
03-20	40,343.85	0.00	9,635.45	49,979.30
06-21	28,095.98	0.00	0.00	28,095.98
09-23	43,001.08	0.00	5,710.41	48,711.49
12-21	71,476.54	0.00	59.39	71,535.93

表 33 波动电价对应问题 2 的指定日期电量

日期	原计划／kWh	有效承诺／kWh	紧急量／kWh	日初 SOC	日末 SOC
03-20	65,597.91	65,597.91	1,568.24	4,455.01	5,022.53
06-21	54,650.63	54,650.63	0.00	6,713.89	6,924.29
09-23	66,346.38	66,346.38	892.92	1,264.82	1,423.94
12-21	93,257.09	93,257.09	7.61	10,800.00	7,333.47

表 34 波动电价对应问题 3 的指定日期费用

日期	计划费／元	调增费／元	紧急费／元	总费／元
03-20	41,544.79	993.57	3,682.89	46,221.25
06-21	23,273.11	204.76	0.00	23,477.87
09-23	42,364.17	444.85	6,704.69	49,513.71
12-21	68,148.28	1,523.70	789.71	70,461.69

表 35 波动电价对应问题 3 的指定日期电量

日期	原计划／kWh	有效承诺／kWh	紧急量／kWh	日初 SOC	日末 SOC
03-20	67,010.34	67,855.39	672.43	4,462.58	5,022.53
06-21	42,629.28	42,772.41	0.00	6,016.62	5,327.04
09-23	65,563.60	65,845.55	1,198.09	1,256.09	1,374.85
12-21	89,177.56	90,236.88	101.69	10,795.81	6,524.96

以上为 2025 年指定日期的对应计算结果。五个正式工作簿都保存题目要求的全部计划、储能和紧急区间；波动电价下指定时段原计划和储能明细另列附录 C。当前结果没有将 Oracle 数值写入正式工作簿。

两类方案在 9 月 23 日仍发生较高紧急费用，说明实时调整无法完全消除供需预测误差；6 月 21 日没有紧急购电。冬至附近日初 SOC 较高并不意味着全天没有紧急风险，储能还受到每个十分钟 833.333 kWh 放电上限约束。

7 模型检验与稳健性

7.1 独立的物理与结算核验

从原始附件和十分钟明细独立重算能量平衡、SOC 递推、非负性、功率上限与充放电互斥，不把汇总文件当作计算源。问题二至四各核对 48,096 个时段；Q3 和 Q4 之三分别按 72,144 条阶段记录恢复全部承诺，核对每笔调增、调减与价格费用。工作簿按日期和时段全量回读，不仅比较总和。

表 36 算法和结果核验范围

检查	证据或结果
Q1／Q2／Q3／Q4 自动测试	12／17／24／29 项，共 82 项通过
能量平衡与 SOC 递推	正式数据最大残差在 10^{-12} kWh 量级
充放电互斥和功率限制	逐十分钟检查通过
跨日 SOC 连续	问题二至四无重置断点
信息隔离反事实	未来数据变化不影响此前计划与动作
多次调整与结算	上一承诺、最终承诺和逐笔费用相符
输出合同	五个工作簿按模板工作表全量回读

测试只能验证覆盖到的性质，不等同于无条件证明所有模型假设正确。当前十分钟聚合反馈、效率解释、调减计费及价格发布时间均保留明确假设。

7.2 效率 风险和状态边界

Q1 往返效率 90%的替代费用为 33,801.50 元；Q2 同解释下总费比主口径低约 3.12%，表明效率不是可忽略的表述差异。风险分位提高可减少部分紧急购电，但也增加计划冗余，历史经济选参用于平衡两者。

Q4 联合场景与独立价格残差重排、不同 CVaR 权重、执行时域及电价模型均保留对照。独立重排只改变价格场景与供需场景的配对，不能笼统称为所有变量、所有时段完全独立。CVaR 的引入也不保证平均费用必然降低。

Q2 和 Q3 正式期初／末 SOC 分别为 10,800／3,453.75 和 8,170.60／3,390.75 kWh，现金比较并非等状态实验。按同一终端代理价值校正后，总费用分别为 14,761,978.64 元与 14,394,775.18 元；库存校正不是实际账单。问题四两方案同为 10,800 kWh 期初，但期末不同，同样分别报告同边界完全信息参考。

8 模型评价与结论

8.1 优点与局限

统一物理方程使四问的差别明确对应信息和交易权限；原计划、承诺变更和紧急购电分层保留，使输出能够逐笔解释。预报不以全年误差事后选优，而以过去完成日的闭环经济效果选择；线性计划与有限候选控制便于计算和复现。

主要局限包括：仅有一个年度、单一系统的数据，缺少天气与节假日外生信息；十分钟反馈不能刻画区间内部瞬时波动；当前未计储能老化和并网设备约束；历史窗口及候选范围限定了优化质量。除问题一确定性模型外，问题二至四不宣称原随机控制问题的全局最优。信息消融是在给定策略族内的数值实验，不等同于统计意义上的普遍因果结论。

8.2 结论

问题一获得 35,126.95 元的确定性最优日费用。问题二通过因果预测和历史经济选参得到 14,758,543.05 元的正式区间费用。问题三日内机制将费用降至 14,392,539.81 元，在保持调整权限的旧 PV 对照中，新预报对应 70,110.23 元改善，但单个时点的新预报不一定有正边际收益。

波动电价下对应问题二和三的费用分别为 15,399,318.84 元和 15,008,011.51 元。经济收益需要在信息合法可得、承诺不可追溯修改、结算一致及 SOC 连续的条件下评价。实际方案应同时关注紧急购电量、发生频率和高价暴露，而不能仅追求平均预测误差或最低样本内费用。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于问题背景与文献检索辅助、建模思路讨论、代码及结果核验、复现程序适配、论文草稿生成与排版，详细使用情况见支撑材料。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略及附件[Z]. 2026.

[2] SILVA V A, AOKI A R, LAMBERT-TORRES G. Optimal day-ahead scheduling of microgrids with battery energy storage system[J]. Energies, 2020, 13(19): 5188. DOI:10.3390/en13195188.

[3] LAGO J, MARCJASZ G, DE SCHUTTER B, et al. Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark[J]. Applied Energy, 2021, 293: 116983. DOI:10.1016/j.apenergy.2021.116983.

[4] ROCKAFELLAR R T, URYASEV S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21-41. DOI:10.21314/JOR.2000.038.

[5] SciPy Developers. scipy.optimize.milp[EB/OL]. [2026-09-13]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html.

[6] ZHANG Y, MENG F, WANG R, et al. A stochastic MPC based approach to integrated energy management in microgrids[J]. Sustainable Cities and Society, 2018, 41: 349-362. DOI:10.1016/j.scs.2018.05.044.

附录 A 支撑材料文件列表与复现说明

本文附录与摘要、正文连续编号。下列路径均相对于支撑材料压缩包的顶层目录“支撑材料/”。全部完整源程序列于附录 B；指定日期波动电价明细列于附录 C。代码读取赛题官方附件，自主检索文献只用于方法参考，未引入任何外部数值数据训练正式模型。

先在 code/附件/放入附件 1 至 4 以及附件 5 的五个空模板，再按 README.md 运行。Python 数值计算依赖 NumPy、SciPy、openpyxl 和 Matplotlib；问题四提供公开 Python 导出器 export_workbooks_public.py，原可选 Node 导出器保留。公开导出器写入数值汇总，因此数值结果等价，不要求绘图、文件时间戳或 Excel 压缩字节完全相同。

001 AI 工具使用详情.pdf
002 README.md
003 code/Q1/plot_preprocessing.py
004 code/Q1/solve_q1.py
005 code/Q1/test_q1.py
006 code/Q1/time_mapping.py
007 code/Q2/forecast_q2.py
008 code/Q2/report_q2.py
009 code/Q2/solve_q2.py
010 code/Q2/strategy_q2.py
011 code/Q2/test_q2.py
012 code/Q3/audit_q3.py
013 code/Q3/diagnostics_q3.py
014 code/Q3/prepare_q3.py
015 code/Q3/q3_forecast.py
016 code/Q3/q3_model.py
017 code/Q3/report_q3.py
018 code/Q3/solve_q3.py
019 code/Q3/test_q3.py
020 code/Q4/audit_q4.py
021 code/Q4/export_workbooks.mjs
022 code/Q4/export_workbooks_public.py
023 code/Q4/literature_q4.py
024 code/Q4/prepare_q4.py
025 code/Q4/price_diagnostics.py
026 code/Q4/q4_execution_mpc.py
027 code/Q4/q4_features.py
028 code/Q4/q4_model.py
029 code/Q4/q4_price_forecast.py
030 code/Q4/q4_risk.py
031 code/Q4/q4_utils.py
032 code/Q4/report_q4.py
033 code/Q4/solve_q4.py
034 code/Q4/supplement_q4.py
035 code/Q4/test_q4.py
036 code/microgrid_adjustment.py
037 code/microgrid_common.py
038 code/microgrid_forecasts.py
039 code/microgrid_online.py
040 code/reproduce.py
041 config/requirements.txt
042 figures/Q1/dispatch.png
043 figures/Q1/input_daily_patterns.png
044 figures/Q2/figures/2025-03-20.png
045 figures/Q2/figures/2025-06-21.png
046 figures/Q2/figures/2025-09-23.png
047 figures/Q2/figures/2025-12-21.png
048 figures/Q2/figures/forecast_errors.png

figures/Q2/figures/monthly_costs.png
figures/Q2/figures/quantile_window_sensitivity.png
figures/Q3/figures/2025-03-20.png
figures/Q3/figures/2025-06-21.png
figures/Q3/figures/2025-09-23.png
figures/Q3/figures/2025-12-21.png
figures/Q3/figures/forecast_and_value.png
figures/Q3/figures/monthly_costs.png
figures/Q4/figures/price_forecast_precheck.png
figures/Q4/figures/q42_2025-03-20.png
figures/Q4/figures/q42_2025-06-21.png
figures/Q4/figures/q42_2025-07-03.png
figures/Q4/figures/q42_2025-08-15.png
figures/Q4/figures/q42_2025-09-05.png
figures/Q4/figures/q42_2025-09-23.png
figures/Q4/figures/q42_2025-12-02.png
figures/Q4/figures/q42_2025-12-14.png
figures/Q4/figures/q42_2025-12-21.png
figures/Q4/figures/q42_monthly_costs.png
figures/Q4/figures/q42_risk_pareto.png
figures/Q4/figures/q43_2025-03-20.png
figures/Q4/figures/q43_2025-06-21.png
figures/Q4/figures/q43_2025-07-03.png
figures/Q4/figures/q43_2025-08-15.png
figures/Q4/figures/q43_2025-09-05.png
figures/Q4/figures/q43_2025-09-23.png
figures/Q4/figures/q43_2025-12-01.png
figures/Q4/figures/q43_2025-12-14.png
figures/Q4/figures/q43_2025-12-21.png
figures/Q4/figures/q43_monthly_costs.png
figures/Q4/figures/q43_risk_pareto.png
intermediate/Q1/dispatch_10min.csv
intermediate/Q1/efficiency_sensitivity.csv
intermediate/Q1/input_statistics.json
intermediate/Q1/summary.json
intermediate/Q1/table1.csv
intermediate/Q1/table2.csv
intermediate/Q1/time_mapping.csv
intermediate/Q2/daily_summary.csv
intermediate/Q2/emergency_purchase_merged.csv
intermediate/Q2/experiments/candidate_summary.csv
intermediate/Q2/experiments/formal_metrics.json
intermediate/Q2/experiments/monthly_selection.csv
intermediate/Q2/experiments/oracle.json
intermediate/Q2/experiments/scenario_pressure.csv
intermediate/Q2/experiments/sensitivities.json
intermediate/Q2/forecast_metrics.csv
intermediate/Q2/information_audit.csv
intermediate/Q2/selected_days/2025-03-20_table1.csv
intermediate/Q2/selected_days/2025-03-20_table2.csv
intermediate/Q2/selected_days/2025-06-21_table1.csv
intermediate/Q2/selected_days/2025-06-21_table2.csv
intermediate/Q2/selected_days/2025-09-23_table1.csv
intermediate/Q2/selected_days/2025-09-23_table2.csv
intermediate/Q2/selected_days/2025-12-21_table1.csv
intermediate/Q2/selected_days/2025-12-21_table2.csv
intermediate/Q2/selected_days/summary.csv
intermediate/Q2/summary.json
intermediate/Q2/validation.json
intermediate/Q3/daily_summary.csv
intermediate/Q3/emergency_purchase_merged.csv
intermediate/Q3/experiments/calibration.csv
intermediate/Q3/experiments/candidate_summary.csv
intermediate/Q3/experiments/information_ablation.csv
intermediate/Q3/experiments/monthly_selection.csv

intermediate/Q3/experiments/sensitivities.csv
intermediate/Q3/experiments/stage_value.csv
intermediate/Q3/experiments/supplemental.csv
intermediate/Q3/forecast_metrics.csv
intermediate/Q3/selected_days/2025-03-20_table1.csv
intermediate/Q3/selected_days/2025-03-20_table2.csv
intermediate/Q3/selected_days/2025-06-21_table1.csv
intermediate/Q3/selected_days/2025-06-21_table2.csv
intermediate/Q3/selected_days/2025-09-23_table1.csv
intermediate/Q3/selected_days/2025-09-23_table2.csv
intermediate/Q3/selected_days/2025-12-21_table1.csv
intermediate/Q3/selected_days/2025-12-21_table2.csv
intermediate/Q3/selected_days/summary.csv
intermediate/Q3/summary.json
intermediate/Q3/validation.json
intermediate/Q4/experiments/q42_candidate_summary.csv
intermediate/Q4/experiments/q42_comparisons.json
intermediate/Q4/experiments/q42_extreme_days.json
intermediate/Q4/experiments/q42_monthly_selection.csv
intermediate/Q4/experiments/q43_candidate_summary.csv
intermediate/Q4/experiments/q43_comparisons.json
intermediate/Q4/experiments/q43_extreme_days.json
intermediate/Q4/experiments/q43_monthly_selection.csv
intermediate/Q4/price_daily.csv
intermediate/Q4/price_interval_metrics.csv
intermediate/Q4/price_metrics.csv
intermediate/Q4/price_profile.csv
intermediate/Q4/q42_daily_summary.csv
intermediate/Q4/q42_emergency_purchase_merged.csv
intermediate/Q4/q42_summary.json
intermediate/Q4/q43_daily_summary.csv
intermediate/Q4/q43_emergency_purchase_merged.csv
intermediate/Q4/q43_summary.json
intermediate/Q4/selected_days/q42_summary.csv
intermediate/Q4/selected_days/q43_summary.csv
intermediate/Q4/validation.json
references/文献来源与使用范围.md
references_used.md
results/result1.xlsx
results/result2.xlsx
results/result3.xlsx
results/result4-2.xlsx
results/result4-3.xlsx

附录 B 完整源程序

以下为支撑材料中全部 Python 与 JavaScript 源文件的完整文本，包括求解、报告、输出、测试和关键审计程序。代码以文件标题分隔；长行在排版时自动换行，实际运行应使用 ZIP 中的原文件。源文件不以省略号替代。报告程序内的独立分问说明为复现辅助文本，正式论文结论以本文正文为准。

B1 code/Q1/plot_preprocessing.py

```
"""生成问题 1 预处理阶段的三组输入数据日内特征图，不读取或绘制优化结果。"""
from pathlib import Path
import json
import os
import sys

ROOT = Path(__file__).resolve().parents[1]
OUT = Path(__file__).resolve().parent
os.environ.setdefault('MPLCONFIGDIR', str(OUT/'.matplotlib'))
sys.path.insert(0, str(ROOT))

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
import numpy as np

from microgrid_common import read_single_day
from time_mapping import canonical_periods


def _font():
    path = Path('/System/Library/Fonts/STHeiti Medium.ttc')
    return font_manager.FontProperties(fname=str(path)) if path.exists() else None


def _clock(index):
    minutes = index*10
    return f'{minutes//60:02d}:{minutes%60:02d}'


def input_statistics(data):
    price = np.asarray(data['price'])
    load_kw = np.asarray(data['load'])*6
    pv_kw = np.asarray(data['pv'])*6
    return {
        'record_count': len(price),
        'period_start': canonical_periods(data['end_minutes'])[0],
        'period_end': canonical_periods(data['end_minutes'])[-1],
        'price_min_yuan_per_kwh': float(price.min()),
        'price_min_period': canonical_periods(data['end_minutes'])[int(price.argmin())],
        'price_max_yuan_per_kwh': float(price.max()),
        'price_max_period': canonical_periods(data['end_minutes'])[int(price.argmax())],
        'load_max_kw': float(load_kw.max()),
        'load_max_period': canonical_periods(data['end_minutes'])
[int(load_kw.argmax())],
        'pv_max_kw': float(pv_kw.max()),
        'pv_max_period': canonical_periods(data['end_minutes'])[int(pv_kw.argmax())],
    }


def plot_input_profiles(data, output_dir=OUT):
    """只接收附件 1 预处理后的输入字段，输出 PNG、SVG 和描述统计 JSON。"""
```

```
    output_dir = Path(output_dir)
    price = np.asarray(data['price'])
    load_kw = np.asarray(data['load'])*6
    pv_kw = np.asarray(data['pv'])*6
    ends = np.asarray(data['end_minutes'])
    if len(price) != 144 or not (len(load_kw) == len(pv_kw) == len(ends)):
        raise ValueError('输入图必须使用完整的144个十分钟时段')
    x = (ends-5)/60  # 每个十分钟区间的中心
    cn = _font()
    plt.rcParams.update({'axes.unicode_minus': False, 'font.size': 10})
    colors = ['#334155', '#d99a13', '#be185d']
    fig, axes = plt.subplots(3, 1, figsize=(11.2, 8.2), sharex=True,
                             gridspec_kw={'hspace': .13})
    series = [
        (load_kw, '小区负荷', '功率（kW）', colors[0]),
        (pv_kw, '光伏预测', '功率（kW）', colors[1]),
        (price, '外网电价', '电价（元/kWh）', colors[2]),
    ]
    for ax, (values, label, ylabel, color) in zip(axes, series):
        ax.plot(x, values, color=color, linewidth=1.8, label=label)
        ax.fill_between(x, values, color=color, alpha=.12)
        ax.set_ylabel(ylabel, fontproperties=cn)
        ax.legend(loc='upper right', frameon=False, prop=cn)
        ax.grid(axis='both', alpha=.2, linewidth=.7)
        ax.set_xlim(0, 24)
        for hour in (6, 12, 18):
            ax.axvline(hour, color='#94a3b8', linestyle='--', linewidth=.8, alpha=.75)
    annotations = [
        (int(load_kw.argmax()), f'日最大负荷{load_kw.max():.1f} kW'),
        (int(pv_kw.argmax()), f'光伏峰值{pv_kw.max():.1f} kW'),
        (int(price.argmin()), f'最低电价{price.min():.4f} 元/kWh'),
    ]
    for ax, values, (index, label), color in zip(
            axes, [load_kw, pv_kw, price], annotations, colors):
        ax.scatter([x[index]], [values[index]], color=color, s=28, zorder=3)
        ax.annotate(label, (x[index], values[index]), xytext=(8, 10),
                    textcoords='offset points', fontproperties=cn, color=color,
                    arrowprops={'arrowstyle':'-', 'color':color, 'lw':.8})
    axes[-1].set_xticks(np.arange(0, 25, 2))
    axes[-1].set_xlabel('自然日时刻', fontproperties=cn)
    fig.suptitle('问题1三组输入数据的日内变化特征', fontproperties=cn, fontsize=16)
    fig.text(.5, .01, '虚线标示6:00、12:00 和18:00；曲线仅来自附件1输入数据',
             ha='center', fontproperties=cn, fontsize=9, color='#475569')
    fig.subplots_adjust(left=.10, right=.98, top=.92, bottom=.08)
    fig.savefig(output_dir/'input_daily_patterns.png', dpi=220)
    fig.savefig(output_dir/'input_daily_patterns.svg')
    plt.close(fig)
    stats = input_statistics(data)
    (output_dir/'input_statistics.json').write_text(
        json.dumps(stats, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
    return stats


if __name__ == '__main__':
    result = plot_input_profiles(read_single_day(ROOT/'附件/附件1.xlsx'))
    print(json.dumps(result, ensure_ascii=False, indent=2))
```

B2 code/Q1/solve_q1.py

```
"""运行本文件可重建问题1的全部结果、图件、论文表格及检验记录。"""
from pathlib import Path
import csv
import hashlib
import json
import os
import sys
```

```
from dataclasses import asdict, replace

ROOT = Path(__file__).resolve().parents[1]
OUT = Path(__file__).resolve().parent
os.environ.setdefault('MPLCONFIGDIR', str(OUT / '.matplotlib'))
sys.path.insert(0, str(ROOT))
import numpy as np
import scipy
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from openpyxl import load_workbook
from microgrid_common import Battery, clock_label, read_single_day, optimize, validate
from time_mapping import canonical_periods, period_bounds, validate_export_periods
from plot_preprocessing import plot_input_profiles


def csv_write(name, header, rows):
    with (OUT/name).open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.writer(f); writer.writerow(header); writer.writerows(rows)


def main():
    if any(OUT.glob('freeze_manifest*.json')):
        raise RuntimeError('Q1 已冻结，禁止原地覆盖。请按冻结说明在独立解压副本中复现。')
    data = read_single_day(ROOT/'附件/附件 1.xlsx')
    input_stats = plot_input_profiles(data, OUT)
    b = Battery()
    lp = optimize(data['price'],data['load'],data['pv'],b)
    s = optimize(data['price'],data['load'],data['pv'],b,integer=True)
    checks = validate(data,s,b)
    checks['milp_minus_lp_yuan'] = float(s['cost']-lp['cost'])
    if abs(checks['milp_minus_lp_yuan']) > 1e-5:
        raise AssertionError('MILP 与 LP 下界不一致，需检查最优性')
    template = ROOT/'附件/附件 5/result1.xlsx'
    template_wb = load_workbook(template, data_only=False)
    template_purchase = template_wb['计划购电量']
    template_labels = [template_purchase.cell(t,1).value for t in range(2,146)]
    template_headers = {
        ws.title: [ws.cell(1,c).value for c in range(1,ws.max_column+1)]
        for ws in template_wb
    }
    template_dimensions = {
        ws.title: (ws.max_row,ws.max_column)
        for ws in template_wb
    }
    template_styles = {
        ws.title: [[ws.cell(r,c).style_id for c in range(1,ws.max_column+1)]
                   for r in range(1,ws.max_row+1)]
        for ws in template_wb
    }
    template_wb.close()
    if len(template_labels) != 144 or len(set(template_labels)) != 144:
        raise AssertionError('官方模板的计划购电时间标签应为 144 个且不得重复')
    model_labels = canonical_periods(data['end_minutes'])
    template_offsets = [period_bounds(old)[0]-period_bounds(new)[0]
                        for old,new in zip(template_labels,model_labels)]
    if set(template_offsets) not in ({0}, {10}):
        raise ValueError('模板时间标签出现未知变化，需要重新核对，不能静默按行覆盖')
    for label,offset,end in zip(template_labels,template_offsets,data['end_minutes']):
        if period_bounds(label) != (int(end)-10+offset,int(end)+offset):
            raise ValueError('模板区间长度或结束时刻异常')
    base_grid = np.maximum(data['load']-data['pv'],0)
    base_cost = float(data['price']@base_grid)
    groups = [(f'{h}:00-{h+4}:00', float(s['charge'][h*6:(h+4)*6].sum()),
               float(s['discharge'][h*6:(h+4)*6].sum())) for h in range(0,24,4)]
```

```
    selected_indices = [60,72,84,96,108,120]
    selected = [(model_labels[t],float(s['grid'][t])) for t in selected_indices]
    rows = [[model_labels[t], data['price'][t],data['load'][t]/b.dt,data['pv'][t]/b.dt,
             s['grid'][t],s['charge'][t],s['discharge'][t],s['curtail'][t],
             s['energy'][t],s['energy'][t+1],data['price'][t]*s['grid'][t],base_grid[t]]
            for t in range(144)]
    header = ['时间段','电价_元每kWh','负载_kW','光伏预测_kW','购电_kWh','充电_母线侧_kWh',
              '放电_母线侧_kWh','弃光_kWh','期初储电_kWh','期末储电_kWh','购电费_元','无储能购电
_kWh']
    csv_write('dispatch_10min.csv',header,rows)
    csv_write('table1.csv',['时间段','购电量_kWh'],selected+[('全
天',float(s['grid'].sum()))])
    csv_write('table2.csv',['时间段','充电量_kWh','放电量_kWh'],groups)
    wb = load_workbook(template)
    ws = wb['计划购电量']
    for t in range(144):
        row = t+2
        if ws.cell(row,1).value != template_labels[t]:
            raise AssertionError('导出前模板时间标签发生变化')
        ws.cell(row,1,model_labels[t])
        ws.cell(row,2,float(s['grid'][t]))
    ws = wb['充放电量']
    for i,(_,c,d) in enumerate(groups,2):
        ws.cell(i,2,c); ws.cell(i,3,d)
    ws['E2']=float(s['energy'][0]); ws['E3']=float(s['energy'][-1])
    wb.save(OUT/'result1.xlsx')
    exported=load_workbook(OUT/'result1.xlsx',data_only=True)
    assert exported.sheetnames==['计划购电量','充放电量']
    assert {
        ws.title:(ws.max_row,ws.max_column) for ws in exported
    } == template_dimensions
    assert {
        ws.title:[ws.cell(1,c).value for c in range(1,ws.max_column+1)]
        for ws in exported
    } == template_headers
    assert {
        ws.title:[[ws.cell(r,c).style_id for c in range(1,ws.max_column+1)]
                  for r in range(1,ws.max_row+1)]
        for ws in exported
    } == template_styles
    for t in range(144):
        row = t+2
        assert exported['计划购电量'].cell(row,1).value==model_labels[t]
        assert abs(exported['计划购电量'].cell(row,2).value-s['grid'][t])<1e-8
    validate_export_periods([exported['计划购电量'].cell(row,1).value
                             for row in range(2,146)],data['end_minutes'])
    for i,(_,c,d) in enumerate(groups,2):
        assert abs(exported['充放电量'].cell(i,2).value-c)<1e-8
        assert abs(exported['充放电量'].cell(i,3).value-d)<1e-8
    assert exported['充放电量']['E2'].value==exported['充放电量']['E3'].value==6000
    by_period = {exported['计划购电量'].cell(row,1).value:
                 exported['计划购电量'].cell(row,2).value for row in range(2,146)}
    for label,value in selected:
        assert abs(by_period[label]-value)<1e-8
    exported.close()
    csv_write('time_mapping.csv',['模型索引','附件1行号','附件1时刻_按结束点解释','输出行号',
              '原模板标签','模型自然日区间','输出Excel标签','原标签偏移_分钟','购电量_kWh'],
              [[t,t+2,clock_label(int(data['end_minutes'][t])),t+2,template_labels[t],
                model_labels[t],model_labels[t],template_offsets[t],float(s['grid'][t])]
for t in range(144)])
    checks['template_match']='工作表、行列数、表头及样式沿用附件5；仅修正144个时间标签并填入所需数
值'
    checks['time_label_match']='144个Excel标签逐项匹配模型及CSV，连续覆盖0:00至24:00；指定时
段按标签查找核验'
    checks['unit_match']='购电量、充放电量和储电量均为kWh，购电费为元'
    checks['excel_export_order']='模型grid[0:144]按原始顺序逐项写入模板第2至145行，无循环平移'
```

```
    checks['table_indices']='表1索引为60、72、84、96、108、120；表2为六组连续24时段自然日汇总'
    checks['export_recheck']='144个购电量、6个指定时段、6组四小时充放电量及首末电量均已回读核对'
    alt_b=replace(b,eta_charge=np.sqrt(.9),eta_discharge=np.sqrt(.9))
    alt=optimize(data['price'],data['load'],data['pv'],alt_b,integer=True)
    validate(data,alt,alt_b)
    csv_write('efficiency_sensitivity.csv',['效率解释','充电效率','放电效率','购电费_元','购电
量_kWh'],
              [('各90%',.9,.9,s['cost'],float(s['grid'].sum())),
               ('往返
90%',alt_b.eta_charge,alt_b.eta_discharge,alt['cost'],float(alt['grid'].sum()))])
    metrics=dict(baseline_id='Q1_final',time_semantics='end_of_interval',
        model_period='0:00-
24:00',excel_export_order=list(range(144)),cyclic_shift=False,
        official_template_labels_preserved=False,internal_checks_passed=True,
        unittest_execution='not_run_by_solve_q1; run Q1/test_q1.py separately',

time_semantics_status='modeling_assumption_supported_by_data_and_report_not_explicit_in_
problem',
        template_label_offset_minutes=sorted(set(template_offsets)),
        export_label_policy='canonical_natural_day_labels_no_value_shift',
        terminal_constraint_basis='问题1明确要求E144=E0；结合附录1的初始6000 kWh取
E0=E144=6000',
        efficiency_interpretation='主模型将充放电效率90%解释为各90%；往返90%仅作敏感性比较',

cost_yuan=s['cost'],grid_kwh=float(s['grid'].sum()),baseline_cost_yuan=base_cost,
        baseline_grid_kwh=float(base_grid.sum()),saving_yuan=base_cost-
s['cost'],saving_rate=1-s['cost']/base_cost,
        load_kwh=float(data['load'].sum()),pv_kwh=float(data['pv'].sum()),

curtail_kwh=float(s['curtail'].sum()),charge_kwh=float(s['charge'].sum()),discharge_kwh=
float(s['discharge'].sum()),
        storage_loss_kwh=float(s['charge'].sum()-s['discharge'].sum()),
        energy_min_kwh=float(s['energy'].min()),energy_max_kwh=float(s['energy'].max()),
        initial_kwh=float(s['energy'][0]),terminal_kwh=float(s['energy'][-1]),

lp_lower_bound_yuan=lp['cost'],mip_gap=s['mip_gap'],parameters=asdict(b),checks=checks,
        sensitivity_roundtrip_cost_yuan=alt['cost'],

sources_sha256={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest()
                        for p in [ROOT/'附件/附件1.xlsx',template,ROOT/'C题.pdf',ROOT/'C
题建模分析报告.docx']
                        if p.exists()},

versions={'python':sys.version,'numpy':np.__version__,'scipy':scipy.__version__})

(OUT/'summary.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding
='utf-8')
    plot(data,s,b)
    report(metrics,selected,groups,header,rows,input_stats)
    audit_report(metrics)
    print(json.dumps({k:v for k,v in metrics.items() if k in
['cost_yuan','grid_kwh','saving_rate','checks']},ensure_ascii=False,indent=2))


def plot(data,s,b):
    x=np.arange(144)/6
    fig,axes=plt.subplots(4,1,figsize=(12,12),sharex=True,layout='constrained')
    axes[0].stairs(data['load']/b.dt,np.arange(145)/6,label='Load',color='#334155')
    axes[0].stairs(data['pv']/b.dt,np.arange(145)/6,label='PV forecast',color='#d99a13')
    axes[0].stairs(s['grid']/b.dt,np.arange(145)/6,label='Grid
purchase',color='#2563eb')
    axes[0].set_ylabel('Power (kW)');axes[0].legend(ncol=3)

axes[1].bar(x,s['charge']/b.dt,width=1/6,align='edge',label='Charge',color='#0d9488')

axes[1].bar(x,-s['discharge']/b.dt,width=1/6,align='edge',label='Discharge',color='#ef75
```

```
42')
    axes[1].set_ylabel('Battery power (kW)');axes[1].legend(ncol=2)
    axes[2].plot(np.arange(145)/6,s['energy'],color='#7c3aed')
    for e in
[1200,6000,10800]:axes[2].axhline(e,color='#94a3b8',linestyle='--',linewidth=.8)
    axes[2].set_ylabel('Stored energy (kWh)');axes[2].set_ylim(0,12000)
    axes[3].stairs(data['price'],np.arange(145)/6,color='#be185d')
    axes[3].set_ylabel('Price (CNY/kWh)');axes[3].set_xlabel('Hour of day')
    for a in axes:a.grid(alpha=.2);a.set_xlim(0,24);a.set_xticks(np.arange(0,25,2))
    fig.suptitle('Q1: Optimal day-ahead microgrid dispatch',fontsize=16)

fig.savefig(OUT/'dispatch.png',dpi=180);fig.savefig(OUT/'dispatch.svg');plt.close(fig)


def report(m,selected,groups,header,rows,input_stats):
    lines=['# 问题1：微电网确定性日前购电优化','',
        '> 基准版本：`Q1_final`。本文档、表格、图片、Excel结果和检验记录均由 `solve_q1.py` 从原始附件一键生成。','',
        '## 1 问题分析','',
        '问题1给定次日144个十分钟时段的电价、负荷与光伏预测，需要同时确定外网购电和储能充放电计划，使全天购电费用最低。决策的核心不是逐时段独立选择最低购电量，而是利用储能在时间维度转移电能：低价或光伏富余时充电，在高价时放电；同时满足功率平衡、储能效率、容量、功率、充放电互斥及日末电量回归约束。因此，该问题可建成确定性混合整数线性规划模型。','',
        '## 2 模型假设','',
        '1. 将一天均分为144个十分钟时段。原题未明示时间字段是起点还是终点；结合附件1从0:10至0:00+1的144条记录、自然日边界及建模报告，本文将时刻解释为区间结束点，并将功率视为区间平均功率，乘以1/6 h换算为电量。此为明确采用的离散化假设。',
        '2. 充电量与放电量均采用交流母线侧口径。附录1原文为“储能设备的充放电效率为90%”，未明示往返效率。本文采用充、放电效率各90%的主解释，往返效率为81%，另检验往返90%的解释。',
        '3. 微电网可从外网购电但不能向外网售电；题目未规定外网购电容量，故不人为添加购电上限。',
        '4. 允许弃光，不允许负荷削减；忽略储能自放电、寿命损耗费用及其他运维费用。',
        '5. 问题1明确要求0:00与24:00储电量相同，故日循环约束是题目硬约束。结合附录1给出的2025年1月1日0:00初始电量6000 kWh，本文以该初值求解，得到$E_0=E_{144}$=6000 kWh；并非仅为防止终端透支而自行添加末状态约束。','',
        '## 3 数据预处理','',
        '附件1包含144条十分钟记录。首先检查列名、记录数、时间连续性以及电价、负荷和光伏数据的有限性与非负性；随后将0:10至0:00+1解释为各区间结束时刻，构造0:00–0:10至23:50–24:00的自然日区间。负荷和光伏功率乘以1/6 h转换为优化所需的时段电量，电价保持元/kWh。','',
        '![图1 问题1三组输入数据的日内变化特征](input_daily_patterns.png)','',
        f"图1只使用附件1的三组输入数据。光伏在{input_stats['pv_max_period']}达到{input_stats['pv_max_kw']:.1f} kW；负荷在白天保持较高水平，最大值为{input_stats['load_max_kw']:.1f} kW；最低电价{input_stats['price_min_yuan_per_kwh']:.4f}元/kWh出现在{input_stats['price_min_period']}。三者峰谷在时间上明显错开。","",
        '## 4 储能“谷充峰放”的数据依据','',
        '夜间光伏为零但电价较低，白天负荷处于高平台且光伏在正午达到峰值，傍晚至晚间电价明显升高。由此，储能可以在夜间低价时段或正午光伏富余时吸收电能，并在晚间高价时段释放电能。该观察仅用于解释优化机制，具体充放电量仍由后续约束模型统一决定。','',
        '## 5 符号说明','',
        '|符号|含义|单位|',
        '|---|---|---|',
        '|$t$|十分钟时段编号，$t=0,1,\\ldots,143$|–|',
        '|$\\lambda_t$|时段电价|元/kWh|',
        '|$L_t,P_t$|负荷电量、光伏预测电量|kWh|',
        '|$G_t,C_t,D_t,W_t$|购电量、母线侧充电量、母线侧放电量、弃光量|kWh|',
        '|$E_t$|时段$t$开始时的储电量|kWh|',
        '|$z_t$|充电状态变量，充电为1、放电为0|–|',
        '|$\\eta_c,\\eta_d$|充电效率、放电效率，均取0.9|–|',
        '|$\\Delta t$|时段长度，取$1/6$|h|','',
        '## 6 模型建立','',
        '### 6.1 目标函数','',
        '以全天从外网购电的实际费用最小为目标，不额外添加会改变原问题最优解的储能吞吐惩罚：','',
        '$$\\min F=\\sum_{t=0}^{143}\\lambda_tG_t.\\tag{1}$$','',
        '### 6.2 功率平衡约束','',
        '每个时段内，外网购电、光伏和储能放电共同满足负荷、储能充电与弃光需求：','',
        '$$G_t+P_t+D_t=L_t+C_t+W_t,\\quad \\forall t.\\tag{2}$$','',
        '### 6.3 储能状态方程','',
```

```
        '考虑充放电效率后，储电量按下式连续演化：','',
        '$$E_{t+1}=E_t+\\eta_cC_t-\\frac{D_t}{\\eta_d},\\quad \\forall t.\\tag{3}$$','',
        '### 6.4 SOC、功率和互斥约束','',
        '储能容量为 12000 kWh，SOC 允许范围为 10%至 90%。问题 1 规定首末储电量相同，结合附录 1 初始值
6000 kWh，得到','',
        '$$1200\\le E_t\\le10800,\\quad E_0=E_{144}=6000.\\tag{4}$$','',
        '最大充放电功率为 5000 kW。用二进制变量显式禁止同一十分钟时段同时充、放电：','',
        '$$0\\le C_t\\le5000\\Delta t\\,z_t,\\quad '
        '0\\le D_t\\le5000\\Delta t(1-z_t),\\quad z_t\\in\\{0,1\\}.\\tag{5}$$','',
        '购电和弃光边界为','',
        '$$G_t\\ge0,\\quad0\\le W_t\\le P_t.\\tag{6}$$','',
        '## 7 求解方法','',
        '采用 SciPy 调用 HiGHS 求解上述 MILP；另去掉式(5)中的二进制互斥变量，求解连续 LP 作为成本下界。
MILP 目标值与 LP 下界在数值容差内一致，且导出结果通过逐时段功率平衡、状态递推、容量、功率、互斥和 Excel 回
读检查，因此本数据下获得全局最优方案。',
        f"LP 下界为 {m['lp_lower_bound_yuan']:.8f} 元，MILP 最优费用为 {m['cost_yuan']:.8f}
元，相对 MIP 间隙为 {m['mip_gap']:.3g}。","",
        '## 8 求解结果','',
        '### 表 1 指定时段购电量与全天汇总','',
        '|时间段|购电量（kWh）|时间段|购电量（kWh）|时间段|购电量（kWh）|',
        '|---|---:|---|---:|---|---:|']
    for k in (0,3): lines.append('|'+'|'.join(f'{a}|{v:.6f}' for a,v in selected[k:k+3])
+'|')
    lines += [f"|全天购电量|{m['grid_kwh']:.6f}|全天购电费（元）|{m['cost_yuan']:.6f}|||","",
        '### 表 2 储能充放电量与首末储电量','',
        '|时间段|充电量（kWh）|放电量（kWh）|时间段|充电量（kWh）|放电量（kWh）|',
        '|---|---:|---:|---|---:|---:|']
    for k in (0,2,4):lines.append('|'+'|'.join(f'{a}|{c:.6f}|{d:.6f}' for a,c,d in
groups[k:k+2])+'|')
    lines += ['|0:00 储电量|6000||24:00 储电量|6000||','',
        '![图 2 负荷、光伏、购电、储能与电价联合调度结果](dispatch.png)','',
        '图 2 给出了同一时间轴上的四联图。调度策略与电价、光伏及 SOC 边界一致：凌晨电价较低，系统在满足负
荷的同时以最高 5000 kW 功率阶段性充电；清晨电价上升后储能放电，6:00–9:00 附近显著减少外网购电；中午光伏出
力达到高位，系统优先消纳光伏并将可用余量存入储能；18:00–21:00 处于全天高价区间，储能集中放电，其中 19 时
附近外网购电降为 0；21 时以后电价下降，系统重新充电，使日末储电量严格回到 6000 kWh。','',
        '### 8.1 经济性分析','',
        f"无储能基准按 max(负载−光伏,0) 购电，费用为 {m['baseline_cost_yuan']:.6f} 元；优化节省
{m['saving_yuan']:.6f} 元，节省率 {m['saving_rate']:.4%}。",
        f"全天负载 {m['load_kwh']:.6f} kWh，光伏 {m['pv_kwh']:.6f} kWh，弃光
{m['curtail_kwh']:.6f} kWh。",
        f"母线侧总充电 {m['charge_kwh']:.6f} kWh，总放电 {m['discharge_kwh']:.6f} kWh，储能损耗
{m['storage_loss_kwh']:.6f} kWh。储电量范围 {m['energy_min_kwh']:.6f}–
{m['energy_max_kwh']:.6f} kWh。",
        '全天电量可通过“购电+光伏=负载+弃光+储能损耗”复核。费用下降主要来自储能跨时段价差套利与光伏就
地消纳，而非减少必须供给的负荷。','',
        '### 8.2 三项边界说明','',
        '1. **高购电尖峰没有越界。** 购电量由式(2)决定。在低价充电时，外网需同时承担小区净负荷和储能充
电功率，因此图中的购电尖峰约等于“负荷减光伏+5000 kW 最大充电功率”。5000 kW 限制约束的是储能充电功率，
不是外网总购电功率。',
        '2. **四小时汇总表中充、放电量均为正不代表同时充放电。** 表 2 汇总了一个四小时窗口内 24 个十分钟
决策；不同子时段可先充后放。式(5)在每个十分钟时段强制互斥，校验结果中的同时充放电时段数为 0。',
        '3. **外网购电不设人为上限。** 题目仅给出储能容量和充放电功率限制，没有给出并网点容量。若擅自增
加购电上限会改变可行域和最优费用，因此主模型只保留非负购电约束；如后续题目补充并网容量，再将其作为场景参数
加入。','',
        '## 9 模型检验与敏感性分析','',
        '### 9.1 效率口径敏感性','',
        f"若将题目 90%解释为往返效率，充放电效率各取√0.9，最优费用为
{m['sensitivity_roundtrip_cost_yuan']:.6f} 元。该结果仅作敏感性比较，result1.xlsx 采用各 90%的
主模型。","",
        '### 9.2 官方模板与结果一致性校验','',
        '原题附录 2 仅明确 0:00+1 等于当天 24:00，没有明文规定所有记录均是区间结束点。本文根据数据序列和
自然日要求采用结束点解释。原模板标签从 0:10–0:20 至 0:00+1–0:10+1，按字面缺少当天首十分钟且多出次日首十
分钟；它与本文采用的区间定义相差 10 分钟。',
        '本次输出保留两张工作表、行列数、表头及样式，仅把计划购电量的时间列修正为 0:00–0:10 至 23:50–
24:00，购电数值不循环平移。第一行 0:00–0:10 对应 1406.6411 kWh，Excel、CSV、论文按同一自然日区间解
释。time_mapping.csv 保留原模板标签、输入行号、模型区间、输出标签和购电量的完整逐行对照。原始附件不修
```

```
改。',
        '该选择消除了交付文件内部的标签与数据矛盾，但现有本地资料不能确认外部评分程序的时间约定，也不构成官方勘误。若获得官方明确说明，应依据说明统一调整输入映射、统计和输出，不能只循环移动购电列。',
        '购电量、充放电量和储电量均以 kWh 填写，购电费以元计。程序保存后重新打开工作簿，逐项核对 144 个购电量、6 个指定时段、6 组四小时充放电累计值以及 0:00 和 24:00 储电量。',
        'summary.json 记录输入文件 SHA256、运行版本、约束误差、LP 下界、模板一致性和 Excel 回读校验。dispatch_10min.csv 保留未人为舍入的完整决策；表格显示小数不影响内部精度。',
        'summary.json 中的 internal_checks_passed 仅表示生成脚本内置检查通过。生成脚本不自动执行 unittest；回归测试需另行运行 test_q1.py，不再写入未经本次测试证明的 tests_passed 字段。',
        '表 1/表 2 以本报告为论文排版版本，CSV 用于进一步排版。', '',
        '## 10 模型评价','',
        '模型结构简洁、物理约束完整，并通过 MILP 与 LP 下界对照证明本算例最优性；同时保留十分钟粒度结果，避免四小时汇总掩盖时序行为。其局限在于采用确定性预测并忽略储能衰减和并网容量。问题 2–4 若引入预测误差、实时修正或新设备，应在本公共储能物理模型之上扩展信息与费用逻辑，而不改变问题 1 的 `Q1_final` 基准。','',
        '## 参考资料','',
        '- C 题.pdf：问题 1、附录 1、附录 2 及表 1/表 2。',
        '- C 题建模分析报告.docx：第 2 至 4 节及附录假设。',
        '- Silva 等（2020），Optimal Day-Ahead Scheduling of Microgrids with Battery Energy Storage System，第 3.1.3–3.1.4 节：平衡、效率与储能约束。',
        '- Šimić 等（2022），Optimal Scheduling of Battery Energy Storage in Microgrid to Minimize Electricity and Fuel Costs，第 III 节：互斥和日初日末相等。',
        '- 参考资料/问题 1 相关论文/文献目录与摘录.md：采用购电成本目标，不照搬售电、柴油机或寿命成本。']
    text = '\n'.join(lines)+'\n'
    (OUT/'Q1_final.md').write_text(text,encoding='utf-8')
    (OUT/'问题 1 结果与模型说明.md').write_text(text,encoding='utf-8')


def audit_report(m):
    text = f'''# 问题 1 口径核验

本说明由 solve_q1.py 生成，依据本地 C 题.pdf、附件 1、附件 5 及建模分析报告。事实、建模解释和无法确认的评分约定分开记录。

## 1 十分钟时间映射

- 原题第 2 页附录 2 明文：0:00+1 表示次日 0:00，即当天 24:00。未明文说明功率记录是区间起点、终点还是瞬时采样。
- 附件 1 A2:A145 共 144 条记录，为 0:10 至 0:00+1；建模报告第 2.1 节建议按结束点解释。
- 原题第 1 页要求当天计划和 0:00/24:00 边界，表 2 覆盖 0:00 至 24:00。结束点及区间平均功率解释与之相容，本文继续采用该假设。
- 原模板 A2 为 0:10-0:20，A145 为 0:00+1-0:10+1。按字面整体覆盖 0:10 至次日 0:10，无法原样同时表达本文自然日首末时段。
- 旧输出虽然按行号对应，但标签和数值不一致。新输出修正时间标签为 0:00-0:10 至 23:50-24:00，保留数据顺序和模板结构、样式。详细证据链见 time_mapping.csv。
- 不循环平移：移动购电量而不同步负载、光伏、价格、储能状态会破坏物理对应；当前也没有足够依据把当天策略延伸到次日首十分钟。
- 尚未取得官方勘误或评分程序说明。修正标签是本地交付的一致性选择，不声称已消除外部评分约定的不确定性。

## 2 效率

原题第 2 页附录 1 原文：“储能设备的充放电效率为 90%”。它没有使用“综合效率”“往返效率”，也没有分别列出两个效率。主模型按各 90%处理，与建模报告第 3.2 节建议一致；此为采用的解释，不能表述成原题分别明示两个 90%。

|解释|充电效率|放电效率|往返效率|购电费（元）|
|---|---:|---:|---:|---:|
|主模型，各 90%|0.9|0.9|0.81|{m['cost_yuan']:.6f}|
|敏感性，往返 90%|√0.9|√0.9|0.9|{m['sensitivity_roundtrip_cost_yuan']:.6f}|

两种解释成本相差 {m['cost_yuan']-m['sensitivity_roundtrip_cost_yuan']:.6f} 元。result1.xlsx 只使用主模型，敏感性结果不混入提交列。

## 3 日末电量

原题第 1 页问题 1 明文：“储能设备在 0:00 和 24:00 的储电量相同”。因此 E144=E0 是硬约束。
```

附录 1 给出 2025 年 1 月 1 日 0:00 电量 6000 kWh。以该初始状态和题目指定的循环边界结合，取 E0=E144=6000 kWh。原题不是在同一句中直接写出两个 6000；论文已说明两项依据。仅因避免终端透支而将此列为额外假设的表述不准确。

4 检查状态及复现

summary.json 改用 internal_checks_passed，含义仅限物理约束、最优性和生成文件内置回读检查通过；unittest_execution 明确注明生成器未运行独立测试。

```bash
.venv/bin/python Q1/solve_q1.py
.venv/bin/python Q1/test_q1.py
```

回归测试按区间标签检查 Excel/CSV/表 1 数值对应，并包含故意偏移或循环移动标签的反例。仅“工作表结构与原模板一致”不再被视为时间正确的证明。
'''
    (OUT/'口径核验.md').write_text(text,encoding='utf-8')

if __name__=='__main__':main()
```

### B3 code/Q1/test_q1.py

```
"""可手算的小算例，检验效率、互斥、功率限值、弃光和时间解析。"""
import sys
import unittest
from pathlib import Path
from dataclasses import replace
from datetime import time
sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
import numpy as np
from microgrid_common import Battery, optimize, validate, end_minutes
from time_mapping import canonical_periods, validate_export_periods
Q1=Path(__file__).resolve().parent
HAS_Q1_OUTPUTS=(Q1/'result1.xlsx').exists() and (Q1/'summary.json').exists()

class DispatchTests(unittest.TestCase):
    def solve(self,price,load,pv,b):
        data={k:np.array(v,float) for k,v in zip(['price','load','pv'],[price,load,pv])}
        s=optimize(**data,battery=b,initial=0,terminal=0,integer=True)
        validate(data,s,b,initial=0,terminal=0)
        return s

    def test_efficiency_and_arbitrage(self):

b=Battery(capacity=10,minimum=0,maximum=10,power=20,dt=1,eta_charge=.9,eta_discharge=.9)
        s=self.solve([1,3],[0,8.1],[0,0],b)
        self.assertAlmostEqual(s['cost'],10)
        np.testing.assert_allclose(s['charge'],[10,0],atol=1e-7)
        np.testing.assert_allclose(s['discharge'],[0,8.1],atol=1e-7)

    def test_power_limit(self):

b=Battery(capacity=10,minimum=0,maximum=10,power=5,dt=1,eta_charge=1,eta_discharge=1)
        s=self.solve([1,3],[0,8],[0,0],b)
        self.assertAlmostEqual(s['cost'],14)

    def test_curtailment_and_no_sale(self):
        b=Battery(capacity=10,minimum=0,maximum=10,power=5,dt=1)
        s=self.solve([1],[0],[20],b)
        self.assertAlmostEqual(s['cost'],0)
        self.assertAlmostEqual(s['curtail'][0],20)

    def test_equal_prices_do_not_cycle(self):
```
```

```
        b=Battery(capacity=10,minimum=0,maximum=10,power=5,dt=1)
        s=self.solve([1,1],[4,4],[0,0],b)
        self.assertAlmostEqual(s['cost'],8)
        self.assertAlmostEqual(s['charge'].sum(),0)

    def test_time_formats(self):
        self.assertEqual(end_minutes(time(0,10)),10)
        self.assertEqual(end_minutes('10:10'),610)
        self.assertEqual(end_minutes('0:00+1'),1440)

    @unittest.skipUnless(HAS_Q1_OUTPUTS,'Run Q1/solve_q1.py before output checks')
    def test_generated_final_report_contract(self):
        report = Path(__file__).with_name('Q1_final.md')
        self.assertTrue(report.exists())
        text = report.read_text(encoding='utf-8')
        for required in (
            '基准版本：`Q1_final`',
            '## 1 问题分析',
            '## 2 模型假设',
            '## 3 数据预处理',
            'input_daily_patterns.png',
            '## 4 储能“谷充峰放”的数据依据',
            '## 5 符号说明',
            '\\tag{1}',
            '\\tag{6}',
            '高购电尖峰没有越界',
            '四小时汇总表中充、放电量均为正不代表同时充放电',
            '外网购电不设人为上限',
        ):
            self.assertIn(required, text)
        self.assertEqual(
            text,
            Path(__file__).with_name('问题1结果与模型说明.md').read_text(encoding='utf-8'),
        )

    @unittest.skipUnless(HAS_Q1_OUTPUTS,'Run Q1/solve_q1.py before output checks')
    def test_preprocessing_figure_contains_inputs_only(self):
        import json
        q1=Path(__file__).resolve().parent
        for name in
('input_daily_patterns.png','input_daily_patterns.svg','input_statistics.json'):
            self.assertTrue((q1/name).is_file())
            self.assertGreater((q1/name).stat().st_size,100)
        svg=(q1/'input_daily_patterns.svg').read_text(encoding='utf-8')
        for required in ('小区负荷','光伏预测','外网电价'):
            self.assertIn(required,svg)
        for forbidden in ('购电','充电','放电','储电','Grid purchase','Battery'):
            self.assertNotIn(forbidden,svg)
        stats=json.loads((q1/'input_statistics.json').read_text(encoding='utf-8'))
        self.assertEqual(stats['record_count'],144)
        self.assertEqual(stats['period_start'],'0:00-0:10')
        self.assertEqual(stats['period_end'],'23:50-24:00')

    @unittest.skipUnless(HAS_Q1_OUTPUTS,'Run Q1/solve_q1.py before output checks')
    def test_official_workbook_contract(self):
        from openpyxl import load_workbook
        q1 = Path(__file__).resolve().parent
        template = q1.parent/'附件/附件5/result1.xlsx'
        source = load_workbook(template,data_only=False)
        result = load_workbook(q1/'result1.xlsx',data_only=True)
        self.assertEqual(source.sheetnames,result.sheetnames)
        for source_sheet,result_sheet in zip(source,result):
            self.assertEqual(
                (source_sheet.max_row,source_sheet.max_column),
                (result_sheet.max_row,result_sheet.max_column),
            )
```

```
            self.assertEqual(
                [source_sheet.cell(1,c).value for c in
range(1,source_sheet.max_column+1)],
                [result_sheet.cell(1,c).value for c in
range(1,result_sheet.max_column+1)],
            )
            self.assertEqual(
                [[source_sheet.cell(r,c).style_id
                  for c in range(1,source_sheet.max_column+1)]
                 for r in range(1,source_sheet.max_row+1)],
                [[result_sheet.cell(r,c).style_id
                  for c in range(1,result_sheet.max_column+1)]
                 for r in range(1,result_sheet.max_row+1)],
            )
        source_labels = [source['计划购电量'].cell(r,1).value for r in range(2,146)]
        result_labels = [result['计划购电量'].cell(r,1).value for r in range(2,146)]
        validate_export_periods(result_labels,range(10,1441,10))
        self.assertEqual(result_labels[0],'0:00-0:10')
        self.assertEqual(result_labels[-1],'23:50-24:00')
        self.assertEqual(source_labels[0],'0:10-0:20')
        self.assertEqual(source_labels[-1],'0:00+1-0:10+1')
        self.assertTrue(all(
            isinstance(result['计划购电量'].cell(r,2).value,(int,float))
            for r in range(2,146)
        ))
        source.close()
        result.close()

    @unittest.skipUnless(HAS_Q1_OUTPUTS,'Run Q1/solve_q1.py before output checks')
    def test_final_natural_day_mapping_and_metrics(self):
        import csv
        import json
        from openpyxl import load_workbook

        q1 = Path(__file__).resolve().parent
        data = __import__('microgrid_common').read_single_day(q1.parent/'附件/附件
1.xlsx')
        battery = Battery()
        milp = optimize(data['price'],data['load'],data['pv'],battery,integer=True)
        lp = optimize(data['price'],data['load'],data['pv'],battery)
        checks = validate(data,milp,battery)
        self.assertEqual(len(milp['grid']),144)
        self.assertAlmostEqual(milp['cost'],lp['cost'],places=5)
        self.assertEqual(milp['mip_gap'],0)
        self.assertAlmostEqual(milp['cost'],35126.948589289634,places=5)
        self.assertAlmostEqual(milp['grid'].sum(),59482.69899835391,places=5)
        self.assertAlmostEqual(milp['energy'][0],6000)
        self.assertAlmostEqual(milp['energy'][-1],6000)
        self.assertGreaterEqual(milp['energy'].min(),1200-1e-6)
        self.assertLessEqual(milp['energy'].max(),10800+1e-6)
        self.assertEqual(checks['simultaneous_charge_discharge_count'],0)
        self.assertLessEqual(checks['power_bound_violation_kw'],1e-6)
        self.assertLessEqual(checks['balance_max_error_kwh'],1e-6)
        self.assertLessEqual(checks['state_max_error_kwh'],1e-6)

        workbook = load_workbook(q1/'result1.xlsx',read_only=True,data_only=True)
        exported = np.array([
            workbook['计划购电量'].cell(row,2).value for row in range(2,146)
        ])
        np.testing.assert_allclose(exported,milp['grid'],atol=1e-8)
        self.assertAlmostEqual(exported[0],milp['grid'][0])
        self.assertAlmostEqual(exported[-1],milp['grid'][143])
        workbook.close()

        with (q1/'table1.csv').open(encoding='utf-8-sig') as f:
            table1 = list(csv.DictReader(f))
```

```
        indices = [60,72,84,96,108,120]
        self.assertEqual(
            [float(row['购电量_kWh']) for row in table1[:-1]],
            [float(milp['grid'][t]) for t in indices],
        )
        with (q1/'table2.csv').open(encoding='utf-8-sig') as f:
            table2 = list(csv.DictReader(f))
        for group,row in enumerate(table2):
            start = group*24
            stop = start+24
            self.assertAlmostEqual(float(row['充电量_kWh']),milp['charge']
[start:stop].sum())
            self.assertAlmostEqual(float(row['放电量_kWh']),milp['discharge']
[start:stop].sum())

        summary = json.loads((q1/'summary.json').read_text(encoding='utf-8'))
        self.assertEqual(summary['time_semantics'],'end_of_interval')
        self.assertEqual(summary['model_period'],'0:00-24:00')
        self.assertEqual(summary['excel_export_order'],list(range(144)))
        self.assertFalse(summary['cyclic_shift'])
        self.assertFalse(summary['official_template_labels_preserved'])
        self.assertTrue(summary['internal_checks_passed'])
        self.assertNotIn('tests_passed',summary)
        self.assertEqual(summary['unittest_execution'],
                         'not_run_by_solve_q1; run Q1/test_q1.py separately')

    def test_shifted_labels_are_rejected(self):
        # 旧版首行标签错误即使数值未平移，也必须判定为不一致。
        from microgrid_common import clock_label
        labels=[f'{clock_label(t)}-{clock_label(t+10)}' for t in range(10,1450,10)]
        with self.assertRaises(ValueError):
            validate_export_periods(labels,range(10,1441,10))

    def test_rotated_labels_are_rejected(self):
        labels=canonical_periods(range(10,1441,10))
        with self.assertRaises(ValueError):
            validate_export_periods(labels[1:]+labels[:1],range(10,1441,10))

    @unittest.skipUnless(HAS_Q1_OUTPUTS,'Run Q1/solve_q1.py before output checks')
    def test_excel_csv_table1_agree_by_time_label(self):
        import csv
        from openpyxl import load_workbook
        q1=Path(__file__).resolve().parent
        with (q1/'dispatch_10min.csv').open(encoding='utf-8-sig') as f:
            dispatch=list(csv.DictReader(f))
        with (q1/'table1.csv').open(encoding='utf-8-sig') as f:
            table1=list(csv.DictReader(f))
        with (q1/'time_mapping.csv').open(encoding='utf-8-sig') as f:
            mapping=list(csv.DictReader(f))
        w=load_workbook(q1/'result1.xlsx',data_only=True)
        actual={label:value for label,value in list(w['计划购电量'].values)[1:]}
        expected={row['时间段']:float(row['购电_kWh']) for row in dispatch}
        self.assertEqual(set(actual),set(expected))
        for label in expected:
            self.assertAlmostEqual(actual[label],expected[label],places=8)
        for row in table1[:-1]:
            self.assertAlmostEqual(actual[row['时间段']],float(row['购电量
_kWh']),places=8)
        self.assertEqual(len(mapping),144)
        for index,row in enumerate(mapping):
            self.assertEqual(int(row['模型索引']),index)
            self.assertEqual(int(row['附件1行号']),index+2)
            self.assertEqual(row['模型自然日区间'],row['输出Excel标签'])
            self.assertAlmostEqual(actual[row['输出Excel标签']],float(row['购电量
_kWh']),places=8)
        w.close()
```

```
if __name__=='__main__':unittest.main()
```

B4 code/Q1/time_mapping.py

```
"""问题1自然日时间映射；标签和数据同时校验，禁止仅凭行号判定一致。"""
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from microgrid_common import clock_label, end_minutes


def canonical_periods(ends):
    ends = [int(v) for v in ends]
    if ends != list(range(10, 1441, 10)):
        raise ValueError('问题1必须覆盖当天0:00至24:00的144个连续十分钟区间')
    return [f'{clock_label(end-10)}-{clock_label(end)}' for end in ends]


def period_bounds(label):
    start, end = str(label).split('-')
    return end_minutes(start), end_minutes(end)


def validate_export_periods(labels, ends):
    expected = [period_bounds(v) for v in canonical_periods(ends)]
    actual = [period_bounds(v) for v in labels]
    if actual != expected:
        raise ValueError('Excel时间标签与模型自然日区间不一致，存在偏移、遗漏或顺序错误')
    return True
```

B5 code/Q2/forecast_q2.py

```
"""逐日起点、多个时距预测。只接收已经完成日；所有数值为kWh。"""
from dataclasses import dataclass
from datetime import date, timedelta
import hashlib
import numpy as np

METHODS=('last','mean7','blend')


def forecast(history, day, method, horizon=3):
    """history包含恰好day个已完成日；输出horizon×144×(load,pv)。"""
    history=np.asarray(history,float)
    assert len(history)==day and day>0
    recent=history[-7:].mean(axis=0)
    out=[]
    for j in range(horizon):
        if method=='last':
            y=history[-1]
        elif method=='mean7':
            y=recent
        elif method=='blend':
            inds=[k for k in range(day) if (day+j-k)%7==0][-4:]
            weekly=history[inds].mean(axis=0) if inds else recent
            y=.5*recent+.5*weekly
        else:
            raise ValueError(method)
        out.append(np.maximum(y,0))
    return np.asarray(out)


@dataclass(frozen=True)
class Features:
    day:int
```

```
    prediction:dict
    residual:dict
    residual_dates:dict
    training_max:int
    history_sha256:str


class OnlineForecast:
    def __init__(self):
        self.history=[]
        self.issued=[]

    def issue(self):
        day=len(self.history)
        assert len(self.issued)==day
        pred={m:forecast(self.history,day,m) if day else np.zeros((3,144,2)) for m in
METHODS}
        residual={}; residual_dates={}
        for method in METHODS:
            residual[method]=[];residual_dates[method]=[]
            for j in range(3):
                # origin>=1 excludes intentionally non-forecast startup day
                origins=list(range(max(1,day-j-30),day-j))
                r=[self.history[k+j]-self.issued[k].prediction[method][j] for k in
origins]
                residual[method].append(np.asarray(r).reshape(-1,144,2))
                residual_dates[method].append([k+j for k in origins])
        a=np.asarray(self.history,dtype=float)

features=Features(day,pred,residual,residual_dates,day-1,hashlib.sha256(a.tobytes()).hex
digest())
        self.issued.append(features)
        return features

    def observe(self,actual):
        assert len(self.issued)==len(self.history)+1
        a=np.asarray(actual,float).copy()
        assert a.shape==(144,2) and np.isfinite(a).all() and (a>=0).all()
        self.history.append(a)


def build_features(actual):
    bank=OnlineForecast()
    for day in actual:
        bank.issue()
        bank.observe(day)
    return bank.issued


def risk_net(features,method,q,horizon):
    pred=features.prediction[method][:horizon]
    net=pred[:,:,0]-pred[:,:,1]
    margin=np.zeros_like(net)
    if q>0:
        for j in range(horizon):
            r=features.residual[method][j]
            if len(r):
                margin[j]=np.maximum(0,np.quantile(r[:,:,0]-r[:,:,1],q,axis=0))
    return (net+margin).ravel(),margin
```

B6 code/Q2/report_q2.py

```
"""从本次原始计算数组生成全部 CSV、官方模板工作簿、论文和图件。"""
from pathlib import Path
from copy import copy
from datetime import datetime
```

```
import json,csv,sys,os
import numpy as np
from openpyxl import load_workbook
from openpyxl.styles import Alignment
from openpyxl.utils import get_column_letter
from solve_q2 import ROOT,OUT,write_json,write_csv,summary_of,check_q1
from microgrid_online import read_year,PERIODS,merge_emergency,terminal_value
from microgrid_common import Battery


def read_csv(path):
    with path.open(encoding='utf-8-sig') as f:return list(csv.DictReader(f))


def table(headers,rows):
    return '\n'.join(['|'+'|'.join(headers)+'|','|'+'|'.join(['---']*len(headers))+'|']+
                     ['|'+'|'.join(str(x) for x in row)+'|' for row in rows])


def export_workbook(dates,actual,price,s):
    # user explicitly requested a reproducible Python generator using official template
    template=ROOT/'附件/附件5/result2.xlsx'
    wb=load_workbook(template)
    ws=wb['计划购电量'];old=[ws.cell(1,c).value for c in range(2,146)]
    for t,label in enumerate(PERIODS,2):ws.cell(1,t,label)
    for day in range(31,365):
        row=day-29
        ws.cell(row,1,datetime.combine(dates[day],datetime.min.time()))
        for t in range(144):ws.cell(row,t+2,float(s['G'][day,t]))
        ws.cell(row,146,float(s['G'][day].sum()))
        ws.cell(row,147,float(price@s['G'][day]+5*price@s['H'][day]))
    ws.freeze_panes='B2';ws.auto_filter.ref='A1:EQ335'
    for col in range(1,148):ws.column_dimensions[get_column_letter(col)].width=17 if col
in (1,146,147) else 15
    for row in ws.iter_rows(min_row=2):
        row[0].number_format='yyyy-mm-dd'
        for cell in row[1:]:cell.number_format='#,##0.0000'
    ws.row_dimensions[1].height=25
    # expand examples by copying actual template styles, not creating similar sheets
    ws=wb['充放电量'];styles=[[copy(ws.cell(r,c)._style) for c in range(1,7)] for r in
range(2,8)]
    ws.delete_rows(2,ws.max_row-1)
    for day in range(31,365):
        for j in range(6):
            r=2+(day-31)*6+j
            for c in range(1,7):ws.cell(r,c)._style=copy(styles[j][c-1])
            if
j==0:ws.cell(r,1,datetime.combine(dates[day],datetime.min.time())).number_format='yyyy-
mm-dd'
            ws.cell(r,2,f'{j*4}:00-{(j+1)*4}:00')
            ws.cell(r,3,float(s['C'][day,j*24:(j+1)*24].sum()))
            ws.cell(r,4,float(s['D'][day,j*24:(j+1)*24].sum()))
            if j<2:
                ws.cell(r,5,'0:00' if j==0 else '24:00')
                ws.cell(r,6,float(s['E'][day,0 if j==0 else -1]))
            for c in (3,4,6):ws.cell(r,c).number_format='#,##0.0000'
    ws.freeze_panes='C2'
    for c in range(1,7):ws.column_dimensions[get_column_letter(c)].width=18
    ws=wb['紧急购电量'];styles=[copy(ws.cell(2,c)._style) for c in range(1,4)]
    ws.delete_rows(2,ws.max_row-1);merged=merge_emergency(dates[31:],s['H'][31:]);row=2
    for day in range(31,365):
        events=[r for r in merged if r['date']==dates[day].isoformat()]
        if not events:events=[dict(period='无',kwh=0.)]
        for j,event in enumerate(events):
            for c in range(1,4):ws.cell(row,c)._style=copy(styles[c-1])
            if
```

```
j==0:ws.cell(row,1,datetime.combine(dates[day],datetime.min.time())).number_format='yyyy
-mm-dd'

ws.cell(row,2,event['period']);ws.cell(row,3,float(event['kwh'])).number_format='#,##0.0
000';row+=1
    ws.freeze_panes='B2'
    for c in range(1,4):ws.column_dimensions[get_column_letter(c)].width=22
    for sheet in wb:
        sheet.sheet_view.zoomScale=85
        sheet.print_options.horizontalCentered=True
        sheet.print_title_rows='1:1'
    wb.save(OUT/'result2.xlsx');wb.close()
    write_csv(OUT/'time_mapping.csv',[dict(index=t,input_end=PERIODS[t].split('-')[1],
        original_template=old[t],output_period=PERIODS[t],output_column=t+2) for t in
range(144)])
    return merged


def data_exports(dates,a,p,s):
    daily=[];dispatch=[];selected=[]
    wanted={'2025-03-20','2025-06-21','2025-09-23','2025-12-21'}
    for day,date in enumerate(dates):
        g,h=s['G'][day],s['H'][day];pc=float(p@g);hc=float(5*p@h)
        daily.append(dict(date=date.isoformat(),formal=day>=31,config=str(s['config']
[day]),q=float(s['q'][day]),

plan_kwh=float(g.sum()),emergency_kwh=float(h.sum()),total_grid_kwh=float((g+h).sum()),
             plan_cost_yuan=pc,emergency_cost_yuan=hc,total_cost_yuan=pc+hc,
             initial_kwh=float(s['E'][day,0]),terminal_kwh=float(s['E'][day,-1]),
             charge_kwh=float(s['C'][day].sum()),discharge_kwh=float(s['D'][day].sum()),
             emergency_periods=int((h>1e-7).sum()),waste_kwh=float(s['W'][day].sum()),
             pv_curtail_kwh=float(s['pv_curtail']
[day].sum()),load_kwh=float(a[day,:,0].sum()),pv_kwh=float(a[day,:,1].sum()),
             net_mae_kw=float(np.abs((a[day,:,0]-a[day,:,1])-(s['prediction'][day,:,0]-
s['prediction'][day,:,1])).mean()*6),
             solve_seconds=float(s['seconds'][day])))
        if day<31:continue
        local=[]
        for t in range(144):

row=dict(date=date.isoformat(),period=PERIODS[t],index=t,price_yuan_kwh=float(p[t]),
                     load_kw=float(a[day,t,0]*6),pv_kw=float(a[day,t,1]*6),
                     load_forecast_kw=float(s['prediction']
[day,t,0]*6),pv_forecast_kw=float(s['prediction'][day,t,1]*6),
                     margin_kw=float(s['margin'][day,t]*6),G_plan_kwh=float(g[t]),
                     C_plan_kwh=float(s['C_plan'][day,t]),D_plan_kwh=float(s['D_plan']
[day,t]),
                     E_plan_start_kwh=float(s['E_plan']
[day,t]),E_plan_end_kwh=float(s['E_plan'][day,t+1]),
                     C_real_kwh=float(s['C'][day,t]),D_real_kwh=float(s['D']
[day,t]),H_real_kwh=float(h[t]),
                     W_real_kwh=float(s['W']
[day,t]),unused_plan_kwh=float(s['unused_grid'][day,t]),
                     pv_curtail_kwh=float(s['pv_curtail']
[day,t]),pv_curtail_upper_kwh=float(s['pv_curtail_upper'][day,t]),
                     E_real_start_kwh=float(s['E'][day,t]),E_real_end_kwh=float(s['E']
[day,t+1]),

plan_cost_yuan=float(p[t]*g[t]),emergency_cost_yuan=float(5*p[t]*h[t]))
            dispatch.append(row);local.append(row)
        if date.isoformat() in wanted:
            write_csv(OUT/'selected_days'/f'{date.isoformat()}_dispatch.csv',local)
            selected.append(daily[-1])
            write_csv(OUT/'selected_days'/f'{date.isoformat()}_table1.csv',
                [dict(period=PERIODS[t],plan_kwh=float(g[t])) for t in
(60,72,84,96,108,120)]+
```

```
                [dict(period='全天计划购电量',plan_kwh=float(g.sum()))])
            write_csv(OUT/'selected_days'/f'{date.isoformat()}_table2.csv',
                [dict(period=f'{j*4}:00-{(j+1)*4}:00',charge_kwh=float(s['C'][day,j*24:
(j+1)*24].sum()),
                      discharge_kwh=float(s['D'][day,j*24:(j+1)*24].sum())) for j in
range(6)])

write_csv(OUT/'daily_summary.csv',daily[31:]);write_csv(OUT/'warmup_january.csv',daily[:
31])
    write_csv(OUT/'dispatch_10min.csv',dispatch)
    write_csv(OUT/'emergency_purchase_10min.csv',
[dict(date=r['date'],period=r['period'],H_real_kwh=r['H_real_kwh']) for r in dispatch])
    write_csv(OUT/'selected_days/summary.csv',selected)
    return daily,selected


def figures(dates,a,p,s,daily,comparisons):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family':'DejaVu
Sans','font.size':10,'axes.spines.top':False,'axes.spines.right':False})
    figdir=OUT/'figures';figdir.mkdir(exist_ok=True)
    def save(fig,name):

fig.savefig(figdir/f'{name}.png',dpi=160,bbox_inches='tight');fig.savefig(figdir/
f'{name}.svg',bbox_inches='tight');plt.close(fig)
    months=np.arange(2,13);pc=[];hc=[];ek=[]
    for m in months:
        ids=[i for i,d in enumerate(dates) if d.month==m]
        pc.append(float((s['G'][ids]@p).sum())/1e6);hc.append(float((s['H']
[ids]@(5*p)).sum())/1e6);ek.append(s['H'][ids].sum()/1000)
    fig,ax=plt.subplots(2,1,figsize=(10,7),layout='constrained')
    ax[0].bar(months,pc,label='Day-ahead
cost',color='#2563eb');ax[0].bar(months,hc,bottom=pc,label='Emergency
cost',color='#e66a36')
    ax[0].set_ylabel('Cost (million
CNY)');ax[0].legend();ax[1].bar(months,ek,color='#e66a36')
    ax[1].set_ylabel('Emergency energy
(MWh)');ax[1].set_xlabel('Month');ax[1].set_xticks(months)
    fig.suptitle('Causal policy: monthly realized costs and emergency
purchases');save(fig,'monthly_costs')
    for day,date in enumerate(dates):
        if date.isoformat() not in ('2025-03-20','2025-06-21','2025-09-23','2025-12-
21'):continue
        x=(np.arange(144)+.5)/6;edges=np.arange(145)/6
        fig,axes=plt.subplots(5,1,figsize=(12,13),sharex=True,layout='constrained')
        axes[0].plot(x,a[day,:,0]*6,label='Actual load',color='#334155')
        axes[0].plot(x,s['prediction'][day,:,0]*6,'--',label='Load
forecast',color='#94a3b8')
        axes[0].plot(x,a[day,:,1]*6,label='Actual PV',color='#c98b0e')
        axes[0].plot(x,s['prediction'][day,:,1]*6,'--',label='PV
forecast',color='#e8b958');axes[0].legend(ncol=4);axes[0].set_ylabel('Power (kW)')
        error=((a[day,:,0]-a[day,:,1])-(s['prediction'][day,:,0]-s['prediction']
[day,:,1]))*6
        axes[1].plot(x,error,label='Net-load
error',color='#475569');axes[1].plot(x,s['margin'][day]*6,label='Risk
margin',color='#9333ea')
        axes[1].axhline(0,color='gray',lw=.6);axes[1].set_ylabel('Error / margin
(kW)');axes[1].legend(ncol=2)
        axes[2].stairs(s['G'][day]*6,edges,label='Fixed day-ahead plan',color='#2563eb')
        axes[2].bar(x,s['H']
[day]*6,width=1/6,label='Emergency',color='#dc2626');axes[2].set_ylabel('Grid power
(kW)');axes[2].legend(ncol=2)
        axes[3].bar(x,(s['C'][day]-s['D']
[day])*6,width=1/6,color='#0d9488',label='Charge (+) / discharge (-)')
```

```
        other=axes[3].twinx();other.plot(edges,s['E']
[day],color='#7c3aed',label='Energy');other.set_ylabel('Stored energy
(kWh)');other.set_ylim(0,12000)
        for level in (1200,10800):other.axhline(level,color='#7c3aed',alpha=.3,ls='--')
        axes[3].set_ylabel('Battery power (kW)');axes[3].legend(loc='upper left')
        axes[4].stairs(p,edges,color='#be185d',label='Price');axes[4].set_ylabel('Price
(CNY/kWh)')
        other=axes[4].twinx();other.plot(edges,np.r_[0,np.cumsum(p*(s['G'][day]+5*s['H']
[day]))]/1000,color='#334155');other.set_ylabel('Cumulative cost (1000 CNY)')
        for ax in axes:ax.set_xlim(0,24);ax.grid(alpha=.15)
        axes[-1].set_xticks(range(0,25,2));axes[-1].set_xlabel('Hour')
        fig.suptitle(f'{date}: {s["config"][day]}');save(fig,str(date))
    fig,axes=plt.subplots(1,2,figsize=(12,4.5),layout='constrained')
    err=((a[31:,:,0]-a[31:,:,1])-(s['prediction'][31:,:,0]-s['prediction'][31:,:,1]))*6
    axes[0].hist(err.ravel(),bins=65,color='#64748b');axes[0].set_xlabel('Net-load error
(kW)');axes[0].set_ylabel('10-minute periods')
    cover=(err<=s['margin'][31:]*6+1e-6).mean(axis=0)

axes[1].plot(np.arange(144)/6,cover,color='#9333ea');axes[1].set_ylim(0,1);axes[1].set_x
label('Hour');axes[1].set_ylabel('Empirical upper-margin coverage')
    save(fig,'forecast_errors')
    fig,axes=plt.subplots(1,2,figsize=(12,4.5),layout='constrained')
    for h,term in [(24,'cycle'),(24,'value'),(48,'cycle'),(72,'cycle')]:
        rows=sorted([r for r in comparisons if r['method']=='blend' and float(r['q'])>0
and int(r['horizon_hours'])==h and r['terminal']==term and
r['scenario']=='False'],key=lambda r:float(r['q']))
        axes[0].plot([float(r['q']) for r in rows],[float(r['total_cost_yuan'])/1e6 for
r in rows],marker='o',label=f'{h}h {term}')
        axes[1].plot([float(r['q']) for r in rows],[float(r['emergency_kwh'])/1000 for r
in rows],marker='o',label=f'{h}h {term}')
    axes[0].set_ylabel('Total cost (million CNY)');axes[1].set_ylabel('Emergency energy
(MWh)')
    for ax in axes:ax.set_xlabel('Quantile q');ax.legend();ax.grid(alpha=.15)
    fig.suptitle('Fixed-policy diagnostics; not used to select past official
plans');save(fig,'quantile_window_sensitivity')


def generate_documents(dates,a,p,s,summary,daily,selected,comparisons):
    sel=[r for r in read_csv(OUT/'experiments/monthly_selection.csv') if
r['selected']=='True']
    forecast=read_csv(OUT/'forecast_metrics.csv')
    sensitivity=json.loads((OUT/'experiments/sensitivities.json').read_text())
    oracle=json.loads((OUT/'experiments/oracle.json').read_text())
    metadata=json.loads((OUT/'experiments/run_metadata.json').read_text())
    byname={r['config']:r for r in comparisons}

representative=['last_q0.00_24h_cycle','mean7_q0.00_24h_cycle','blend_q0.00_24h_value',

'blend_q0.80_24h_cycle','blend_q0.80_48h_cycle','blend_q0.80_72h_cycle','blend_q0.80_24h
_value','scenario_blend_48']
    cmp_rows=[]
    for name in representative:

r=byname[name];cmp_rows.append([name,f'{float(r["total_cost_yuan"]):,.2f}',f'{float(r["e
mergency_kwh"]):,.2f}',r['emergency_periods'],f'{float(r["terminal_kwh"]):.2f}'])
    cmp_rows.insert(0,['正式月度因果选
择',f'{summary["total_cost_yuan"]:,.2f}',f'{summary["emergency_kwh"]:,.2f}',summary['emer
gency_periods'],f'{summary["terminal_kwh"]:.2f}'])
    cmp_rows.append(['无储能 mean7',f'{sensitivity["no_storage_mean7"]
["total_cost_yuan"]:,.2f}',f'{sensitivity["no_storage_mean7"]
["emergency_kwh"]:,.2f}',sensitivity['no_storage_mean7']
['emergency_periods'],f'{sensitivity["no_storage_mean7"]["terminal_kwh"]:.2f}'])
    cmp_rows.append(['同边界全年 oracle 下
界',f'{oracle["objective"]:,.2f}',0,0,f'{summary["terminal_kwh"]:.2f}'])
    fmt=lambda x:f'{x:,.4f}'
    text=r'''# 问题 2 因果预测与历史经济验证驱动的微网日前购电
```

1 问题分析与证据边界

问题 2 要求每天 0:00 承诺当天购电，在未知实际负载与光伏的条件下利用储能减少 5 倍电价紧急补电。本文采用“因果预测—候选计划优化—实时物理反馈—跨日状态传递”的分层策略，并在每月首日用已完成历史的经济费用选择配置。原题规定费用与设备参数；日内储能可反馈、十分钟均值解释及终端价值属于本文假设/设计。最终方案由逐月历史验证产生，不从事后全年费用排名选一个赢家回填。

2 假设与符号

不售电、不削负荷、不添加题目未给的并网功率上限和衰减费用。允许弃光及已付款计划电量未利用。各流量均为交流母线侧 kWh，E 为内部储电量。

符号	定义
d,t	日期与十分钟时段，t=0,…,143
L,P	实际负载与光伏时段电量，输入功率除以 6
p,G,H	正常电价、0:00 计划购电、实际紧急购电
C,D,W,E	实际充电、放电、未利用能量、储电量
带横线 C,D,E	计划层参考轨迹，非实际执行结果
q,v	净负荷上尾裕度参数、终端电量代理价值

容量 12000，能量界 1200–10800，最大每时段充放电量 5000/6；主效率 ηc=ηd=0.9，往返为 0.81。题面“90%”解释存在歧义，另测往返 90%。每日首末相等只属于 Q1 条件，不继承为 Q2 硬约束。

3 信息集与非预见性

F[d,0]包含固定日价格、日历、d 日前全部已完成负载/光伏、真实初始 E 及已完成历史验证。G[d,:]仅由该信息决定。执行器的 F+[d,t]额外包含当前实时供需观测，不包含 t 之后的真实值。十分钟平均数据模拟时段内即时平衡，不意味着开区间前已知其均值。

代码将 make_plan 的输入限制为 Features、初始 E、固定价格及配置；Features 逐日起点构建，所有残差满足实际观测日<d。反馈函数 feedback_step 仅接收五个当前量，无法看到未来数组。反事实测试分别修改当天和未来月份数据，检查当天计划不变；另修改日内未来段，检查此前动作不变。

4 预测模型及误差校准

比较昨日同刻、过去 7 日均值及组合预测。组合预测为

$$\hat X_{d+j,t}=\tfrac12\operatorname{mean}_{k\in[d-7,d-1]}X_{k,t}+
\tfrac12\operatorname{mean}_{k<d,\,k\equiv d+j\pmod7}^{最近 4 日}X_{k,t}.\tag{1}$$

不足时使用可用样本，同星期无样本时回退近期均值。L 和 P 分别预测后相减。即使预测第二、三天，所有 k 仍严格小于当前 d。每日发出的多时距预测独立存档，只有对应实际日完成后才加入残差池。最近 30 个有效残差形成 m=max(0,Qq(e))。q=0 配置表示不加裕度。正截断后的曲线是保守裕度曲线，不保证具有精确 q 覆盖。

原报告将固定 7 日/4 周均值称扩展窗口不够准确；本文称历史滑窗预测与逐日起点验证。未采用缺少本地天气数据支持的复杂预测器。三类可解释预测器已联合参与经济比较，不能从本实验推出所有更复杂方法都无价值。

5 风险分位的数学依据

忽略储能的单时段费用 J(G)=pG+5p E[(N−G)+]。连续分布下 J'(G)=p−5pPr(N>G)，内点最优为 F_N(G)=0.8。直观上缺购的相对损失为 4p，过购损失为 p。非负约束存在时还需投影。

动态储能的效率、SOC 界、功率限制、价差及跨日影子价值破坏单时段分离，因此 0.8 不再严格最优。本文事先注册 q=0、0.65、0.70、0.75、0.80、0.85、0.90、0.95，通过过去实际费用而非全年误差选择。

6 计划层与终端条件

对 T=144/288/432、风险净负荷 n=(预测负载−预测光伏)+m，求

$$\min\sum_{t=0}^{T-1}p_tG_t-vE_T,\tag{2}$$
$$G_t+\bar D_t-\bar C_t-\bar W_t=n_t,\quad G_t,\bar W_t\ge0,\tag{3}$$
$$E_{t+1}=E_t+\eta_c\bar C_t-\bar D_t/\eta_d,\quad1200\le E_t\le10800,\tag{4}$$
$$0\le\bar C_t,\bar D_t\le5000/6,\quad E_0=E^{real}_{d,0}.\tag{5}$$

闭合候选 v=0 且 E_T=E_0；24h 终端价值候选不强制末态，v=日价格最低 24 点均价/ηc，是低价补回内部电量的代理成本。式(2)是设计计划的代理目标；正式账单始终是式(8)，不把终端价值算成真实收入。

LP 允许的同时充放电可精确消除。令 δ=min(C,D/(ηcηd))，取 C'=C−δ、D'=D−ηcηdδ、W'=W+(1−ηcηd)δ，G 与 E 及目标均不变，功率界满足且至少一项为零。因此此计划模型存在互斥 LP 最优解，不必全年反复求解二进制 MILP。小算例与代表日期另用 MILP 验证费用一致。这一证明依赖自由弃电，不能用于不允许弃电的其他模型。

7 实际执行与费用

令 r=G+P−L，若 r≥0 则

$$C=\min(r,5000/6,(10800-E)/\eta_c),\quad D=H=0,\quad W=r-C.\tag{6}$$

若 r<0 则

$$D=\min(-r,5000/6,(E-1200)\eta_d),\quad H=-r-D,\quad C=W=0.\tag{7}$$

更新状态，下一日初值等于本日实际末值。该执行器保障当期供电且不使用未来，但属于贪心反馈，不声称未来紧急费用的动态最优。正式紧急定义是最大可行储能补缺后仍剩余的供给缺口。

$$F=\sum_{d,t}(p_tG_{d,t}+5p_tH_{d,t}).\tag{8}$$

计划 G 当日不变，即使被弃用仍收费。W 包括两类电量；主口径光伏优先，未利用计划电量=min(W,G)，弃光=max(W−G,0)。不作来源优先假设时弃光界为[max(W−G,0),min(W,P)]，因此 W 本身不能代表弃光。

8 跨日启动与历史模型选择

1 月 1 日无历史：G=0、储能静置、光伏直接供负载、缺口应急；初始 6000 保持。此日是公开的保守回退假设。1 月 2 日起按 mean7、q=0.8、48h 闭合共同预运行，月底真实状态传给 2 月。1 月费用独立保存。

预先注册 96 个预测×分位×终端组合及 1 个有限策略场景候选。每月首日以最近 21 个已完成日作为验证段，从该段已知正式初值出发，候选各自连续运行；评分为 Σ 真实现金费−v(E 末−E 初)，避免偏好透支末态。选最小评分配置，月内固定配置，每天更新历史。参数筛选本身也是可执行策略；2 月选择仅用 1 月，12 月选择不使用 12 月结果。

场景候选用过去至多 12 条联合完整日残差曲线，在 7 种 q 计划中最小化平均实际反馈费用与末态价值。储能由同一反馈映射逐点决定，无场景内自由未来储能变量，因此非预见性成立。这是有限策略样本平均优化，而非精确多阶段随机规划。场景压力另用 200 次整日抽样对照逐点独立重排，作为经验诊断，不是严格分布外置信保证。

9 求解结果与参数选择

```
'''
    text+='\n'+table(['选择时刻','配置','历史截止','历史调整评分（元）'],
[[str(dates[int(r['selection_day'])]),r['config'],str(dates[int(r['end'])]),f'{float(r["
score"]):,.2f}'] for r in sel])+'\n\n'
    text+=f'终端代理价值 v={terminal_value(p):.8f}元/内部 kWh。所有月份选择都发生在月初历史数据上；全年固定候选排名不参与过去计划。正式统计区间为 334 天，共 48,096 时段。计划费**{fmt(summary["plan_cost_yuan"])}元**，紧急费 **{fmt(summary["emergency_cost_yuan"])}元**，总费 **{fmt(summary["total_cost_yuan"])}元**。紧急购电 **{fmt(summary["emergency_kwh"])}kWh**，共{summary["emergency_periods"]}个时段、{summary["emergency_days"]}日，最大单时段{fmt(summary["max_emergency_kwh"])}kWh，日费用 95%分位{fmt(summary["daily_p95_yuan"])}元。\n\n'
    text+=f'期初/期末储电量分别{fmt(summary["initial_kwh"])}、{fmt(summary["terminal_kwh"])}kWh；储电量范围{fmt(summary["energy_min"])}–{fmt(summary["energy_max"])}kWh。总充电{fmt(summary["charge_kwh"])}kWh，总放电{fmt(summary["discharge_kwh"])}kWh。总未利用电量{fmt(summary["waste_kwh"])}kWh，光伏优先口径弃光{fmt(summary["pv_curtail_kwh"])}kWh，来源分配不确定时弃光上界{fmt(summary["pv_curtail_upper_kwh"])}kWh，主口径光伏消纳率{summary["pv_utilization"]:.4%}。\n\n'
    text+='![月度实际费用与紧急购电](figures/monthly_costs.png)\n\n## 10 预测评价\n\n'
    text+=table(['预测器','对象','MAE kW','RMSE kW','真实值减预测偏差 kW'],
[[r['method'],r['object'],f'{float(r["mae_kw"]):.3f}',f'{float(r["rmse_kw"]):.3f}',f'{fl
oat(r["bias_kw"]):.3f}'] for r in forecast if r['group']=='all'])+'\n\n'
    cover=next(r['coverage'] for r in forecast if r['method']=='formal' and
r['group']=='all' and r['object']=='net')
    text+=f'正式上尾裕度经验覆盖率{float(cover):.4%}。该覆盖率不等于不紧急购电概率，因电池能在时段间吸收误差；也不要求等于固定 80%，因为 q 按月变化且裕度正截断。分月及分小时 MAE/RMSE 保存在 forecast_metrics.csv。\n\n![误差与覆盖](figures/forecast_errors.png)\n\n## 11 候选对照与oracle\n\n'
```

```
    text+=table(['策略','现金总费元','紧急电量kWh','紧急时段数','末态kWh'],cmp_rows)+'\n\n'
    for name in
('mean7_q0.00_24h_cycle','blend_q0.00_24h_value','blend_q0.80_48h_cycle','scenario_blend
_48'):
        cost=float(byname[name]['total_cost_yuan']);text+=f'相对{name}，正式费用变化为
{(summary["total_cost_yuan"]/cost-1):+.4%}。\n\n'
    best=min(comparisons,key=lambda r:float(r['total_cost_yuan']))
    text+=f'事后固定候选最低现金费用为{best["config"]}的{float(best["total_cost_yuan"]):,.2f}
元。这只是事后诊断，不得据此倒改任何早期月度选择。正式策略不承诺超过每个事后固定候选。候选终点不同，完整表
另列终端价值调整成本；评价不能只选对某方案有利的现金口径。\n\n'
    text+=f'oracle直接使用全部334日真实净负荷，并严格匹配正式2月初和12月末状态，一次求解整段LP，最
优费用为{oracle["objective"]:,.4f}元，正式费用相对下界差距为{(summary["total_cost_yuan"]/
oracle["objective"]-1):.4%}。这是同边界确定性信息松弛下界，不是可执行计划，也未混入result2。单日
oracle若起终状态不同则不能用来证明逐日高低。\n\n'
    text+='## 12 四个指定日期\n\n'
    text+=table(['日期','负载kWh','光伏kWh','计划费元','紧急费元','总费元','紧急kWh','日初/末
kWh'],
[[r['date'],fmt(r['load_kwh']),fmt(r['pv_kwh']),fmt(r['plan_cost_yuan']),fmt(r['emergenc
y_cost_yuan']),fmt(r['total_cost_yuan']),fmt(r['emergency_kwh']),f'{r["initial_kwh"]:.2f
}/{r["terminal_kwh"]:.2f}'] for r in selected])+'\n\n'
    merged=merge_emergency(dates[31:],s['H'][31:])
    event_columns=[[x for x in merged if x['date']==r['date']] for r in selected]
    text+='### 表3 四个指定日期的紧急购电\n\n'
    headers=[label for r in selected for label in (r['date']+' 时间段','购电量kWh')]
    event_rows=[]
    for i in range(max(1,max(map(len,event_columns)))):
        event_rows.append([value for events in event_columns for value in
                           ([events[i]['period'],fmt(events[i]['kwh'])] if i<len(events)
else (['无',0] if not events and i==0 else ['',''']))])
    text+=table(headers,event_rows)+'\n\n'
    for r in selected:
        day=dates.index(datetime.fromisoformat(r['date']).date());g=s['G'][day]
        text+=f'### {r["date"]}\n\n'
        text+=table(['时间段','计划购电kWh','时间段','计划购电kWh','时间段','计划购电kWh'],
          [[PERIODS[t],fmt(g[t]),PERIODS[t+12],fmt(g[t+12]),PERIODS[t+24],fmt(g[t+24])]
for t in (60,96)])+'\n\n'
        text+=f'全天计划购电{fmt(g.sum())}kWh，含紧急的实际总购电{fmt((g+s["H"][day]).sum())}
kWh；全天购电费{fmt(r["total_cost_yuan"])}元。\n\n'
        text+=table(['时段','实际充电kWh','实际放电kWh','时段','实际充电kWh','实际放电kWh'],
          [[f'{j*4}:00-{j*4+4}:00',fmt(s['C'][day,j*24:(j+1)*24].sum()),fmt(s['D']
[day,j*24:(j+1)*24].sum()),
            f'{(j+1)*4}:00-{(j+2)*4}:00',fmt(s['C'][day,(j+1)*24:
(j+2)*24].sum()),fmt(s['D'][day,(j+1)*24:(j+2)*24].sum())] for j in (0,2,4)])+'\n\n'
        text+=f'0:00储电量{fmt(s["E"][day,0])}kWh，24:00储电量{fmt(s["E"][day,-1])}kWh。
\n\n'
        events=[x for x in merged if x['date']==r['date']]
        text+=table(['紧急购电时间段','购电量kWh'],[[x['period'],fmt(x['kwh'])] for x in
events] or [['无',0]])+'\n\n'
        max_t=int(np.argmax(s['H'][day]));surplus=np.maximum(a[day,:,1]-
a[day,:,0],0).sum()
        error=(a[day,:,0]-a[day,:,1])-(s['prediction'][day,:,0]-s['prediction']
[day,:,1])
        text+=f'当日实际净负荷{fmt(r["load_kwh"]-r["pv_kwh"])}kWh、未计储能前光伏时段性过剩
{fmt(surplus)}kWh，净负荷预测MAE为{r["net_mae_kw"]:.2f}kW。上尾裕度平均{s["margin"]
[day].mean()*6:.2f}kW，实际误差超过裕度的时段数为{int((error>s["margin"][day]
+1e-7).sum())}。'
        if s['H'][day,max_t]>1e-7:
            text+=f'最大紧急购电发生于{PERIODS[max_t]}，电量{s["H"][day,max_t]:.4f}kWh；该段初
始储电{s["E"][day,max_t]:.2f}kWh、放电{s["D"][day,max_t]:.2f}kWh。该信息直接区分SOC不足与功率
限制，不把所有补电都归因于天气。'
        else:text+='当日没有紧急购电，说明本日计划与可用储能共同覆盖了实际缺口，并不保证其他日也无风
险。'
        load_bias=(a[day,:,0]-s['prediction'][day,:,0]).sum()
        pv_bias=(a[day,:,1]-s['prediction'][day,:,1]).sum()
        text+=f'实际减预测的日负载偏差为{load_bias:+.2f}kWh，日光伏偏差为{pv_bias:+.2f}kWh。'
        if r['date']=='2025-06-21':
```

```
            text+=f'该日为周六，四个指定日中负载最低、光伏时段性富余最大；组合预测仍混入近期较高负载，加上 q=0.85 裕度，导致计划偏保守。虽然没有补电，总未利用电量达到{r["waste_kwh"]:.2f}kWh，说明“无应急”不能单独代表经济最优。'
        elif r['date']=='2025-12-21':
            text+='该日光伏电量为四日最低、净负荷最高，购电费用主要由较高的基础购电需求决定；风险裕度与多轮充放电共同覆盖供需误差，因此负荷高也不必然出现紧急购电。'
        else:
            text+='此日光伏集中于白天，而较高电价及补电风险延续至晚间，白天总发电充足不能替代晚间功率与 SOC 可用性。'
        text+=f'本反馈规则对已形成缺口优先释放储能，没有人为保留备用的机制；日内保留电量是此前供需及 SOC 约束的结果。\n\n![当日联合调度](figures/{r["date"]}.png)\n\n'
    text+='## 13 敏感性与场景检验\n\n![风险与窗口对照](figures/quantile_window_sensitivity.png)\n\n'
    text+=table(['替代解释','总费用 元','相对主解释','紧急购电 kWh'],[[name,fmt(v['total_cost_yuan']),f'{v["total_cost_yuan"]/summary["total_cost_yuan"]-1:+.4%}',fmt(v['emergency_kwh'])] for name,v in sensitivity.items() if name!='no_storage_mean7'])+'\n\n'
    text+='效率与固定储能实验固定主策略已经选出的配置日历，仅重算计划和执行，隔离口径变化，不宣称该口径重新历史调参后的最优性能。固定储能解释可能在实际不足时为预定充电紧急补电，故不作为主解释。\n\n'
    pressure=read_csv(OUT/'experiments/scenario_pressure.csv')
    text+=table(['日期','残差构造','平均费元','95%费元','平均紧急 kWh'],[[r['date'],r['kind'],f'{float(r["mean_cost"]):.2f}',f'{float(r["p95_cost"]):.2f}',f'{float(r["mean_emergency"]):.2f}'] for r in pressure])+'\n\n'
    text+='这些场景来自决策日前历史，而非当天真实未来；因也用于校准，不能称完全独立的样本外测试。真正时序样本外评价是 334 日实际逐日运行。独立重排改变相关性，差异用于说明相关性对风险评价的重要性，不能预设它一定低估风险。\n\n## 14 模型检验与复杂度\n\n'
    text+=f'逐时能量平衡最大误差{summary["checks"]["balance"]:.3e}kWh，状态误差{summary["checks"]["state"]:.3e}kWh，跨日连续误差{summary["continuity_error"]:.3e}kWh，所有边界和互斥检查通过。144/288/432 时段 LP 分别有 721/1441/2161 变量及 288/576/864 等式，另有变量界。正式日均计划求解{summary["avg_plan_solve_seconds"]:.6f}秒；含历史选参、97 个候选全年诊断及敏感性等总计算耗时{metadata["total_seconds"]:.2f}秒，测试和制图另计。\n\n'
    text+='测试包括未来反事实、月首参数选择、当期执行前缀不变、物理小算例、费用不退款、紧急区间合并、全量工作簿回读、同边界 oracle 及 Q1 接手哈希。最终实际通过数量与日志见 validation.json、tests.log；不把只检查文件存在当作结果验证。\n\n## 15 模型评价与局限\n\n'
    text+='优势在于信息边界可测试、费用可重算、终端条件可比较、复杂度适中，且正式策略的每次选择都能追溯到过去数据。创新点是将预测器、风险裕度及终端处理放入同一历史经济验证框架，并通过可执行反馈评价有限策略场景，避免全知二阶段偏差。\n\n局限包括：仅一年数据、1 月早期样本少；未给天气特征，光伏预测能力有限；历史 21 日/30 日窗口与低价终端价值是预先固定的设计而非普适最优；贪心执行未优化跨时段应急机会成本；十分钟平均模型不能验证更细时间粒度；不计衰减、无并网容量；时间标签与 90%效率仍有题面歧义；光伏来源分摊影响消纳率但不影响总费用。月度历史选择未提供分布漂移下收益保证，不能用事后最低候选夸大算法能力。\n\n## 参考文献与依据\n\n[1] 原题 C 题.pdf 及附件 1、2、5：参数、费用、输出合同。\n\n[2] Panda 等，2024，Sustainable Energy, Grids and Networks 38:101334，DOI:10.1016/j.segan.2024.101334：日前承诺与实时实施分层，第 5-8 页；当前一步执行，第 6 页；跨日比较，第 8-10 页。\n\n[3] Herding 等，2023，Applied Energy 336:120847，DOI:10.1016/j.apenergy.2023.120847：统一第一阶段及锁定后评价，第 5、7 页；模拟样本外边界，第 7 页。其全天第二阶段简化、售电和机组不照搬。\n\n两份建模报告是本地方案参考，不把它们的旧数值或“最终”称谓当作实验依据。具体取舍见方案审计.md 及文献方法映射.md。\n'
    (OUT/'Q2_final.md').write_text(text,encoding='utf-8')
    decision='# Q2 最终决策报告\n\n'+f'正式算法是月度过去 21 日经济验证选择的因果预测及储能计划策略。实际 11 个月均选择组合预测与 24h 终端价值，q 按月变化，不固定 80%，未选择 48h 或场景候选。主结果费用{summary["total_cost_yuan"]:,.4f}元；这是原始附件重算结果。\n\n'
    decision+='## 为什么这样选择\n\n每月选择最小历史未态调整费用，不根据未来月份挑参。候选全年对比仅作诊断。24h 终端价值避免强制闭合，同时用代理电量价值限制透支；它在 11 次实际历史验证中胜出，不能据此推导所有数据上都优于 48h。0.8 有无储能单时段理论支持，但动态系统允许其他 q 更好。\n\n'
    decision+=table(['日期','预测/风险/终端配置'],[[str(dates[int(r['selection_day'])]),r['config']] for r in sel])+'\n\n'
    decision+='## 对现有报告与论文的取舍\n\n保留固定日前承诺、跨日 SOC、5 倍结算、历史残差、物理反馈与场景相关性。修改固定 0.8 和预设 48h、信息集时点矛盾、弃电等同弃光以及场景内全知储能。Panda 真正启发是承诺/执行分层与跨日比较；Herding 是统一第一阶段和锁定后样本评价。两篇的售电、机组、备用市场、随机价格、竞价曲线和 DRL 没有本题依据，不采用。其文字中聚类数据范围、第二阶段完美信息等限制也已记录。\n\n'
    decision+='## 可执行性与信息证明\n\n0:00 只用历史实际、日历、固定电价、实际初始 E、过去验证和已实现残差。计划函数不接受当天实际；当前反馈函数不接受未来数组。测试篡改当天/未来月份后比较计划，另检查月首选择与当前动作前缀。证据见 test_q2.py 和最终测试日志。\n\n'
    decision+='## 数值与对照\n\n'+table(['指标','值'],[[k,fmt(summary[k]) if isinstance(summary[k],float) else summary[k]] for k in ('plan_cost_yuan','emergency_cost_yuan','total_cost_yuan','emergency_kwh','emergency_periods','emergency_days')])+'\n\n'+table(['策略','费用元','紧急 kWh','紧急时段','末
```

```
E'],cmp_rows)+'\n\n'
    decision+='## 预测及四个日期\n\n'+table(['对象','MAE kW','RMSE kW'],
[[r['object'],f'{float(r["mae_kw"]):.3f}',f'{float(r["rmse_kw"]):.3f}'] for r in
forecast if r['method']=='formal' and r['group']=='all'])+'\n\n'
    decision+=table(['日期','计划费元','紧急费元','总费元','紧急 kWh'],
[[r['date'],fmt(r['plan_cost_yuan']),fmt(r['emergency_cost_yuan']),fmt(r['total_cost_yua
n']),fmt(r['emergency_kwh'])] for r in selected])+'\n\n'
    decision+='## 核验与仍需说明的边界\n\nresult2.xlsx 按官方模板展开，计划表数值为 0:00 计划，充放
电/H 为实际执行；全量回读检查由 test_q2.py 进行，最终状态见 validation.json。紧急
H=max(0,L−PV−G−Dmax_available)，全年 SOC 连续。场景进入候选及压力测试，但实际月度选择未采用场景候选。
\n\nQ1 接手后 39 个文件保持不变，当前 Q1_frozen_v2 清单与 ZIP 完整校验通过；Q1_frozen_v1 保留为历史。工
作文件相对 v1 的 8 项差异属于既有版本演进，任一 Q1 清单均未改写。当前 Q1 测试结果另保存。\n\n 时间结束点、实
时储能反馈、初日回退、效率解释、终端价值、光伏来源分摊均须作为假设公开。所有替代口径及数值、完整表 1/2/3、
图件、模型局限见 Q2_final.md，复现命令见 README.md。\n'
    (OUT/'Q2 最终决策报告.md').write_text(decision,encoding='utf-8')


def deliver():

check_q1();dates,a,p=read_year();s=dict(np.load(OUT/'experiments/formal_arrays.npz'))
    summary=summary_of(s,a,p)
    summary.update(model='monthly_past_only_economic_selection',period='2025-02-01/2025-
12-31',
                   official_is_oracle=False,price_rule='plan*p + emergency*5*p',
                   time_semantics='assumed_end_of_interval',efficiency='charge=.9
discharge=.9',
                   input_data='附件 1 价格、附件 2 实际；不使用附件 3/4',
                   full_verification='see validation.json after actual independent test
execution')
    write_json(OUT/'summary.json',summary)
    merged=export_workbook(dates,a,p,s)
    write_csv(OUT/'emergency_purchase_merged.csv',merged)
    daily,selected=data_exports(dates,a,p,s)
    comparisons=read_csv(OUT/'experiments/candidate_summary.csv')
    figures(dates,a,p,s,daily,comparisons)
    generate_documents(dates,a,p,s,summary,daily,selected,comparisons)
    print('已生成工作簿、完整明细、论文稿、决策报告和图件')


if __name__=='__main__':deliver()
```

B7 code/Q2/solve_q2.py

```
"""问题 2 唯一重建入口：原始附件→历史选择→全年执行→实验→交付→测试→冻结。

python Q2/solve_q2.py --compute 仅计算；--deliver 用本次计算产物生成图表/论文；
无参数完整重建（已有冻结时拒绝覆盖）。所有派生文件生成代码均保留。
"""
from pathlib import Path
import os,sys,time,json,csv,hashlib,argparse
from dataclasses import asdict,replace
ROOT=Path(__file__).resolve().parents[1]
OUT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT))
os.environ.setdefault('MPLCONFIGDIR',str(OUT/'.matplotlib'))
import numpy as np
from microgrid_common import Battery
from microgrid_online import
read_year,execute_day,plan_net,terminal_value,validate_execution,merge_emergency
from forecast_q2 import build_features,METHODS
from strategy_q2 import Config,CANDIDATES,WARMUP,make_plan,select_config


def write_json(path,obj):
    path.parent.mkdir(parents=True,exist_ok=True)
    path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item()
```

```
if isinstance(x,np.generic) else str(x))+'\n',encoding='utf-8')


def write_csv(path,rows):
    rows=list(rows);path.parent.mkdir(parents=True,exist_ok=True)
    if not rows:return
    with path.open('w',encoding='utf-8-sig',newline='') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)


def check_q1():
    handoff=OUT/'q1_handoff_hashes.json'
    if not handoff.exists():
        return 0  # Submission copy omits freeze-only protection metadata.
    known=json.loads(handoff.read_text())
    changes=[p for p,h in known.items() if not (ROOT/p).exists() or
hashlib.sha256((ROOT/p).read_bytes()).hexdigest()!=h]
    assert not changes, f'Q1接手后发生变化: {changes}'
    return len(known)


def blank_result(n):
    result={k:np.zeros((n,144)) for k in
('G','C','D','H','W','unused_grid','pv_curtail','pv_curtail_upper','C_plan','D_plan','ma
rgin')}
    result['E']=np.zeros((n,145));result['E_plan']=np.zeros((n,145))

result['prediction']=np.zeros((n,144,2));result['q']=np.zeros(n);result['seconds']=np.ze
ros(n)
    result['config']=np.full(n,'',dtype='<U80')
    return result


def store_day(out,day,real,plan):
    for k in ('G','C','D','H','W','E','unused_grid','pv_curtail','pv_curtail_upper'):
        out[k][day]=real[k]
    if plan is not None:
        for k in ('C','D','E'):out[k+'_plan'][day]=plan[k][:145 if k=='E' else 144]
        for k in ('prediction','margin','q','seconds','config'):out[k][day]=plan[k]
    else:
        out['E_plan'][day]=real['E'];out['config'][day]='startup_idle_zero_plan'


def
simulate(dates,actual,price,features,config=None,battery=Battery(),fixed_actions=False,
             no_storage=False,stop=None,candidates=CANDIDATES,write_progress=False):
    """环境读取actual，日前make_plan仅能读取因果features和当前E。"""
    n=len(actual) if stop is None else stop
    out=blank_result(n);e=6000.;known=[e];selection=[];current=WARMUP
    for day in range(n):
        if day==0:
            plan=None;g=np.zeros(144)
        else:
            if day<31:current=WARMUP
            elif config is not None:current=config
            elif dates[day].day==1:

current,rows=select_config(features[:day],actual[:day],known,day,price,battery,candidate
s)
                selection.extend(rows)
                if write_progress:
                    print(f'月首选择{dates[day]} {current.name}',flush=True)

write_csv(OUT/'experiments/monthly_selection_progress.csv',selection)
            plan=make_plan(features[day],e,price,current,battery)
            g=plan['G'][:144].copy()
```

```
            if no_storage and day>=31:
                g=np.maximum(plan['prediction'][:,0]-plan['prediction'][:,1],0)
                plan['C'][:144]=0;plan['D'][:144]=0;plan['E'][:145]=e
                plan['config']='no_storage_'+current.method
        fixed=(plan['C'][:144],plan['D'][:144]) if fixed_actions and day>=31 else None
        real=execute_day(g,actual[day],e,battery,fixed=fixed,idle=day==0 or (no_storage
and day>=31))
        store_day(out,day,real,plan);e=float(real['E'][-1]);known.append(e)
    return out,selection


def summary_of(s,actual,price,battery=Battery(),start=31):
    g,h,e=s['G'][start:],s['H'][start:],s['E'][start:]
    planned=g@price;emergency=h@(5*price);daily=planned+emergency
    errors=[]
    for d in range(start,len(s['G'])):
        errors.append(validate_execution(actual[d],{k:s[k][d] for k in
('G','C','D','H','W','E')},battery))

result=dict(days=len(g),plan_cost_yuan=float(planned.sum()),emergency_cost_yuan=float(em
ergency.sum()),

total_cost_yuan=float(daily.sum()),daily_mean_yuan=float(daily.mean()),daily_p95_yuan=fl
oat(np.quantile(daily,.95)),

plan_kwh=float(g.sum()),emergency_kwh=float(h.sum()),total_grid_kwh=float(g.sum()
+h.sum()),

emergency_periods=int((h>1e-7).sum()),emergency_days=int(np.any(h>1e-7,axis=1).sum()),

max_emergency_kwh=float(h.max()),energy_min=float(e.min()),energy_max=float(e.max()),
        initial_kwh=float(e[0,0]),terminal_kwh=float(e[-1,-1]),
        charge_kwh=float(s['C'][start:].sum()),discharge_kwh=float(s['D']
[start:].sum()),
        waste_kwh=float(s['W'][start:].sum()),unused_plan_kwh=float(s['unused_grid']
[start:].sum()),
        pv_curtail_kwh=float(s['pv_curtail']
[start:].sum()),pv_curtail_upper_kwh=float(s['pv_curtail_upper'][start:].sum()),

pv_utilization=float(1-s['pv_curtail'][start:].sum()/actual[start:len(s['G']),:,1].sum()
),
        continuity_error=float(np.max(np.abs(e[1:,0]-e[:-1,-1]))) if len(e)>1 else 0.,

lower_bound_states=int((e<=battery.minimum+1e-6).sum()),upper_bound_states=int((e>=batte
ry.maximum-1e-6).sum()),
        avg_plan_solve_seconds=float(s['seconds'][start:].mean()),
        terminal_adjusted_cost_yuan=float(daily.sum()-
terminal_value(price,battery)*(e[-1,-1]-e[0,0])),
        checks={k:max(r[k] for r in errors) for k in errors[0]})
    return result


def forecasting_metrics(dates,actual,features,formal):
    rows=[]
    for method in (*METHODS,'formal'):
        pred=np.asarray([f.prediction[method][0] for f in features]) if method!='formal'
else formal['prediction']
        for group in ['all']+[f'month_{m:02d}' for m in range(2,13)]+[f'hour_{h:02d}'
for h in range(24)]:
            ds=np.arange(31,len(dates));ts=np.arange(144)
            if group.startswith('month'):ds=np.array([d for d in ds if
dates[d].month==int(group[-2:])])
            if group.startswith('hour'):ts=np.arange(int(group[-2:])*6,(int(group[-2:])
+1)*6)
            for obj,j in [('load',0),('pv',1),('net',2)]:
                v=actual[:,:,j]-pred[:,:,j] if j<2 else (actual[:,:,0]-actual[:,:,1])-
```

```
(pred[:,:,0]-pred[:,:,1])
                err=v[np.ix_(ds,ts)]*6

row=dict(method=method,group=group,object=obj,mae_kw=float(np.abs(err).mean()),rmse_kw=f
loat(np.sqrt((err**2).mean())),bias_kw=float(err.mean()),coverage='')
                if j==2 and
method=='formal':row['coverage']=float((v[np.ix_(ds,ts)]<=formal['margin']
[np.ix_(ds,ts)]+1e-7).mean())
                rows.append(row)
    return rows


def
fixed_path_sensitivity(dates,actual,price,features,formal,configs,battery=Battery(),fixe
d=False):
    """冻结已选日历配置重放；用于口径影响，非重新挑参后的最优值。"""
    out=blank_result(len(actual));e=6000.
    for d in range(len(actual)):
        if d==0:plan=None;g=np.zeros(144)
        else:
            conf=WARMUP if d<31 else configs[formal['config'][d]]
            plan=make_plan(features[d],e,price,conf,battery);g=plan['G'][:144]
        real=execute_day(g,actual[d],e,battery,
                         fixed=(plan['C'][:144],plan['D'][:144]) if fixed and d>=31 else
None,idle=d==0)
        store_day(out,d,real,plan);e=real['E'][-1]
    return out


def pressure_tests(dates,actual,price,features,formal):
    """历史经验整日场景对照独立重排；不声称独立真实分布置信保证。"""
    rng=np.random.default_rng(20260911);rows=[]
    selected=[i for i,d in enumerate(dates) if d.isoformat() in ('2025-03-20','2025-06-
21','2025-09-23','2025-12-21')]
    for day in selected:
        conf=next(c for c in CANDIDATES if c.name==formal['config'][day])
        r=features[day].residual[conf.method][0];pred=formal['prediction'][day]
        for kind in ('whole_day','independent_points'):
            sims=[];hs=[];eds=[]
            for k in range(200):
                error=r[rng.integers(len(r))] if kind=='whole_day' else
r[rng.integers(len(r),size=144),np.arange(144)]
                scene=np.maximum(pred+error,0)
                real=execute_day(formal['G'][day],scene,formal['E'][day,0])
                sims.append(float(price@real['G']
+5*price@real['H']));hs.append(real['H'].sum());eds.append(real['E'][-1])

rows.append(dict(date=dates[day].isoformat(),kind=kind,samples=200,source_max_day=max(fe
atures[day].residual_dates[conf.method][0]),

mean_cost=float(np.mean(sims)),p95_cost=float(np.quantile(sims,.95)),mean_emergency=floa
t(np.mean(hs)),mean_terminal=float(np.mean(eds))))
    return rows


def compute():
    check_q1();tic=time.perf_counter()
    dates,actual,price=read_year();features=build_features(actual)
    configs={c.name:c for c in CANDIDATES}
    write_json(OUT/'experiments/protocol.json',dict(candidates=[asdict(c) for c in
CANDIDATES],

validation_days=21,residual_days=30,selection='monthly_past_only',warmup=asdict(WARMUP),
        pricing='p*G + 5*p*H',formal_start='2025-02-01',seed=20260911))
    formal,selection=simulate(dates,actual,price,features,write_progress=True)
    np.savez_compressed(OUT/'experiments/formal_arrays.npz',**formal)
```

```
    write_csv(OUT/'experiments/monthly_selection.csv',selection)
    fm=summary_of(formal,actual,price)
    write_json(OUT/'experiments/formal_metrics.json',fm)

write_csv(OUT/'forecast_metrics.csv',forecasting_metrics(dates,actual,features,formal))
    print('正式策略现金费用',fm['total_cost_yuan'],flush=True)
    comparisons=[];candidate_daily=[]
    for i,conf in enumerate(CANDIDATES):
        s,_=simulate(dates,actual,price,features,config=conf)
        m=summary_of(s,actual,price)

comparisons.append(dict(config=conf.name,method=conf.method,q=conf.q,horizon_hours=24*co
nf.horizon,
                                terminal=conf.terminal,scenario=conf.scenario,**{k:v for
k,v in m.items() if k!='checks'}))
        for d in range(31,365):
            candidate_daily.append(dict(config=conf.name,date=dates[d].isoformat(),
                plan_cost=float(price@s['G'][d]),emergency_cost=float(5*price@s['H']
[d]),
                emergency_kwh=float(s['H'][d].sum()),initial=float(s['E']
[d,0]),terminal=float(s['E'][d,-1])))
        if i%8==0:print(f'固定候选诊断{i+1}/{len(CANDIDATES)} {conf.name}
{m["total_cost_yuan"]:.2f}',flush=True)
    write_csv(OUT/'experiments/candidate_summary.csv',comparisons)
    write_csv(OUT/'experiments/candidate_daily.csv',candidate_daily)
    s,_=simulate(dates,actual,price,features,config=Config('mean7',0,1),no_storage=True)
    sensitivities={'no_storage_mean7':summary_of(s,actual,price)}
    for name,b,fixed in [('fixed_battery',Battery(),True),
('roundtrip_90',replace(Battery(),eta_charge=np.sqrt(.9),eta_discharge=np.sqrt(.9)),Fals
e)]:
        s=fixed_path_sensitivity(dates,actual,price,features,formal,configs,b,fixed)
        sensitivities[name]=summary_of(s,actual,price,b)
    write_json(OUT/'experiments/sensitivities.json',sensitivities)
    print('计算整段同边界完美信息下界',flush=True)
    oracle=plan_net((actual[31:,:,0]-actual[31:,:,1]).ravel(),np.tile(price,334),
                    formal['E'][31,0],terminal=float(formal['E'][-1,-1]))
    np.savez_compressed(OUT/'experiments/oracle_arrays.npz',**{k:oracle[k] for k in
('G','C','D','E','W')})
    write_json(OUT/'experiments/oracle.json',{k:v for k,v in oracle.items() if k not in
('G','C','D','E','W')})
    assert oracle['objective']<=fm['total_cost_yuan']+1e-5

write_csv(OUT/'experiments/scenario_pressure.csv',pressure_tests(dates,actual,price,feat
ures,formal))
    audit=[]
    for d in range(365):
        f=features[d];
audit.append(dict(date=dates[d].isoformat(),training_max=(dates[d-1].isoformat() if d
else ''),

history_days=d,history_sha256=f.history_sha256,config=str(formal['config']
[d]),q=float(formal['q'][d]),
                    max_residual_observed_index=max([-1]+[x for m in METHODS for js in
f.residual_dates[m] for x in js])))
    write_csv(OUT/'information_audit.csv',audit)

write_json(OUT/'experiments/run_metadata.json',dict(total_seconds=time.perf_counter()-
tic,

source_hashes={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p
in [ROOT/'附件/附件1.xlsx',ROOT/'附件/附件2.xlsx',ROOT/'附件/附件5/result2.xlsx',ROOT/'C
题.pdf'] if p.exists()},

q1_unchanged_files=check_q1(),gate_a_b='written_before_solver',gate_c='prototype.json',
        official_method='monthly past-only selection among predeclared causal policies',
        fixed_candidate_ranking='retrospective diagnostics only; NOT used for official
```

```
past plans'))
    print('全部计算完成',time.perf_counter()-tic,flush=True)


def prototype():
    """重建 Gate C 小规模证明；全年正式结果不参与本检查。"""
    dates,a,p=read_year();features=build_features(a[:35]);e=6000.;records=[]
    for day in range(1,5):
        s=make_plan(features[day],e,p,Config())
        r=execute_day(s['G'][:144],a[day],e)
        m=plan_net(s['net'],np.tile(p,2),e,integer=True)
        assert abs(s['objective']-m['objective'])<1e-5
        records.append(dict(day=day,lp_milp_difference=s['objective']-m['objective'],
                            checks=validate_execution(a[day],r),seconds=s['seconds']))
        e=r['E'][-1]
    changed=a[:35].copy();changed[20:]*=2
    alt=build_features(changed)
    x=make_plan(features[20],6000,p,Config());y=make_plan(alt[20],6000,p,Config())
    np.testing.assert_array_equal(x['G'],y['G'])

write_json(OUT/'experiments/prototype.json',dict(days=records,counterfactual_plan_unchan
ged=True))


if __name__=='__main__':
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--compute',action='store_true');parser.add_argument('--
deliver',action='store_true')
    parser.add_argument('--prototype',action='store_true')
    args=parser.parse_args()
    if (OUT/'freeze_manifest.json').exists():raise RuntimeError('Q2 已冻结；请在独立副本解除
清单后复现')
    if args.prototype:
        prototype();sys.exit(0)
    if not args.deliver:compute()
    if not args.compute:
        from report_q2 import deliver
        deliver()
```

B8 code/Q2/strategy_q2.py

```
"""预先注册的候选集合；计划层完全不接收当天真实数据。"""
from dataclasses import dataclass,asdict
import numpy as np
from microgrid_common import Battery
from microgrid_online import plan_net,execute_day,terminal_value
from forecast_q2 import METHODS,risk_net

QUANTILES=(0.,.65,.70,.75,.80,.85,.90,.95)
WINDOWS=((1,'cycle'),(1,'value'),(2,'cycle'),(3,'cycle'))


@dataclass(frozen=True)
class Config:
    method:str='blend'
    q:float=.8
    horizon:int=2
    terminal:str='cycle'
    scenario:bool=False

    @property
    def name(self):
        return 'scenario_blend_48' if self.scenario else f'{self.method}_q{self.q:.2f}
_{24*self.horizon}h_{self.terminal}'
```

```
CANDIDATES=tuple(Config(m,q,h,t) for m in METHODS for q in QUANTILES for h,t in WINDOWS)
+(Config(scenario=True),)
WARMUP=Config('mean7',.8,2,'cycle')


def make_plan(features,initial,price,config,battery=Battery()):
    assert features.training_max<features.day
    if config.scenario:
        return scenario_plan(features,initial,price,battery)
    net,margin=risk_net(features,config.method,config.q,config.horizon)
    result=plan_net(net,np.tile(price,config.horizon),initial,config.terminal,battery)

result.update(q=config.q,margin=margin[0],prediction=features.prediction[config.method]
[0],
                  config=config.name,net=net)
    return result


def scenario_plan(features,initial,price,battery=Battery()):
    """有限策略 SAA；每条场景按同一 feedback_step 执行，无场景全知储能。"""
    r=features.residual['blend'][0]
    if not len(r):
        return make_plan(features,initial,price,Config(),battery)
    pred=features.prediction['blend'][0]
    scenes=np.maximum(pred[None,:,:]+r,0)
    # 至多 12 条按时间均匀保留的完整日轨迹；没有逐时点独立抽样。

scenes=scenes[np.unique(np.linspace(0,len(scenes)-1,min(12,len(scenes))).astype(int))]
    value=terminal_value(price,battery)
    best=None;best_score=np.inf;seconds=0
    for q in QUANTILES[1:]:
        plan=make_plan(features,initial,price,Config('blend',q,2,'cycle'),battery)
        seconds+=plan['seconds']
        scores=[]
        for scene in scenes:
            s=execute_day(plan['G'][:144],scene,initial,battery)
            scores.append(float(price@s['G']+5*price@s['H']-value*(s['E'][-1]-initial)))
        score=float(np.mean(scores))
        if score<best_score-1e-8:
            best,best_score=plan,score
    best.update(config='scenario_blend_48',scenario_score=best_score,seconds=seconds)
    return best


def
select_config(features,past_actual,known_energy,start_day,price,battery=Battery(),candid
ates=CANDIDATES):
    """月首只用过去 21 日，验证段共同起点、各候选独立连续 E，终点价值调整。"""
    assert len(past_actual)==start_day and len(features)>=start_day
    begin=max(1,start_day-21)
    initial=float(known_energy[begin])
    value=terminal_value(price,battery)
    rows=[]
    for conf in candidates:
        e=initial;cash=0.;emergency=0.;seconds=0.
        for k in range(begin,start_day):
            plan=make_plan(features[k],e,price,conf,battery)
            real=execute_day(plan['G'][:144],past_actual[k],e,battery)
            cash+=float(price@real['G']+5*price@real['H'])
            emergency+=float(real['H'].sum());seconds+=plan['seconds'];e=real['E'][-1]

rows.append(dict(selection_day=start_day,begin=begin,end=start_day-1,config=conf.name,
                         cash_cost=cash,score=cash-value*(e-
initial),initial=initial,terminal=float(e),
                         emergency_kwh=emergency,solve_seconds=seconds))
    idx=min(range(len(rows)),key=lambda i:(rows[i]['score'],i))
```

```
    for i,row in enumerate(rows):row['selected']=i==idx
    return candidates[idx],rows
```

B9 code/Q2/test_q2.py

```
"""独立反事实/物理/结算/输出测试；不修改 Q1 和正式结果。"""
import sys,json,csv,hashlib,unittest
from pathlib import Path
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
import numpy as np
from openpyxl import load_workbook
from microgrid_common import Battery
from microgrid_online import
read_year,plan_net,execute_day,feedback_step,merge_emergency,PERIODS,validate_execution,
terminal_value
from forecast_q2 import build_features,forecast
from strategy_q2 import make_plan,Config,select_config,CANDIDATES
from solve_q2 import simulate,check_q1
Q2=Path(__file__).resolve().parent
HAS_Q2_OUTPUTS=(Q2/'experiments/formal_arrays.npz').exists() and
(Q2/'summary.json').exists() and (Q2/'result2.xlsx').exists()


class Q2Tests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.dates,cls.actual,cls.price=read_year()
        cls.features=build_features(cls.actual[:65])

    def test_dates_time_units(self):
        self.assertEqual(len(self.dates),365)
        self.assertEqual(len(self.dates[31:]),334)
        self.assertEqual(str(self.dates[31]),'2025-02-01')
        self.assertEqual(PERIODS[0],'0:00-0:10');self.assertEqual(PERIODS[-1],'23:50-
24:00')
        self.assertEqual(PERIODS[60],'10:00-10:10')
        wb=load_workbook(ROOT/'附件/附件 2.xlsx',read_only=True,data_only=True)

self.assertAlmostEqual(wb.worksheets[0]['B2'].value/6,self.actual[0,0,0]);wb.close()

    def test_future_counterfactual_all_candidate_types(self):
        changed=self.actual[:65].copy();changed[40:,:,0]*=1.8;changed[40:,:,1]*=.1
        bank=build_features(changed)
        for conf in
(Config('last',0,1),Config('blend',.7,1,'value'),Config('mean7',.9,3),Config(scenario=Tr
ue)):
            a=make_plan(self.features[40],6000,self.price,conf)
            b=make_plan(bank[40],6000,self.price,conf)
            np.testing.assert_array_equal(a['G'],b['G'])
            x=execute_day(a['G'][:144],self.actual[40],6000)
            y=execute_day(b['G'][:144],changed[40],6000)
            self.assertGreater(abs(self.price@(x['H']-y['H'])),1)

    def test_future_month_does_not_change_earlier_features(self):
        changed=self.actual[:65].copy();changed[59:]*=20
        bank=build_features(changed)
        self.assertEqual(bank[31].history_sha256,self.features[31].history_sha256)
        for m in ('last','mean7','blend'):

np.testing.assert_array_equal(bank[31].prediction[m],self.features[31].prediction[m])

    def test_residual_observation_date_and_horizon(self):
        for day,f in enumerate(self.features):
            self.assertLess(f.training_max,day)
            for method in ('last','mean7','blend'):
                for j in range(3):
```

```
                    ds=f.residual_dates[method][j]
                    self.assertTrue(all(k<day for k in ds))
                    for i,k in enumerate(ds):
                        origin=k-j
                        np.testing.assert_allclose(f.residual[method][j]
[i],self.actual[k]-self.features[origin].prediction[method][j])

    def test_month_selection_causality_and_whole_simulation(self):
        # 包含2月1日参数选择与后续执行，篡改当天和未来只能改变执行。

candidates=(Config('last',0,1),Config('mean7',.8,2),Config('blend',.7,1,'value'))

original,_=simulate(self.dates,self.actual,self.price,self.features,stop=33,candidates=c
andidates)
        changed=self.actual[:65].copy();changed[31:,:,0]*=1.5

mutated,_=simulate(self.dates,changed,self.price,build_features(changed),stop=33,candida
tes=candidates)
        np.testing.assert_array_equal(original['G'][:32],mutated['G'][:32])
        self.assertEqual(original['config'][31],mutated['config'][31])
        self.assertGreater(np.abs(original['H'][31]-mutated['H'][31]).sum(),1)
        self.assertTrue(np.array_equal(original['E'][1:,0],original['E'][:-1,-1]))

    def test_execution_prefix_nonanticipativity(self):
        g=np.full(144,400.);a=self.actual[40].copy();b=a.copy();b[70:,0]*=3
        x=execute_day(g,a,6000);y=execute_day(g,b,6000)
        for k in ('C','D','H','W'):np.testing.assert_array_equal(x[k][:70],y[k][:70])
        np.testing.assert_array_equal(x['E'][:71],y['E'][:71])

    def test_feedback_bounds_and_no_mutation(self):
        g=np.array([0.,20000.,0.]);a=np.array([[10000.,0],[0,2000.],[10000.,0]])
        before=g.copy();s=execute_day(g,a,6000)
        np.testing.assert_array_equal(g,before)
        self.assertAlmostEqual(s['D'][0],5000/6)
        self.assertAlmostEqual(s['C'][1],5000/6)
        self.assertAlmostEqual(s['H'][0],10000-5000/6)
        validate_execution(a,s)

    def test_cost_five_times_and_no_refund(self):
        b=Battery();a=np.array([[100.,0.],[0.,0.]])
        s=execute_day(np.array([50.,100.]),a,1200,idle=True)
        p=np.array([2.,3.])
        self.assertEqual(p@s['G'],400.)
        self.assertEqual(5*p@s['H'],500.)
        self.assertEqual(s['unused_grid'][1],100.)

    def test_lp_milp_agree_and_mutex(self):
        records=[]
        for day,conf in [(2,Config()),(20,Config('blend',.75,1,'value')),
(40,Config('mean7',.8,3))]:
            p=make_plan(self.features[day],6000,self.price,conf)

m=plan_net(p['net'],np.tile(self.price,conf.horizon),6000,conf.terminal,integer=True)
            self.assertAlmostEqual(p['objective'],m['objective'],places=5)
            self.assertFalse(np.any((p['C']>1e-6)&(p['D']>1e-6)))
            records.append(p['objective']-m['objective'])

    def test_state_efficiency_hand_case(self):
        s=execute_day(np.array([100.,0.]),np.array([[0.,0.],[81.,0.]]),6000)
        np.testing.assert_allclose(s['C'],[100,0]);np.testing.assert_allclose(s['D'],
[0,81])
        self.assertAlmostEqual(s['E'][-1],6000)

    def test_emergency_merge_no_midnight_join(self):
        h=np.zeros((2,144));h[0,60:63]=[1,2,3];h[0,-1]=4;h[1,0]=5
        r=merge_emergency(self.dates[:2],h)
```

```
        self.assertEqual([x['period'] for x in r],['10:00-10:30','23:50-24:00','0:00-
0:10'])
        self.assertEqual([x['kwh'] for x in r],[6,4,5])

    def test_pv_attribution_bounds(self):
        s=execute_day(np.array([1000.]),np.array([[100.,50.]]),10800)
        self.assertEqual(s['W'][0],950);self.assertEqual(s['pv_curtail'][0],0)
        self.assertEqual(s['pv_curtail_upper'][0],50)

    def test_q1_protected(self):
        self.assertEqual(check_q1(),0)

    @unittest.skipUnless(HAS_Q2_OUTPUTS,'Run Q2 solve and report commands before output
checks')
    def test_complete_saved_physics_and_cost(self):
        s=dict(np.load(Q2/'experiments/formal_arrays.npz'))
        for day in range(365):
            validate_execution(self.actual[day],{k:s[k][day] for k in
('G','C','D','H','W','E')})
        np.testing.assert_array_equal(s['E'][1:,0],s['E'][:-1,-1])
        self.assertGreater(np.unique(np.round(s['E'][31:,0],4)).size,1)
        summary=json.loads((Q2/'summary.json').read_text())
        self.assertAlmostEqual(float(((s['G'][31:]+5*s['H']
[31:])@self.price).sum()),summary['total_cost_yuan'],places=5)
        self.assertFalse(any('oracle' in v for v in s['config'][31:]))

    @unittest.skipUnless(HAS_Q2_OUTPUTS,'Run Q2 solve and report commands before output
checks')
    def test_excel_csv_full_readback(self):
        s=dict(np.load(Q2/'experiments/formal_arrays.npz'))
        wb=load_workbook(Q2/'result2.xlsx',read_only=True,data_only=True)
        self.assertEqual(wb.sheetnames,['计划购电量','充放电量','紧急购电量'])
        rows=list(wb.worksheets[0].values)
        self.assertEqual(len(rows),335);self.assertEqual(list(rows[0][1:145]),PERIODS)
        for i,row in enumerate(rows[1:],31):
            self.assertEqual(row[0].date(),self.dates[i])
            np.testing.assert_allclose(row[1:145],s['G'][i],atol=1e-8,rtol=0)
            self.assertAlmostEqual(row[145],s['G'][i].sum(),places=6)
            self.assertAlmostEqual(row[146],self.price@s['G'][i]+5*self.price@s['H']
[i],places=6)
        battery=list(wb.worksheets[1].values)[1:]
        self.assertEqual(len(battery),334*6)
        for i in range(334):
            block=battery[i*6:(i+1)*6];d=i+31
            self.assertEqual(block[0][0].date(),self.dates[d])
            for j,row in enumerate(block):
                self.assertEqual(row[1],f'{j*4}:00-{(j+1)*4}:00')
                self.assertAlmostEqual(row[2],s['C'][d,j*24:(j+1)*24].sum(),places=6)
                self.assertAlmostEqual(row[3],s['D'][d,j*24:(j+1)*24].sum(),places=6)
            self.assertAlmostEqual(block[0][5],s['E'][d,0],places=6)
            self.assertAlmostEqual(block[1][5],s['E'][d,-1],places=6)
        erows=list(wb.worksheets[2].values)
[1:];expected=merge_emergency(self.dates[31:],s['H'][31:])
        # 导出仅非零区间；无紧急日也有“无”占位。
        out=[];current=None
        for row in erows:
            if row[0] is not None:current=row[0].date().isoformat()
            if row[1]!='无':out.append((current,row[1],row[2]))
        self.assertEqual(len(out),len(expected))
        for x,y in zip(out,expected):
            self.assertEqual(x[:2],
(y['date'],y['period']));self.assertAlmostEqual(x[2],y['kwh'],places=6)
        wb.close()
        with (Q2/'dispatch_10min.csv').open(encoding='utf-8-sig') as f:
            dispatch=list(csv.DictReader(f))
        self.assertEqual(len(dispatch),334*144)
```

```
        g=np.array([float(r['G_plan_kwh']) for r in dispatch]).reshape(334,144)
        h=np.array([float(r['H_real_kwh']) for r in dispatch]).reshape(334,144)
        np.testing.assert_allclose(g,s['G'][31:],atol=1e-8,rtol=0)
        np.testing.assert_allclose(h,s['H'][31:],atol=1e-8,rtol=0)

    @unittest.skipUnless(HAS_Q2_OUTPUTS,'Run Q2 solve and report commands before output
checks')
    def test_selection_records_only_history(self):
        with (Q2/'experiments/monthly_selection.csv').open(encoding='utf-8-sig') as
f:rows=list(csv.DictReader(f))
        dates=sorted(set(int(r['selection_day']) for r in rows))
        self.assertEqual(len(dates),11)
        for d in dates:
            group=[r for r in rows if int(r['selection_day'])==d]
            self.assertEqual(len(group),len(CANDIDATES))
            self.assertTrue(all(int(r['end'])==d-1 for r in group))
            chosen=[r for r in group if
r['selected']=='True'];self.assertEqual(len(chosen),1)
            self.assertAlmostEqual(float(chosen[0]['score']),min(float(r['score']) for r
in group),places=5)

    @unittest.skipUnless(HAS_Q2_OUTPUTS,'Run Q2 solve and report commands before output
checks')
    def test_oracle_same_boundaries_lower_cost(self):
        oracle=dict(np.load(Q2/'experiments/oracle_arrays.npz'))
        formal=dict(np.load(Q2/'experiments/formal_arrays.npz'))
        self.assertAlmostEqual(oracle['E'][0],formal['E'][31,0],places=6)
        self.assertAlmostEqual(oracle['E'][-1],formal['E'][-1,-1],places=6)
        net=(self.actual[31:,:,0]-self.actual[31:,:,1]).ravel()
        np.testing.assert_allclose(oracle['G']+oracle['D']-oracle['C']-
oracle['W'],net,atol=1e-5)
        self.assertLessEqual(np.tile(self.price,334)@oracle['G'],((formal['G']
[31:]+5*formal['H'][31:])@self.price).sum()+1e-5)


if __name__=='__main__':unittest.main(verbosity=2)
```

B10 code/Q3/audit_q3.py

```
"""独立验收：从 CSV 底层和原始输入重新聚合，不用 summary 作为计算源。"""
import csv,json,sys,subprocess,time,hashlib
from pathlib import Path
from decimal import Decimal
import numpy as np
ROOT=Path(__file__).resolve().parents[1];OUT=Path(__file__).resolve().parent;sys.path.in
sert(0,str(ROOT))
from microgrid_online import read_year,PERIODS,merge_emergency
from solve_q3 import write_json,check_protected

def rows(path):
    with path.open(encoding='utf-8-sig') as f:return list(csv.DictReader(f))

def audit(run_tests=True):

tic=time.perf_counter();dates,a,p=read_year();data=rows(OUT/'dispatch_10min.csv');ledger
=rows(OUT/'adjustment_ledger.csv')
    assert len(data)==334*144 and len(ledger)==334*216
    def arr(k):return np.array([float(r[k]) for r in data]).reshape(334,144)
    for d in range(334):
        part=data[d*144:(d+1)*144]
        assert [r['date'] for r in part]==[str(dates[d+31])]*144
        assert [r['period'] for r in part]==PERIODS and [int(r['index']) for r in
part]==list(range(144))
    np.testing.assert_allclose(arr('load_kw')/6,a[31:,:,0],rtol=0,atol=1e-9)
    np.testing.assert_allclose(arr('pv_kw')/6,a[31:,:,1],rtol=0,atol=1e-9)
    np.testing.assert_allclose(arr('price_yuan_kwh'),np.tile(p,
```

```
(334,1)),rtol=0,atol=1e-12)

g=arr('G_effective');c=arr('C_real');d=arr('D_real');h=arr('H_real');w=arr('W_real');e=a
rr('E_start');ee=arr('E_end')
    balance=g+a[31:,:,1]+d+h-a[31:,:,0]-c-w
    state=ee-e-.9*c+d/.9

np.testing.assert_allclose(balance,0,atol=1e-7);np.testing.assert_allclose(state,0,atol=
1e-7)
    np.testing.assert_array_equal(ee.ravel()[:-1],e.ravel()[1:])
    assert min(e.min(),ee.min())>=1200-1e-7 and max(e.max(),ee.max())<=10800+1e-7
    assert max(c.max(),d.max())<=5000/6+1e-7 and not np.any((c>1e-7)&(d>1e-7))
    short=a[31:,:,0]-a[31:,:,1]-g
    dhat=np.minimum(np.minimum(np.maximum(short,0),5000/6),np.maximum(0,(e-1200)*.9))
    np.testing.assert_allclose(h,np.maximum(0,short-dhat),atol=1e-7)
    # 财务从最底层 p、G0、每笔 U/R、H 重算，Decimal 避免累积浮点解释混淆。
    D=Decimal;zero=D(0)
    plan=sum((D(r['price_yuan_kwh'])*D(r['G0']) for r in data),zero)
    emergency=sum((D(5)*D(r['price_yuan_kwh'])*D(r['H_real']) for r in data),zero)
    up=sum((D('1.5')*D(r['normal_price'])*D(r['increase_U']) for r in ledger),zero)
    down=sum((D('.5')*D(r['normal_price'])*D(r['decrease_R']) for r in ledger),zero)
    increase=sum((D(r['increase_U']) for r in
ledger),zero);decrease=sum((D(r['decrease_R']) for r in ledger),zero)
    hp=sum((D(r['H_real']) for r in data),zero);total=plan+up+down+emergency

values=dict(plan_cost_yuan=plan,up_cost_yuan=up,penalty_yuan=down,refund_yuan=zero,emerg
ency_cost_yuan=emergency,total_cost_yuan=total,

increase_kwh=increase,decrease_kwh=decrease,emergency_kwh=hp,emergency_periods=int(np.su
m(h>1e-7)))
    summary=json.loads((OUT/'summary.json').read_text())
    for k,v in values.items():assert abs(float(v)-summary[k])<1e-6,(k,v,summary[k])
    # 不使用 NPZ，独立按账本逐笔恢复承诺，最终与 CSV 有效 G 逐点核对。
    commitment={(r['date'],int(r['index'])):D(r['G0']) for r in data}
    seen=set();max_ledger_error=0
    for row in ledger:
        key=(row['delivery_date'],int(row['index']));k=int(row['stage']);stamp=(key,k)
        assert stamp not in seen;seen.add(stamp)
        assert int(row['index'])>=k*36 and row['decision_date']==row['delivery_date']

old=D(row['previous_commitment']);new=D(row['new_commitment']);u=D(row['increase_U']);r=
D(row['decrease_R'])
        assert u>=0 and r>=0 and not(u>D('1e-7') and r>D('1e-7'))
        err=max(abs(commitment[key]-old),abs(new-old-
u+r));max_ledger_error=max(max_ledger_error,float(err));assert err<D('1e-6')
        commitment[key]=new
        assert abs(D(row['adjustment_charge'])-
D(row['normal_price'])*(D('1.5')*u+D('.5')*r))<D('1e-6')
    for row in data:assert abs(commitment[(row['date'],int(row['index']))]-
D(row['G_effective']))<D('1e-6')

merged=merge_emergency(dates[31:],h);saved=rows(OUT/'emergency_purchase_merged.csv');ass
ert len(merged)==len(saved)
    for x,y in zip(merged,saved):
        assert x['date']==y['date'] and x['period']==y['period'];assert abs(x['kwh']-
float(y['kwh']))<1e-6
    # 正式配置日历内，每个选参训练区间必须严格在月首前；核验不据摘要声称通过。
    chosen=rows(OUT/'experiments/monthly_selection.csv')
    for row in chosen:assert int(row['end'])<int(row['selection_day'])
    check_protected();tests={}
    if run_tests:
        for name,cmd in [('q1_tests',[sys.executable,str(ROOT/'Q1/test_q1.py')]),
('q2_tests',[sys.executable,str(ROOT/'Q2/test_q2.py')]),('q3_tests',
[sys.executable,str(OUT/'test_q3.py')]),
                         ('q1_freeze',[sys.executable,str(ROOT/'Q1/freeze_q1.py')]),
('q2_freeze',[sys.executable,str(ROOT/'Q2/freeze_q2.py')])]:
```

```
            if name.endswith('_freeze') and not Path(cmd[1]).exists():
                tests[name]='not_packaged';continue
            result=subprocess.run(cmd,capture_output=True,text=True)

(OUT/(name+'.log')).write_text(result.stdout+result.stderr);tests[name]=result.returncod
e
            if result.returncode:raise RuntimeError(name+' failed; see log')
    protected=check_protected()
    result=dict(status='PASS',independent_source='original inputs + dispatch_10min.csv +
adjustment_ledger.csv; summary used only after recomputation',
      decimal_aggregates={k:str(v) for k,v in
values.items()},formal_days=334,intervals=len(data),ledger_rows=len(ledger),

balance_max_kwh=float(np.abs(balance).max()),state_max_kwh=float(np.abs(state).max()),le
dger_max_kwh=max_ledger_error,
      excel_full_readback='q3_tests.log
OutputTests.test_excel_all_values',regression_exit_codes=tests,protected_files=protected
,seconds=time.perf_counter()-tic,
      warnings=['A/B/B_net settlement ambiguity','10min endpoint and efficiency each90%
assumptions','instantaneous feedback aggregate','different initial/end SOC compared with
Q2','shared-calendar vs independently selected ablations','local realized VOI is not
additive annual contribution'])
    write_json(OUT/'validation.json',result)
    lines=['# Q3 最终验收审计','','结论：PASS。独立读取原始输入、48,096 条交付时段和 72,144 条调整账
本，用十进制数重算下表；summary 仅在聚合后用于交叉对照，不作为计算源。',
      '','|项目|底层重新计算|','|---|---|']
    lines.extend(f'|{k}|{v}|' for k,v in values.items())
    lines+=['',f'逐时平衡最大残差{result["balance_max_kwh"]:.3e} kWh，状态递推最大残差
{result["state_max_kwh"]:.3e} kWh；逐笔承诺恢复最大误差{max_ledger_error:.3e} kWh。日期、144
时段、阶段时点、跨日 SOC、U/R 非负互斥、累计承诺及紧急区间全部核对。',
      '',f'Q1、Q2、Q3 全部测试和 Q1/Q2 冻结核验实际运行返回码均 0；接手快照{protected}个文件保持字节一
致。日志保存在 Q3，不写入 Q1/Q2。Excel 逐值回读覆盖原计划 G0、有效 G、费用、4h 充放电、首末 SOC 和紧急事
件。',
      '','## WARNING 与结论边界','',
      '1. 主 A 不退费并另收调减违约费不是唯一官方解释；B 与 B_net 是分开的替代实验。A 无调减符合支配逻
辑。',
      '2. 时间结束点、效率各 90%、十分钟实时反馈均为继承假设；不能写成题目逐字规定。',
      '3. Q2 和 Q3 正式期初 SOC 不同；现金和末态调整费用同时披露，后者不是账单收入。',
      '4. 8 子集共享正式配置日历；独立仅 0 时历史选择作为另一个基准。逐阶段新预报屏蔽保持调整权限，用于区
分新 PV 信息和 SOC 反馈。',
      '5. estimated VOI 是事前模型估计，realized_value 是余日冻结对照，多个余日重叠不可直接加总年度
收益。',
      '6. 更少紧急电量不代表更少紧急时段；正式 Q3 存在更多小缺口时段。',
      '7. 审计代码与求解器在本次同一任务中编写，但验算路径独立，不是外部第三方认证；数学模型仍仅为有限策
略类中的历史选择。',
      '','数值验收通过后可创建 Q3 冻结；视觉检查另记录 visual_review.json。未改动任何 Q1/Q2 既有日志或
结果。']
    (OUT/'Q3 最终验收审计.md').write_text('\n'.join(lines)+'\n');print('Independent audit
PASS',float(total),flush=True)
    return result

if __name__=='__main__':audit()
```

B11 code/Q3/diagnostics_q3.py

```
"""补充公平性诊断：仅 0 时独立历史选参；同权限逐阶段屏蔽新 PV；18h 次日信息。

在基础消融结果后添加的诊断，不改变正式选择和原预注册候选。
"""
from dataclasses import replace
from datetime import datetime,timedelta
import numpy as np
from solve_q3 import OUT,inputs,write_csv,write_json,replay,select_config
from q3_forecast import FeatureFactory
from q3_model import Config,CANDIDATES,run_day,stack_days,summarize
```

```
def supplemental():
    dates,a,p,records=inputs();f=FeatureFactory(a,records)
    formal=dict(np.load(OUT/'experiments/formal_arrays.npz'))
    warmup=[{k:v[d] for k,v in formal.items()} for d in range(31)]
    calendar=[Config(**v) for v in
__import__('json').loads((OUT/'experiments/calendar.json').read_text())]

write_json(OUT/'experiments/supplement_protocol.json',dict(added_after_initial_ablations
=True,reason='separate adjustment access, new PV information, and retuning of
nonadjustable baseline',changes_to_formal_policy=False,zero_only_selection='same 16
candidates, past 21 days monthly',freshness='same calendar and all adjustment rights;
withhold one new release',next_day18='preserve today forecast; replace next-day18
release targets with prior available prediction'))
    rows=[]
    for k in (1,2,3):
        ss,_=replay(f,p,warmup,calendar,stale=(k,))
        np.savez_compressed(OUT/f'experiments/withhold_new_{6*k}.npz',**ss)
        rows.append(dict(policy=f'withhold_new_{6*k}_same_rights',**summarize(ss,p)))
    # 独立仅0时基准：不是将可调整低q日历机械用于不可调整方案。
    known=[float(formal['E'][0,0])]+[float(formal['E'][d,-1]) for d in
range(31)];e=known[-1]
    days=list(warmup);selection=[];conf=Config()
    for d in range(31,365):
        if dates[d].day==1:
            conf,rr=select_config(f,d,known,p,updates=());selection.extend(rr)
            print('0h-only monthly selection',dates[d],conf.name,flush=True)
        ss,_=run_day(f,d,e,p,conf,updates=());days.append(ss);e=float(ss['E']
[-1]);known.append(e)

ss=stack_days(days);np.savez_compressed(OUT/'experiments/zero_only_selected.npz',**ss)
    write_csv(OUT/'experiments/zero_only_monthly_selection.csv',selection)
    rows.append(dict(policy='zero_only_independent_past_selection',**summarize(ss,p)))
    # 18时只保留当日目标，次日目标用12时之前已知曲线补齐，不能使用次日实际。
    changed=dict(records)
    for d in range(365):
        issue=datetime(2025,1,1)+timedelta(days=d,hours=18);rec=records[issue]
        old=f.raw(d,3,'linear',0.,2);powers=list(rec.power)
        for h in range(7,25):powers[h-1]=float(old[h*6-1,1]*6)
        changed[issue]=replace(rec,power=tuple(powers))
    ff=FeatureFactory(a,changed)
    ss,_=replay(ff,p,warmup,Config(horizon='R48'))
    rows.append(dict(policy='R48_18_nextday_old_information',**summarize(ss,p)))
    np.savez_compressed(OUT/'experiments/R48_18_nextday_old.npz',**ss)
    write_csv(OUT/'experiments/supplemental.csv',rows)
    print('supplemental causal diagnostics complete',flush=True)

if __name__=='__main__':supplemental()
```

B12 code/Q3/prepare_q3.py

```
"""重建Gate A/B文档、附件结构审计、预注册协议；在正式计算前运行。"""
from pathlib import Path
from dataclasses import asdict
import json,csv,hashlib,sys
ROOT=Path(__file__).resolve().parents[1];OUT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT))
from microgrid_forecasts import read_pv_forecasts
from q3_model import CANDIDATES,WARMUP

TEXTS={
'题意与信息集审计.md':'''# 问题3题意与信息集审计

## 原题明文规定

Q3新增两项：附件3每日0/6/12/18时的未来24小时整点PV预报，以及按规定价格调整普通购电的权限。0时制定当天计划，其他发布时间可调整；增加部分1.5倍价，减少部分50%违约价，紧急购电继承Q2的5倍价。Q3没有要求日首
```

末 SOC 相等。结果保存 2 月 1 日至 12 月 31 日，四指定日为 3/20、6/21、9/23、12/21。

附件能够确认

0 时预报目标为当日 1 时至次日 0 时；6 时为 7 时至次日 6 时；12 时为 13 时至次日 12 时；18 时为 19 时至次日 18 时。日期空白需向下继承，预报列名给出提前时长，不是当日固定小时。result3 的调整表只有每日 144 点，没有为三次发行分别留行；内部必须额外保存全版本与账本。

继承 Q2 冻结口径

每 10 分钟结束点功率/6 近似区间电量；母线侧充放电、效率各 0.9、SOC1200–10800、功率 5000 kW。不售电、不削负荷、不设题外并网容量。实时储能反馈使用当前观测及 E，属于允许反馈的建模解释，十分钟均值近似时段内反馈而非开始前已知均值。SOC 跨日传递，不能每天重置。Q2 正式方案是月度历史选择和 24 小时价值，不是综合报告的 48 小时执行 MPC。

本文新增建模解释

令 F(d,k)为在阶段开始前的已完成实际记录、当前 SOC、固定电价、日历、已发布预报与已完成历史误差。0 时包含当日 0 时预报，不含 6/12/18 时预报或当天实际未来；6 时新增 0–6 已发生记录及 6 时预报，12/18 同理。G0 必须利用 0 时官方 PV；负载继承 Q2 组合预测。调增/调减是预先改变未来正常承诺；H 是在实际供给仍不足时产生的补购，二者价格和信息时点不同。

各阶段只能改变尚未执行的承诺，过去动作不可追溯。已承诺/支付费用进入独立账本，不能因重优化消失。储能继续实时反馈，不把预测参考轨迹当实际充放电。18 时信息对当天日落后 PV 可能作用小，但若前瞻跨日，次日 PV 可改变当晚能量准备；须实测，不能预先宣布有用或无用。

是否需要后续预报由相同初始边界、相同执行器、相同费用规则的 8 个预报子集及旧预报重优化对照回答。MAE 改进不等于现金节省。计划表存 G0，调整表存逐时最终生效 G，充放电表存实际 C/D 和首末 E，紧急表存实际 H 合并区间。

1 月 1 日无历史负载，沿用 Q2 的零计划、静置电池启动；仍登记官方预报。1 月 2 日起所有候选共享 Q3 固定 linear/q=.8/T24 热启动；这与 Q2 的热启动策略不同，比较会披露 2 月初及年底 SOC 并作终端价值修正。

存在歧义与替代验证

调减退费、多次调整基准和模板“全天购电费”未被原文唯一解释，单独审计。时间终点与效率各 90%保留既有假设属性。主结果采用统一解释，不能按日期挑口径。
'''，
'调整结算口径审计.md':'''# 调整结算口径审计

主解释 A

原题在 Q2 规定除紧急费外按计划购电计费，Q3 又说总费包括计划费与调整相关费用，但没有明确退款。因此保守继承原计划费不退，每次针对上一有效承诺增量结算：C=ΣpG0+Σk,t(1.5pU+0.5pR)+Σ5pH。p 取对应交付十分钟的附件 1 价格，不取 6 时一个价格统一覆盖所有未来时段。每笔 U=max(Gnew−Gold,0)、R=max(Gold−Gnew,0)。

此解释不是唯一官方规则。模板没有退款字段，也没有逐阶段字段，不能据此证明原文排除了退款。

若允许不花费额外成本地弃用剩余电量，A 下付费减少承诺没有天然收益：可保留更多正常供电并弃用，至少存在不劣的物理策略。本题执行器为单向贪心反馈，多供电不增加剩余缺口；而调减额外付费。因此 A 下 R=0 应是合理结果而非算法缺陷。合成反例与全年账本另检验。

替代 B 与 B_net

B 按上一承诺结算：先冲销 pR，再收 0.5pR，净调减费用−0.5pR；调增仍 1.5pU。总费=ΣpG0+Σ(1.5pU+0.5pR−pR)+Σ5pH。对之前以 1.5p 增购的电量也按基准 p 退款，是明确采用的统一基价简化；不是实际市场规则。

B_net 把未交付时段对 0 时原计划的净偏差当作可修订结算账面：F(g)=1.5p(g−G0)+−0.5p(G0−g)+，每次只记 F(new)−F(old)。旧财务分项允许负冲销；物理 U/R 仍按上一有效计划记录。此口径允许取消尚未交付的早先追加订单而冲销费用，不允许消除已交付账单。与逐事件 B 不同，属于需要披露的替代假设，不能把每次 F(new)全部再收费。

手算例（p=1）

原计划 100，依次 80、110、90：物理变化为 R20、U30、R20。A 总非紧急费 100+10+45+10=165；B 为 100−10+45−10=125；B_net 为 100−5=95，三个净变更收费−10、25、−20；每笔冲销明确保留。若 100→120→100，A 为 140、B 为 120、B_net 为 100。不能用终态相同声称 A/B 没有交易费。

主方案、B、B_net 分别完整重放实际执行，绝不把同一轨迹仅换费率冒充另一口径下最优策略。敏感性沿用主方案已按历史形成的配置日历，重新求解计划/调整与 SOC；不声称替代口径已经重新全局调参。
''',
'方案审计.md':'''# 两份综合报告的方案审计

A=继承；B=实验决定；C=替代解释；D=不作为正式方案。

事项	判定	处理
0/6/12/18 时间线	A	预报发行前不可见，过去承诺冻结
插值	B/C	常值、线性、PCHIP 进入预注册候选，保持原始整点锚点
官方 PV 偏差校正	B	原始与过去完整残差均值校正比较；不得全年拟合
负载预测	A/B	继承组合预测，日内过去 6 小时误差指数衰减修正为候选
PV 预测	A	优先官方，24h 外用同一起点历史预测补齐
24h/48h	B	T24 自然日末价值、R24/R48 滚动终端价值比较，不预设 48h
18h 跨日价值	B	T24 与 R48 分别消融，次日购电仅内部前瞻
多阶段 MPC	A/B	按阶段更新信息，有限策略类，不宣称全局随机最优
场景生成	B	过去完整 48h 负载/PV 联合残差，最多 8 条等时间距经验代表轨迹
完整残差相关性	A	不逐点独立噪声；只收齐全部目标后入库
K-medoids	B	当前只 8 条无需聚类，不凭复杂度增加依赖
非预见性	A	当前候选 G 跨路径共享，逐点反馈不接收路径编号
自由场景储能	D	拒绝把提前知道整条路径的优化轨迹当执行动作
CVaR	B	本轮风险中性均值，风险以分位和经验路径反映，不声称已证明 CVaR 无效
VOI	A/C	同一当前路径与费用基准；不调为可行选项；实际收益另记
调整费	C	主 A，另重放 B 及 B_net，禁止混用
终端 SOC	A/C	v 取完整固定电价最低 24 点均值/0.9，仅策略比较代理，非账单收入
实时执行	A	完整沿用 Q2 反馈，不加入执行 MPC 掩盖预报增益
紧急购电	A	仅剩余缺口 5 倍，不用于主动充电
result3	A/C	官方四表模板，G0 与有效 G 分开，费用语义明示

最新报告仍称 Q1 v1 且 Q2 为 48h 执行 MPC，与冻结实现不同。冻结 Q1 v2/Q2 v1 优先。报告关于“18 时必然有价值”“新预报更准所以省钱”的推断只能成为假说；没有实验不得写成结论。报告式 VOI 若已含调整费，不应再重复减一次调整费。
''',
'文献方法映射.md':'''# 文献方法映射

依据本地全文提取及相关正文核对；页面为 PDF 文件页序。未新增在线文献，不把检索目录的摘要核验冒充全文证据。

Polimeni 等 2021 Processes 9 323

本地 20 页期刊稿已阅读全文（含附录），研究孤岛多能源微网，含 PV、电池、燃气机组与制水。第 3–4 页：日前预报 11AM/PM 每日两次 24h，日内每小时修正未来 3h，另 30min nowcasting。第 5–6 页：上层 MILP 定机组承诺与储能计划，下层 PI 追踪 SOC 并分配实时功率。第 8 页：24h 时域、15min 离散与滚动；工作站 20 秒通信、PLC100ms。第 10 页：末态能量按等效燃料修正，不强制所有实验结束 SOC 相同。第 10–15 页：三天实物运行，每天其中一种 EMS 实测，其余模拟；第 2 页所称四日比较不能误写成四天均实测。第 15–16 页：备用约束可限制更准预报的收益，不是越短期越经济。第 17 页附录同时约束备用功率和持续能量。

借鉴固定 EMS 比较预报、计划/执行分层、末态能量调整；不能搬入制水、燃气、允许未满足负荷、外部天气或 nowcasting。该研究不是本题全年严格逐日起点验证，不能把实验燃料降幅当成本题节费。

Wen 等 2021 Mathematics and Computers in Simulation 183 116–128

本地 13 页未校正作者稿已阅读全文，非正式排版版；文件页序与印刷页码差 1。第 3–6 页：城市微网三台微燃气轮机、半小时 48 阶段、日前机组组合，DP 处理非凸费用；按负荷/PV 净偏差概率确定备用，备用即使未使用也有成本。第 9–11 页：PV 预测区间上下界下重新调度并分析费用范围，95%区间和 5%失负荷风险是论文设定。第 10 页误差样本来自三年数据中参考日及前后 7 日（共 45 样本）的季节邻域；本题单年严格因果实现不能直接读取当年未来 7 天。第 13 页把储能处理不确定性列为后续研究，因此它没有直接提供本题电池策略或终端 SOC 公式。

借鉴误差分布、风险水平和经济后果关联；不搬入机组备用市场、失负荷容忍、正态分布或固定 95%保证。本文使用历史经验残差，无统计覆盖率理论保证，实际缺口全部补电。

Panda 等 2024 SEGAN 38 101334

重新核对本地 12 页稿中第 3–8 页及结论的承诺、实时控制和时域部分。日前共享外网承诺/备用与实时控制分层；比较规则、MPC、DRL 和完美预报基准。第 5 页终端 E 约束含预设值，不能据此要求本题每日回归。第 5–6 页场景中的自由未来机组/储能是规划近似，不能在本题真实执行中提前揭示未来路径。借鉴当前一步反馈和对照；不采用售电、柴油机、

调频收入或其效率写法替换 Q2 物理。

Herding 等 2023 Applied Energy 336 120847

重新核对第 4–7 页及附录 A：小时级日前市场投标曲线，共享第一阶段；电池末能量不小于初始，日初固定 SOC。第 7 页先锁定投标，再在不放回抽取的另一组模拟场景评价；原文明说样本外是模拟实现而非真实观察。其评估在场景揭示后重解整天确定性模型，不能直接称本题十分钟因果执行。借鉴锁定决策再评价，拒绝把模拟样本外等同真实滚动年度检验；不采用售电、机组、每日重置或未来场景全知储能。

本题方法定位

场景候选仅在有限 G 集合中以同一逐点反馈器计算经验均值，各场景当前 G 相同，不建立隐含全知第二阶段。最多 8 条完整历史轨迹，无 K-medoids 必要；β=0，不机械引入 CVaR。所有 50%/1.5/5 系数只来自题面，退费和多次交易基准由本文公开假设，文献无法唯一确定。
'''
'候选数学模型.md':r'''# 候选数学模型与预注册协议

原问题及策略类

随机控制目标为最小化策略的期望实际现金费。缺少可靠全年联合分布，本文只在预注册有限策略类中做历史选择，不声称全年随机控制全局最优。d 为日期，k=0,1,2,3，τk=0,36,72,108，t 为 10 分钟交付区间，所有流量 kWh。

普通承诺更新：$G^k=G^{k-1}+U^k-R^k$，U/R 非负、同轮互斥；$t<\tau k$ 不可改变。G0 可测于 F(d,0)，G^k 可测于 F(d,k)。实际平衡 G+PV+D+H=L+C+W；E'=E+0.9C−D/0.9；E 在 1200–10800、C/D≤5000/6 且互斥，G/H/W≥0。H 仅补执行层剩余缺口。E[d+1,0]=E[d,144]。

预测：Q2 的最近 7 日与最近最多 4 个同星期各半组合给出负载；PV 使用最新可用官方预报，发行时最后一个已知 PV 作为首端锚点。constant 是向后保持整点值、linear 为线性、pchip 形状保持；取区间结束点功率/6，与既有结束点离散化一致。更精细区间积分不是本轮主模型。校正候选用最近 30 个完整 48h 误差，分发行阶段与相对时距；不按月份再分稀疏样本。不足 30 用已有，零样本为 0。为避免尾部未实现，原始残差必须全部 288 点已实现后才可用，即历史发行日至少早 2 天；索引精确检查 end≤now。

负载修正候选：已发生最近最多 36 点误差均值乘 gain∈{0,.5,1}，未来按 exp(−j/36)衰减。PV bias 为过去对应点误差均值并截断非负；风险裕度 m=max(0,Qq(eL−ePV))。q=0 特指点预测无裕度，不是 0 分位数。

T24 优化自然日剩余部分；R24 优化发行后 24h；R48 优化发行后 48h，超过官方覆盖以同一历史起点的 Q2 预测补齐。所有窗口用固定全日电价最低 24 点均值/0.9 的 v，目标减 vE 末；不规定日末回到初值。v 是补能代理，不是收入。

0 时提案 LP：min ΣpG−vE 末。调整提案 LP：min Σ 当日(1.5pU+0.5pR)+Σ 次日及以后 pG−vE 末，B 把 R 系数改为 −0.5p，B_net 相对原始 G0 线性化。均满足 G+D−C−W=预测净负荷+风险裕度与储能约束。W 不错误限制为 PV，因此可消除同充同放循环而保持 G 和 E，代表性算例用 MILP 交叉检验。

VOI 及场景

每次更新把 KEEP 与提案放到相同当前评价路径，用实际同型 feedback_step 逐点仿真，不采用 LP 参考储能轨迹。单路径候选用风险修正预测，场景候选用最多 8 条等时间距历史完整联合误差加当前预测形成非负路径，并从 q∈{0,.65,.8,.9}的共享 G 提案选择最小平均成本。路径只揭示当前点给反馈器，场景编号不能进入动作。无场景树，不宣称严格多阶段随机最优。

J=本次增量交易费+预测 5pH+前瞻普通购电费−v(E 末−E 当前)。KEEP 本次交易费为 0，其他既有现金支出对二者相同且略去。对于跨日候选，先在当前平均路径推演至午夜，再从该预测 E 以同一当前信息生成前瞻 G；该 G 在所有评价场景共享，并不提前签订次日合同。VOI=Jkeep−Jupdate，门限 1e−6 元仅数值容差；选择 KEEP 优先处理平分。无门控候选总采用其提案。VOI 已含全部调整费，不二次扣除。事后价值为实际剩余日冻结 KEEP/采用方案的费用差加末态价值差，不含后续再调整，是诊断而非在线决策或可加总全年因果贡献。

正式选择与实验顺序

预注册 16 个候选见 experiments/protocol.json：linear T24 的 q=0/.65/.7/.8/.85/.9，constant/pchip，PV bias，load gain .5/1，R24/R48，T24 场景，取消门控，R48 场景。月首仅用过去 21 个完成日，所有候选从该验证段正式策略已知 SOC 开始并连续运行。score=现金费−v(E 末−E 初)，最小者当月固定，稳定平分按注册顺序；插值/风险/校正/时域一起按过去经济性选择。

先检查连续季节原型及合成反例，再正式月度选择全年顺序执行。之后固定候选全年排名仅为诊断，不能回填历史选择。8 个信息子集及旧预报对照沿用同一正式因果配置日历，单独推进各自 SOC，隔离预报权限；它们不各自再调参，须作为结论范围公开。替代 B/B_net 同样按已选日历重放。另 T24/R48 分别比较移除 18 时信息，区分当天影响和跨日准备。Q2 仅作为冻结基准，比较现金和首末 SOC 调整值，不宣称所有差异都来自预报。

```
''',
'口径核验.md':'''# 问题3输出与物理口径

主结算A按交付时段p、上一承诺增量收费；模板计划表存0时G0，其全天量sum G0、费sum pG0（原计划账单）；调整表存最终逐时有效普通购电G，其全天量sum G、费为最终实际总费（含原计划、所有调整、紧急）。两表费含义不同，不能相加。为完整披露，与Q2计划表费用包含紧急费的既有写法不同，本问主结果明确区分原计划账单和最终结算；日明细另保留全部费用。四指定日表1最终购电量为G，不含另列H，总费用含H。

模板四表名、表头与布局保留，只修正十分钟偏移标签并展开示意行。电量为kWh，价格元/kWh，费用元，显示四位小数、内部双精度，SOC内部能量而非比例。实际无紧急日写“无、0”，有紧急时按日内连续H>1e−7合并。总未利用W按光伏优先分摊：未用普通电min(W,G)，PV弃用下界max(W−G,0)、上界min(W,PV)，不能全把W叫弃光。

原题没有并网容量、寿命费或负荷削减，不引入。实时储能反馈、时间终点和效率各90%是继承假设。1月初6000，2月初来自共同预运行，年末不强制回归，经济比较附末态修正，现金账单不掺入末态价值。
'''}

def prepare():
    OUT.mkdir(exist_ok=True);(OUT/'experiments').mkdir(exist_ok=True)
    for name,txt in TEXTS.items():(OUT/name).write_text(txt.strip()+'\n')
    records=read_pv_forecasts(ROOT/'附件/附件3.xlsx')
    with (OUT/'forecast_versions.csv').open('w',encoding='utf-8-sig',newline='') as
file:

w=csv.writer(file);w.writerow(['issue_date','issue_time','target_datetime','lead_time_ho
urs','forecast_power_kw'])
        for rec in records.values():
            for h,(dt,p) in
enumerate(zip(rec.targets,rec.power),1):w.writerow([rec.issue.date(),rec.issue.strftime(
'%H:%M'),dt.isoformat(),h,p])
    (OUT/'附件3结构审计.md').write_text('''# 附件3与result3结构审计

原始附件3为Sheet1，1461行26列。读取器以列名“预报h小时”解析h，要求1..24完整；日期向下继承后要求2025-01-01起1460个连续6小时发行，无缺失/重复/负数/非有限值。35040个目标按issue+lead构造显式时间，含跨日；最后一期18时预报覆盖2026-01-01 18时，其2026真实值不可用于验证。forecast_versions.csv逐值保存原始键与功率。

首目标分别1/7/13/19时，不是发行时刻。发行时锚点取刚结束的已知PV，1月1日0时缺少前点公开取0。每种插值在每个整点严格等于原预报，偏差校正后的曲线另外标记，不能冒充原始锚点。

result3模板四表：计划购电量335×147、调整购电量335×147、充放电量26×6、紧急购电量11×3。后两张含省略示例须展开。前两表时间从0:10-0:20起，与既有自然日口径差10分钟；修正标签而不平移数据。调整表只有每天一行，没有三个更新版本位置，最终有效量入表，全部阶段版本与交易账本另存。
''')
    protocol=dict(candidates=[asdict(c) for c in
CANDIDATES],warmup=asdict(WARMUP),selection='monthly_previous_21_complete_days',residual
_days=30,

scenarios=8,rule='A',value='mean(lowest24(full_day_price))/.9',formal_selection_before_p
osthoc_rankings=True,
      information_subsets='all eight subsets; same causal configuration
calendar',created_before_compute=True)
    p=OUT/'experiments/protocol.json'
    encoded=json.dumps(protocol,ensure_ascii=False,indent=2)+'\n'
    if p.exists():assert p.read_text()==encoded,'预注册协议已存在且不一致'
    else:p.write_text(encoded)
    print('Gate A/B materials ready; candidates',len(CANDIDATES),flush=True)

if __name__=='__main__':prepare()
```

B13 code/Q3/q3_forecast.py

```
"""因果特征工厂：控制器只能收到当前特征，不收到全年数据。"""
from dataclasses import dataclass
from datetime import datetime,timedelta
from functools import lru_cache
import sys
```

```
from pathlib import Path
import numpy as np
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'Q2'))
from forecast_q2 import forecast
from microgrid_forecasts import interpolate_pv

@dataclass(frozen=True)
class StageFeatures:
    now:int
    source_issue:int
    prediction:np.ndarray
    residual:np.ndarray
    residual_origins:tuple
    residual_latest_end:int
    correction:np.ndarray

def causal_prediction(history,record,source_anchor,day,k,method,gain):
    """history只包含截至当前阶段开始已结束的十分钟记录。"""
    t=k*36;now=day*144+t
    assert history.shape==(now,2)
    assert record.issue<=datetime(2025,1,1)+timedelta(minutes=10*now)
    complete=history[:day*144].reshape(day,144,2)
    pred=forecast(complete,day,'blend',3).reshape(-1,2) if day else np.zeros((432,2))
    # 日内只用最近最多6小时已实现负载误差，衰减时间固定6小时。
    if t and gain:
        delta=(history[day*144+max(0,t-36):now,0]-pred[max(0,t-36):t,0]).mean()
        pred[t:,0]=np.maximum(pred[t:,0]+gain*delta*np.exp(-np.arange(432-t)/36),0)
    curve=interpolate_pv(record,source_anchor,method)/6
    origin=int((record.issue-datetime(2025,1,1)).total_seconds()/600)-day*144
    lo=max(0,origin);hi=min(432,origin+144)
    pred[lo:hi,1]=curve[lo-origin:hi-origin]
    return pred[t:t+288].copy()

class FeatureFactory:
    """环境拥有数据；只有此工厂按时间切片后调用纯函数。缓存不用于模型选参。"""
    def __init__(self,actual,records):
        self.actual=np.asarray(actual);self.flat=self.actual.reshape(-1,2)
        self.records=records

    @lru_cache(maxsize=8192)
    def raw(self,day,k,method,gain,source):
        now=day*144+k*36;issue=day*144+source*36
        dt=datetime(2025,1,1)+timedelta(minutes=issue*10)
        rec=self.records[dt]
        anchor=self.flat[issue-1,1]*6 if issue else 0.
        return causal_prediction(self.flat[:now],rec,anchor,day,k,method,gain)

    @lru_cache(maxsize=768)
    def feature(self,day,k,method='linear',gain=0.,bias=False,source=None):
        source=k if source is None else source
        assert source<=k
        now=day*144+k*36
        pred=self.raw(day,k,method,gain,source).copy()
        # 完整48小时残差窗口必须全部实现；即使只用自然日剩余部分也不借用未完成尾部。
        origins=tuple(range(max(1,day-31),max(1,day-1)))
        residual=[]
        for d in origins:
            start=d*144+k*36;end=start+288
            assert end<=now
            residual.append(self.flat[start:end]-self.raw(d,k,method,gain,source))
        r=np.array(residual).reshape(-1,288,2)
        correction=np.zeros(288)
        if bias and len(r):
            correction=r[:,:,1].mean(axis=0)
            adjusted=np.maximum(0,pred[:,1]+correction)
```

```
            correction=adjusted-pred[:,1];pred[:,1]=adjusted;r[:,:,1]-=correction
        latest=(origins[-1]*144+k*36+288) if origins else -1
        return StageFeatures(now,day*144+source*36,pred,r,origins,latest,correction)

def risk_net(f,q,n):
    margin=np.zeros(n)
    if q and len(f.residual):margin=np.maximum(0,np.quantile(f.residual[:,:n,0]-
f.residual[:,:n,1],q,axis=0))
    return f.prediction[:n,0]-f.prediction[:n,1]+margin,margin

def scenarios(f,n,count=8):
    r=f.residual
    if not len(r):return f.prediction[None,:n].copy()
    indices=np.unique(np.linspace(0,len(r)-1,min(count,len(r))).astype(int))
    return np.maximum(0,f.prediction[None,:n]+r[indices,:n])
```

B14 code/Q3/q3_model.py

```
"""预注册策略类、当前信息 VOI 与严格逐时环境。"""
from dataclasses import dataclass,replace,asdict
from datetime import datetime,timedelta
import time
import numpy as np
from microgrid_common import Battery
from microgrid_online import plan_net,feedback_step,validate_execution,terminal_value
from microgrid_adjustment import settlement,adjustment_plan
from q3_forecast import risk_net,scenarios

@dataclass(frozen=True)
class Config:
    interp:str='linear'
    q:float=.8
    horizon:str='T24'
    gain:float=0.
    bias:bool=False
    scenario:bool=False
    gate:bool=True
    @property
    def name(self):
        return f'{self.interp}_q{self.q:.2f}_{self.horizon}_g{self.gain:g}
_b{int(self.bias)}_s{int(self.scenario)}_gate{int(self.gate)}'

CANDIDATES=tuple(Config(q=q) for q in (0.,.65,.7,.8,.85,.9))+(
    Config(interp='constant'),Config(interp='pchip'),Config(bias=True),
    Config(gain=.5),Config(gain=1.),Config(horizon='R24'),Config(horizon='R48'),
    Config(scenario=True),Config(gate=False),Config(scenario=True,horizon='R48'))
WARMUP=Config()

def horizon_size(k,conf):
    return 144-36*k if conf.horizon=='T24' else (144 if conf.horizon=='R24' else 288)

def simulate_paths(g,paths,initial,battery=Battery()):
    """向量化多个路径，但各路径的动作只用当前观测和自身 E；没有自由场景储能变量。"""
    paths=np.asarray(paths);s,n,_=paths.shape;b=battery
    e=np.full(s,float(initial));h=np.zeros((s,n))
    for t in range(n):
        r=g[t]+paths[:,t,1]-paths[:,t,0]
        c=np.minimum(np.minimum(np.maximum(r,0),b.power*b.dt),np.maximum(0,(b.maximum-
e)/b.eta_charge))
        d=np.minimum(np.minimum(np.maximum(-r,0),b.power*b.dt),np.maximum(0,(e-
b.minimum)*b.eta_discharge))
        h[:,t]=np.maximum(0,-r-d);e+=b.eta_charge*c-d/b.eta_discharge
    return h,e

def
choose_stage(f,initial,price,conf,k,old=None,original=None,rule='A',threshold=1e-6,batte
```

```
ry=Battery()):
    """唯一决策接口只收到当前 StageFeatures；不能访问环境或未来发行版本。"""
    tic=time.perf_counter();t=k*36;m=144-t;n=horizon_size(k,conf)
    assert f.now%144==t and f.source_issue<=f.now and f.residual_latest_end<=f.now
    p=np.tile(price,3)[t:t+n];value=terminal_value(price,battery)
    qs=(0.,.65,.8,.9) if conf.scenario else (conf.q,)
    # 同一个当前信息下的相同评价路径用于所有 KEEP/UPDATE 候选。
    if conf.scenario:paths=scenarios(f,n)
    else:
        _,margin=risk_net(f,conf.q,n)
        path=f.prediction[:n].copy();path[:,0]+=margin;paths=path[None]
    nominal=paths.mean(axis=0)
    tailnet=nominal[m:,0]-nominal[m:,1]

    def evaluate(g):
        g=np.asarray(g[:m]).copy()
        # 前瞻购电只是一条当前信息生成的辅助策略，不进入实际承诺/账本。
        h0,e0=simulate_paths(g,nominal[None,:m],initial,battery)
        if n>m:
            # 未来规划使用固定全日电量价值，通过专用无承诺函数。
            future=free_plan(tailnet,p[m:],float(e0[0]),value,battery)
            full=np.r_[g,future['G']]
        else:full=g
        h,e=simulate_paths(full,paths,initial,battery)
        fee=float(p[:m]@g) if old is None else
float(settlement(old,g,p[:m],rule,original)['charge'].sum())
        scores=fee+float(p[m:]@full[m:])+h@(5*p)-value*(e-initial)
        return float(scores.mean())

    choices=[]
    if old is not None:choices.append((old.copy(),evaluate(old),'KEEP'))
    for q in qs:
        net,margin=risk_net(f,q,n)
        sol=free_plan(net,p,initial,value,battery) if old is None else
adjustment_plan(net,p,initial,old,value,rule,original,battery)
        g=sol['G'][:m];choices.append((g,evaluate(g),f'q={q}'))
    # 稳定平分优先 KEEP；无门控候选强制使用提出的方案。
    start=1 if old is not None and not conf.gate else 0
    best=min(range(start,len(choices)),key=lambda i:(choices[i][1],i))
    keep=choices[0][1] if old is not None else np.nan
    proposed=min(choices[1:],key=lambda v:v[1]) if old is not None else choices[best]
    voi=keep-proposed[1] if old is not None else np.nan
    if old is not None and conf.gate and voi<=threshold:best=0
    g,score,label=choices[best]
    return
dict(G=g.copy(),proposed=proposed[0].copy(),keep_score=keep,update_score=proposed[1],voi
=voi,
                accepted=old is not None and bool(np.max(np.abs(g-old))>1e-7),
                choice=label,seconds=time.perf_counter()-
tic,source_issue=f.source_issue,
                residual_latest_end=f.residual_latest_end,score=score)

def free_plan(net,p,initial,value,battery=Battery()):
    # 与 Q2 普通计划 LP 相同，用显式 value 避免截断价格改变终端代理价值。
    from scipy.optimize import linprog
    from microgrid_online import balance_matrix
    n=len(net);b=battery;obj=np.zeros(5*n+1);obj[:n]=p;obj[-1]=-value
    lo=np.zeros(5*n+1);hi=np.full(5*n+1,np.inf)
    hi[n:3*n]=b.power*b.dt;lo[4*n:]=b.minimum;hi[4*n:]=b.maximum
    lo[4*n]=hi[4*n]=np.clip(initial,b.minimum,b.maximum)

res=linprog(obj,A_eq=balance_matrix(n,b.eta_charge,b.eta_discharge),b_eq=np.r_[net,np.ze
ros(n)],bounds=np.column_stack([lo,hi]),method='highs')
    if not res.success:raise RuntimeError(res.message)
    return dict(G=res.x[:n].copy(),E=res.x[4*n:].copy(),objective=float(res.fun))
```

```
def
run_day(factory,day,initial,price,conf,updates=(1,2,3),rule='A',stale=False,battery=Batt
ery(),details=False):
    """环境逐段揭示真实值；控制器看到的特征由 factory 严格截断。"""
    actual=factory.actual[day];g=np.zeros(144);versions=np.zeros((4,144))
    s={key:np.zeros(144) for key in ('G','C','D','H','W')};s['E']=np.zeros(145);s['E']
[0]=initial
    fees={key:np.zeros(144) for key in ('up_cost','penalty','refund')}
    u=np.zeros((3,144));r=u.copy();pred=np.full((4,144,2),np.nan);events=[];latest=0
    for k in range(4):
        t=36*k;e=float(s['E'][t]);old=g.copy()
        if k==0 or k in updates:
            if stale is True:latest=0
            elif not isinstance(stale,tuple) or k not in stale:latest=k
            f=factory.feature(day,k,conf.interp,conf.gain,conf.bias,latest)
            pred[k,t:]=f.prediction[:144-t]
            if day==0:
                decision=dict(G=np.zeros(144-t),proposed=np.zeros(144-
t),keep_score=0.,update_score=0.,voi=0.,accepted=False,choice='startup',seconds=0.,sourc
e_issue=f.source_issue,residual_latest_end=f.residual_latest_end)
            else:decision=choose_stage(f,e,price,conf,k,None if k==0 else old[t:],None
if k==0 else versions[0,t:],rule,battery=battery)
            g[t:]=decision['G']
            if k:
                bill=settlement(old[t:],g[t:],price[t:],rule,versions[0,t:])
                u[k-1,t:]=bill['U'];r[k-1,t:]=bill['R']
                for key in fees:fees[key][t:]+=bill[key]
                ev={key:decision[key] for key in
('keep_score','update_score','voi','accepted','choice','seconds','source_issue','residua
l_latest_end')}

ev.update(day=day,stage=k,initial=e,increase_kwh=float(bill['U'].sum()),decrease_kwh=flo
at(bill['R'].sum()),adjustment_charge=float(bill['charge'].sum()))
                if details:
                    # 事后固定余日对照，只作诊断；在决策已完成之后读取实际未来，绝不返回控制器。
                    hk,ek=simulate_paths(old[t:],actual[None,t:],e,battery)
                    hu,eu=simulate_paths(g[t:],actual[None,t:],e,battery)
                    ev['realized_value']=float(5*price[t:]@(hk[0]-hu[0])-
bill['charge'].sum()+terminal_value(price,battery)*(eu[0]-ek[0]))
                events.append(ev)
        versions[k]=g
        assert np.array_equal(g[:t],old[:t])
        for j in range(t,t+36):
            s['G'][j]=g[j]
            if day==0:
                rr=g[j]+actual[j,1]-actual[j,0]
                s['H'][j]=max(-rr,0);s['W'][j]=max(rr,0);s['E'][j+1]=s['E'][j]
            else:
                s['C'][j],s['D'][j],s['H'][j],s['W'][j],s['E']
[j+1]=feedback_step(g[j],*actual[j],s['E'][j],battery)
    validate_execution(actual,s,battery)
    s.update(versions=versions,U=u,R=r,prediction=pred,**fees)
    s['plan_cost']=versions[0]*price;s['emergency_cost']=5*price*s['H']
    s['cash']=s['plan_cost']+s['up_cost']+s['penalty']-s['refund']+s['emergency_cost']
    s['config']=conf.name
    return s,events

def stack_days(days):
    return {key:np.asarray([s[key] for s in days]) for key in days[0]}

def summarize(s,price,start=31):
    x={k:v[start:] for k,v in
s.items()};e=x['E'];g=x['G'];h=x['H'];cash=x['cash'].sum(axis=1)
    result={k+'_yuan':float(x[k].sum()) for k in
('plan_cost','up_cost','penalty','refund','emergency_cost')}
```

```
result.update(total_cost_yuan=float(cash.sum()),terminal_adjusted_cost_yuan=float(cash.s
um()-terminal_value(price)*(e[-1,-1]-e[0,0])),
        days=len(e),initial_kwh=float(e[0,0]),terminal_kwh=float(e[-1,-
1]),emergency_kwh=float(h.sum()),

emergency_periods=int((h>1e-7).sum()),emergency_days=int(np.any(h>1e-7,axis=1).sum()),ma
x_emergency_kwh=float(h.max()),

increase_kwh=float(x['U'].sum()),decrease_kwh=float(x['R'].sum()),plan_kwh=float(x['vers
ions'][:,0].sum()),effective_kwh=float(g.sum()),
        adjustment_events=int(np.any(x['U']
+x['R']>1e-7,axis=2).sum()),adjustment_days=int(np.any(x['U']
+x['R']>1e-7,axis=(1,2)).sum()),

daily_p95_yuan=float(np.quantile(cash,.95)),energy_min=float(e.min()),energy_max=float(e
.max()),

charge_kwh=float(x['C'].sum()),discharge_kwh=float(x['D'].sum()),waste_kwh=float(x['W'].
sum()),
        lower_bound_states=int((e<=1200+1e-6).sum()),upper_bound_states=int((e>=10800-
1e-6).sum()),
        continuity_error=float(np.max(np.abs(e[1:,0]-e[:-1,-1]))) if len(e)>1 else 0.)
    return result
```

B15 code/Q3/report_q3.py

```
"""从原始附件与本次运算数组重建 CSV、官方 Excel、图件及论文文字。"""
from pathlib import Path
from copy import copy
from datetime import datetime,timedelta
from math import factorial
import csv,json,sys,os
import numpy as np
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter
from solve_q3 import ROOT,OUT,write_json,write_csv,inputs,check_protected
from microgrid_online import PERIODS,merge_emergency,terminal_value
from q3_forecast import FeatureFactory,risk_net
from q3_model import Config,summarize

WANTED=('2025-03-20','2025-06-21','2025-09-23','2025-12-21')
def read_csv(path):
    with path.open(encoding='utf-8-sig') as f:return list(csv.DictReader(f))
def table(headers,rows):
    return '\n'.join(['|'+'|'.join(headers)+'|','|'+'|'.join(['---']*len(headers))+'|']+
['|'+'|'.join(map(str,r))+'|' for r in rows])+'\n'
def num(x):return f'{float(x):,.4f}'

def workbook(dates,p,s):
    wb=load_workbook(ROOT/'附件/附件 5/result3.xlsx');original=None
    for i,name in enumerate(('计划购电量','调整购电量')):
        ws=wb[name]
        if original is None:original=[ws.cell(1,c).value for c in range(2,146)]
        for t,label in enumerate(PERIODS,2):ws.cell(1,t,label)
        for d in range(31,365):
            row=d-29;g=s['versions'][d,0] if i==0 else s['G'][d]

ws.cell(row,1,datetime.combine(dates[d],datetime.min.time())).number_format='yyyy-mm-dd'
            for t in range(144):ws.cell(row,t+2,float(g[t])).number_format='#,##0.0000'
            ws.cell(row,146,float(g.sum())).number_format='#,##0.0000'
            ws.cell(row,147,float(p@g) if i==0 else float(s['cash']
[d].sum())).number_format='#,##0.0000'

ws.freeze_panes='B2';ws.auto_filter.ref='A1:EQ335';ws.row_dimensions[1].height=27
        for c in range(1,148):ws.column_dimensions[get_column_letter(c)].width=18 if c
in (1,146,147) else 15
```

```
    ws=wb['充放电量'];styles=[[copy(ws.cell(r,c)._style) for c in range(1,7)] for r in
range(2,8)]
    for merged in list(ws.merged_cells.ranges):ws.unmerge_cells(str(merged))
    ws.delete_rows(2,ws.max_row-1)
    for d in range(31,365):
        for k in range(6):
            r=2+(d-31)*6+k
            for c in range(1,7):ws.cell(r,c)._style=copy(styles[k][c-1])
            if
k==0:ws.cell(r,1,datetime.combine(dates[d],datetime.min.time())).number_format='yyyy-mm-
dd'
            ws.cell(r,2,f'{k*4}:00-{k*4+4}:00')
            for c,key in ((3,'C'),(4,'D')):ws.cell(r,c,float(s[key][d,k*24:
(k+1)*24].sum())).number_format='#,##0.0000'
            if k<2:
                ws.cell(r,5,'0:00' if k==0 else '24:00');ws.cell(r,6,float(s['E'][d,0 if
k==0 else -1])).number_format='#,##0.0000'
    for c in range(1,7):ws.column_dimensions[get_column_letter(c)].width=19
    ws.freeze_panes='C2'
    ws=wb['紧急购电量'];styles=[copy(ws.cell(2,c)._style) for c in range(1,4)]
    for merged in list(ws.merged_cells.ranges):ws.unmerge_cells(str(merged))
    ws.delete_rows(2,ws.max_row-1);events=merge_emergency(dates[31:],s['H'][31:]);r=2
    for d in range(31,365):
        ev=[e for e in events if e['date']==dates[d].isoformat()] or
[dict(period='无',kwh=0.)]
        for j,e in enumerate(ev):
            for c in range(1,4):ws.cell(r,c)._style=copy(styles[c-1])
            if
j==0:ws.cell(r,1,datetime.combine(dates[d],datetime.min.time())).number_format='yyyy-mm-
dd'

ws.cell(r,2,e['period']);ws.cell(r,3,float(e['kwh'])).number_format='#,##0.0000';r+=1
    for c in range(1,4):ws.column_dimensions[get_column_letter(c)].width=23
    ws.freeze_panes='B2'
    for ws in wb:
        ws.sheet_view.zoomScale=85;ws.print_title_rows='1:1'
    wb.save(OUT/'result3.xlsx');wb.close()
    write_csv(OUT/'time_mapping.csv',
[dict(index=t,original_template=original[t],period=PERIODS[t],input_end=PERIODS[t].split
('-')[1],column=t+2) for t in range(144)])
    write_csv(OUT/'emergency_purchase_merged.csv',events)
    return events

def data_exports(dates,a,p,s):
    daily=[];dispatch=[];ledger=[]
    for d,date_ in enumerate(dates):
        row=dict(date=date_.isoformat(),config=s['config'][d],initial_kwh=s['E']
[d,0],terminal_kwh=s['E'][d,-1],
          plan_kwh=s['versions'][d,0].sum(),effective_kwh=s['G']
[d].sum(),emergency_kwh=s['H'][d].sum(),
          increase_kwh=s['U'][d].sum(),decrease_kwh=s['R']
[d].sum(),emergency_periods=int((s['H'][d]>1e-7).sum()),
          adjustment_events=int(np.any(s['U'][d]+s['R']
[d]>1e-7,axis=1).sum()),load_kwh=a[d,:,0].sum(),pv_kwh=a[d,:,1].sum(),
          pv_curtail_lower_kwh=np.maximum(0,s['W'][d]-s['G']
[d]).sum(),pv_curtail_upper_kwh=np.minimum(s['W'][d],a[d,:,1]).sum())
        row.update({k+'_yuan':s[k][d].sum() for k in
('plan_cost','up_cost','penalty','refund','emergency_cost','cash')});daily.append(row)
        if d<31:continue
        local=[]
        for t in range(144):

rr=dict(date=date_.isoformat(),index=t,period=PERIODS[t],price_yuan_kwh=p[t],load_kw=a[d
,t,0]*6,pv_kw=a[d,t,1]*6,
                G0=s['versions'][d,0,t],G6=s['versions'][d,1,t],G12=s['versions']
[d,2,t],G18=s['versions'][d,3,t],G_effective=s['G'][d,t],
```

```
                C_real=s['C'][d,t],D_real=s['D'][d,t],H_real=s['H'][d,t],W_real=s['W']
[d,t],E_start=s['E'][d,t],E_end=s['E'][d,t+1],
                pv_curtail_lower=max(0,s['W'][d,t]-s['G']
[d,t]),pv_curtail_upper=min(s['W'][d,t],a[d,t,1]))
            for k in range(4):rr[f'PV_forecast_{k*6}_kw']=s['prediction'][d,k,t,1]*6 if
t>=k*36 else ''
            for k in range(3):rr[f'U{6*(k+1)}']=s['U'][d,k,t];rr[f'R{6*(k+1)}']=s['R']
[d,k,t]
            rr.update({key+'_yuan':s[key][d,t] for key in
('plan_cost','up_cost','penalty','refund','emergency_cost','cash')})
            dispatch.append(rr);local.append(rr)
        for k in (1,2,3):
            for t in range(k*36,144):
                u=s['U'][d,k-1,t];r=s['R'][d,k-1,t]

ledger.append(dict(decision_date=date_.isoformat(),decision_time=f'{k*6}:00',stage=k,del
ivery_date=date_.isoformat(),delivery_interval=PERIODS[t],index=t,
                    previous_commitment=s['versions']
[d,k-1,t],new_commitment=s['versions']
[d,k,t],increase_U=u,decrease_R=r,normal_price=p[t],

increase_charge=1.5*p[t]*u,decrease_penalty=.5*p[t]*r,refund=0.,adjustment_charge=1.5*p[
t]*u+.5*p[t]*r,settlement_rule='A_incremental',forecast_version=date_.isoformat()
+f'T{k*6:02d}:00'))
        if date_.isoformat() in WANTED:
            write_csv(OUT/'selected_days'/f'{date_}_dispatch.csv',local)
            write_csv(OUT/'selected_days'/f'{date_}_table1.csv',
[dict(period=PERIODS[t],G0=s['versions'][d,0,t],effective=s['G'][d,t],emergency=s['H']
[d,t]) for t in (60,72,84,96,108,120)])
            write_csv(OUT/'selected_days'/f'{date_}_table2.csv',[dict(period=f'{4*j}:00-
{4*j+4}:00',charge=s['C'][d,24*j:24*j+24].sum(),discharge=s['D'][d,24*j:24*j+24].sum())
for j in range(6)])

write_csv(OUT/'daily_summary.csv',daily[31:]);write_csv(OUT/'warmup_january.csv',daily[:
31]);write_csv(OUT/'dispatch_10min.csv',dispatch)
    write_csv(OUT/'adjustment_ledger.csv',ledger)
    write_csv(OUT/'emergency_purchase_10min.csv',
[dict(date=r['date'],index=r['index'],period=r['period'],H_real=r['H_real']) for r in
dispatch])
    write_csv(OUT/'selected_days/summary.csv',[r for r in daily if r['date'] in WANTED])
    return daily

def forecast_diagnostics(dates,a,p,records,s):
    f=FeatureFactory(a,records);rows=[];dayrows=[];matched=[];flat=a.reshape(-1,2)
    # 固定方法全年误差是事后诊断，正式方法由此前 21 日经济选择。
    for method in ('constant','linear','pchip'):
        for k in range(4):
            errors=[];targets=[];energies=[];peaks=[]
            for d in range(31,365):
                start=d*144+k*36;n=min(144,len(flat)-start)
                pred=f.raw(d,k,method,0.,k)[:n,1]*6;real=flat[start:start+n,1]*6
                err=pred-real;errors.append(err);targets.append((np.arange(n)+k*36)%144)
                energies.append(err.sum()/6);peaks.append(pred.max()-real.max())

dayrows.append(dict(date=str(dates[d]),method=method,issue_hour=k*6,mae_kw=np.abs(err).m
ean(),rmse_kw=np.sqrt((err**2).mean()),energy_bias_kwh=err.sum()/
6,peak_bias_kw=pred.max()-real.max()))
            error=np.concatenate(errors);clock=np.concatenate(targets)
            dawn=((clock>=30)&(clock<54))|((clock>=96)&(clock<120))

rows.append(dict(method=method,issue_hour=k*6,lead_hour='all',n=len(error),mae_kw=np.abs
(error).mean(),rmse_kw=np.sqrt((error**2).mean()),bias_kw=error.mean(),mean_energy_bias_
kwh=np.mean(energies),dawn_dusk_mae_kw=np.abs(error[dawn]).mean(),mean_peak_bias_kw=np.m
ean(peaks)))
            for h in range(24):
                e=np.concatenate([er[h*6:(h+1)*6] for er in errors if len(er)>h*6])
```

```
rows.append(dict(method=method,issue_hour=k*6,lead_hour=h+1,n=len(e),mae_kw=np.abs(e).me
an(),rmse_kw=np.sqrt((e**2).mean()),bias_kw=e.mean(),mean_energy_bias_kwh='',dawn_dusk_m
ae_kw='',mean_peak_bias_kw=''))
    for d in range(31,365):
        for k in (1,2,3):
            t=k*36;old=f.raw(d,k,'linear',0.,k-1)[:144-
t,1]*6;new=f.raw(d,k,'linear',0.,k)[:144-t,1]*6;real=a[d,t:,1]*6

matched.append(dict(date=str(dates[d]),issue_hour=k*6,previous_issue_hour=(k-1)*6,change
_kwh=np.abs(new-old).sum()/6,
              old_mae_kw=np.abs(old-real).mean(),new_mae_kw=np.abs(new-
real).mean(),mae_improvement_kw=np.abs(old-real).mean()-np.abs(new-real).mean()))
    calibration=[]
    calendar=[Config(**c) for c in
json.loads((OUT/'experiments/calendar.json').read_text())]
    for k in range(4):
        for obj in (0,1):
            err=[];coverage=[]
            for d in range(31,365):
                c=calendar[d];ff=f.feature(d,k,c.interp,c.gain,c.bias);n=144-k*36
                er=(ff.prediction[:n,obj]-a[d,k*36:,obj])*6;err.extend(er)
                _,m=risk_net(ff,c.q,n);actualnet=a[d,k*36:,0]-a[d,k*36:,1]
                coverage.extend(actualnet<=ff.prediction[:n,0]-
ff.prediction[:n,1]+m+1e-7)
            er=np.array(err);calibration.append(dict(object='load' if obj==0 else
'pv',issue_hour=k*6,mae_kw=np.abs(er).mean(),rmse_kw=np.sqrt((er**2).mean()),bias_kw=er.
mean(),net_margin_coverage=np.mean(coverage)))
    write_csv(OUT/'forecast_metrics.csv',rows);write_csv(OUT/'experiments/
forecast_daily.csv',dayrows);write_csv(OUT/'experiments/
forecast_matched_updates.csv',matched);write_csv(OUT/'experiments/
calibration.csv',calibration)
    return rows,matched,calibration

def information_value():
    ab=read_csv(OUT/'experiments/information_ablation.csv');mapping={int(r['mask']):r
for r in ab if int(r['mask'])>=0};events=read_csv(OUT/'voi_events.csv');rows=[]
    for k in (1,2,3):
        bit=1<<(k-1);values={key:0. for key in
('total_cost_yuan','terminal_adjusted_cost_yuan')}
        for mask in range(8):
            if mask&bit:continue
            count=mask.bit_count();weight=factorial(count)*factorial(2-count)/6
            for key in values:values[key]+=weight*(float(mapping[mask][key])-
float(mapping[mask|bit][key]))
        ev=[r for r in events if int(r['stage'])==k];accepted=[r for r in ev if
r['accepted']=='True']

rows.append(dict(issue_hour=k*6,shapley_cash_yuan=values['total_cost_yuan'],shapley_adju
sted_yuan=values['terminal_adjusted_cost_yuan'],
          leave_one_out_cash_yuan=float(mapping[7^bit]['total_cost_yuan'])-
float(mapping[7]['total_cost_yuan']),
          accepted=len(accepted),opportunities=len(ev),acceptance_rate=len(accepted)/
len(ev),voi_positive_rate=np.mean([float(r['voi'])>1e-6 for r in ev]),
          mean_estimated_voi=np.mean([float(r['voi']) for r in
ev]),sum_local_realized_value=sum(float(r['realized_value']) for r in ev),
          increase_kwh=sum(float(r['increase_kwh']) for r in
ev),decrease_kwh=sum(float(r['decrease_kwh']) for r in ev),
          mean_adjustment_kwh=np.mean([float(r['increase_kwh'])+float(r['decrease_kwh'])
for r in ev]),
          mean_solve_seconds=np.mean([float(r['seconds']) for r in
ev]),max_solve_seconds=max(float(r['seconds']) for r in ev)))
    write_csv(OUT/'experiments/stage_value.csv',rows)
    return ab,rows

def figures(dates,a,p,s,daily,metrics,stage):
```

```
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt

plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False,'f
igure.dpi':130})
    directory=OUT/'figures';directory.mkdir(exist_ok=True)
    def save(fig,name):

fig.savefig(directory/(name+'.png'),dpi=160,bbox_inches='tight');fig.savefig(directory/
(name+'.svg'),bbox_inches='tight');plt.close(fig)
    x=(np.arange(144)+.5)/6
    for date_ in WANTED:
        d=next(i for i,dt in enumerate(dates) if str(dt)==date_)
        fig,ax=plt.subplots(6,1,figsize=(12,16),sharex=True,layout='constrained')
        ax[0].plot(x,a[d,:,0]*6,label='Actual
load',color='#334155');ax[0].plot(x,a[d,:,1]*6,label='Actual PV',color='#f59e0b',lw=2)
        for k in range(4):ax[0].plot(x,s['prediction'][d,k,:,1]*6,label=f'PV issue
{k*6:02d}:00',lw=1,alpha=.8)
        ax[0].set_ylabel('Power
(kW)');ax[0].legend(ncol=3,fontsize=8);ax[0].set_title(date_+' | Forecasts, commitments
and actual operation',loc='left')
        for k in range(4):ax[1].step(x,s['versions']
[d,k],where='mid',label=f'G{k*6}',alpha=.8)
        ax[1].plot(x,a[d,:,0]-a[d,:,1],color='#64748b',lw=.7,label='Actual net
load');ax[1].set_ylabel('kWh / 10 min');ax[1].legend(ncol=5,fontsize=8)
        for k in range(3):ax[2].step(x,s['U'][d,k]-s['R'][d,k],label=f'Update {6*(k+1)}
h',where='mid')
        ax[2].axhline(0,color='#94a3b8',lw=.5);ax[2].set_ylabel('Adjustment
(kWh)');ax[2].legend(ncol=3,fontsize=8)
        ax[3].fill_between(x,0,s['C']
[d]*6,label='Charge',alpha=.5,color='#10b981');ax[3].fill_between(x,0,-s['D']
[d]*6,label='Discharge',alpha=.5,color='#3b82f6')
        ax[3].step(x,s['H']
[d]*6,label='Emergency',color='#dc2626',where='mid');ax[3].set_ylabel('Power
(kW)');ax[3].legend(ncol=3,fontsize=8)
        ax[4].plot(np.arange(145)/6,s['E']
[d],color='#7c3aed');ax[4].axhline(1200,ls=':',color='#94a3b8');ax[4].axhline(10800,ls='
:',color='#94a3b8');ax[4].set_ylabel('Stored energy (kWh)')
        ax[5].plot(x,np.cumsum(s['cash'][d]),color='#0f766e',label='Cash allocated by
delivery');ay=ax[5].twinx();ay.plot(x,p,color='#ef4444',alpha=.6);ay.set_ylabel('Price
(yuan/kWh)');ax[5].set_ylabel('Cumulative yuan');ax[5].set_xlabel('Hour of delivery
(costs allocated, not payment timestamp)')
        for axes in ax:
            axes.set_xlim(0,24);axes.set_xticks(np.arange(0,25,2));axes.grid(alpha=.15)
            for tick in (6,12,18):axes.axvline(tick,color='#94a3b8',ls='--',lw=.6)
        save(fig,date_)
    fig,ax=plt.subplots(1,2,figsize=(12,4.4),layout='constrained')
    for k in (0,6,12,18):
        rr=[r for r in metrics if r['method']=='linear' and r['issue_hour']==k and
r['lead_hour']!='all']
        ax[0].plot([r['lead_hour'] for r in rr],[r['mae_kw'] for r in rr],label=f'Issue
{k}h')
    ax[0].set(xlabel='Lead time (h)',ylabel='PV MAE (kW)',title='Raw linear forecasts;
comparable lead groups');ax[0].legend()
    ax[1].bar([str(r['issue_hour']) for r in stage],[r['shapley_cash_yuan']/1000 for r
in stage],color=['#0f766e','#3b82f6','#f59e0b'])
    ax[1].set(xlabel='Additional forecast issue hour',ylabel='Cash saving (thousand
yuan)',title='Shapley allocation across all eight
subsets');ax[1].axhline(0,color='#64748b',lw=.5);save(fig,'forecast_and_value')

fig,ax=plt.subplots(figsize=(11,4.5),layout='constrained');months=range(2,13);bottom=np.
zeros(11)
    for key,label,color in [('plan_cost_yuan','Original plan','#334155'),
('up_cost_yuan','Increase','#38bdf8'),('emergency_cost_yuan','Emergency','#ef4444')]:
        vals=np.array([sum(float(r[key]) for r in daily if r['date'][5:7]==f'{m:02d}')
```

```
for m in months])/1e6
        ax.bar(list(months),vals,bottom=bottom,label=label,color=color);bottom+=vals
    ax.set(xlabel='Month',ylabel='Million yuan',title='Actual cash cost by
month');ax.legend(ncol=3);save(fig,'monthly_costs')

def documents(dates,a,p,s,daily,merged,metrics,matched,calibration,ablation,stage):
    summary=summarize(s,p);q2=json.loads((ROOT/'Q2/summary.json').read_text())
    subsets={int(r['mask']):r for r in ablation};only0=subsets[0]

candidates=read_csv(OUT/'experiments/candidate_summary.csv');sensitive=read_csv(OUT/'exp
eriments/sensitivities.csv')
    choices=[r for r in read_csv(OUT/'experiments/monthly_selection.csv') if
r['selected']=='True'];events=read_csv(OUT/'voi_events.csv')
    supplemental=read_csv(OUT/'experiments/supplemental.csv');supp={r['policy']:r for r
in supplemental}

independent0=supp['zero_only_independent_past_selection'];independent_gain=float(indepen
dent0['total_cost_yuan'])-summary['total_cost_yuan']
    cash=summary['total_cost_yuan'];gain_q2=q2['total_cost_yuan']-
cash;gain0=float(only0['total_cost_yuan'])-cash
    policy_counts={c:int(np.sum(s['config'][31:]==c)) for c in np.unique(s['config']
[31:])}
    state_effect=float(subsets[-1]['total_cost_yuan'])-cash
    text=f'''# 问题3 基于已发布光伏预报的多阶段购电调整

## 1 问题分析与结论

问题2在0时冻结全天普通购电，问题3增加0/6/12/18时官方光伏预报，并允许在日内按规定价格改变未交付承诺。新信息应在节省紧急购电与支付调整费用之间权衡。本研究保留已验收Q2的实时储能反馈，采用预注册候选的月度历史经济选择，在每次预报发布时形成或调整未来承诺。

主解释为原计划费用不退、调增按1.5倍、调减另付0.5倍，按上一有效承诺逐事件结算。2025-02-01至12-31共334天实际费用为{num(cash)}元，相对冻结Q2现金节省{num(gain_q2)}元（{gain_q2/
q2['total_cost_yuan']:.2%}）。紧急购电{num(summary['emergency_kwh'])} kWh，共{summary['emergency_periods']}个十分钟时段；Q2为146312.1414 kWh、923时段。因此费用及缺口电量改善不等于所有可靠性指标同时改善。

独立历史选参的仅0时预报对照费用{num(independent0['total_cost_yuan'])}元，完整日内更新比它节省{num(independent_gain)}元；它相对Q2费用增加{num(float(independent0['total_cost_yuan'])-
q2['total_cost_yuan'])}元，不能声称官方0时预报单独必然优于历史预测。另有共享正式配置日历的仅0时对照{num(only0['total_cost_yuan'])}元，用于隔离同一风险配置下的调整权限。以下结论只适用于声明的结算、执行器、候选集和历史数据。正式方法是在线选择器，不能把事后全年最低固定候选替换成2月份已提交计划。

## 2 事实 假设与信息集

原题规定每日四次未来24小时整点PV预报、调整价与紧急价、储能上下界和结果日期。沿用Q2建模解释：10分钟记录为区间结束点、功率/6近似电量、充放电效率各0.9、实时反馈、不售电、不削负荷、无额外并网容量。实时反馈使用当前时段观测的聚合近似，不是时段开始前就知道其完整平均值。Q3没有重复Q1每日首末相等要求，全年SOC连续。

F(d,k)包括阶段开始前全部已实现记录、真实当前SOC、已发布预报、固定价格、日历、已完成历史误差和月初已确定配置；不包括尚未发布预报或未来实际曲线。0时只能读取当日0时预报；6时增加0–6实际与6时预报；12/18同理。实际未来只进入逐时执行环境和完成后的评价，不进入choose_stage接口。

1月1日初值6000 kWh。因缺历史负载，采用Q2同类保守启动日：G=0、电池静置、缺口补电；1月2日起固定Q3热启动，1月费用另存。所有Q3实验共享这条1月轨迹。2月1日SOC为{num(summary['initial_kwh'])} kWh、年底为{num(summary['terminal_kwh'])} kWh；Q2相应为10800和3453.7471 kWh，不能隐去这一边界差异。

## 3 附件3结构及整点映射

附件3含365×4=1460次发行、每次24个点，显式键为发行日期、发行时间、目标时间和提前小时。0时目标为1时至次日0时；6/12/18时依次平移，年末部分目标落在2026年，未提供的实际值不参与误差评价。原始35040个点保存在forecast_versions.csv。

比较向后保持常值、线性和PCHIP。发行时最后已知PV为左端锚点，首次1月1日0时取0；原始整点值必须保持，插值为本文选择。区间结束点采样除以6近似电量，不声称小时功率预报已给出小时总能量。实际月度选择及误差见下表；全年误差仅诊断，正式插值由过去21日经济回放选择。
```

```
'''
    text+=table(['方法','发行时刻','PV MAE kW','RMSE kW','预报减实际 bias kW','24h 平均能量偏差 kWh','晨昏 MAE kW'],
[[r['method'],r['issue_hour'],num(r['mae_kw']),num(r['rmse_kw']),num(r['bias_kw']),num(r['mean_energy_bias_kwh']),num(r['dawn_dusk_mae_kw'])] for r in metrics if r['lead_hour']=='all'])
    text+=r'''
```

最后一日的部分 24h 目标不可观测，因此能量统计使用可见目标子集，完整日诊断另见 forecast_daily.csv；晨昏定义为 5–9 及 16–20 时。首小时常值使用发行锚点向后保持，可能有晨昏滞后，不能把不同插值误差视为同一算法的时间错位。

4 负载预测 官方预报校准与风险

负载沿用 Q2 组合预测：最近 7 日同刻均值与过去最多 4 个同星期同刻均值各半，整个未来窗口固定同一历史截断点。候选日内修正使用最近最多 6 小时已观测负载误差均值，以 6 小时衰减时间指数递减；增益 0/.5/1 由历史验证选择。

官方 PV 优先覆盖其 24h 有效范围，之外由 Q2 历史 PV 预测补齐。偏差校正候选用过去对应发行阶段、相对时距的误差均值；零样本不校正，不按全年数据拟合。误差档案仅收录已完整实现的 48h 负载/PV 联合路径，最多 30 条；这比只等待目标点实现更保守，牺牲最新尾部信息以保证实现简洁。分位裕度为

$$m_{d,k,t}=\max\{0,Q_q(e^L-e^{PV})\},\qquad \widetilde N=\widehat L-\widehat P+m. \tag{1}$$

q=0 是无裕度点预测的标记，不表示 0 分位数。固定加性 PV bias 若对应净误差分位仍为正，会被同量分位平移抵消；因此该组合下费用一致有代数原因，不能据此断言所有 PV 校准无效。经验覆盖率不具有独立同分布或 conformal 严格保证。

```
'''
    text+=table(['对象','阶段 h','正式预测 MAE kW','RMSE kW','净负荷裕度经验覆盖'],
[[r['object'],r['issue_hour'],num(r['mae_kw']),num(r['rmse_kw']),f"{r['net_margin_coverage']:.2%}"] for r in calibration])
    text+=r'''
```

5 数学模型

日期 d、阶段 k、交付时段 t；G0 为 0 时原计划，G^k 为当前承诺，U/R 为本轮调增/调减，C/D/H/W/E 为真实充电、放电、紧急购电、未利用能量和储电量。所有流量 kWh，p 元/kWh。

$$G_t^k=G_t^{k-1}+U_t^k-R_t^k,\quad U_t^k,R_t^k\ge0;\quad G_t^k=G_t^{k-1},\ t<\tau_k. \tag{2}$$

$$G_t+P_t+D_t+H_t=L_t+C_t+W_t,\qquad E_{t+1}=E_t+0.9C_t-D_t/0.9. \tag{3}$$

$$1200\le E_t\le10800,\quad 0\le C_t\le(5000/6)z_t,\quad0\le D_t\le(5000/6)(1-z_t),\quad z_t\in\{0,1\}. \tag{4}$$

G/H/W 非负，实际 H 只补剩余缺口。预测提案层用 G+D−C−W=风险修正净负荷，且不预先把实际 H 当作普通购电。0 时提案目标为 ΣpG−vE 末；调整提案主 A 目标为当前日 Σ(1.5pU+0.5pR)+未来日 ΣpG−vE 末。G0 在 F(d,0)可测，G^k 在 F(d,k)可测，跨日 E[d+1,0]=E[d,144]。

终端代理 v=当日固定电价最低 24 个点均值/0.9=0.4676666667 元/内部 kWh。T24 至自然日末，R24/R48 分别从发行时刻向后 24/48h；均采用同一 v，不每日闭合。未来日购电仅前瞻辅助，次日 0 时重新形成正式 G0，不能把今晚前瞻重复记账。

LP 计划允许 W 作为总未利用能量，若同充同放，令 δ=min(C,D/(ηcηd))，扣减 C 的 δ、D 的 ηcηdδ 并增加 W 的 (1−ηcηd)δ，G、E 与费用不变，恢复互斥。代表性调整 LP 与显式 MILP 目标对照，实际反馈天然互斥。该结论针对允许未利用电量的本模型，不应推广到其他弃电边界。

6 实时执行与调整账本

每时段令 r=G+PV−L。r≥0 时 C=min(r,5000/6,(10800−E)/0.9)，D=H=0，W=r−C；r<0 时 D=min(−r,5000/6,(E−1200)0.9)，C=W=0，H=−r−D。此为继承 Q2 的反馈器，不是全局最优储能控制，不向电池馈入未来场景。紧急购电只满足负载缺口，不能用于主动充电。

主账单为

$$C_A=\sum_t p_tG_t^0+\sum_{k=1}^3\sum_{t\ge\tau_k}(1.5p_tU_t^k+0.5p_tR_t^k)+
\sum_t5p_tH_t. \tag{5}$$

每笔调整相对上一有效承诺，不再对完整新 G 收一次计划费。调减后电量不再参加物理平衡，即使原费用已沉没。主 A 下付费调减被保留更多承诺并弃用的策略支配，本次 R=0 与这一逻辑一致。

替代 B 退回 pR 再收 0.5pR，调减净费−0.5pR；B_net 相对原始 G0 净偏差重新计账并只收新旧账面差，允许取消未交付早先调整，但保留每笔冲销。原计划 100 依次 80、110、90，p=1 时 A/B/B_net 非紧急总费分别 165/125/95。题面未唯一消除退费及基准歧义，主口径不是官方认证。

7 更新价值与有限场景方法

在当前相同预测路径、SOC 和费用基础下，同时评价 KEEP 与 UPDATE。对给定 G 逐点调用同型反馈器，计算

$$J=\Delta C_{\rm adj}+\mathbb E_s\big[5\sum pH_s+C_{\rm future}-v(E_{T,s}-E_{\rm now})
\big],\quad VOI=J_{\rm keep}-J_{\rm update}.\tag{6}$$

已沉没计划费对两者相同，不重复计入比较。门限 1e−6 元仅为浮点容差；相等先 KEEP。J 已包含调整费，不能再从 VOI 扣一次。跨日前瞻在当前平均路径得到预计午夜 E 后产生一条共享未来普通购电曲线，所有场景使用同一曲线；不是场景实现后自由重解整个路径。该有限策略近似没有未来发行预报的场景树，不声称多阶段随机最优。

场景候选使用最多 8 条历史完整联合残差加当前预测，不逐点独立噪声；从 q=0/.65/.8/.9 提案中选择平均 J 最小者。当前 G 跨所有场景共享；实际储能始终逐点反馈，因此没有全知场景电池。当前规模无需 K-medoids；未添加 CVaR 权重，不代表已验证所有 CVaR 方法无收益。

estimated VOI 只使用当前信息。realized_value 是决策提交后用真实余日分别冻结 KEEP 与已采用方案计算的现金差和终端值差，不模拟后续再更新，不能把多个事件的这一重叠诊断值相加当全年预报贡献。

8 候选选择与严格滚动验证

16 个配置在正式全年排名之前注册，包含点预测、不同风险分位、插值、偏差修正、负载更新、T24/R24/R48、有限场景与取消门控。每月首日用过去 21 个完整日，所有候选从同一个已知验证起点 SOC 独立连续回放，评分为现金费−v(E 末−E 初)，最小者月内固定。正式未来实现仍按顺序揭示。以下为实际选择，并非全年事后最优回填。

```
    '''
    text+=table(['月首','选择配置','历史窗口','历史评分元'],
[[str(dates[int(r['selection_day'])]),r['config'],f"{dates[int(r['begin'])]}–
{dates[int(r['end'])]}",num(r['score'])] for r in choices])
    text+='\n 固定候选全年诊断（各候选相同 Q3 热启动，未用于回选过去）：\n\n'
    text+=table(['配置','现金元','末态调整元','紧急 kWh','期末 E'],
[[r['config'],num(r['total_cost_yuan']),num(r['terminal_adjusted_cost_yuan']),num(r['eme
rgency_kwh']),num(r['terminal_kwh'])] for r in candidates])
    text+='\n## 9 全年费用与运行结果\n\n'
    text+=table(['指标','结果'],[[k,num(v)] for k,v in summary.items()])
    lower=sum(r['pv_curtail_lower_kwh'] for r in
daily[31:]);upper=sum(r['pv_curtail_upper_kwh'] for r in daily[31:]);pv=a[31:,:,1].sum()
    text+=f'\n 未利用能量不等于弃光。按光伏优先分摊，弃光为下界{num(lower)} kWh，消纳
率{1-lower/pv:.2%}；不固定来源时弃光可在{num(lower)}至{num(upper)} kWh 之间。总费用和供需平衡不依
赖该来源分配。\n\n![月度费用](figures/monthly_costs.png)\n'
    text+='\n## 10 后续预报信息消融\n\n'
    text+=table(['信息/策略','现金元','末态调整元','紧急 kWh','调整次数','期初 E','期末 E'],
[[r['policy'],num(r['total_cost_yuan']),num(r['terminal_adjusted_cost_yuan']),num(r['eme
rgency_kwh']),r['adjustment_events'],num(r['initial_kwh']),num(r['terminal_kwh'])] for r
in ablation])
    text+=f'''\n8 个预报子集沿用正式策略的因果配置日历，拥有各自连续 SOC，并非分别再调参后的最优子策
略。旧预报对照在同样 6/12/18 时更新 SOC、允许同样付费调整，但 PV 保持 0 时发行版本。全预报相对旧预报重优化节
省{num(state_effect)}元，才更接近同权限下新 PV 信息的增量贡献；全预报相对全天不调整的{num(gain0)}元同
时包含重新观测 SOC 和合同调整机会。

对 8 子集用三变量 Shapley 平均边际节省分配 6/12/18 的交互影响，三者之和严格等于仅 0 时费用减全预报费用。
leave-one-out 为保留另两个时刻时加入该时刻的边际；二者均是事后策略比较，不作为当天 VOI。每日现金分配到交
付时段，图中累计不是交易付款时间。

'''
    text+=table(['时刻 h','Shapley 现金节省元','末态调整节省元','移除此时刻增加费用元','采用次
数/334','VOI 正比例','平均预计 VOI 元','平均/最大求解秒'],
```

```
[[r['issue_hour'],num(r['shapley_cash_yuan']),num(r['shapley_adjusted_yuan']),num(r['lea
ve_one_out_cash_yuan']),f"{r['accepted']}/
334",f"{r['voi_positive_rate']:.2%}",num(r['mean_estimated_voi']),f"{r['mean_solve_secon
ds']:.5f}/{r['max_solve_seconds']:.5f}"] for r in stage])
    text+='\n### 同调整权限下的新 PV 信息与公平基准\n\n'
    text+=table(['补充对照','现金元','紧急 kWh','末态调整元'],
[[r['policy'],num(r['total_cost_yuan']),num(r['emergency_kwh']),num(r['terminal_adjusted
_cost_yuan'])] for r in supplemental])
    text+='\n 逐个屏蔽该时刻的新 PV 但保留当前 SOC 更新及全部调整权限，其余时刻正常发行。以“屏蔽费−完整
费”为新 PV 的条件现金价值：'
    fresh=[]
    for k in (6,12,18):
        val=float(supp[f'withhold_new_{k}_same_rights']['total_cost_yuan'])-
cash;fresh.append((k,val))
        text+=f'{k}时{num(val)}元；'
    text+='新 PV 边际存在交互，不与前三个 Shapley 权限值相加。6 时新 PV 在这组条件下现金边际为负但可降低紧
急电量，不能声称每份新预报都值得采用，也不能把它的事后负值用于撤销历史正式决策。独立仅 0 时基准仍只用过去
21 日选参，属于发现共享低 q 配置对不可调整基准不公平后增加的诊断，不改变预注册正式策略。\n'
    text+='\n![预报误差与经济价值](figures/forecast_and_value.png)\n\n## 11 18 时预报与替代口径
\n\n'
    text+=table(['策略','现金元','末态调整元','调增 kWh','调减 kWh','紧急 kWh','末 E'],
[[r['policy'],num(r['total_cost_yuan']),num(r['terminal_adjusted_cost_yuan']),num(r['inc
rease_kwh']),num(r['decrease_kwh']),num(r['emergency_kwh']),num(r['terminal_kwh'])] for
r in sensitive])
    sens={r['policy']:r for r in sensitive}
    for horizon in ('T24','R48'):
        delta=float(sens[horizon+'_without18']['total_cost_yuan'])-
float(sens[horizon+'_with18']['total_cost_yuan'])
        text+=f'\n{horizon}固定 q=.8 对照中，加入 18 时更新的全年现金节省为{num(delta)}元。'
    nextvalue=float(supp['R48_18_nextday_old_information']['total_cost_yuan'])-
float(sens['R48_with18']['total_cost_yuan'])
    text+=f'\n 进一步保持 18 时当日目标、SOC 与调整权限不变，仅将次日预报目标替换为 12 时已知预测/因果补
齐；R48 的次日新信息现金价值为{num(nextvalue)}元。本数据和该候选下没有观测到额外的次日准备节省，不能照
搬报告的“18 时次日 PV 必然有用”。\n'
    text+='''\nT24 的固定终端价值不依赖次日 PV，因此仅有次日预报变化时无法使 T24 决策改变；不能给 T24 虚
构跨日天气价值。R48 可使用次日 PV，以上 R48 对照同时含 18 时 SOC 更新与调整权限的效应，不能仅凭它证明“次日
PV 本身”导致节省。正式新/旧预报同权限对照和匹配目标误差共同约束解释。B/B_net 数值是相同历史配置日历下重
新优化和执行，不是只对主轨迹换收费公式，也未对替代口径重新寻找所有最优超参数。

## 12 四个指定日期

表 1 报告最后实际生效普通购电 G，原计划 G0 并列以展示调整；H 另列，不与普通调整混淆。全天费用包括所有费用；四
小时段的充放电均为真实执行汇总。日首末 SOC 可能不同，不能把透支储能产生的表面收益归因于预报。
'''
    selected=[]
    for date_ in WANTED:
        d=next(i for i,dt in enumerate(dates) if
str(dt)==date_);row=daily[d];selected.append(row)
        text+=f'\n### {date_}\n\n 表 1 指定时段购电量（kWh）\n\n'
        text+=table(['时间段','原计划 G0','有效 G','紧急 H'],[[PERIODS[t],num(s['versions']
[d,0,t]),num(s['G'][d,t]),num(s['H'][d,t])] for t in (60,72,84,96,108,120)])
        text+=f"\n 全天原计划{num(row['plan_kwh'])} kWh，有效普通购电
{num(row['effective_kwh'])} kWh，紧急{num(row['emergency_kwh'])} kWh；原计划费
{num(row['plan_cost_yuan'])}元、调增费{num(row['up_cost_yuan'])}元、调减费
{num(row['penalty_yuan'])}元、紧急费{num(row['emergency_cost_yuan'])}元，总费
{num(row['cash_yuan'])}元。\n\n 表 2 真实储能充放电（kWh）\n\n"
        text+=table(['时间段','充电','放电'],[[f'{4*k}:00-{4*k+4}:00',num(s['C'][d,k*24:
(k+1)*24].sum()),num(s['D'][d,k*24:(k+1)*24].sum())] for k in range(6)])
        text+=f"\n0:00 储电量{num(s['E'][d,0])} kWh，24:00 储电量{num(s['E'][d,-1])} kWh。
\n\n"
        ev=[r for r in events if r['date']==date_];mt=[r for r in matched if
r['date']==date_]
        text+=table(['更新 h','PV 曲线变化 kWh','匹配目标 MAE 改善 kW','预计 VOI 元','采用','局部事后
价值元'],[[r['issue_hour'],num(r['change_kwh']),num(r['mae_improvement_kw']),num(ev[j]
['voi']),ev[j]['accepted'],num(ev[j]['realized_value'])] for j,r in enumerate(mt)])
        accurate_but_keep=[r['issue_hour'] for j,r in enumerate(mt) if
```

```
r['mae_improvement_kw']>1e-7 and ev[j]['accepted']=='False']
        text+=f'\n本日“匹配目标预报更准但未采用购电调整”的时刻：{accurate_but_keep if
accurate_but_keep else "没有出现"}。若没有出现也不预设它必然存在；全年事件可在
forecast_matched_updates与voi_events按日期阶段联合复核。\n'
        largest=max(mt,key=lambda r:r['change_kwh']);best=max(mt,key=lambda
r:r['mae_improvement_kw']);economic=max(ev,key=lambda r:float(r['realized_value']))
        text+=f"\n本日PV变化量最大在{largest['issue_hour']}时，匹配目标误差改善最大在
{best['issue_hour']}时，已采用策略相对冻结KEEP的局部事后价值最大在{economic['decision_time']}。
这些排名不要求相同：电池余量、未来缺口、1.5倍增购价与5倍紧急价共同决定是否调整。局部事后价值可为负，不能
反过来撤销当时已提交决策。\n\n![{date_}调度](figures/{date_}.png)\n"
    text+='\n### 表3 四指定日紧急购电\n\n'
    evs=[[r for r in merged if r['date']==date_] for date_ in WANTED];tabrows=[]
    for j in range(max(1,max(map(len,evs)))):
        row=[]
        for ev in evs:row.extend([ev[j]['period'],num(ev[j]['kwh'])] if j<len(ev) else
['无' if j==0 else '',0 if j==0 else ''])
        tabrows.append(row)
    text+=table([x for date_ in WANTED for x in (date_+' 时段','kWh')],tabrows)
    text+='''
## 13 验证与结果文件
```

原型覆盖1/29–2/1、3/19–3/22、6/19–6/22、12/18–12/21四个连续窗口，每窗16候选。核心测试包括整点锚点、功率单位、预报跨日、残差全部实现、修改未来发行版本和未来实际曲线的反事实、过去计划冻结、A/B/B_net手算多交易、缺电增购/富余不调整、SOC边界、LP/MILP对照与继承Q2执行器一致。正式后另全量验证365天物理、334×144计划/调整、充放电和紧急合并，独立从CSV重算费用，并重跑Q1/Q2/Q3全部测试，状态见validation.json与Q3最终验收审计.md。

result3由官方模板填写，计划表144点及全天量是G0，计划费列为原计划pG0；调整表144点及全天量是最终有效G，费用列是最终实际总账单。两表费用不能相加。这一主语义与Q2计划表费用含紧急的既有写法不同，已在口径核验公开，底层费用无混用。内部保留G0/G6/G12/G18、U/R、交易账本和真实执行，任何表均可重建。时间列统一自然日，无循环移动数值；原始模板不改。

14 模型评价与局限

该策略的优点是发行时间隔离、冻结承诺与实际执行分层、逐笔账本、历史选参及完整输出核验。它优化的是有限候选类，不是全年随机控制全局最优；贪心储能未优化未来应急机会成本。48h补齐和末态值均为代理，无未来价值精确保证；1月启动、人为残差窗口与候选集设计使用了题目总体结构，因此只称严格时间样本外回放，不称外部独立数据集验证。

最主要WARNING是调减退费/多次调整基准、时间与效率歧义、十分钟即时反馈假设、预报子集共享配置日历、末态SOC不同以及局部VOI与全年价值不可混用。主口径A无调减是结构性经济结果，不表示题目50%条款被遗忘。更低紧急电量伴随更多小缺口时段，需根据实际偏好判断可靠性取舍。当前实验不证明CVaR、复杂MPC或机器学习在所有数据上无用，只是不凭名称增加未验证复杂度。

参考资料

[1] C题.pdf，问题2/3、附录1/2与result3合同；附件1/2/3/5。

[2] Q2冻结决策报告、正式论文与最终验收审计，Q2_frozen_v1。

[3] Polimeni et al. Implementation of Different PV Forecast Approaches in a MultiGood MicroGrid: Modeling and Experimental Results. Processes, 2021, 9:323. DOI:10.3390/pr9020323。固定执行框架比较预报、末态校正及精度与经济性区别，见第5/10/15页。

[4] Wen et al. Modeling of photovoltaic power uncertainties for impact analysis on generation scheduling and cost of an urban micro grid. Mathematics and Computers in Simulation, 2021,183:116–128. DOI:10.1016/j.matcom.2020.02.023。本地未校正稿第4–6、9–11页，借鉴误差与风险经济性，不照搬当年未来样本或机组备用。

[5] Panda et al. Methods for Mitigating Uncertainty in Real-Time Operations of a Connected Microgrid. SEGAN,2024,38:101334. DOI:10.1016/j.segan.2024.101334。承诺/实时执行分层。

[6] Herding et al. Stochastic programming approach for optimal day-ahead market bidding curves of a microgrid. Applied Energy,2023,336:120847. DOI:10.1016/j.apenergy.2023.120847。锁定第一阶段及模拟样本外局限，第7页。

```
'''
    (OUT/'Q3_final.md').write_text(text)
    decision=f'''# Q3 最终决策报告

正式策略为 16 个预注册候选中的月度过去 21 日经济选择，不使用全年排名回填。实际配置及使用天数：
{json.dumps(policy_counts,ensure_ascii=False)}。

0 时官方 PV 进入 G0；负载继承 Q2 组合；日内根据当前 SOC、已发布预报和历史残差提出调整，在同一信息下比较 KEEP/UPDATE。实际执行完全复用 Q2 feedback_step。根目录新增 microgrid_forecasts.py 及 microgrid_adjustment.py 供 Q4 复用，原公共模块/Q1/Q2 未修改。

总费{num(cash)}元：原计划{num(summary['plan_cost_yuan'])}，调增 {num(summary['up_cost_yuan'])}，调减违约{num(summary['penalty_yuan'])}，退款 {num(summary['refund_yuan'])}，紧急{num(summary['emergency_cost_yuan'])}。紧急电量 {num(summary['emergency_kwh'])} kWh、{summary['emergency_periods']}时段；调增 {num(summary['increase_kwh'])} kWh、调减{num(summary['decrease_kwh'])}， {summary['adjustment_events']}次阶段调整。

独立历史选择 Q3 仅 0 时相对 Q2 现金变化（正为节省）{num(q2['total_cost_yuan']- float(independent0['total_cost_yuan']))}元；完整 Q3 相对独立仅 0 时节省{num(independent_gain)}元。共享同一正式配置日历的仅 0 时消融费用为{num(only0['total_cost_yuan'])}元，完整 Q3 比它节省 {num(gain0)}元；这不是两个均独立调优的对照。完整 Q3 相对 Q2 节省{num(gain_q2)}元。不同期初/末 SOC 应同时查看末态调整费用，不能全部归因于预测。

'''
    decision+=table(['阶段 h','Shapley 现金价值元','移除此阶段增费元','采用次数'],
[[r['issue_hour'],num(r['shapley_cash_yuan']),num(r['leave_one_out_cash_yuan']),r['accepted']] for r in stage])
    decision+='\n 在保持调整权限和 SOC 更新的逐个新 PV 屏蔽对照中，6/12/18 时新预报条件现金价值分别为'+'、'.join(num(v)+'元' for _,v in fresh)+'。6 时该边际略负，不能宣称所有新 PV 都降低费用。18 时次日 PV 专项 R48 对照的现金价值为'+num(nextvalue)+'元，未观察到报告预设的跨日收益。\n'
    decision+='''
预报使用不等于每次必须改变 G。estimated VOI 完全事前；realized value 只在提交后计算，且是冻结余日的局部诊断，不能加总为年度因果贡献。是否保留 6/12/18 参见上表的子集比较与论文 18 时专节，不以太阳落山直接判断。

原始整点锚点保持，插值、bias、负载更新、时域、场景及门控由月首历史选择决定。R24/R48 是候选，未经选择不得宣布它们主用。有限场景不是全知电池的自由两阶段规划；无 K-medoids 与 CVaR，不声称已证明所有复杂扩展无收益。全年固定候选排名、bias 代数抵消和窗口结果见 Q3_final 第 8 节。

主 A 不退款，调增 1.5p、调减+.5p，针对上一承诺按交付价格记账；B/B_net 另重算，结果不能混用。A 下没有调减有经济支配理由。result3 基于原始模板，计划表为 G0，调整表为有效 G，充放电与 H 均来自实际执行；全量回读与底层独立汇总的最终状态以 validation.json 和 Q3 最终验收审计.md 为准。

四指定日期：

'''
    decision+=table(['日期','原计划费','调增费','紧急费','总费','紧急 kWh','首/末 E'],
[[r['date'],num(r['plan_cost_yuan']),num(r['up_cost_yuan']),num(r['emergency_cost_yuan']),num(r['cash_yuan']),num(r['emergency_kwh']),f"{num(r['initial_kwh'])}/{num(r['terminal_kwh'])}"] for r in selected])
    best=max([ev for ev in events if ev['date'] in WANTED],key=lambda ev:float(ev['realized_value']))
    decision+=f"\n 四指定日中，单次余日冻结对照的最大局部更新价值发生在{best['date']} {best['decision_time']}，为{num(best['realized_value'])}元；不把同一天重叠余日价值直接相加当成日因果收益。\n"
    decision+='\nWARNING：结算歧义、效率/时间假设、实时反馈聚合、终端代理与边界不同、同日历消融及局部 VOI 不可加总，均在论文中披露。紧急电量下降但时段次数增加，不应写成所有可靠性指标改善。\n'
    (OUT/'Q3 最终决策报告.md').write_text(decision)
    (OUT/'README.md').write_text('''# 问题 3 程序与交付

正式文件 result3.xlsx，论文 Q3_final.md，选择与结算说明见 Q3 最终决策报告.md 及各审计。主 A，不退款、逐事件上一承诺结算；B/B_net 仅敏感性。Q1/Q2 保持冻结。所有求解、表格、图、文档和验证生成代码保留。

在 C 题目录，用 Python 及 numpy/scipy/openpyxl/matplotlib：

```bash
```
```

```
.venv/bin/python Q3/solve_q3.py
.venv/bin/python Q3/audit_q3.py
.venv/bin/python Q3/freeze_q3.py --create
```

首次无冻结时入口顺序执行 Gate A/B 文档、原型、核心测试、正式历史选择、全部实验和交付。可单独使用--prototype、--compute、--deliver；--compute 需要先通过原型和核心测试。首次接手快照由 snapshot_q3.py 保存；已有快照不覆盖。当前冻结后生成器拒绝原地覆盖；请在独立副本移走 Q3 冻结清单再重建，不改变 Q1/Q2 清单。

audit_q3.py 独立读取 CSV/Excel 复算并运行 Q1/Q2/Q3 测试，日志全部在 Q3。freeze_q3.py 创建或只读校验；环境精确版本 requirements.freeze.txt。冻结 ZIP 保存代码、输出、输入及 Q1/Q2 依赖快照，不包含虚拟环境/node_modules/绘图库缓存。图表英文以避免中文字体缺失，中文解释在论文。

verify_workbook.mjs 是额外只读 Excel 视觉核验，使用@oai/artifact-tool；不属于 Python 求解必需依赖。使用工作区依赖加载器的 Node 和 node_modules，运行 node Q3/verify_workbook.mjs；--template 查看原始模板。全部 Python 输出可不依赖该渲染器重建。

重要输出：dispatch_10min.csv 保存 G0/G6/G12/G18/有效 G、U/R 和实际 C/D/H/E；adjustment_ledger.csv 逐事件账本（含 0 调整记录）；forecast_versions.csv 原始 35040 预报；voi_events.csv 事前 VOI 与局部事后诊断；experiments 保存预注册、月首验证、固定候选诊断、8 个信息子集、旧预报、三种结算和 18h 时域对照。selected_days 为四指定日，figures 为 PNG/SVG。
''')

```
def deliver():
    check_protected();dates,a,p,r=inputs();s=dict(np.load(OUT/'experiments/
formal_arrays.npz'))
    daily=data_exports(dates,a,p,s)
    metrics,matched,calibration=forecast_diagnostics(dates,a,p,r,s)
    ab,stage=information_value();merged=workbook(dates,p,s)

figures(dates,a,p,s,daily,metrics,stage);documents(dates,a,p,s,daily,merged,metrics,matc
hed,calibration,ab,stage)

summary=summarize(s,p);summary.update(pv_curtail_lower_kwh=sum(v['pv_curtail_lower_kwh']
for v in daily[31:]),pv_curtail_upper_kwh=sum(v['pv_curtail_upper_kwh'] for v in
daily[31:]))
    write_json(OUT/'summary.json',summary);print('Delivery files generated; independent
audit still required.',flush=True)

if __name__=='__main__':deliver()
```

B16 code/Q3/solve_q3.py

```
"""原始附件→预审→原型→因果月度选择→正式全年→消融→图表/论文→验收。
所有问题 3 派生文件由此入口和 audit_q3.py/freeze_q3.py 重建。
"""
import sys,os,json,csv,time,argparse,hashlib
from pathlib import Path
from datetime import datetime,timedelta
from dataclasses import asdict
ROOT=Path(__file__).resolve().parents[1];OUT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT));os.environ.setdefault('MPLCONFIGDIR',str(OUT/'.matplotlib')
)
import numpy as np
from microgrid_online import read_year,terminal_value
from microgrid_forecasts import read_pv_forecasts
from q3_forecast import FeatureFactory
from q3_model import Config,CANDIDATES,WARMUP,run_day,stack_days,summarize

def write_json(path,obj):
    path.parent.mkdir(parents=True,exist_ok=True)
    path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item()
if isinstance(x,np.generic) else str(x))+'\n')

def write_csv(path,rows,fields=None):
```

```
    rows=list(rows);path.parent.mkdir(parents=True,exist_ok=True)
    with path.open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=fields or list(rows[0]) if rows else fields or
['empty']);w.writeheader();w.writerows(rows)

def check_protected():
    handoff=OUT/'handoff_hashes.json'
    if not handoff.exists():
        return 0  # Submission copy omits freeze-only protection metadata.
    known=json.loads(handoff.read_text())
    changed=[p for p,h in known.items() if not (ROOT/p).is_file() or
hashlib.sha256((ROOT/p).read_bytes()).hexdigest()!=h]
    assert not changed,changed
    return len(known)

def inputs():
    dates,actual,price=read_year();records=read_pv_forecasts(ROOT/'附件/附件3.xlsx')
    return dates,actual,price,records

def prototype():
    from prepare_q3 import prepare
    prepare();check_protected();dates,a,p,r=inputs();factory=FeatureFactory(a,r);rows=[]
    tic=time.perf_counter()
    for begin in (28,77,169,351):
        for conf in CANDIDATES:
            e=6000.;days=[]
            for d in range(begin,begin+4):

s,events=run_day(factory,d,e,p,conf,details=True);days.append(s);e=s['E'][-1]
            agg=summarize(stack_days(days),p,start=0)
            rows.append(dict(begin=dates[begin].isoformat(),config=conf.name,**agg))
        print('prototype continuous window',dates[begin],flush=True)
    write_csv(OUT/'experiments/prototype.csv',rows)

write_json(OUT/'experiments/prototype.json',dict(windows=4,days_each=4,candidates=len(CA
NDIDATES),physics=True,seconds=time.perf_counter()-tic))

def select_config(factory,day,known,price,candidates=CANDIDATES,updates=(1,2,3)):
    # 选参接口显式截断真实数据和预报档案，验证日每个阶段再按发行时间隔离。
    cutoff=datetime(2025,1,1)+timedelta(days=day)
    past=FeatureFactory(factory.actual[:day],{t:v for t,v in factory.records.items() if
t<cutoff})
    assert len(past.actual)==day and all(t<cutoff for t in past.records)
    begin=max(1,day-21);start=float(known[begin]);rows=[]
    for conf in candidates:
        e=start;cash=0.;h=0.
        for d in range(begin,day):

s,_=run_day(past,d,e,price,conf,updates=updates);cash+=s['cash'].sum();h+=s['H'].sum();e
=s['E'][-1]
        score=cash-terminal_value(price)*(e-start)

rows.append(dict(selection_day=day,begin=begin,end=day-1,config=conf.name,cash=float(cas
h),score=float(score),initial=start,terminal=float(e),emergency_kwh=float(h)))
    idx=min(range(len(rows)),key=lambda i:(rows[i]['score'],i))
    for i,row in enumerate(rows):row['selected']=i==idx
    # lru_cache是实例方法全局缓存，清理避免保存此前factory历史副本。
    past.feature.cache_clear();past.raw.cache_clear()
    return candidates[idx],rows

def
replay(factory,p,warmup,calendar,updates=(1,2,3),rule='A',stale=False,details=False):
    days=list(warmup);events=[];e=float(days[-1]['E'][-1])
    for d in range(31,len(factory.actual)):
        conf=calendar[d] if isinstance(calendar,list) else calendar
        s,ev=run_day(factory,d,e,p,conf,updates,rule,stale,details=details)
```

```
        days.append(s);events.extend(ev);e=s['E'][-1]
    return stack_days(days),events

def compute():
    check_protected();assert (OUT/'experiments/prototype.json').exists()
    import subprocess
    core=subprocess.run([sys.executable,str(OUT/'test_q3.py'),'--
core'],capture_output=True,text=True)
    (OUT/'experiments/core_tests.log').write_text(core.stdout+core.stderr)
    assert core.returncode==0,'核心测试未通过，拒绝正式全年运行'
    dates,a,p,r=inputs();f=FeatureFactory(a,r);tic=time.perf_counter()

days=[];warmup=[];known=[6000.];e=6000.;selection=[];events=[];calendar=[];conf=WARMUP
    for d in range(365):
        if d>=31 and dates[d].day==1:
            conf,rows=select_config(f,d,known,p);selection.extend(rows)
            print('monthly selection',dates[d],conf.name,flush=True)
            write_csv(OUT/'experiments/monthly_selection.csv',selection)
        s,ev=run_day(f,d,e,p,conf,details=True);days.append(s);events.extend(ev)
        if d<31:warmup.append(s)
        calendar.append(conf);e=float(s['E'][-1]);known.append(e)

formal=stack_days(days);np.savez_compressed(OUT/'experiments/formal_arrays.npz',**formal
)
    write_json(OUT/'experiments/calendar.json',[asdict(c) for c in calendar])
    write_csv(OUT/'voi_events.csv',
[dict(date=dates[v['day']].isoformat(),decision_time=f"{v['stage']*6}:00",**v) for v in
events if v['day']>=31])
    write_json(OUT/'summary.json',summarize(formal,p))

write_json(OUT/'experiments/run_metadata.json',dict(formal_completed_before_fixed_rankin
gs=True,formal_seconds=time.perf_counter()-tic))
    print('formal complete',summarize(formal,p)['total_cost_yuan'],flush=True)
    # 一旦正式轨迹完成才查看固定策略的全年事后排名。
    candidates=[];daily=[]
    for conf in CANDIDATES:
        ss,_=replay(f,p,warmup,conf)
        candidates.append(dict(config=conf.name,**summarize(ss,p)))

daily.extend(dict(date=dates[d].isoformat(),config=conf.name,cash=float(ss['cash']
[d].sum()),initial=float(ss['E'][d,0]),terminal=float(ss['E']
[d,-1]),emergency=float(ss['H'][d].sum())) for d in range(31,365))
        write_csv(OUT/'experiments/candidate_summary.csv',candidates)
        print('fixed diagnostic',conf.name,candidates[-1]['total_cost_yuan'],flush=True)
    write_csv(OUT/'experiments/candidate_daily.csv',daily)
    results=[]
    for mask in range(8):
        subset=tuple(k for k in (1,2,3) if mask & (1<<(k-1)))
        ss,_=replay(f,p,warmup,calendar,subset)
        name='0'+''.join('_'+str(k*6) for k in subset)
        np.savez_compressed(OUT/f'experiments/subset_{mask}.npz',**ss)
        results.append(dict(policy=name,mask=mask,**summarize(ss,p)))
        print('information subset',name,results[-1]['total_cost_yuan'],flush=True)
    ss,_=replay(f,p,warmup,calendar,stale=True)
    np.savez_compressed(OUT/'experiments/stale_arrays.npz',**ss)

results.append(dict(policy='old_forecast_same_adjustment_rights',mask=-1,**summarize(ss,
p)))
    write_csv(OUT/'experiments/information_ablation.csv',results)
    sensitivities=[]
    for rule in ('B','B_net'):
        ss,_=replay(f,p,warmup,calendar,rule=rule)
        np.savez_compressed(OUT/f'experiments/settlement_{rule}.npz',**ss)
        sensitivities.append(dict(policy='settlement_'+rule,**summarize(ss,p)))
        print('settlement sensitivity',rule,sensitivities[-1]
['total_cost_yuan'],flush=True)
```

```
    for horizon in ('T24','R48'):
        conf=Config(horizon=horizon)
        for subset in ((1,2),(1,2,3)):
            ss,_=replay(f,p,warmup,conf,updates=subset)
            sensitivities.append(dict(policy=horizon+('_with18' if 3 in subset else
'_without18'),**summarize(ss,p)))
    write_csv(OUT/'experiments/sensitivities.csv',sensitivities)
    from diagnostics_q3 import supplemental
    supplemental()

write_json(OUT/'experiments/run_metadata.json',dict(formal_completed_before_fixed_rankin
gs=True,total_seconds=time.perf_counter()-
tic,protected_files=check_protected(),configuration='monthly_past_only',posthoc_not_used
_for_selection=True))
    print('all experiments complete',time.perf_counter()-tic,flush=True)

if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--
prototype',action='store_true');parser.add_argument('--
compute',action='store_true');parser.add_argument('--
deliver',action='store_true');args=parser.parse_args()
    if (OUT/'freeze_manifest.json').exists():raise RuntimeError('Q3 已冻结，请在独立复现副本
移走清单后运行')
    if args.prototype:prototype()
    elif args.compute:compute()
    elif args.deliver:
        from report_q3 import deliver
        deliver()
    else:
        prototype()
        import subprocess
        subprocess.run([sys.executable,str(OUT/'test_q3.py'),'--core'],check=True)
        compute()
        from report_q3 import deliver
        deliver()
```

B17 code/Q3/test_q3.py

```
"""独立重算、手算、未来预报/实际反事实和全量交付回读。"""
import sys,json,csv,unittest
from pathlib import Path
from datetime import datetime,timedelta
from dataclasses import replace
ROOT=Path(__file__).resolve().parents[1];OUT=Path(__file__).resolve().parent;sys.path.in
sert(0,str(ROOT))
import numpy as np
from openpyxl import load_workbook
from microgrid_forecasts import read_pv_forecasts,interpolate_pv,released_curve
from microgrid_adjustment import settlement,adjustment_plan
from microgrid_online import
read_year,execute_day,validate_execution,PERIODS,merge_emergency,terminal_value
from q3_forecast import FeatureFactory,StageFeatures
from q3_model import Config,choose_stage,run_day,simulate_paths,CANDIDATES
from solve_q3 import check_protected

class CoreTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.dates,cls.a,cls.p=read_year();cls.r=read_pv_forecasts(ROOT/'附件/附件
3.xlsx');cls.f=FeatureFactory(cls.a,cls.r)

    def test_forecast_keys_dates_and_units(self):
        self.assertEqual(len(self.r),1460)
        for issue,rec in self.r.items():

self.assertEqual(rec.targets[0],issue+timedelta(hours=1));self.assertEqual(rec.targets[-
```

```
1],issue+timedelta(hours=24))

self.assertEqual(self.r[datetime(2025,1,1,18)].targets[-1],datetime(2025,1,2,18))
        self.assertEqual(PERIODS[36],'6:00-6:10');self.assertEqual(PERIODS[108],'18:00-
18:10')
        rec=self.r[datetime(2025,3,20,6)]
        for method in ('constant','linear','pchip'):
            v=interpolate_pv(rec,10.,method)

self.assertEqual(len(v),144);np.testing.assert_allclose(v[5::6]/6,np.array(rec.power)/
6,atol=1e-8)
            self.assertGreaterEqual(v.min(),0)

    def test_reject_future_release(self):
        now=datetime(2025,3,20,6)
        with
self.assertRaises(AssertionError):released_curve([self.r[now+timedelta(hours=6)]],now,0)

    def test_residual_targets_have_all_arrived(self):
        for day in (1,2,10,31,100):
            for k in range(4):
                f=self.f.feature(day,k)
                self.assertLessEqual(f.residual_latest_end,f.now)
                for j,d in enumerate(f.residual_origins):
                    lo=d*144+k*36
                    np.testing.assert_allclose(f.residual[j],self.f.flat[lo:lo+288]-
self.f.raw(d,k,'linear',0.,k))

    def test_publication_time_counterfactual(self):
        day=60;base=datetime(2025,1,1)+timedelta(days=day)
        for cutoff in (1,2,3):
            records=dict(self.r)
            for k in range(cutoff,4):

dt=base+timedelta(hours=6*k);rec=records[dt];records[dt]=replace(rec,power=tuple(np.arra
y(rec.power)*.01+5000))
            x,_=run_day(self.f,day,6000,self.p,Config())
            y,_=run_day(FeatureFactory(self.a,records),day,6000,self.p,Config())
            np.testing.assert_array_equal(x['versions'][:cutoff],y['versions'][:cutoff])
            np.testing.assert_array_equal(x['G'][:cutoff*36],y['G'][:cutoff*36])

    def test_actual_future_counterfactual_all_stage_decisions(self):
        day=60
        for cutoff in (0,36,72,108):
            a=self.a.copy();a[day,cutoff:,0]*=2;a[day,cutoff:,1]*=.01;a[day+1:]*=1.7
            x,_=run_day(self.f,day,6000,self.p,Config(gain=.5,bias=True))

y,_=run_day(FeatureFactory(a,self.r),day,6000,self.p,Config(gain=.5,bias=True))
            np.testing.assert_array_equal(x['versions'][:cutoff//36+1],y['versions']
[:cutoff//36+1])
            for key in ('C','D','H','W'):np.testing.assert_array_equal(x[key]
[:cutoff],y[key][:cutoff])
            self.assertGreater(np.abs(x['H'][cutoff:]-y['H'][cutoff:]).sum(),1.)

    def test_nonretroactivity_and_adjustment_identity(self):
        for rule in ('A','B','B_net'):
            s,_=run_day(self.f,80,6000,self.p,Config(),rule=rule)
            for k in (1,2,3):
                np.testing.assert_array_equal(s['versions'][k,:k*36],s['versions']
[k-1,:k*36])
                np.testing.assert_allclose(s['versions'][k],s['versions'][k-1]+s['U']
[k-1]-s['R'][k-1],atol=1e-8)
            self.assertFalse(np.any((s['U']>1e-7)&(s['R']>1e-7)))

    def test_settlement_hand_multiple_transactions(self):
        for rule,expected in [('A',165),('B',125),('B_net',95)]:
```

```
            g=np.array([100.]);base=g.copy();total=100.
            for new in (80.,110.,90.):

bill=settlement(g,np.array([new]),np.ones(1),rule,base);total+=bill['charge'].sum();g[:]
=new
            self.assertAlmostEqual(total,expected)
        for rule,expected in [('A',140),('B',120),('B_net',100)]:
            total=100.;g=np.array([100.]);base=g.copy()
            for new in (120.,100.):
                total+=settlement(g,np.array([new]),np.ones(1),rule,base)
['charge'].sum();g[:]=new
            self.assertAlmostEqual(total,expected)

    def synthetic(self,pv=0.,load=100.,initial=1200.,old=50.,rule='A',threshold=1e-6):
        pred=np.column_stack([np.full(288,load),np.full(288,pv)])
        f=StageFeatures(36,36,pred,np.zeros((0,288,2)),(),-1,np.zeros(288))
        return
choose_stage(f,initial,np.ones(144),Config(q=0),1,np.full(108,old),np.full(108,old),rule
,threshold)

    def test_synthetic_lower_pv_and_emergency_tradeoff(self):
        normal=self.synthetic(pv=50);low=self.synthetic(pv=0)

self.assertFalse(normal['accepted']);self.assertTrue(low['accepted']);self.assertGreater
(low['G'].sum(),normal['G'].sum())
        self.assertGreater(low['voi'],0)

    def test_synthetic_high_pv_no_refund_vs_refund(self):
        a=self.synthetic(pv=200);b=self.synthetic(pv=200,rule='B')

self.assertFalse(a['accepted']);self.assertTrue(b['accepted']);self.assertLess(b['G'].su
m(),a['G'].sum())

    def test_synthetic_small_value_high_threshold(self):
        s=self.synthetic(pv=49.999,threshold=1e6)

self.assertFalse(s['accepted']);np.testing.assert_array_equal(s['G'],np.full(108,50.))

    def test_synthetic_battery_boundaries(self):
        low=self.synthetic(initial=1200);high=self.synthetic(initial=10800)
        self.assertGreaterEqual(low['G'].sum(),high['G'].sum()-1e-6)
        excess=self.synthetic(pv=300,initial=10800)
        self.assertFalse(excess['accepted'])

    def test_lp_milp_adjustment_equivalence(self):
        f=self.f.feature(80,1);net=f.prediction[:108,0]-f.prediction[:108,1]
        for rule in ('A','B'):

a=adjustment_plan(net,self.p[36:],6000,np.full(108,400.),terminal_value(self.p),rule)

b=adjustment_plan(net,self.p[36:],6000,np.full(108,400.),terminal_value(self.p),rule,int
eger=True)
            self.assertAlmostEqual(a['objective'],b['objective'],places=5)
            self.assertFalse(np.any((a['C']>1e-6)&(a['D']>1e-6)))

    def test_path_execution_matches_q2(self):
        g=np.full(144,400.);h,e=simulate_paths(g,self.a[60:63],6000)
        for j in range(3):
            x=execute_day(g,self.a[60+j],6000)
            np.testing.assert_allclose(h[j],x['H']);self.assertAlmostEqual(e[j],x['E']
[-1])

    def test_no_update_identity_and_voi_gate(self):
        s,events=run_day(self.f,60,6000,self.p,Config(),updates=())
        for k in (1,2,3):np.testing.assert_array_equal(s['versions'][k],s['versions']
[0])
```

```
        self.assertEqual(s['U'].sum()+s['R'].sum(),0)
        s,events=run_day(self.f,60,6000,self.p,Config(scenario=True))
        for ev in events:
            self.assertAlmostEqual(ev['voi'],ev['keep_score']-ev['update_score'])
            if ev['accepted']:self.assertGreater(ev['voi'],1e-6)
            else:self.assertEqual(ev['increase_kwh']+ev['decrease_kwh'],0)

    def test_protected(self):self.assertEqual(check_protected(),0)

    def test_month_selection_future_counterfactual(self):
        from solve_q3 import select_config
        actual=self.a.copy();actual[31:,:,0]*=3;records=dict(self.r)
        for dt,rec in records.copy().items():
            if
dt>=datetime(2025,2,1):records[dt]=replace(rec,power=tuple(np.array(rec.power)*.1))
        candidates=(Config(q=0),Config(q=.65),Config(horizon='R48'))
        x,rx=select_config(self.f,31,[6000.]*32,self.p,candidates)
        y,ry=select_config(FeatureFactory(actual,records),31,
[6000.]*32,self.p,candidates)
        self.assertEqual(x,y);self.assertEqual(rx,ry)

    def test_scenario_execution_no_future_path_information(self):

g=np.full(144,400.);paths=self.a[60:62].copy();changed=paths.copy();changed[:,72:,0]*=3
        x,_=simulate_paths(g,paths,6000);y,_=simulate_paths(g,changed,6000)
        np.testing.assert_array_equal(x[:,:72],y[:,:72])

_,e1=simulate_paths(g[:72],paths[:,:72],6000);_,e2=simulate_paths(g[:72],changed[:,:72],
6000)
        np.testing.assert_array_equal(e1,e2)

    def test_T24_18_nextday_forecast_not_visible_to_objective(self):
        day=80;ff=self.f.feature(day,3);pred=ff.prediction.copy();pred[36:,1]*=20
        mutated=replace(ff,prediction=pred)
        old=np.full(36,400.)
        x=choose_stage(ff,6000,self.p,Config(),3,old,old)
        y=choose_stage(mutated,6000,self.p,Config(),3,old,old)
        np.testing.assert_array_equal(x['G'],y['G']);self.assertEqual(x['voi'],y['voi'])

class OutputTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.dates,cls.a,cls.p=read_year();cls.s=dict(np.load(OUT/'experiments/
formal_arrays.npz'))

    def test_all_physics_versions_costs(self):
        s=self.s
        for d in range(365):
            validate_execution(self.a[d],{k:s[k][d] for k in ('G','C','D','H','W','E')})
            np.testing.assert_allclose(s['cash'][d],s['versions']
[d,0]*self.p+s['up_cost'][d]+s['penalty'][d]-s['refund'][d]+5*self.p*s['H'][d])
            for k in (1,2,3):
                np.testing.assert_array_equal(s['versions'][d,k,:k*36],s['versions']
[d,k-1,:k*36])
                np.testing.assert_allclose(s['versions'][d,k],s['versions']
[d,k-1]+s['U'][d,k-1]-s['R'][d,k-1],atol=1e-7)
        np.testing.assert_array_equal(s['E'][1:,0],s['E'][:-1,-1])
        np.testing.assert_array_equal(s['G'],s['versions'][:,3])
        self.assertAlmostEqual(float(s['R'][31:].sum()),0,places=5)

    def test_excel_all_values(self):
        s=self.s;w=load_workbook(OUT/'result3.xlsx',read_only=True,data_only=True)
        self.assertEqual(w.sheetnames,['计划购电量','调整购电量','充放电量','紧急购电量'])
        for idx in (0,1):

rows=list(w.worksheets[idx].values);self.assertEqual(len(rows),335);self.assertEqual(lis
```

```
t(rows[0][1:145]),PERIODS)
            for d,row in enumerate(rows[1:],31):
                self.assertEqual(row[0].date(),self.dates[d]);g=s['versions'][d,0] if
idx==0 else s['G'][d]
                np.testing.assert_allclose(row[1:145],g,atol=1e-8,rtol=0)
                self.assertAlmostEqual(row[145],g.sum(),places=6)
                self.assertAlmostEqual(row[146],float(g@self.p) if idx==0 else
float(s['cash'][d].sum()),places=6)
        rows=list(w.worksheets[2].values)[1:];self.assertEqual(len(rows),334*6)
        for d in range(31,365):
            block=rows[(d-31)*6:(d-30)*6];self.assertEqual(block[0]
[0].date(),self.dates[d])
            for k,row in enumerate(block):
                self.assertEqual(row[1],f'{4*k}:00-{4*k+4}:00')
                self.assertAlmostEqual(row[2],s['C'][d,k*24:(k+1)*24].sum(),places=6)
                self.assertAlmostEqual(row[3],s['D'][d,k*24:(k+1)*24].sum(),places=6)
            self.assertAlmostEqual(block[0][5],s['E']
[d,0],places=6);self.assertAlmostEqual(block[1][5],s['E'][d,-1],places=6)
        events=merge_emergency(self.dates[31:],s['H'][31:]);got=[];date_=None
        for row in list(w.worksheets[3].values)[1:]:
            if row[0]:date_=row[0].date().isoformat()
            if row[1]!='无':got.append((date_,row[1],row[2]))
        self.assertEqual(len(events),len(got))
        for x,y in zip(events,got):

self.assertEqual((x['date'],x['period']),y[:2]);self.assertAlmostEqual(x['kwh'],y[2],pla
ces=6)
        w.close()

    def test_monthly_selection_only_past(self):
        with (OUT/'experiments/monthly_selection.csv').open(encoding='utf-8-sig') as
f:rows=list(csv.DictReader(f))
        groups=sorted({int(r['selection_day']) for r in
rows});self.assertEqual(len(groups),11)
        for d in groups:
            g=[r for r in rows if
int(r['selection_day'])==d];self.assertEqual(len(g),len(CANDIDATES))
            self.assertTrue(all(int(r['end'])==d-1 for r in g))
            selected=[r for r in g if
r['selected']=='True'];self.assertEqual(len(selected),1)
            self.assertAlmostEqual(float(selected[0]['score']),min(float(r['score']) for
r in g),places=5)

    def test_ledger_reconstructs_versions(self):
        with (OUT/'adjustment_ledger.csv').open(encoding='utf-8-sig') as
f:rows=list(csv.DictReader(f))
        self.assertEqual(len(rows),334*(108+72+36))
        for row in rows:
            day=(datetime.fromisoformat(row['decision_date']).date()-
self.dates[0]).days;k=int(row['stage']);t=int(row['index'])
            self.assertAlmostEqual(float(row['previous_commitment']),self.s['versions']
[day,k-1,t])
            self.assertAlmostEqual(float(row['new_commitment']),self.s['versions']
[day,k,t])
            u=float(row['increase_U']);r=float(row['decrease_R']);p=self.p[t]
            self.assertAlmostEqual(float(row['adjustment_charge']),1.5*p*u+.5*p*r)

    def test_alternative_settlement_and_physics_all_days(self):
        for rule in ('B','B_net'):
            s=dict(np.load(OUT/f'experiments/settlement_{rule}.npz'))
            np.testing.assert_array_equal(s['E'][1:,0],s['E'][:-1,-1])
            self.assertGreater(s['R'][31:].sum(),0)
            for d in range(31,365):
                validate_execution(self.a[d],{k:s[k][d] for k in
('G','C','D','H','W','E')})
                fees=np.zeros(144)
```

```
                for k in (1,2,3):
                    fees+=settlement(s['versions'][d,k-1],s['versions']
[d,k],self.p,rule,s['versions'][d,0])['charge']
                np.testing.assert_allclose(s['cash'][d],self.p*s['versions']
[d,0]+fees+5*self.p*s['H'][d],atol=1e-7)

    def test_reject_ten_minute_shift_and_missing_intervals(self):
        with (OUT/'dispatch_10min.csv').open(encoding='utf-8-sig') as
f:rows=list(csv.DictReader(f))[:144]
        labels=[r['period'] for r in rows]
        self.assertEqual(labels,PERIODS)
        self.assertNotEqual(labels[1:]+labels[:1],PERIODS)
        self.assertNotEqual(labels[1:],PERIODS)

if __name__=='__main__':
    core='--core' in sys.argv

suite=unittest.TestSuite([unittest.defaultTestLoader.loadTestsFromTestCase(CoreTests)])
    if not
core:suite.addTests(unittest.defaultTestLoader.loadTestsFromTestCase(OutputTests))
    result=unittest.TextTestRunner(verbosity=2).run(suite);sys.exit(not
result.wasSuccessful())
```

B18 code/Q4/audit_q4.py

```
"""Independent original-input/CSV/Decimal reconciliation and complete Excel readback."""
import subprocess,tempfile,shutil,time
from decimal import Decimal as D
from openpyxl import load_workbook
from q4_utils import *
from microgrid_online import PERIODS,merge_emergency

def close(x,y,tol=1e-6):np.testing.assert_allclose(x,y,rtol=0,atol=tol)
def audit_kind(kind,dates,a,p):
    rows=read_csv(OUT/f'q{kind}_dispatch_10min.csv');assert len(rows)==334*144
    def arr(k):return np.array([float(r[k]) for r in rows]).reshape(334,144)
    for day in range(334):
        part=rows[day*144:(day+1)*144]
        assert [r['date'] for r in part]==[str(dates[day+31])]*144
        assert [int(r['index']) for r in part]==list(range(144)) and [r['period'] for r
in part]==PERIODS

close(arr('load_kw')/6,a[31:,:,0]);close(arr('pv_kw')/6,a[31:,:,1]);close(arr('price_rea
lized'),p[31:],1e-12)
    g,c,d,h,w,e,ee=(arr(k) for k in
('G_effective','C_real','D_real','H_real','W_real','E_start','E_end'))
    balance=g+a[31:,:,1]+d+h-a[31:,:,0]-c-w;state=ee-e-.9*c+d/.9
    close(balance,0);close(state,0);close(ee.ravel()[:-1],e.ravel()[1:],1e-8)
    assert min(e.min(),ee.min())>=1200-1e-7 and max(e.max(),ee.max())<=10800+1e-7
    assert min(x.min() for x in (g,c,d,h,w))>=-1e-7
    assert max(c.max(),d.max())<=5000/6+1e-7 and not np.any((c>1e-7)&(d>1e-7))
    short=a[31:,:,0]-a[31:,:,1]-g
    assert np.all(c<=np.maximum(-short,0)+1e-7) and np.all(d<=np.maximum(short,0)+1e-7)
    close(h,np.maximum(short-d,0));close(w,np.maximum(-short-c,0))
    versions=np.stack([arr(k) for k in ('G0','G6','G12','G18')],axis=1)
    for k in (1,2,3):close(versions[:,k,:k*36],versions[:,k-1,:k*36],0)
    for k in range(4):close(g[:,k*36:(k+1)*36],versions[:,k,k*36:(k+1)*36],0)
    ledger=read_csv(OUT/'q43_adjustment_ledger.csv') if kind==43 else []
    assert len(ledger)==(334*216 if kind==43 else 0)
    if kind==42:close(versions,np.repeat(versions[:,:1],4,axis=1),0)
    commit={(r['date'],int(r['index'])):D(r['G0']) for r in
rows};seen=set();up=np.zeros_like(g);down=np.zeros_like(g)
    for r in ledger:

date=r['delivery_date'];t=int(r['index']);k=int(r['stage']);idx=(date,t);stamp=(date,t,k
)
```

```
        assert stamp not in seen;seen.add(stamp)
        assert date==r['decision_date'] and t>=k*36 and int(r['decision_hour'])==k*6
        old,new,u,v=(D(r[x]) for x in
('previous_commitment','new_commitment','increase_U','decrease_R'))
        assert abs(commit[idx]-old)<D('1e-6') and abs(new-old-u+v)<D('1e-6')
        assert u>=0 and v>=0 and min(u,v)<D('1e-7');commit[idx]=new
        day=next(j for j,x in enumerate(dates[31:]) if str(x)==date)
        close(float(r['price_realized']),p[day+31,t],1e-12)
        close(float(new),versions[day,k,t]);up[day,t]+=float(u);down[day,t]+=float(v)
        assert abs(D(r['adjustment_charge'])-
D(r['price_realized'])*(D('1.5')*u+D('.5')*v))<D('1e-6')
    for r in rows:assert abs(commit[(r['date'],int(r['index']))]-
D(r['G_effective']))<D('1e-6')

costs=dict(plan_cost=p[31:]*arr('G0'),up_cost=1.5*p[31:]*up,penalty=.5*p[31:]*down,refun
d=np.zeros_like(g),emergency_cost=5*p[31:]*h)
    cash=sum(costs.values());close(arr('cash_yuan'),cash)
    for k,x in costs.items():close(arr(k+'_yuan'),x)
    zero=D(0)
    dec=dict(plan_cost_yuan=sum((D(r['price_realized'])*D(r['G0']) for r in rows),zero),
      emergency_cost_yuan=sum((D(5)*D(r['price_realized'])*D(r['H_real']) for r in
rows),zero),
      up_cost_yuan=sum((D('1.5')*D(r['price_realized'])*D(r['increase_U']) for r in
ledger),zero),
      penalty_yuan=sum((D('.5')*D(r['price_realized'])*D(r['decrease_R']) for r in
ledger),zero))
    dec['total_cost_yuan']=sum(dec.values(),zero)
    summary=json.loads((OUT/f'q{kind}_summary.json').read_text())
    for k,x in dec.items():close(float(x),summary[k])
    daily=read_csv(OUT/f'q{kind}_daily_summary.csv');assert len(daily)==334
    close([float(r['total_cost_yuan']) for r in daily],cash.sum(1))
    merged=merge_emergency(dates[31:],h);saved=read_csv(OUT/f'q{kind}
_emergency_purchase_merged.csv');assert len(merged)==len(saved)
    for x,y in zip(merged,saved):assert x['date']==y['date'] and
x['period']==y['period'];close(x['kwh'],float(y['kwh']))
    wb=load_workbook(OUT/f'result4-{kind-40}.xlsx',data_only=True,read_only=True)
    template=load_workbook(ROOT/f'附件/附件5/result4-{kind-40}.xlsx',read_only=True)
    assert wb.sheetnames==template.sheetnames
    cells=0
    for name in wb.sheetnames:
        actual=list(wb[name].values);head=list(template[name].values)[0]
        if name in ('计划购电量','调整购电量'):
            assert list(actual[0][1:145])==PERIODS
            assert actual[0][0]==head[0] and actual[0][145:]==head[145:]
        else:assert actual[0]==head,(name,actual[0],head)
        for row in actual:
            assert not any(isinstance(v,str) and
v.startswith(('#REF!','#DIV/0!','#VALUE!','#NAME?','#N/A','#NUM!','#SPILL!')) for v in
row)
        if name in ('计划购电量','调整购电量'):
            assert len(actual)==335
            target=arr('G0') if name=='计划购电量' else g
            for j,row in enumerate(actual[1:]):
                assert row[0].date()==dates[j+31]
                close(row[1:145],target[j]);close(row[145],target[j].sum())
                fee=cash[j].sum() if kind==42 or name=='调整购电量' else
costs['plan_cost'][j].sum()
                close(row[146],fee);cells+=147
        elif name=='充放电量':
            assert len(actual)==2005
            for j in range(334):
                for b in range(6):
                    row=actual[1+j*6+b]
                    if b==0:assert row[0].date()==dates[j+31]
                    else:assert row[0] is None
                    assert row[1]==f'{b*4}:00-{(b+1)*4}:00'
```

```
                    close(row[2],c[j,b*24:(b+1)*24].sum());close(row[3],d[j,b*24:
(b+1)*24].sum())
                    if b<2:assert row[4]==('0:00' if b==0 else
'24:00');close(row[5],e[j,0] if b==0 else ee[j,-1])
                    else:assert row[4] is None and row[5] is None
                    cells+=6
        else:
            expected=[]
            for date in dates[31:]:
                group=[r for r in merged if r['date']==str(date)]
                expected.extend([(date if i==0 else None,r['period'],r['kwh']) for i,r
in enumerate(group)] if group else [(date,'无',0.)])
            assert len(actual)==len(expected)+1
            for row,x in zip(actual[1:],expected):
                assert (row[0].date() if row[0] else None)==x[0] and
row[1]==x[1];close(row[2],x[2]);cells+=3
    wb.close();template.close()
    selection=read_csv(OUT/f'experiments/q{kind}_monthly_selection.csv')
    assert len(selection)==26*11 and sum(r['selected']=='True' for r in selection)==11
    cal=json.loads((OUT/f'experiments/q{kind}_calendar.json').read_text())
    from q4_model import Config
    for r in selection:
        day=int(r['selection_day']);assert int(r['end'])==day-1 and
int(r['begin'])==day-21
        if r['selected']=='True':
            group=[v for v in selection if v['selection_day']==r['selection_day']]
            close(float(r['score']),min(float(v['score']) for v in group))
            assert Config(**cal[day]).name==r['config']
    formal=dict(np.load(OUT/f'experiments/q{kind}_formal.npz'));close(formal['E'][30,-
1],e[0,0])
    for k in ('G','C','D','H','W'):close(formal[k][31:],{'G':g,'C':c,'D':d,'H':h,'W':w}
[k])
    return dict(decimal_totals={k:str(v) for k,v in
dec.items()},intervals=len(rows),ledger_rows=len(ledger),excel_cells_checked=cells,balan
ce_max=float(abs(balance).max()),state_max=float(abs(state).max()))

def regressions():
    results={}
    for q in (1,2,3,4):
        cmd=[sys.executable,str(ROOT/f'Q{q}/test_q{q}.py')]
        r=subprocess.run(cmd,capture_output=True,text=True);(OUT/f'q{q}
_regression.log').write_text(r.stdout+r.stderr)
        results[f'q{q}_tests']=r.returncode;assert r.returncode==0,f'Q{q} tests failed'
    for q in (1,2,3):
        if not (ROOT/f'Q{q}/freeze_q{q}.py').exists():
            results[f'q{q}_freeze']='not_packaged';continue

r=subprocess.run([sys.executable,str(ROOT/f'Q{q}/freeze_q{q}.py')],capture_output=True,t
ext=True)
        (OUT/f'q{q}_freeze_check.log').write_text(r.stdout+r.stderr);results[f'q{q}
_freeze']=r.returncode;assert r.returncode==0
    # Frozen Q3 audit writes its own report: redirect a complete minimal read context to
a temporary copy.
    with tempfile.TemporaryDirectory(prefix='q4_old_audit_') as td:
        temp=Path(td)
        for item in ROOT.iterdir():
            if item.name in ('Q3','Q4','.git','.venv','.agents','.codex'):continue
            (temp/item.name).symlink_to(item,target_is_directory=item.is_dir())

shutil.copytree(ROOT/'Q3',temp/'Q3',ignore=shutil.ignore_patterns('__pycache__','node_mo
dules','.matplotlib','*.zip'))
        r=subprocess.run([sys.executable,'-c',"import sys;sys.path.insert(0,'Q3');from
audit_q3 import audit;audit(run_tests=False)"],cwd=temp,capture_output=True,text=True)

(OUT/'q3_transaction_audit.log').write_text(r.stdout+r.stderr);results['q3_transaction_a
udit']=r.returncode;assert r.returncode==0
```

```
    return results

def audit():
    tic=time.perf_counter();dates,a,fixed,p,records=read_inputs()
    kinds={str(k):audit_kind(k,dates,a,p) for k in
(42,43)};tests=regressions();protected=check_protected()

result=dict(status='PASS',results=kinds,regression_exit_codes=tests,protected_files=prot
ected,seconds=time.perf_counter()-tic)
    write_json(OUT/'validation.json',result)
    (OUT/'Q4 最终验收审计.md').write_text('# Q4 最终验收审计\n\n独立从原始附件、CSV 逐点流量与逐笔
调整账本重算，全部通过。summary 仅用于核算后交叉检查。
\n\n'+json.dumps(result,ensure_ascii=False,indent=2)+'\n\n两份 Excel 的所有正式日期、144 点计
划/有效承诺、每日费用、4 小时充放电、日首末 SOC 及紧急购电全部回读。263 个接手保护文件保持字节一致。Q3 交易
审计在临时副本执行，原有审计文件不改写。\n\n限制：未来价格未知、当期聚合可见、交付价结算为建模解释；E1 的
阶段 VOI 采用 E0 代理；334 日单市场测试不构成普适统计优势。哈希及数值验算不是外部独立认证。视觉核对另见
visual_review.json。\n')
    print('Q4 independent audit PASS',flush=True)
if __name__=='__main__':audit()
```

B19 code/Q4/export_workbooks.mjs

```
// Official templates populated from Python's verified numerical payload.
import fs from 'node:fs/promises';
import path from 'node:path';
import {fileURLToPath} from 'node:url';
import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
const dir=path.dirname(fileURLToPath(import.meta.url));
const template=process.argv.includes('--template');
const previewOnly=process.argv.includes('--preview-only');
for(const kind of [42,43]) {
 const filename=`result4-${kind-40}.xlsx`;
 const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(path.join(dir,
(previewOnly?'':`../附件/附件 5/`)+filename)));
 const out=path.join(dir,'previews',template?'template':`q${kind}`);await fs.mkdir(out,
{recursive:true});
 if(!template&&!previewOnly) {
  const data=JSON.parse(await fs.readFile(path.join(dir,`q${kind}
_workbook_payload.json`),'utf8'));
  for(const [name,rows] of Object.entries(data)) {
   const sheet=wb.worksheets.getItem(name);
   if(name==='充放电量'||name==='紧急购电量') {
    sheet.getUsedRange().unmerge();
    sheet.getUsedRange().clear({applyTo:'contents'});
   }
   const values=rows.map((r,i)=>r.map((v,j)=>j===0&&i>0&&typeof v==='string'&&/^2025-
\d\d-\d\d$/.test(v)?new Date(`${v}T00:00:00Z`):v));
   sheet.getRangeByIndexes(0,0,values.length,values[0].length).values=values;
   sheet.getRangeByIndexes(1,0,values.length-1,1).setNumberFormat('yyyy-mm-dd');

sheet.getRangeByIndexes(1,1,values.length-1,values[0].length-1).setNumberFormat('#,##0.0
000');

sheet.getRangeByIndexes(0,0,values.length,values[0].length).format.font={name:'Arial',si
ze:11};
   sheet.getRangeByIndexes(0,0,1,values[0].length).format.rowHeight=34;
   sheet.getRangeByIndexes(0,0,1,values[0].length).format.wrapText=true;
   sheet.getRangeByIndexes(0,0,values.length,1).format.columnWidth=15;
   sheet.freezePanes.freezeRows(1);
   if(name.includes('购电量')&&name!=='紧急购电量') {
    sheet.getRange('B1:EO335').format.columnWidth=17;
    sheet.getRange('EP1:EQ335').format.columnWidth=21;
    sheet.getRange('EP2:EP335').formulas=Array.from({length:334},(_,i)=>[`=SUM(B$
{i+2}:EO${i+2})`]);
   } else if(name==='充放电量') {
    sheet.getRange(`B1:F${rows.length}`).format.columnWidth=22;
```

```
    for(let d=0;d<334;d++)sheet.getRange(`A${2+d*6}:A${7+d*6}`).merge();
   } else {
    sheet.getRange(`B1:C${rows.length}`).format.columnWidth=24;
   }
  }
 }
 const ranges=[['计划购电量','A1:F7','plan'],['计划购电量','EN1:EQ7','plan_totals'],['充放电
量','A1:F13','battery'],['紧急购电量','A1:C13','emergency']];
 if(kind===43)ranges.push(['调整购电量','A1:F7','adjustment'],['调整购电
量','EN1:EQ7','adjustment_totals']);
 for(const [sheetName,range,name] of ranges) {
  const img=await wb.render({sheetName,range,scale:1.5,format:'png'});
  await fs.writeFile(path.join(out,`${template?kind+'_':''}${name}.png`),new
Uint8Array(await img.arrayBuffer()));
 }
 const inspection=await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|
#NAME\\?|#N/A|#NUM!',options:{useRegex:true,maxResults:30}});
 await fs.writeFile(path.join(out,`${kind}_inspection.ndjson`),inspection.ndjson);
 if(!template&&!previewOnly) {
  const result=await SpreadsheetFile.exportXlsx(wb);await
result.save(path.join(dir,filename));
 }
 console.log(template?'template':previewOnly?'preview':'export',filename);
}
```

B20 code/Q4/export_workbooks_public.py

```
"""Public Python exporter for the verified Q4 payload; no numerical policy changes."""
from pathlib import Path
from datetime import datetime
import json
from openpyxl import load_workbook
from openpyxl.styles import Font, Alignment
from openpyxl.utils import get_column_letter
OUT=Path(__file__).resolve().parent

def export():
    for kind in (42,43):
        name=f'result4-{kind-40}.xlsx'
        wb=load_workbook(OUT.parent/'附件/附件5'/name)
        payload=json.loads((OUT/f'q{kind}_workbook_payload.json').read_text())
        for title,rows in payload.items():
            ws=wb[title]
            for merged in list(ws.merged_cells.ranges):ws.unmerge_cells(str(merged))
            for row in ws:
                for cell in row:cell.value=None
            for i,row in enumerate(rows,1):
                for j,value in enumerate(row,1):
                    if i>1 and j==1 and isinstance(value,str) and len(value)==10:
                        value=datetime.strptime(value,'%Y-%m-%d')
                    cell=ws.cell(i,j,value)
                    cell.font=Font(name='Arial',size=11)
                    if i>1:cell.number_format='yyyy-mm-dd' if j==1 else '#,##0.0000'
                    if i==1:cell.alignment=Alignment(wrap_text=True)
            ws.row_dimensions[1].height=34
            ws.column_dimensions['A'].width=15
            for j in range(2,len(rows[0])
+1):ws.column_dimensions[get_column_letter(j)].width=17 if len(rows[0])>10 else 24
            ws.freeze_panes='A2'
            if title=='充放电量':
                for d in
range(334):ws.merge_cells(start_row=2+d*6,end_row=7+d*6,start_column=1,end_column=1)
        wb.properties.creator='CUMCM Team';wb.properties.lastModifiedBy='CUMCM Team'
        wb.save(OUT/name)
if __name__=='__main__':export()
```

B21 code/Q4/literature_q4.py

```
"""Reproducible source-to-method notes, based on the four local full texts."""
from q4_utils import *
def generate():
    source=ROOT/'参考资料/问题 4 相关论文/PDF 原文'
    write_json(OUT/'literature_sources.json',
{p.name:dict(sha256=digest(p),source=str(p.relative_to(ROOT))) for p in
sorted(source.glob('*.pdf'))})
    (OUT/'文献方法映射.md').write_text('''# 文献方法映射
```

以下依据本地四篇 PDF 正文的模型、实验和局限整理；PDF 页码包含封面，不把原文结果当成本题收益。原始 PDF 留在参考资料目录，literature_sources.json 记录输入哈希。

Zhang 等，2018，A Stochastic MPC based Approach to Integrated Energy Management in Microgrids

Sustainable Cities and Society 41, 349–362; DOI: https://doi.org/10.1016/j.scs.2018.05.044 。本地 Stochastic_MPC_Microgrids_2018.pdf，34 页作者稿。第 9 页及场景部分将净负载和价格误差分别建模，独立生成并组合概率；场景削减的两个步骤不等于两阶段非前视决策树。模型包含柴油机、燃料电池、需求侧和买卖电，采用随机滚动预调度与实际信息到达后的修正。

第 18–20 页的滚动控制和当步实际修正启发 Q4 的信息时序；本题不引入发电机、售电或需求响应。本文同步抽取负载/PV/价格整条残差，以保留它们的历史相关性，与原文独立组合有明确差别。第 29–30 页的比较来自约 4 日实验，不能迁移为本题节省百分比。原文风险中性期望费用不是 CVaR 模型；其日末库存设定也不覆盖本题连续跨日要求。本文只在有限共享 G 提案类内进行场景评价，不能借文献声称解得完整随机最优。

Hossain 等，2021，Energy management of community energy storage in grid-connected microgrid under uncertain real-time prices

Sustainable Cities and Society 66, 102658; DOI: https://doi.org/10.1016/j.scs.2020.102658 。本地 Community_Storage_Uncertain_Real_Time_Prices_2021_AAM.pdf 共 26 页。必须保留版本警告：机构封面标记接受稿，但内页题名为 Energy Management of Community Microgrids Considering Uncertainty using Particle Swarm Optimisation，并带 2020 年 6 月预印本及 NOT PEER-REVIEWED 水印，另列预印本 DOI 10.20944/preprints202006.0205.v1。因此本次依据的是所存正文，不能断言与期刊最终版本逐字一致。

正文用 PSO 预调度和实时修正、正态误差 Monte Carlo 及少量代表场景，示例约 48 小时。其小时提前价格通知、无功率限制的外网交换、相同购售价、40kWh 电池，均不同于本题。可借鉴“区分预测计划与实际执行”，不能照搬售电利润，也不能由其特定条件下负载/PV 不确定性简化推出本题无需联合风险。本题 5 倍紧急电价和调整费用直接改变这种结论。该文不提供采用 CVaR 或 DRO 的直接依据。

Alves 等，2023，Battery Storage Systems Control Strategies with Intelligent Algorithms in Microgrids with Dynamic Pricing

Energies 16(14), 5262; DOI: https://doi.org/10.3390/en16145262 。本地 Battery_Control_Dynamic_Pricing_2023.pdf 共 30 页，期刊排版正文。非线性自回归 ANN 使用 3 个滞后、15 个隐层节点，结合以 SOC、PV、负载和价格为输入的 Mamdani 模糊控制。实验采用法国 Nord Pool 小时价格和晴/阴日 24 小时示例。

可以借鉴“电池是否放电需考虑价格与库存”的执行层思想，因此设置固定 G 的 2/6 小时 MPC 候选；本文并未复现其 ANN/模糊系统。原文购售电和无本题调整费的设定不适用。第 22 页比较的期末库存存在显著差别，不能只比较现金就泛化方法优势；本文同时披露正式期首末 SOC 和库存价值校正。短日示例不构成本题年度样本外证据，也不证明复杂预测或控制必然优于简单方法。

Lago 等，2021，Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark

Applied Energy 293, 116983; DOI: https://doi.org/10.1016/j.apenergy.2021.116983 。本地 Forecasting_Day_Ahead_Electricity_Prices_2021.pdf 共 21 页，为 CC BY 期刊正文。第 2–9 页提出五市场各六年、末两年测试的开放基准，比较 LASSO 自回归 LEAR 和 DNN；本文 ridge 不是其 LEAR 复现，也没有复现该多市场基准。

第 10–13 页及第 17–18 页强调简单而强的基线、每日重估、测试与调参分离、公开代码、MAE/rMAE/RMSE 及显著性评估，反对只靠短样本和 MAPE 宣布胜出。这里采用昨日同刻、7 日均值、同星期混合、季节中位数和两种 ridge，月首仅已完成 21 日按真实费用选参。rMAE 分母明确取昨日基线，区别于文献优先建议的 7 日前基线。附件仅一年，1 月预运行后测试 334 日，未满足其至少一年测试和多市场建议，因此结论限定于本数据与策略类。费用比较是滚动样本外实测，不

是文献意义的普适显著性结论。

推导与实施归属

价格加权分位来自本题 pG+5p(N-G)+的直接一阶条件；CVaR 采用有限分布的标准辅助阈值定义，代码枚举原子精确求该有限分布值。二者不是上述论文已证明适用于本题的结论。储能 MPC、联合经验场景和价格更新必须接受同一信息集、同一结算和年度真实执行检验。未实施 DRO、GBDT 或 Conformal，不凭术语增添结论。
''')
if __name__=='__main__':generate()
```

### B22 code/Q4/prepare_q4.py

```
"""Generate Gate A/B evidence and locked protocol before optimization."""
from dataclasses import asdict
from datetime import datetime,timedelta,timezone
import numpy as np
from q4_utils import *
from q4_model import CANDIDATES

def prepare():
    writable();dates,a,fixed,p,records=read_inputs()

protocol=dict(created_utc=datetime.now(timezone.utc).isoformat(),candidates=[asdict(c)
for c in CANDIDATES],
        selection='monthly previous 21 completed days; common validation initial SOC;
cash minus historical terminal inventory proxy',
        official_period='2025-02-01/2025-12-31',startup='Jan 1 G=0 battery idle; Jan 2-
31 mean7 q=.8, independent Q42/Q43 SOC',
        price_information='strictly before stage; current price only for E1 current
aggregate action',
        risk='finite shared-G candidates, causal frozen feedback scenario evaluation; no
free scenario-specific future optimization',
        execution='E1 is independently tested execution challenger; stage proposal/VOI
uses E0 proxy and historical selection evaluates actual E1',
        scenarios='synchronized complete 48h residuals, <=30 origins, deterministic
evenly spaced subsample',
        price_floor=.001,ridge_penalty=10.,price_stage_decay_periods=12,
        sensitivity_predeclared=['Oracle-P today only','Oracle-NP','full same-boundary
LP','frozen Q2/Q3 repricing','E0/E1 paired',
          'joint price residual permutation','B/B_net','decision-time lock uses last
completed price','only0','stale_price','stale_pv','current-price lag','Q42 official
PV0'],
        optional_declined=['DRO','GBDT','conformal formal
predictor'],no_formal_results_seen=True)
    target=OUT/'experiments/protocol.json'
    if target.exists():
        old=json.loads(target.read_text());assert
old['candidates']==protocol['candidates']
    else:write_json(target,protocol)

stats=dict(shape=list(p.shape),count=p.size,min=float(p.min()),max=float(p.max()),mean=f
loat(p.mean()),std=float(p.std()),
      quantiles={str(q):float(np.quantile(p,q)) for q in
(0,.01,.05,.1,.25,.5,.75,.9,.95,.99,1)},

negative=int((p<0).sum()),zero=int((p==0).sum()),missing=int((~np.isfinite(p)).sum()),
      acf={str(lag):float(np.corrcoef(p.ravel()[:-lag],p.ravel()[lag:])[0,1]) for lag in
(1,6,36,144,1008)},
      mean_profile_fixed_mae=float(np.abs(p.mean(0)-
fixed).mean()),mean_profile_fixed_corr=float(np.corrcoef(p.mean(0),fixed)[0,1]),

adjacent_abs_jump_max=float(np.abs(np.diff(p.ravel())).max()),interday_same_slot_mae=flo
at(np.abs(np.diff(p,axis=0)).mean()),

max_mean_day=str(dates[int(p.mean(1).argmax())]),max_range_day=str(dates[int(np.ptp(p,ax
is=1).argmax())]),
```
```

```
      max_single_day=str(dates[int(np.unravel_index(p.argmax(),p.shape)[0])]))
    write_json(OUT/'price_statistics.json',stats)
    write_csv(OUT/'price_monthly.csv',[dict(month=m,mean=float(p[[d.month==m for d in
dates]].mean()),std=float(p[[d.month==m for d in dates]].std())) for m in range(1,13)])
    write_csv(OUT/'price_weekday.csv',[dict(weekday=k,mean=float(p[[d.weekday()==k for d
in dates]].mean())) for k in range(7)])
    write_csv(OUT/'price_profile.csv',
[dict(index=t,fixed=fixed[t],mean=p[:,t].mean(),median=np.median(p[:,t]),q10=np.quantile
(p[:,t],.1),q90=np.quantile(p[:,t],.9),std=p[:,t].std()) for t in range(144)])
    write_csv(OUT/'price_daily.csv',
[dict(date=str(d),mean=p[j].mean(),min=p[j].min(),max=p[j].max(),range=np.ptp(p[j])) for
j,d in enumerate(dates)])
    write_csv(OUT/'time_mapping.csv',[dict(date=str(d),index=t,interval_start=f'{t//6}:
{t%6*10:02}',interval_end=f'{(t+1)//6}:{(t+1)%6*10:02}',price=p[j,t]) for j,d in
enumerate(dates) for t in range(144)])
    docs={
    '题意与价格信息集审计.md':'''# 题意与价格信息集审计

## 原题明文
问题 4 在波动外网电价下重新计算问题 2、3，附件 4 给出全年电价。紧急购电是交易时刻电价 5 倍；Q3 调增 1.5 倍、调减 50%违约。没有明文规定 0 时公布全天价格，没有定义价格预报发布机制，也未明示调整下单时与交付时哪个锁价。

## 附件能够确认
附件 4 有 365 日×144 点实现价格及时间标签，不带发行时间或预测版本。附件 5 要求两套 334 日结果。附件 3 是光伏预报，不能作为电价预报。

## 继承 Q2/Q3 冻结口径
Q42 无官方 PV 预报、0 时普通计划当天固定；Q43 有 0/6/12/18 官方 PV，只有这些节点可修改未执行承诺。主 A 不退款，针对上一有效承诺逐次结算，按交付十分钟价格计费。G/H/C/D/W 母线侧 kWh，效率各.9，SOC1200–10800、功率 5000 kW，跨日连续。实时供需均值是既有反馈聚合假设。

## 本文新增解释
主方案按实时揭示解释：阶段前只知已完成价格，预测未来。计划和调整只用预测/场景价，账单统一用附件 4 实际交付价。6 时调整 18 时交付用 18 时预测计算 VOI，之后用 18 时实际价结算。E0 不读价格；E1 假设当前时段报价可用于当前反馈动作，但后续报价不可见；另检验仅上一完成区间价格可用。E1 只改变储能与缺口 H，不改变 G，不用紧急电充电。

## 存在歧义的替代解释
Oracle-P 当日完整价已知、负载/PV 仍预测，属于信息敏感性，不是唯一官方解释或严格全局下界。决策锁价敏感性使用决策前最后完成价格作为可观察代理；它不是题面给出的真实订单价格。Q42+0 时官方 PV 仅信息增强敏感性。B/B_net 继承 Q3 的独立替代结算，绝不混入正式 A。
''',
    '附件 4 结构审计.md':f'''# 附件 4 结构审计

实际读取并断言全年连续日期与附件 2 完全相等，144 个结束标签为 0:10 至 0:00+1，无重复、缺日、缺值或非有限值；全部为正。精确统计见 price_statistics.json。

价格最小{p.min():.4f}、最大{p.max():.4f}、均值{p.mean():.8f}、标准差{p.std():.8f}元/kWh。价格不乘 1/6。数据结束点解释继承既有假设，输出 time_mapping.csv 保存 52,560 条显式映射。

附件 1 与附件 4 平均日型 MAE={stats['mean_profile_fixed_mae']:.8f}，相关系数
={stats['mean_profile_fixed_corr']:.8f}。相邻十分钟最大绝对跳变
={stats['adjacent_abs_jump_max']:.4f}；相邻日同刻平均绝对变化
={stats['interday_same_slot_mae']:.8f}。同刻均值接近并不意味着每天峰谷完全相同。

price_profile.csv 记录均值、中位数、10/90 分位、波动率与附件 1；price_monthly/weekday/daily 分别保留月份、星期、每日均值峰谷差；ACF 记录 1/6/36/144/1008 步。描述统计使用全年仅作事后说明，不能用于正式预测参数和历史选择。
''',
    '候选价格预测模型.md':'''# 候选价格预测模型

P0 昨日同刻；P1 近 7 日同刻均值；P2 近期与最多 4 个同星期日各半；P3 过去 90 日工作日类型同刻中位数；P4 滚动 30 日 Ridge；P4E 扩展窗口 Ridge。全部在每个起点重新校准，训练目标严格早于起点。Ridge 使用上一日同刻、7 日前同刻、近 7 日同刻、日内一二谐波和周末标记，惩罚 10，不使用外部数据或未来真实负载/PV。少于 9 日退回组合预测。

阶段新增已完成最近一小时价格偏差，用两小时指数衰减更新后续曲线。预测下界.001 只为数值正性，不能由全年最小
```

值倒推。无历史日公开回退.5 且零 G 静置，不影响正式期直接预测。经验残差 50/80/90%区间作为诊断，不声称分布无关覆盖保证。不引入 GBDT、LSTM 或 Conformal 复杂训练，无新增机器学习依赖。
''',
    '候选数学模型.md':r'''# 候选数学模型

共同物理和信息约束
原题流量 G/H/C/D、状态 E，继承剩余能量 W；Q4 新增真实价 p、发行预测 p_hat、场景 p_s。G+PV+D+H=L+C+W，E'=E+.9C-D/.9，C,D 在[0,5000/6]，E 在[1200,10800]，G/H/W 非负，C 与 D 互斥。初值 6000，跨日相接。Q42 的 G0 仅可测于 0 时信息集；Q43 的 Gk 只可测于阶段 k 信息，过去 t<36k 不变，Gnew=Gold+U-R。

价格加权风险推导
单时段 J(G)=E[pG+5p(N-G)+]。连续内点 J'=E[p]-5E[p 1(N>G)]。最优价格加权尾部占总价格质量的 1/5；只有独立 p、N 时退化普通 0.8 分位。离散经验分布用按价格加权的逆 CDF；储能耦合使其成为解释性近似。候选 q=.65/.8 及基线.7、无裕度 0；margin=max(0,weighted_quantile(eL-ePV, max(.001,p_hat+ep),q))。

Q42 普通计划与 Q43 调整提案
0 时 LP：min sum(p_hat G)-vE_T。调整 LP：min sum(1.5 p_hat U+.5 p_hat R)+sum(前瞻正常购电)-vE_T；原计划沉没不重复收费。净负荷为预测加风险裕度。T24 到午夜；R48 到发行后 48h，仅当天成为承诺。v=当前发行未来 144 点最低 24 价均值/.9，为代理价值非收入。LP 提案沿用已有消循环/互斥理论，实际执行单向。

联合经验场景与 CVaR
不同 Q42/Q43 分别记录已发行的 48h 负载/PV/价格预测误差，完整实现后加入过去 30 条库。同步整条三变量残差一起抽取；4/8/12/20 条等时间距代表路径，不用聚类。以 q={0,.65,.8,.9}产生共享 G 提案，同一逐点反馈器揭示场景当前供需并算真实同型 H/E。场景号不能进入动作，不采用未来自由储能全知补救。

J=mean(未来可影响现金-v(E_T-E_0))+beta CVaR_alpha(未来可影响现金)。
CVaR=min_z[z+sum(pi*max(C_s-z,0))/(1-alpha)]，beta=0/.1/.2，alpha=.9/.95；离散小场景尾部可退化为最大场景，须披露。0 时现金含普通与紧急，阶段含增量调整、前瞻正常与紧急，不含沉没原计划。KEEP 和 UPDATE 用同一场景，VOI=Jkeep-Jupdate，含调整费不重复扣。有限提案策略类，不是原随机控制问题全局最优。

价格感知执行候选
固定 G，当前观测净负荷覆盖首点，剩余仅合法预测。每十分钟最小化 sum(5 p_hat H)-vE_T，约束充电不超过 max(G-N,0)，放电不超过 max(N-G,0)，H=N-G-D 在缺口段、盈余段 H=0，故不从紧急渠道充电。时域 12/36 点，只执行第一步，余量免费吸收不劣时直接返回等价反馈。阶段提案/VOI 保留 E0 评价代理，历史候选回放实际执行 E1，不能把代理 VOI 冒充 E1 精确价值。此明确近似降低嵌套计算。

选择与正式账本
26 个配置预注册于 protocol.json；每月首日仅前 21 个完成日、验证段共同已知起始 SOC、候选独立连续回放。评分现金-v_hist(末初 E 差)，v_hist 由选择前价格历史平均日型计算。月内配置固定、每日/阶段预测重校准。正式选择先于全年固定候选诊断。所有正式费用按交付实际 p：Q42=sum(pG0+5pH)；Q43 再加逐事件 sum(1.5pU+.5pR)，零退款。
''',
    '输出与费用口径.md':'''# 输出与费用口径

两文件保留附件 5 官方表名、非时间表头和示意布局。原模板 144 个区间从 0:10–0:20 至 0:00–0:10+1，存在整体偏移；继承 Q2/Q3 已冻结解释，修正标签为自然日 0:00–24:00，不循环平移数值。Q42 计划表=G0，全天量=sum G0，费用=普通+紧急。Q43 计划表=G0 及原始计划账单；调整表=最终有效 G 及完整实际现金费用，两费用不能相加。充放电/SOC 为实际执行；紧急按 H>1e-7 日内连续合并，无事件日写无/0。显示四位，内部双精度。额外预测、场景和账本另存 CSV，不添加官方 sheet。

总 W 含未用计划电，PV 弃用上下界 max(W-G,0)、min(W,PV)。终端价值只放算法对照，绝不写入官方现金。全年从 1 月 1 日 6000 独立推进，2 月起 334 日正式输出；日末不重置。
''',
    '方案审计.md':'''# 方案审计

继承物理、Q42/Q43 权限与 Q3 主 A。修正旧综合蓝图把 48h/MPC/CVaR 预设主用的表述。候选因果价格预测、加权风险、同步场景、CVaR 和 E1 以历史经济选择检验。DRO/GBDT 为可选，本次不引入，不能据此声称无效。

E1 收益采用同一计划算法 E0/E1 完整回放比较，状态路径可反馈影响翌日计划，因此是执行政策改变的总闭环效应。另固定已生成承诺的执行对照用于纯执行诊断。新价格/PV 屏蔽保留调整权限与 SOC 更新。Oracle-P 不是严格下界；Full Oracle 以各正式期首末 SOC 求 48,096 点确定性 LP 作为同边界下界。结算歧义单列敏感性，未来价格绝不随意混入正式信息。
''',
    '口径核验.md':'''# 口径核验

题面未唯一规定未来价格可知性、当前报价公布时点及调减退款。本次新增严格因果价格、当前 E1 可见报价假设；继承

交付时段结算、不退款、逐次上一承诺，替代实验分开。价格乘电量不乘时间，功率先除 6。Q42 无附件 3；Q43 最新已发布 PV。初始 6000，1 月预运行，2 月 SOC 各自继承。预报误差库目标结束<=发行时刻，当前报价与当前聚合供需只影响当前执行，不回写同刻之前形成的普通计划。
'''}
    for name,body in docs.items():(OUT/name).write_text(body)
    print('Gate A/B ready:',len(CANDIDATES),'predeclared candidates',flush=True)

if __name__=='__main__':prepare()
```

## B23  code/Q4/price_diagnostics.py

```
"""Pre-optimization causal forecast experiment and interval diagnostics."""
import os
os.environ.setdefault('MPLCONFIGDIR',str(__import__('pathlib').Path(__file__).parent/'.m
atplotlib'))
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from q4_utils import *
from q4_price_forecast import PriceFactory,MODELS

def price_diagnostics():
    writable();dates,a,fixed,p,records=read_inputs();factory=PriceFactory(p)
    outputs={};rows=[];intervals=[]
    baseline=np.array([factory.raw(d,0,'last')[:144] for d in range(365)])
    for model in MODELS:
        pred=np.array([[factory.raw(d,k,model) for k in range(4)] for d in range(365)])
        outputs[model]=pred
        # Same-day targets only: avoid mixing issuance lead times and cross-year missing
data.
        for k in range(4):
            t=k*36;n=144-t;y=p[31:,t:];z=pred[31:,k,:n];err=z-y
            masks=[('all','all',np.ones(y.shape,bool))]
            masks += [('month',str(m),np.broadcast_to(np.array([d.month==m for d in
dates[31:]])[:,None],y.shape)) for m in range(2,13)]
            masks +=
[('hour',str(h),np.broadcast_to(np.arange(t,144)[None]//6==h,y.shape)) for h in
range(t//6,24)]
            masks +=
[('lead_hour',str(h),np.broadcast_to(np.arange(n)[None]//6==h,y.shape)) for h in
range((n+5)//6)]
            threshold=np.quantile(y,.9)
            masks += [('price_group','top10',y>=threshold),
('price_group','bottom10',y<=np.quantile(y,.1))]
            for group,label,mask in masks:
                er=err[mask];yy=y[mask];zz=z[mask];bm=(baseline[31:,t:]-y)[mask]
                if not len(er):continue

rows.append(dict(model=model,stage=k,group=group,label=label,n=len(er),mae=float(np.abs(
er).mean()),rmse=float(np.sqrt(np.mean(er**2))),

bias=float(er.mean()),rmae=float(np.abs(er).mean()/max(np.abs(bm).mean(),1e-12)),
                    spike_miss_rate=float(np.mean(zz[yy>=threshold]<threshold)) if
np.any(yy>=threshold) else '',

diagnostic_weighted_mae=float(np.average(np.abs(er),weights=np.maximum(a[31:,t:,0]
[mask],1)))))
        coverage={level:[] for level in (.5,.8,.9)}
        for d in range(31,365):
            origins=range(max(1,d-31),d-1)
            r=np.array([p.ravel()[j*144:j*144+288]-pred[j,0] for j in origins])[:,:144]
            for level in coverage:

q=(1-level)/2;lo=np.maximum(.001,pred[d,0,:144]+np.quantile(r,q,axis=0));hi=np.maximum(.
001,pred[d,0,:144]+np.quantile(r,1-q,axis=0))
```
```

```
                residual=p[d]-lo;pinlo=np.maximum(q*residual,(q-1)*residual)
                residual=p[d]-hi;pinhi=np.maximum((1-q)*residual,-q*residual)
                coverage[level].append([np.mean((p[d]>=lo)&(p[d]<=hi)),np.mean(hi-
lo),np.mean((pinlo+pinhi)/2)])
        for level,vals in coverage.items():

v=np.mean(vals,0);intervals.append(dict(model=model,nominal=level,coverage=v[0],width=v[
1],pinball=v[2],method='rolling complete historical residual; no coverage guarantee'))
        print('forecast diagnostics',model,flush=True)
    np.savez_compressed(OUT/'experiments/price_forecast_arrays.npz',**outputs)

write_csv(OUT/'price_metrics.csv',rows);write_csv(OUT/'price_interval_metrics.csv',inter
vals)
    (OUT/'figures').mkdir(exist_ok=True)
    fig,ax=plt.subplots(2,2,figsize=(12,7),layout='constrained')
    for ax_,date_ in zip(ax.ravel(),('2025-03-20','2025-06-21','2025-09-23','2025-12-
21')):
        d=next(i for i,v in enumerate(dates) if str(v)==date_);x=np.arange(144)/6
        ax_.plot(x,p[d],label='Realized',color='black',lw=1)
        for model in ('last','mean7','ridge30'):ax_.plot(x,outputs[model]
[d,0,:144],label=model,lw=1)
        rr=np.array([p[j]-outputs['mean7'][j,0,:144] for j in range(d-30,d)])
        ax_.fill_between(x,np.maximum(.001,outputs['mean7']
[d,0,:144]+np.quantile(rr,.05,axis=0)),outputs['mean7']
[d,0,:144]+np.quantile(rr,.95,axis=0),alpha=.12)

ax_.set(title=date_,xlabel='Hour',ylabel='CNY/kWh');ax_.legend(fontsize=8,ncol=2)

fig.savefig(OUT/'figures/price_forecast_precheck.png',dpi=150);fig.savefig(OUT/'figures/
price_forecast_precheck.svg');plt.close(fig)

if __name__=='__main__':price_diagnostics()
```

B24 code/Q4/q4_execution_mpc.py

```
"""Fixed-commitment storage MPC. H never buys extra energy for charging."""
from functools import lru_cache
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix
from microgrid_online import feedback_step

@lru_cache(maxsize=144)
def state_matrix(n):
    t=np.arange(n)
    return coo_matrix((np.r_[-.9*np.ones(n),np.ones(n)/.9,-np.ones(n),np.ones(n)],
        (np.tile(t,4),np.r_[t,n+t,2*n+t,2*n+t+1])),shape=(n,3*n+1)).tocsr()

def mpc_step(g,net_forecast,price_forecast,energy,value=0.):
    """Current net/price passed explicitly; tail consists only of legal forecasts.
    For fixed G, charge <= surplus and discharge <= deficit. Min 5pH-vE_end.
    Current-period aggregate demand is an inherited feedback approximation.
    """
    g=np.asarray(g);net=np.asarray(net_forecast);p=np.asarray(price_forecast)
    assert g.shape==net.shape==p.shape and np.isfinite(p).all() and (p>0).all()
    r=g-net;n=len(g)
    # With free disposal and positive stored-energy value, absorbing surplus cannot
hurt.
    if r[0]>=0 or energy<=1200+1e-8:
        return feedback_step(g[0],net[0],0.,energy)
    surplus=np.minimum(np.maximum(r,0),5000/6)
    deficit=np.minimum(np.maximum(-r,0),5000/6)
    obj=np.r_[np.zeros(n),-5*p,np.zeros(n),-value]

lo=np.r_[np.zeros(2*n),np.full(n+1,1200.)];hi=np.r_[surplus,deficit,np.full(n+1,10800.)]
    lo[2*n]=hi[2*n]=float(np.clip(energy,1200,10800))
```

```
sol=linprog(obj,A_eq=state_matrix(n),b_eq=np.zeros(n),bounds=np.column_stack([lo,hi]),me
thod='highs')
    if not sol.success:raise RuntimeError(sol.message)
    c=max(0.,sol.x[0]);d=max(0.,min(sol.x[n],-r[0],5000/6,(energy-1200)*.9))
    h=max(0.,-r[0]-d);w=max(0.,r[0]-c)
    return c,d,h,w,energy+.9*c-d/.9
```

B25 code/Q4/q4_features.py

```
"""Q42 and Q43 feature sets and synchronized historical 48-hour residuals."""
from dataclasses import dataclass
from functools import lru_cache
import numpy as np
from q4_utils import *
from forecast_q2 import forecast
from q3_forecast import FeatureFactory
from q4_price_forecast import PriceFactory

@dataclass(frozen=True)
class Features:
    now:int
    source_issue:int
    prediction:np.ndarray
    price_forecast:np.ndarray
    residual:np.ndarray
    origins:tuple
    latest_end:int

class Factory:
    def __init__(self,actual,realized,records,fixed):
        self.actual=np.asarray(actual);self.flat=self.actual.reshape(-1,2)
        self.prices=PriceFactory(realized);self.pv=FeatureFactory(actual,records)
        self.fixed=fixed;self.records=records

    @lru_cache(maxsize=4000)
    def raw_net(self,kind,day,k,source):
        if kind==43:return self.pv.raw(day,k,'linear',0.,source)
        pred=forecast(self.actual[:day],day,'blend',3).reshape(-1,2) if day else
np.zeros((432,2))
        return pred[k*36:k*36+288].copy()

    @lru_cache(maxsize=1024)
    def feature(self,kind,day,k,model,stale_price=False,stale_pv=False):
        now=day*144+k*36;source=0 if stale_pv else k
        pred=self.raw_net(kind,day,k,source).copy()
        def pp(d):
            if model=='fixed':return np.tile(self.fixed,3)[k*36:k*36+288].copy()
            return self.prices.base(d,model).ravel()[k*36:k*36+288].copy() if
stale_price else self.prices.raw(d,k,model).copy()
        price=pp(day)
        origins=tuple(range(max(1,day-31),max(1,day-1)))
        rs=[]
        for d in origins:
            start=d*144+k*36;end=start+288
            assert end<=now
            rs.append(np.column_stack([self.flat[start:end]-
self.raw_net(kind,d,k,source),self.prices.flat[start:end]-pp(d)]))
        residual=np.asarray(rs).reshape(-1,288,3)
        return Features(now,day*144+source*36 if kind==43 else
day*144,pred,price,residual,origins,
                        origins[-1]*144+k*36+288 if origins else -1)
```

B26 code/Q4/q4_model.py

```
"""Finite policy optimization under causal price information; both official permission
sets."""
```

```
from dataclasses import dataclass,replace,asdict
import time
import numpy as np
from q4_utils import *
from microgrid_online import feedback_step,validate_execution,terminal_value
from microgrid_adjustment import adjustment_plan,settlement
from q3_model import free_plan,simulate_paths
from q4_execution_mpc import mpc_step
from q4_risk import weighted_quantile,joint_paths,cvar
from q4_price_forecast import MODELS

@dataclass(frozen=True)
class Config:
    price_model:str='mean7'
    q:float=.7
    risk:str='ordinary'
    scenarios:int=8
    alpha:float=.9
    beta:float=0.
    horizon:str='T24'
    execution:int=0 # number of ten-minute lookahead periods; zero is frozen feedback
    @property
    def name(self):
        return f'{self.price_model}_q{self.q:g}_{self.risk}_S{self.scenarios}
_a{self.alpha:g}_b{self.beta:g}_{self.horizon}_E{self.execution}'

# Locked before formal rolling optimization. Factorial explosion intentionally avoided.
CANDIDATES=tuple(Config(m,q) for m in MODELS for q in (.65,.8)) + (
    Config(q=0),Config(q=.7),Config(risk='weighted'),Config(q=.8,risk='weighted'),

Config(risk='joint',scenarios=4),Config(risk='joint'),Config(risk='joint',scenarios=12),
    Config(risk='joint',scenarios=20),Config(risk='joint',beta=.1),
    Config(risk='joint',beta=.2,alpha=.95),Config(horizon='R48'),

Config(execution=12),Config(execution=36),Config(price_model='ridge30',execution=36))
WARMUP=Config(q=.8)

def horizon(k,conf):return 144-36*k if conf.horizon=='T24' else 288

def risk_net(f,q,n,weighted=False):
    net=f.prediction[:n,0]-f.prediction[:n,1]
    margin=np.zeros(n)
    if q and len(f.residual):
        residual=f.residual[:,:n,0]-f.residual[:,:n,1]
        if weighted:
            weights=np.maximum(.001,f.price_forecast[None,:n]+f.residual[:,:n,2])
            margin=weighted_quantile(residual,weights,q)
        else:margin=np.quantile(residual,q,axis=0)
        margin=np.maximum(0,margin)
    return net+margin,margin

def
evaluate(g,paths,price_scenario,initial,value,old=None,original=None,rule='A',beta=0,alp
ha=.9):
    h,end=simulate_paths(g,paths,initial)
    m=len(g) if old is None else len(old)
    if old is None:fees=price_scenario@g
    else:
        # Forecast cash by scenario; sunk G0 is common and excluded from marginal risk.
        coeff=settlement(old,g[:m],np.ones(m),rule,original)['charge']
        fees=price_scenario[:,:m]@coeff+price_scenario[:,m:]@g[m:]
    cash=fees+(5*price_scenario*h).sum(1)
    score=float(np.mean(cash-value*(end-initial)))+beta*cvar(cash,alpha)
    return score,cash

def
```

```
choose(f,initial,conf,k,old=None,original=None,rule='A',independent=False,locked_price=N
one):
    tic=time.perf_counter();m=144-k*36;n=horizon(k,conf)
    assert f.now%144==k*36 and f.latest_end<=f.now and f.source_issue<=f.now
    p=f.price_forecast[:n];value=terminal_value(f.price_forecast[:144])
    if locked_price is not None:p=np.full(n,locked_price)
    if conf.risk=='joint':
        paths,ps,ix=joint_paths(f.prediction,p,f.residual,conf.scenarios,independent)
        if locked_price is not None:ps[:,:m]=locked_price
        qs=(0.,.65,.8,.9)
    else:
        _,margin=risk_net(f,conf.q,n,conf.risk=='weighted')
        path=f.prediction[:n].copy();path[:,0]+=margin
        paths=path[None];ps=p[None];ix=np.array([-1]);qs=(conf.q,)
    choices=[]
    if old is not None:
        full=old.copy()
        if n>m:
            _,end=simulate_paths(old,paths.mean(0)[None,:m],initial)
            net,_=risk_net(f,conf.q,n,conf.risk=='weighted')
            full=np.r_[old,free_plan(net[m:],p[m:],float(end[0]),value)['G']]

score,cash=evaluate(full,paths,ps,initial,value,old,original,rule,conf.beta,conf.alpha)
        choices.append((full,score,'KEEP'))
    for q in qs:
        net,_=risk_net(f,q,n,conf.risk=='weighted')
        sol=free_plan(net,p,initial,value) if old is None else
adjustment_plan(net,p,initial,old,value,rule,original)

score,cash=evaluate(sol['G'],paths,ps,initial,value,old,original,rule,conf.beta,conf.alp
ha)
        choices.append((sol['G'],score,f'q={q:g}'))
    best=min(range(len(choices)),key=lambda i:(choices[i][1],i))
    keep=choices[0][1] if old is not None else np.nan
    proposal=min(choices[1:],key=lambda v:v[1]) if old is not None else choices[best]
    voi=keep-proposal[1] if old is not None else np.nan
    if old is not None and voi<=1e-6:best=0
    g,score,label=choices[best]
    return
g[:m].copy(),dict(voi=voi,keep_score=keep,update_score=proposal[1],choice=label,
        accepted=old is not None and np.max(np.abs(g[:m]-
old))>1e-7,seconds=time.perf_counter()-tic,

now=f.now,source_issue=f.source_issue,residual_latest_end=f.latest_end,scenario_count=le
n(paths),score=score)

def
run_day(factory,kind,day,initial,conf,updates=(1,2,3),stale_price=False,stale_pv=False,

oracle='',rule='A',locked=False,independent=False,current_known=True,pv0=False):
    tic=time.perf_counter();a=factory.actual[day];realized=factory.prices.realized[day]
    s={k:np.zeros(144) for k in
('G','C','D','H','W','plan_cost','up_cost','penalty','refund','emergency_cost')}
    s['E']=np.zeros(145);s['E'][0]=initial
    versions=np.zeros((4,144));u=np.zeros((3,144));r=u.copy();g=np.zeros(144)
    price_versions=np.full((4,288),np.nan);net_versions=np.full((4,288,2),np.nan)
    events=[];last_features=None;last_start=0
    for k in range(4):
        t=k*36;previous=g.copy()
        active=k==0 or (kind==43 and k in updates)
        if active:
            fk=43 if pv0 and kind==42 else kind
            f=factory.feature(fk,day,k,conf.price_model,stale_price,stale_pv)
            if oracle:

pred=f.prediction.copy();price=f.price_forecast.copy();n=min(288,len(factory.flat)-
```

```
f.now)
                residual=f.residual.copy()
                # Oracle-P knows today's full prices only, cross-day tail remains
causal.
                if oracle in ('P','FULL'):
                    m=144-t;price[:m]=realized[t:]
                    residual[:,:m,2]=0 # revealed price carries no artificial scenario
error
                if oracle in ('NP','FULL'):
                    pred[:n]=factory.flat[f.now:f.now+n]
                    residual[:,:n,:2]=0
                f=replace(f,prediction=pred,price_forecast=price,residual=residual)
            last_features=f;last_start=t
            price_versions[k]=f.price_forecast;net_versions[k]=f.prediction
            issue_price=float(factory.prices.flat[f.now-1]) if f.now else .5
            if day==0:

decision=dict(voi=0.,keep_score=0.,update_score=0.,choice='startup',accepted=False,secon
ds=0.,now=f.now,source_issue=f.source_issue,residual_latest_end=f.latest_end,scenario_co
unt=0,score=0.)
            else:
                g[t:],decision=choose(f,float(s['E'][t]),conf,k,None if k==0 else
g[t:],None if k==0 else versions[0,t:],
                    rule,independent,issue_price if locked else None)
            if k==0:
                s['plan_cost']=g*(issue_price if locked else realized)
            else:
                price_bill=np.full(144-t,issue_price) if locked else realized[t:]
                bill=settlement(previous[t:],g[t:],price_bill,rule,versions[0,t:])
                u[k-1,t:]=bill['U'];r[k-1,t:]=bill['R']
                for key in ('up_cost','penalty','refund'):s[key][t:]+=bill[key]
                events.append(dict(day=day,stage=k,**decision,initial=float(s['E']
[t]),increase=float(bill['U'].sum()),

decrease=float(bill['R'].sum()),realized_adjustment=float(bill['charge'].sum())))
        versions[k]=g
        assert np.array_equal(g[:t],previous[:t])
        for j in range(t,t+36):
            s['G'][j]=g[j];e=s['E'][j]
            if day==0:
                rr=g[j]+a[j,1]-a[j,0];action=(0.,0.,max(-rr,0),max(rr,0),e)
            elif conf.execution:
                n=min(conf.execution,144-j);offset=j-last_start
                pred=last_features.prediction[offset:offset+n].copy()
                pred[0]=a[j] # current aggregate, never later actuals

pp=factory.prices.execution_curve(day,j,conf.price_model,last_features.price_forecast,la
st_start,n)
                pp[0]=realized[j] if current_known else
(factory.prices.flat[day*144+j-1] if day*144+j else .5)
                value=terminal_value(last_features.price_forecast[:144])
                action=mpc_step(g[j:j+n],pred[:,0]-pred[:,1],pp,e,value)
            else:action=feedback_step(g[j],*a[j],e)
            s['C'][j],s['D'][j],s['H'][j],s['W'][j],s['E'][j+1]=action
    validate_execution(a,s)
    s['emergency_cost']=5*realized*s['H'];s['cash']=s['plan_cost']+s['up_cost']
+s['penalty']-s['refund']+s['emergency_cost']

s.update(versions=versions,U=u,R=r,price_forecasts=price_versions,predictions=net_versio
ns,
             config=conf.name,seconds=time.perf_counter()-tic)
    return s,events

def stack(days):return {k:np.array([d[k] for d in days]) for k in days[0]}

def summarize(s,realized,actual,start=31,common_value=.5):
```

```
    x={k:v[start:] for k,v in
s.items()};p=realized[start:];a=actual[start:];cash=x['cash'].sum(1);e=x['E'];h=x['H']
    high=p>=np.quantile(p,.9);low=p<=np.quantile(p,.1);g=x['G'];w=x['W']
    r={k+'_yuan':float(x[k].sum()) for k in
('plan_cost','up_cost','penalty','refund','emergency_cost')}

r.update(total_cost_yuan=float(cash.sum()),daily_mean_yuan=float(cash.mean()),daily_p95_
yuan=float(np.quantile(cash,.95)),
        worst5_mean_yuan=float(np.sort(cash)[-
int(np.ceil(.05*len(cash))):].mean()),max_daily_yuan=float(cash.max()),
        terminal_adjusted_cost_yuan=float(cash.sum()-common_value*(e[-1,-1]-e[0,0])),
        initial_kwh=float(e[0,0]),terminal_kwh=float(e[-1,-
1]),energy_min=float(e.min()),energy_max=float(e.max()),
        plan_kwh=float(x['versions']
[:,0].sum()),effective_kwh=float(g.sum()),emergency_kwh=float(h.sum()),

emergency_periods=int((h>1e-7).sum()),emergency_days=int(np.any(h>1e-7,1).sum()),max_eme
rgency_kwh=float(h.max()),

high_price_emergency_kwh=float(h[high].sum()),increase_kwh=float(x['U'].sum()),decrease_
kwh=float(x['R'].sum()),
        adjustment_events=int(np.any(x['U']
+x['R']>1e-7,2).sum()),charge_kwh=float(x['C'].sum()),discharge_kwh=float(x['D'].sum()),
        waste_kwh=float(w.sum()),pv_curtail_lower_kwh=float(np.maximum(w-
g,0).sum()),pv_curtail_upper_kwh=float(np.minimum(w,a[:,:,1]).sum()),
        pv_utilization_lower=float(1-np.minimum(w,a[:,:,1]).sum()/a[:,:,1].sum()),
        pv_utilization_upper=float(1-np.maximum(w-g,0).sum()/a[:,:,1].sum()),
        lower_bound_states=int((e<=1200+1e-7).sum()),upper_bound_states=int((e>=10800-
1e-7).sum()),
        continuity_error=float(np.max(np.abs(e[1:,0]-e[:-1,-1]))),
        low_price_charge_share=float(x['C'][low].sum()/max(x['C'].sum(),1e-9)),
        high_price_discharge_share=float(x['D'][high].sum()/max(x['D'].sum(),1e-9)),
        grid_weighted_price=float((p*(g+h)).sum()/max((g+h).sum(),1e-9)),
        high_price_grid_share=float((g+h)[high].sum()/max((g+h).sum(),1e-9)),
        execution_seconds=float(x['seconds'].sum()) if 'seconds' in x else
0.,days=len(cash))
    return r
```

B27 code/Q4/q4_price_forecast.py

```
"""Price forecasts issued from a strict historical prefix, never a future realized
vector."""
from functools import lru_cache
import numpy as np

MODELS=('last','mean7','blend','seasonal','ridge30','ridge_expanding')

def daily_price(history,day,model):
    history=np.asarray(history,float)
    assert history.shape==(day,144)
    if day==0:return np.full((3,144),.5)
    last=history[-1];recent=history[-7:].mean(0)
    weekly=[]
    for j in range(3):
        ix=[d for d in range(day) if (day+j-d)%7==0][-4:]
        weekly.append(history[ix].mean(0) if ix else recent)
    if model=='last':out=np.tile(last,(3,1))
    elif model=='mean7':out=np.tile(recent,(3,1))
    elif model=='blend':out=.5*recent+.5*np.array(weekly)
    elif model=='seasonal':
        # Calendar season inferred only from observed days, 90-day rolling median.
        out=[]
        for j in range(3):
            ix=[d for d in range(max(0,day-90),day) if ((d+2)%7<5)==((day+j+2)%7<5)]
            out.append(np.median(history[ix],0) if ix else recent)
        out=np.array(out)
```

```
    elif model in ('ridge30','ridge_expanding'):
        if day<9:return .5*recent+.5*np.array(weekly)
        # Pooled same-slot autoregression; training targets all strictly before issue
day.
        def design(h,k):
            x=np.arange(144)/144*2*np.pi
            return
np.column_stack([np.ones(144),h[k-1],h[max(0,k-7)],h[max(0,k-7):k].mean(0),
                np.sin(x),np.cos(x),np.sin(2*x),np.cos(2*x),np.full(144,((k+2)%7)>=5)])
        begin=max(7,day-30) if model=='ridge30' else 7
        xs=np.concatenate([design(history,k) for k in range(begin,day)])
        ys=history[begin:day].ravel()
        reg=np.eye(xs.shape[1])*10.;reg[0,0]=1e-8
        coef=np.linalg.solve(xs.T@xs+reg,xs.T@ys)
        h=history.copy();out=[]
        for j in range(3):
            y=np.maximum(.001,design(h,day+j)@coef);out.append(y);h=np.vstack([h,y])
        out=np.array(out)
    else:raise ValueError(model)
    assert out.shape==(3,144) and np.isfinite(out).all()
    return np.maximum(.001,out)

class PriceFactory:
    def __init__(self,realized):
        self.realized=np.asarray(realized,float)
        self.flat=self.realized.ravel()

    @lru_cache(maxsize=2400)
    def base(self,day,model):return daily_price(self.realized[:day],day,model)

    @lru_cache(maxsize=12000)
    def raw(self,day,k,model):
        t=k*36;now=day*144+t
        base=self.base(day,model).ravel()
        pred=base[t:t+288].copy()
        if t:
            # Stage price update: last hour deviation; two-hour exponential decay.
            delta=np.mean(self.flat[now-6:now]-base[t-6:t])
            pred+=delta*np.exp(-np.arange(288)/12)
        return np.maximum(.001,pred)

    def execution_curve(self,day,t,model,stage_curve,stage_start,n):
        # Only prices ending strictly before t enter future forecast correction.
        now=day*144+t;j=t-stage_start
        p=stage_curve[j:j+n].copy()
        if j:
            count=min(6,j)
            delta=np.mean(self.flat[now-count:now]-stage_curve[j-count:j])
            p+=delta*np.exp(-np.arange(n)/12)
        return np.maximum(.001,p)
```

B28 code/Q4/q4_risk.py

```
"""Empirical joint scenarios, weighted quantiles and finite-sample CVaR."""
import numpy as np

def weighted_quantile(values,weights,q):
    v,w=np.asarray(values,float),np.asarray(weights,float)
    assert v.shape==w.shape and (w>=0).all() and np.all(w.sum(0)>0)

idx=np.argsort(v,axis=0,kind='stable');v=np.take_along_axis(v,idx,0);w=np.take_along_axi
s(w,idx,0)
    c=np.cumsum(w,0);i=np.argmax(c>=q*c[-1],axis=0)
    return v[i,np.arange(v.shape[1])]

def cvar(loss,alpha=.9,weights=None):
```

```
    x=np.asarray(loss,float);w=np.ones(len(x))/len(x) if weights is None else
np.asarray(weights,float)
    assert 0<=alpha<1 and np.isclose(w.sum(),1) and (w>=0).all()
    # Exact weighted discrete Rockafellar-Uryasev minimum, including fractional tail
atom.
    return float(min(z+np.dot(w,np.maximum(x-z,0))/(1-alpha) for z in x))

def joint_paths(prediction,price_forecast,residual,count=8,independent=False):
    n=len(price_forecast)
    if not len(residual):return
prediction[None,:n].copy(),price_forecast[None].copy(),np.array([-1])
    ix=np.unique(np.linspace(0,len(residual)-1,min(count,len(residual))).astype(int))
    r=residual[ix,:n].copy()
    if independent and len(ix)>1:r[:,:,2]=np.roll(r[:,:,2],1,axis=0)
    # One row index and target timeline shared by all three variables.
    return np.maximum(0,prediction[None,:n]
+r[:,:,:2]),np.maximum(.001,price_forecast[None]+r[:,:,2]),ix
```

B29 code/Q4/q4_utils.py

```
"""Q4 local paths, immutable input protection and serialization."""
from pathlib import Path
import csv, hashlib, json, sys
import numpy as np
ROOT=Path(__file__).resolve().parents[1]
OUT=Path(__file__).resolve().parent
for p in (ROOT,ROOT/'Q2',ROOT/'Q3'):
    if str(p) not in sys.path:sys.path.insert(0,str(p))

def digest(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as f:
        for b in iter(lambda:f.read(1048576),b''):h.update(b)
    return h.hexdigest()

def check_protected():
    snapshot=OUT/'pre_q4_workspace_hashes.json'
    if not snapshot.exists():
        return 0  # Submission copy omits freeze-only protection metadata.
    hashes=json.loads(snapshot.read_text())
    changes=[p for p,h in hashes.items() if not (ROOT/p).exists() or digest(ROOT/p)!=h]
    assert not changes,changes
    return len(hashes)

def writable():
    if (OUT/'freeze_manifest.json').exists():raise RuntimeError('Q4 frozen: use an
independent reproduction copy')
    check_protected()

def write_json(path,obj):
    path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
    path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item()
if isinstance(x,np.generic) else str(x))+'\n')

def write_csv(path,rows,fields=None):
    path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
    rows=list(rows)
    with path.open('w',encoding='utf-8-sig',newline='') as f:
        w=csv.DictWriter(f,fieldnames=fields or list(rows[0]) if rows else (fields or
['empty']))
        w.writeheader();w.writerows(rows)

def read_csv(path):
    with Path(path).open(encoding='utf-8-sig') as f:return list(csv.DictReader(f))

def read_inputs():
```

```
    from microgrid_online import read_year
    from microgrid_forecasts import read_pv_forecasts
    from microgrid_common import end_minutes
    from openpyxl import load_workbook
    dates,a,fixed=read_year(ROOT)
    wb=load_workbook(ROOT/'附件/附件4.xlsx',read_only=True,data_only=True)
    rows=list(wb.active.values);wb.close()
    assert [end_minutes(x) for x in rows[0][1:]]==list(range(10,1441,10))
    assert [r[0].date() for r in rows[1:]]==dates
    realized=np.array([r[1:] for r in rows[1:]],float)
    assert realized.shape==(365,144) and np.isfinite(realized).all() and
(realized>0).all()
    return dates,a,fixed,realized,read_pv_forecasts(ROOT/'附件/附件3.xlsx')
```

B30 code/Q4/report_q4.py

```
"""All CSV, scientific plots, paper and official workbook payload generation."""
import os,shutil,subprocess
from dataclasses import replace
os.environ.setdefault('MPLCONFIGDIR',str(__import__('pathlib').Path(__file__).parent/'.m
atplotlib'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
from q4_utils import *
from q4_model import Config,CANDIDATES
from q4_features import Factory
from q4_risk import joint_paths
from microgrid_online import PERIODS,merge_emergency,terminal_value

WANTED=('2025-03-20','2025-06-21','2025-09-23','2025-12-21')

def table(headers,rows):
    return '\n'.join(['|'+'|'.join(headers)+'|','|'+'|'.join(['---']*len(headers))+'|']+
['|'+'|'.join(str(x) for x in row)+'|' for row in rows])+'\n'
def fmt(x):return f'{float(x):,.4f}'

def exports(kind,dates,a,p,s):
    daily=[];dispatch=[];ledger=[];predrows=[];residualrows=[]
    for d in range(31,365):
        date=str(dates[d]);g0=s['versions'][d,0];g=s['G'][d]
        daily.append(dict(date=date,config=s['config'][d],initial_kwh=s['E']
[d,0],terminal_kwh=s['E'][d,-1],
            **{key+'_yuan':s[key][d].sum() for key in
('plan_cost','up_cost','penalty','refund','emergency_cost')},
            total_cost_yuan=s['cash']
[d].sum(),plan_kwh=g0.sum(),effective_kwh=g.sum(),emergency_kwh=s['H'][d].sum(),
            emergency_periods=int((s['H'][d]>1e-7).sum()),increase_kwh=s['U']
[d].sum(),decrease_kwh=s['R'][d].sum(),
            price_mean=p[d].mean(),price_forecast_mae=np.abs(s['price_forecasts']
[d,0,:144]-p[d]).mean()))
        for t in range(144):

row=dict(date=date,index=t,period=PERIODS[t],load_kw=a[d,t,0]*6,pv_kw=a[d,t,1]*6,price_r
ealized=p[d,t],
                price_forecast_0=s['price_forecasts'][d,0,t],G0=g0[t],G6=s['versions']
[d,1,t],G12=s['versions'][d,2,t],G18=s['versions'][d,3,t],G_effective=g[t],
                C_real=s['C'][d,t],D_real=s['D'][d,t],H_real=s['H'][d,t],W_real=s['W']
[d,t],E_start=s['E'][d,t],E_end=s['E'][d,t+1],
                **{key+'_yuan':s[key][d,t] for key in
('plan_cost','up_cost','penalty','refund','emergency_cost','cash')})
            dispatch.append(row)
        for k in range(4 if kind==43 else 1):
            n=144-k*36;now=d*144+k*36
            for j in range(n):
```

```
                t=k*36+j

predrows.append(dict(kind=kind,issue_date=date,issue_hour=k*6,training_last_index=now-1,
target_date=date,index=t,lead_minutes=(j+1)*10,
                     price_forecast=s['price_forecasts']
[d,k,j],price_realized=p[d,t],load_forecast_kw=s['predictions']
[d,k,j,0]*6,pv_forecast_kw=s['predictions'][d,k,j,1]*6))

residualrows.append(dict(kind=kind,issue_index=now,target_end_index=d*144+t+1,lead=j+1,
                     load_error=a[d,t,0]-s['predictions'][d,k,j,0],pv_error=a[d,t,1]-
s['predictions'][d,k,j,1],
                     price_error=p[d,t]-s['price_forecasts']
[d,k,j],available_after=d*144+t+1))
                if k:
                    u=s['U'][d,k-1,t];r=s['R'][d,k-1,t]

ledger.append(dict(decision_date=date,stage=k,decision_hour=k*6,delivery_date=date,index
=t,period=PERIODS[t],
                        previous_commitment=s['versions']
[d,k-1,t],new_commitment=s['versions'][d,k,t],increase_U=u,decrease_R=r,
                        price_forecast=s['price_forecasts']
[d,k,j],price_realized=p[d,t],up_cost=1.5*p[d,t]*u,penalty=.5*p[d,t]*r,refund=0,
                        adjustment_charge=p[d,t]*(1.5*u+.5*r)))
    write_csv(OUT/f'q{kind}_daily_summary.csv',daily);write_csv(OUT/f'q{kind}
_dispatch_10min.csv',dispatch)
    write_csv(OUT/f'q{kind}_price_forecasts.csv',predrows);write_csv(OUT/f'q{kind}
_price_residuals.csv',residualrows)
    if ledger:write_csv(OUT/f'q{kind}_adjustment_ledger.csv',ledger)
    merged=merge_emergency(dates[31:],s['H'][31:]);write_csv(OUT/f'q{kind}
_emergency_purchase_merged.csv',merged)
    write_csv(OUT/f'q{kind}_warmup_january.csv',[dict(date=str(dates[d]),initial=s['E']
[d,0],terminal=s['E'][d,-1],cash=s['cash'][d].sum()) for d in range(31)])
    selected=[r for r in daily if r['date'] in WANTED]
    write_csv(OUT/f'selected_days/q{kind}_summary.csv',selected)
    for date in WANTED:
        d=next(i for i,v in enumerate(dates) if str(v)==date)
        write_csv(OUT/f'selected_days/q{kind}_{date}_table1.csv',
[dict(period=PERIODS[t],G0=s['versions'][d,0,t],effective=s['G'][d,t]) for t in
(60,72,84,96,108,120)])
        write_csv(OUT/f'selected_days/q{kind}_{date}_table2.csv',
[dict(period=f'{j*4}:00-{(j+1)*4}:00',charge=s['C'][d,j*24:
(j+1)*24].sum(),discharge=s['D'][d,j*24:(j+1)*24].sum()) for j in range(6)])
        write_csv(OUT/f'selected_days/q{kind}_{date}_dispatch.csv',[r for r in dispatch
if r['date']==date])
    payload={};head=['日期\\时间']+PERIODS+['全天购电量','全天购电费']
    payload['计划购电量']=[head]+[[str(dates[d])]+s['versions'][d,0].tolist()+
[float(s['versions'][d,0].sum()),float(s['cash'][d].sum() if kind==42 else
s['plan_cost'][d].sum())] for d in range(31,365)]
    if kind==43:payload['调整购电量']=[head]+[[str(dates[d])]+s['G'][d].tolist()+
[float(s['G'][d].sum()),float(s['cash'][d].sum())] for d in range(31,365)]
    payload['充放电量']=[['日期','时间段','充电量','放电量','时刻','储电量']]
    for d in range(31,365):
        for j in range(6):payload['充放电量'].append([str(dates[d]) if j==0 else
None,f'{j*4}:00-{(j+1)*4}:00',float(s['C'][d,j*24:(j+1)*24].sum()),float(s['D'][d,j*24:
(j+1)*24].sum()),'0:00' if j==0 else '24:00' if j==1 else None,float(s['E'][d,0 if j==0
else -1]) if j<2 else None])
    payload['紧急购电量']=[['日期','购电时间段','购电量']]
    for date in dates[31:]:
        group=[r for r in merged if r['date']==str(date)]
        if group:
            for j,r in enumerate(group):payload['紧急购电量'].append([str(date) if j==0
else None,r['period'],r['kwh']])
        else:payload['紧急购电量'].append([str(date),'无',0.])
    write_json(OUT/f'q{kind}_workbook_payload.json',payload)
    return daily,merged
```

```
def figures(kind,dates,a,p,s,daily):
    out=OUT/'figures';out.mkdir(exist_ok=True)
    def save(fig,name):

fig.savefig(out/f'{name}.png',dpi=145);fig.savefig(out/f'{name}.svg');plt.close(fig)
    extreme={int(p[31:].mean(1).argmax())+31,int(np.ptp(p[31:],axis=1).argmax())
+31,int(np.argmax(s['cash'][31:].sum(1)))+31}
    extreme.add(int(np.unravel_index(p[31:].argmax(),p[31:].shape)[0])+31)
    extreme.add(next(i for i,d in enumerate(dates) if str(d)==max(daily,key=lambda
r:r['price_forecast_mae'])['date']))
    chosen={i for i,d in enumerate(dates) if str(d) in WANTED}|extreme
    for d in sorted(chosen):

x=np.arange(144)/6;fig,axes=plt.subplots(4,1,figsize=(11,10),sharex=True,layout='constra
ined')
        axes[0].plot(x,p[d],color='black',label='Realized
price');axes[0].plot(x,s['price_forecasts'][d,0,:144],label='00:00
forecast');axes[0].set_ylabel('CNY/kWh')
        if kind==43:
            for k in (1,2,3):axes[0].plot(x[k*36:],s['price_forecasts'][d,k,:144-
k*36],label=f'{6*k:02}:00 forecast',lw=.7)
        axes[1].plot(x,(a[d,:,0]-a[d,:,1])*6,label='Actual net load',color='black',lw=1)
        axes[1].step(x,s['versions'][d,0]*6,label='Original
G0',lw=.8);axes[1].step(x,s['G'][d]*6,label='Effective G',lw=.9)
        axes[1].fill_between(x,0,s['H']
[d]*6,alpha=.4,label='Emergency');axes[1].set_ylabel('kW')
        axes[2].plot(np.arange(145)/6,s['E'][d],label='Stored
energy',color='#176b63');axes[2].axhline(1200,color='gray',ls=':');axes[2].axhline(10800
,color='gray',ls=':');axes[2].set_ylabel('kWh')
        axes[3].plot(x,np.cumsum(s['cash'][d]),label='Cumulative
cash',color='#923b36');axes[3].set(ylabel='CNY',xlabel='Hour')
        for ax in axes:ax.legend(fontsize=8,ncol=3);ax.grid(alpha=.15)
        fig.suptitle(f'Q4-{kind-40} / {dates[d]}');save(fig,f'q{kind}_{dates[d]}')
    fig,ax=plt.subplots(figsize=(10,4),layout='constrained')
    for key,label in [('plan_cost','Plan'),('up_cost','Increase'),
('emergency_cost','Emergency')]:
        vals=[sum(s[key][d].sum() for d in range(31,365) if dates[d].month==m) for m in
range(2,13)]
        ax.plot(range(2,13),np.array(vals)/1e6,'o-',label=label)
    ax.set(xlabel='Month',ylabel='Million CNY',title=f'Q4-{kind-40} monthly
cash');ax.legend();save(fig,f'q{kind}_monthly_costs')
    candidates=read_csv(OUT/f'experiments/q{kind}_candidate_summary.csv')
    fig,ax=plt.subplots(figsize=(8,5),layout='constrained')
    for r in candidates:
        joint='_joint_' in
r['config'];ax.scatter(float(r['daily_mean_yuan']),float(r['worst5_mean_yuan']),c='#ae47
41' if joint else '#27756a',s=35)
    ax.set(xlabel='Mean daily cash (CNY)',ylabel='Worst 5% mean daily cash
(CNY)',title=f'Q4-{kind-40} fixed candidates (post-hoc diagnostics)');save(fig,f'q{kind}
_risk_pareto')
    write_json(OUT/f'experiments/q{kind}
_extreme_days.json',dict(max_mean_price=str(dates[np.argmax(p[31:].mean(1))
+31]),max_range_price=str(dates[np.argmax(np.ptp(p[31:],axis=1))
+31]),max_single_price=str(dates[np.unravel_index(p[31:].argmax(),p[31:].shape)
[0]+31]),max_forecast_error=max(daily,key=lambda r:r['price_forecast_mae'])
['date'],max_cost=max(daily,key=lambda r:r['total_cost_yuan'])['date']))

def documents(all_results,all_daily,all_merged):
    summaries={k:json.loads((OUT/f'q{k}_summary.json').read_text()) for k in (42,43)}
    comparisons={k:json.loads((OUT/f'experiments/q{k}_comparisons.json').read_text())
for k in (42,43)}

metrics=read_csv(OUT/'price_metrics.csv');stats=json.loads((OUT/'price_statistics.json')
.read_text())
    text='# 问题 4：波动电价下的因果购电与滚动调整\n\n'
    text+='## 1 递进与结论范围\n\n 原题要求分别重算问题 2、3；本文继承已冻结的物理与权限，增加因果价格
```

预测。题面没有明确全天价预先公布，因此未来未知是本文新增解释，Oracle-P 为替代信息条件。所有正式现金按附件 4 交付十分钟实际价结算。模型是预注册有限策略类中的历史经济选择，不宣称全局随机最优。\n\n'
    delta42=summaries[42]['total_cost_yuan']-comparisons[42]['frozen_repriced']
['total_cost_yuan']
    delta43=summaries[43]['total_cost_yuan']-comparisons[43]['frozen_repriced']
['total_cost_yuan']
    text+=f'关键限制：正式 Q4-2 比冻结 Q2 动态重计价多支出{delta42:,.4f}元；正式 Q4-3 比冻结 Q3 动态重计价费用变化为{delta43:,.4f}元（负值为节省）。因此不能宣称新增价格预测在两问都创造年度收益，更不能宣称复杂方法普遍优越。正式采用的是事前锁定候选、月首历史经济选择规则，未依据全年已知结果回填最优模型。\n\n'
    text+='## 2 价格数据与样本外预测\n\n'+(OUT/'附件 4 结构审计.md').read_text().split('\n',1)
[1]+'\n'
    text+=(OUT/'候选价格预测模型.md').read_text().split('\n',1)[1]+'\n'
    text+=table(['价格模型','0 时 MAE','RMSE','rMAE（昨日）','bias'],
[[r['model'],fmt(r['mae']),fmt(r['rmse']),fmt(r['rmae']),fmt(r['bias'])] for r in
metrics if r['stage']=='0' and r['group']=='all'])
    text+='rMAE 分母是同一测试点上的昨日同刻基线 MAE，不是 Lago 文中优先推荐的 7 日前基线。高价分组阈值用于事后诊断，不进入预测。概率区间基于已完成历史误差，不采用 Conformal，不保证覆盖率。详细月份、时刻、时距和高低价分组见 price_metrics.csv。\n\n'
    text+=table(['预测器','名义覆盖','实际覆盖','平均宽度','pinball'],
[[r['model'],r['nominal'],fmt(r['coverage']),fmt(r['width']),fmt(r['pinball'])] for r in
read_csv(OUT/'price_interval_metrics.csv')])
    text+='![四季价格预测](figures/price_forecast_precheck.png)\n\n## 3 数学模型、价格加权风险、场景与执行\n\n'
    text+=(OUT/'候选数学模型.md').read_text().split('\n',1)[1]
    text+='\n## 4 正式历史选择与全年费用\n\n'
    text+=table(['指标','Q4-2','Q4-3'],[[key,fmt(summaries[42][key]),fmt(summaries[43]
[key])] for key in
('plan_cost_yuan','up_cost_yuan','penalty_yuan','emergency_cost_yuan','total_cost_yuan',
'terminal_adjusted_cost_yuan','emergency_kwh','emergency_periods','increase_kwh','decrea
se_kwh','initial_kwh','terminal_kwh','worst5_mean_yuan','high_price_emergency_kwh')])
    saving=summaries[42]['total_cost_yuan']-summaries[43]['total_cost_yuan']
    text+=f'Q4-3 相对 Q4-2 现金节省{saving:,.4f}元，比例{saving/summaries[42]
["total_cost_yuan"]:.4%}。二者信息与计划调整权限不同，本次正式期初库存分别为{summaries[42]
["initial_kwh"]:.4f}和{summaries[43]["initial_kwh"]:.4f}kWh，期末库存见表，不能全部解释为单一预报价值。终端调整使用 1 月已知价格平均日型最低 24 点均值/.9，属于辅助比较，不是实际收入。\n\n'
    for kind in (42,43):
        selected=[r for r in read_csv(OUT/f'experiments/q{kind}_monthly_selection.csv')
if r['selected']=='True']
        text+=f'### Q4-{kind-40}月度选择\n\n'+table(['月首日索引','配置','历史起止','验证评分'],[[r['selection_day'],r['config'],r['begin']+'–'+r['end'],fmt(r['score'])] for r in
selected])
        text+='配置名 E0 为冻结反馈，E12/E36 为 2/6 小时储能 MPC；q=0 是无风险裕度。阶段提案和 VOI 使用 E0 代理，E1 是否采用由完整真实执行历史费用检验。\n\n'
        comp=comparisons[kind];formal=summaries[kind]['total_cost_yuan']
        text+=f'### Q4-{kind-40}对照与消融\n\n'+table(['对照','真实总费用','相对正式多支出','期初 E','末 E'],[[name,fmt(r['total_cost_yuan']),fmt(r['total_cost_yuan']-
formal),fmt(r.get('initial_kwh',0)),fmt(r.get('terminal_kwh',0))] for name,r in
comp.items()])
        text+=f'冻结 Q{kind-40}按附件 4 重计价与正式差额为{comp["frozen_repriced"]
["total_cost_yuan"]-formal:,.4f}元。该基准保持原 G/U/R/C/D/H 轨迹，绝不反向改写 Q2/Q3。Oracle-P 保持负载/PV 预测、只揭示当天完整价格，差额不保证非负，不能称为严格下界。Full Oracle 放松信息且以正式期首末 SOC 求全 48,096 点 LP，才是相同边界的费用下界。\n\n'
        text+='Oracle-NP 仅揭示负载与 PV（最长 48 小时），价格仍因果预测；两个单类 Oracle 均将已揭示变量的场景残差置零，不再给确定变量添加人工不确定性。它们沿正式选定配置日历重放，而非重新优化全部算法。决策锁价把当时最近已完成价格用于未来普通交易，改变了账单定义，其低费用不代表同一市场的算法优势。\n\n'
        text+=f'配对 E1 相对 E0 现金变化（正为节省）：{comp["paired_E0"]["total_cost_yuan"]-
comp["paired_E1"]["total_cost_yuan"]:,.4f}元；联合场景相对同预测 E0：{comp["paired_E0"]
["total_cost_yuan"]-comp["paired_joint"]["total_cost_yuan"]:,.4f}元；CVaR 相对风险中性：{comp["paired_joint"]["total_cost_yuan"]-comp["paired_CVaR"]["total_cost_yuan"]:,.4f}元。分别是闭环总效应，不能全部归因于当前第一步。\n\n'
        text+=table(['配对方案','均日费','最差 5%日均费','高价紧急 kWh'],[[name,fmt(comp[name]
['daily_mean_yuan']),fmt(comp[name]['worst5_mean_yuan']),fmt(comp[name]
['high_price_emergency_kwh'])] for name in
('paired_E0','paired_E1','paired_joint','paired_CVaR','independent_price')])
        text+='联合场景只在有限 q 提案中优化。价格残差跨场景循环重排是独立近似压力对照，不是证明真实变量独立的统计检验。CVaR 和场景规模结果只适用于本策略类；DRO 与 GBDT 没有实施，不能声称它们一定无效。\n\n'

```
        text+=f'固定正式承诺序列的纯执行诊断：E36 相对 E0 节省{comp["fixed_commitment_E0"]
["total_cost_yuan"]-comp["fixed_commitment_E36"]["total_cost_yuan"]:,.4f}元。两者只使用当时
已发行版本的未来 G，不能看到后续调整；该诊断冻结了外生承诺，因此也不能替代完整闭环选参。\n\n'
        candidates=read_csv(OUT/f'experiments/q{kind}_candidate_summary.csv')
        text+=table(['场景/风险/时域固定候选','总现金','最差 5%日均费'],
[[r['config'],fmt(r['total_cost_yuan']),fmt(r['worst5_mean_yuan'])] for r in candidates
if '_joint_' in r['config'] or '_R48_' in r['config']])
        text+=table(['同 q=.65/E0 价格模型','事后费用','MAE'],[[r['config'].split('_q')
[0],fmt(r['total_cost_yuan']),fmt(next(z['mae'] for z in metrics if z['stage']=='0' and
z['group']=='all' and z['model']==r['config'].split('_q')[0]))] for r in candidates if
'_q0.65_ordinary_' in r['config'] and r['config'].endswith('E0')])
        text+='费用不仅取决于 MAE，还取决于错误出现时的购电量、SOC 和紧急倍价。以上全年固定候选排名是正
式选择完成后的诊断，不能用其最低项替换月度历史日历。\n\n'
        text+=f'![月度费用](figures/q{kind}_monthly_costs.png)\n\n![尾部比较](figures/
q{kind}_risk_pareto.png)\n\n'
        text+='尾部散点中红色为联合场景候选（含 CVaR），绿色为其他候选。横纵轴分别为平均日现金与最差 5%
日平均现金，均越低越好；图是全部固定候选的事后诊断。\n\n'
    events=[r for r in read_csv(OUT/'q43_voi_events.csv') if int(r['day'])>=31]
    local=read_csv(OUT/'q43_voi_realized_local.csv')
    text+='### 阶段调整价值\n\n'+table(['阶段','UPDATE 事件数','平均事前代理 VOI','平均局部真实现
金价值'],[[str(k*6)+':00',sum(r['accepted']=='True' for r in events if
int(r['stage'])==k),fmt(np.mean([float(r['voi']) for r in events if
int(r['stage'])==k])),fmt(np.mean([float(r['realized_E0_proxy_cash_value']) for r in
local if int(r['stage'])==k]))] for k in (1,2,3)])
    text+='局部真实值已扣本次调整费，但保持承诺至日末、使用 E0 反馈，阶段余日区间重叠，不能相加解释全年收
益。新价格屏蔽与新 PV 屏蔽的全年对照见上表。\n\n'
    text+='## 5 四个指定日期与价格极端日\n\n'
    for kind in (42,43):
        selected=[r for r in all_daily[kind] if r['date'] in WANTED]
        text+=f'### Q4-{kind-40}指定日\n\n'+table(['日期','原计划费','调增费','紧急费','总
费','紧急 kWh','日初/末 E'],
[[r['date'],fmt(r['plan_cost_yuan']),fmt(r['up_cost_yuan']),fmt(r['emergency_cost_yuan']
),fmt(r['total_cost_yuan']),fmt(r['emergency_kwh']),fmt(r['initial_kwh'])
+'/'+fmt(r['terminal_kwh'])] for r in selected])
        for date in WANTED:
            text+=f'#### {date}\n\n'
            text+='表 1（普通购电，紧急另列）\n\n'+table(['时段','0 时原计划 kWh','有效普通 kWh'],
[[r['period'],fmt(r['G0']),fmt(r['effective'])] for r in
read_csv(OUT/f'selected_days/q{kind}_{date}_table1.csv')])
            text+='表 2（真实充放电）\n\n'+table(['时段','充电 kWh','放电 kWh'],
[[r['period'],fmt(r['charge']),fmt(r['discharge'])] for r in
read_csv(OUT/f'selected_days/q{kind}_{date}_table2.csv')])
            h=[r for r in all_merged[kind] if r['date']==date]
            text+='表 3（连续紧急购电）\n\n'+table(['时间段','购电 kWh'],
[[r['period'],fmt(r['kwh'])] for r in h] or [['无','0']])
            text+=f'![调度](figures/q{kind}_{date}.png)\n\n'
        extremes=json.loads((OUT/f'experiments/q{kind}_extreme_days.json').read_text())
        text+='极端日期自动选择：'+str(extremes)+'。其中价格最高不必对应费用最高，需同时观察负载、
PV、承诺和 SOC。\n\n'
        for key in
('max_mean_price','max_range_price','max_single_price','max_forecast_error','max_cost'):
            date=extremes[key];r=next(r for r in all_daily[kind] if r['date']==date)
            text+=f'{key}={date}，日均价{r["price_mean"]:.4f}，总费
{r["total_cost_yuan"]:,.4f}元，紧急{r["emergency_kwh"]:,.4f}kWh，首末
SOC{r["initial_kwh"]:.2f}/{r["terminal_kwh"]:.2f}。\n\n'
            text+=f'![极端日调度](figures/q{kind}_{date}.png)\n\n'
    text+='## 6 验证、局限与文件口径\n\n'+(OUT/'输出与费用口径.md').read_text().split('\n',1)
[1]+'\n'
    text+='本模型用一年数据，月首 21 日评分可能受短窗噪声影响；场景少时 CVaR 近似最大样本；风险裕度、终端
代理与执行反馈均是模型设计。当前时段供需均值是聚合反馈假设。更新门控只保证当时代理评价不劣，不能保证真实费
用单调下降。部分替代沿正式配置日历重放，没有各自重新调参。价格误差区间及日费用尾部是经验描述，不是概率保
证。最终验证结果见 Q4 最终验收审计.md；哈希完整性不替代数学模型正确性。\n\n'
    text+='q43_voi_events.csv 保存发行时 KEEP/UPDATE 代理评分；q43_voi_realized_local.csv 事后
以相同当时 SOC，将旧/新承诺分别保持至日末，用真实价格和 E0 反馈评价局部现金差，已经扣除本次调整费。该反事实
不采用后续再次调整，且多个阶段余日区间重叠，不能加总成年收益，也不是 E1 精确 VOI。流程记录：正式滚动前完成
价格预检与 27 项核心测试，Excel 全量验算在正式数组生成后完成；不声称 Excel 验算先于全年求解。\n\n'
```

```
    text+='## 7 文献依据\n\n'+(OUT/'文献方法映射.md').read_text().split('\n',1)[1]
    (OUT/'Q4_final.md').write_text(text)
    # Decision report retains the core interpretation, numeric choices and all
comparison answers.
    start=text.index('## 4 正式历史选择');end=text.index('## 5 四个指定')
    decision='# Q4 最终决策报告\n\n'+text[text.index('## 1 '):text.index('## 3 ')]
+text[start:end]
    decision+='\n 四指定日完整表 1/2/3、极端日及模型公式见 Q4_final.md。无 DRO/GBDT；概率区间是经验
诊断，非 Conformal。价格加权、联合场景和 CVaR 是否主用由逐月配置逐项说明。未来价格未知、当前报价可用于 E1
及交付价结算是新增/继承解释，不是原题唯一官方规定。\n\n 核心测试和全量回读、旧问回归、冻结保护最终状态见
Q4 最终验收审计.md。\n'
    (OUT/'Q4 最终决策报告.md').write_text(decision)

def deliver():
    from literature_q4 import generate
    from supplement_q4 import supplement
    generate();supplement()
    writable();dates,a,fixed,p,records=read_inputs();f=Factory(a,p,records,fixed)
    results={};daily={};merged={};scene_rows=[]
    for kind in (42,43):
        s=dict(np.load(OUT/f'experiments/q{kind}_formal.npz'));results[kind]=s

daily[kind],merged[kind]=exports(kind,dates,a,p,s);figures(kind,dates,a,p,s,daily[kind])
        cal=json.loads((OUT/f'experiments/q{kind}_calendar.json').read_text())
        for date in WANTED:
            d=next(i for i,v in enumerate(dates) if str(v)==date);conf=Config(**cal[d])
            for k in range(4 if kind==43 else 1):
                ff=f.feature(kind,d,k,conf.price_model);n=144-
k*36;paths,ps,ix=joint_paths(ff.prediction,ff.price_forecast[:n],ff.residual,conf.scenar
ios)
                for j,origin in enumerate(ix):
                    for t in
range(n):scene_rows.append(dict(kind=kind,issue=date,stage=k,scenario=j,historical_origi
n=ff.origins[origin] if origin>=0 else -1,target_offset=t,

load_kwh=paths[j,t,0],pv_kwh=paths[j,t,1],price_scenario=ps[j,t],probability=1/
len(ix),historical_target_end=ff.origins[origin]*144+k*36+288 if origin>=0 else -1))
    write_csv(OUT/'joint_scenarios.csv',scene_rows)
    documents(results,daily,merged)
    (OUT/'README.md').write_text('''# 问题 4 完整交付
```

正式文件 result4-2.xlsx 与 result4-3.xlsx；论文 Q4_final.md；模型选择 Q4 最终决策报告.md；验收 Q4 最终验收审计.md。

在 C 题目录执行 `.venv/bin/python Q4/solve_q4.py`，自动准备、预测预检、核心测试、两类全年历史选择、固定候选诊断、敏感性、CSV/图/论文与工作簿、独立审计。可分步使用--prepare、--precheck、--compute-kind 42/43、--diagnostics-kind 42/43、--deliver。全部模型计算与 Excel 填写使用公开 Python 依赖，export_workbooks_public.py 为默认导出器；原 Node 入口用于可选预览。原题、附件、Q1/Q2/Q3 不改动。

仅依赖既有 numpy/scipy/openpyxl/matplotlib 求解和回读，不需商业求解器、外网服务或 scikit-learn。精确环境见 requirements.freeze.txt。--diagnostics-kind 可复用已完成候选 NPZ；若修改算法必须在独立副本清除该副本实验缓存，不得复用旧数组冒充重算。

正式价格未知、当前执行报价可观察是假设；主 A 不退款、逐事件上一承诺、按交付实际价结算。Q42 仅 0 时计划；Q43 只 0/6/12/18 调整。1 月预运行，2–12 月 334 日正式。各策略 SOC 独立连续。相同主日历的敏感性不等于各自重新调优；场景属于有限策略 SAA，不声称全局随机最优。

audit_q4.py 从底层 CSV 和原始输入复算，再全量回读 Excel 并跑 Q1/Q2/Q3/Q4 测试；旧问交易审计在临时独立副本运行，日志仅保存在 Q4。freeze_q4.py 默认只读校验，--create 仅首次冻结；已有冻结拒绝覆盖。视觉检查需查看 figures 和 previews 后记录。

冻结后在原目录只运行 `.venv/bin/python Q4/freeze_q4.py` 做只读验证。要重算，请将 Q4_frozen_v1.zip 解压到独立目录，创建虚拟环境并安装 Q4/requirements.freeze.txt，使用公开 Python 导出器；仅在该副本删除 Q4/freeze_manifest.json 及 Q4/experiments 中的 NPZ 缓存，再运行 solve_q4.py。保留 pre_q4_workspace_hashes.json、protocol.json 以及 Q1/Q2/Q3，旧问保护仍有效。视觉检查图生成：python Q4/visual_q4.py；人工确认后才运行--record-reviewed。最后 python Q4/freeze_q4.py --


```
```

create 建立该副本自己的首次冻结。

最终模型详见 q4_model.py，价格预测见 q4_price_forecast.py，信息集及历史场景见 q4_features.py/q4_risk.py，执行见 q4_execution_mpc.py，CSV/图/论文见 report_q4.py，默认工作簿填写代码见 export_workbooks_public.py。literature_q4.py 保留文献映射生成代码；supplement_q4.py 生成固定承诺执行和局部 VOI 诊断。Oracle 已揭示变量的残差必须为零，refresh_oracles.py 可单独刷新该项对照。
''')
    from export_workbooks_public import export
    export()

if __name__=='__main__':deliver()
```

## B31 code/Q4/solve_q4.py

```
"""Reproduce Q4: preparation, prechecks, causal selection, diagnostics and
deliverables."""
import argparse,subprocess,time
from dataclasses import asdict,replace
from datetime import datetime,timedelta
import numpy as np
from q4_utils import *
from q4_features import Factory
from q4_model import *

def select_config(factory,kind,day,known,candidates=CANDIDATES,window=21):
    begin=max(1,day-window);start=float(known[begin]);rows=[]
    # Explicitly remove future actuals, prices and official forecast releases.
    cutoff=datetime(2025,1,1)+timedelta(days=day)
    past=Factory(factory.actual[:day],factory.prices.realized[:day],{d:r for d,r in
factory.records.items() if d<cutoff},factory.fixed)
    value=terminal_value(factory.prices.realized[max(0,day-7):day].mean(0))
    for conf in candidates:
        e=start;cash=0.;tic=time.perf_counter()
        for d in range(begin,day):
            s,_=run_day(past,kind,d,e,conf);cash+=s['cash'].sum();e=s['E'][-1]

rows.append(dict(kind=kind,selection_day=day,begin=begin,end=day-1,config=conf.name,cash
=float(cash),
            score=float(cash-value*(e-
start)),initial=start,terminal=float(e),value=value,seconds=time.perf_counter()-tic))
    i=min(range(len(rows)),key=lambda j:(rows[j]['score'],j))
    for j,r in enumerate(rows):r['selected']=j==i
    # Drop method cache keys referring to this bounded validation environment.
    Factory.feature.cache_clear();Factory.raw_net.cache_clear()
    return candidates[i],rows

def replay(factory,kind,calendar=None,conf=None,**kwargs):
    e=6000.;days=[];events=[]
    for d in range(len(factory.actual)):
        c=WARMUP if d<31 else (Config(**calendar[d]) if calendar else conf)
        s,ev=run_day(factory,kind,d,e,c,**kwargs);e=s['E']
[-1];days.append(s);events.extend(ev)
    return stack(days),events

def compute_kind(kind):
    writable();assert (OUT/'experiments/core_tests_passed.json').exists()
    dates,a,fixed,p,records=read_inputs();f=Factory(a,p,records,fixed)

common_value=terminal_value(p[:31].mean(0));e=6000.;known=[e];days=[];calendar=[];select
ion=[];events=[];conf=WARMUP
    tic=time.perf_counter()
    for d,date in enumerate(dates):
        if d>=31 and date.day==1:
            conf,rows=select_config(f,kind,d,known);selection.extend(rows)
            write_csv(OUT/f'experiments/q{kind}_monthly_selection.csv',selection)
            print('Q',kind,date,conf.name,flush=True)
```
```

```
        s,ev=run_day(f,kind,d,e,conf);e=float(s['E']
[-1]);known.append(e);days.append(s);calendar.append(asdict(conf));events.extend(ev)
    result=stack(days)
    np.savez_compressed(OUT/f'experiments/q{kind}_formal.npz',**result)
    write_json(OUT/f'experiments/q{kind}_calendar.json',calendar)
    write_csv(OUT/f'q{kind}_voi_events.csv',events,fields=['kind'] if not events else
None)
    write_json(OUT/f'q{kind}
_summary.json',summarize(result,p,a,common_value=common_value))
    write_json(OUT/f'experiments/q{kind}
_formal_complete.json',dict(seconds=time.perf_counter()-
tic,completed_utc=datetime.now().isoformat(),fixed_rankings_not_yet_used=True))
    print('Q',kind,'formal complete',time.perf_counter()-tic,flush=True)

def diagnostics_kind(kind):
    writable();dates,a,fixed,p,records=read_inputs();f=Factory(a,p,records,fixed)
    cal=json.loads((OUT/f'experiments/q{kind}
_calendar.json').read_text());value=terminal_value(p[:31].mean(0))
    summary=[];daily=[]
    # Restartable diagnostics never affect the already selected formal calendar.
    for conf in CANDIDATES:
        file=OUT/f'experiments/q{kind}_candidate_{CANDIDATES.index(conf)}.npz'
        if file.exists():s=dict(np.load(file))
        else:
            s,_=replay(f,kind,conf=conf);np.savez_compressed(file,**s)

summary.append(dict(kind=kind,config=conf.name,**summarize(s,p,a,common_value=value)))

daily.extend(dict(kind=kind,config=conf.name,date=str(dates[d]),cash=float(s['cash']
[d].sum())) for d in range(31,365))
        write_csv(OUT/f'experiments/q{kind}_candidate_summary.csv',summary)
        print('Q',kind,'diagnostic',conf.name,flush=True)
    write_csv(OUT/f'experiments/q{kind}_candidate_daily.csv',daily)
    variations={
      'oracle_P':dict(oracle='P'),'oracle_NP':dict(oracle='NP'),
      'stale_price':dict(stale_price=True),'decision_lock':dict(locked=True),
      'only0':dict(updates=()),'current_lag':dict(current_known=False)}
    if
kind==43:variations.update(stale_pv=dict(stale_pv=True),settlement_B=dict(rule='B'),sett
lement_B_net=dict(rule='B_net'))
    else:variations['official_pv0']=dict(pv0=True)
    comparisons={}
    for name,kwargs in variations.items():
        file=OUT/f'experiments/q{kind}_{name}.npz'
        if file.exists():s=dict(np.load(file))
        else:s,_=replay(f,kind,calendar=cal,**kwargs);np.savez_compressed(file,**s)
        comparisons[name]=summarize(s,p,a,common_value=value)
        print('Q',kind,name,comparisons[name]['total_cost_yuan'],flush=True)
    # Exact matched-family experiments at preregistered mean7 q=.7 baseline.
    for name,conf,kwargs in [('paired_E0',Config(),{}),
('paired_E1',Config(execution=36),{}),
      ('paired_joint',Config(risk='joint'),{}),
('independent_price',Config(risk='joint'),dict(independent=True)),
      ('paired_CVaR',Config(risk='joint',beta=.1),{})]:
        s,_=replay(f,kind,conf=conf,**kwargs);np.savez_compressed(OUT/f'experiments/
q{kind}_{name}.npz',**s)
        comparisons[name]=summarize(s,p,a,common_value=value)
    # Frozen trajectories are read, never reoptimized or overwritten.
    old=dict(np.load(ROOT/f'Q{kind-40}/experiments/formal_arrays.npz'))
    g0=old['G'] if kind==42 else old['versions'][:,0]
    old['plan_cost']=g0*p;old['emergency_cost']=5*p*old['H']
    if kind==42:
        old['versions']=np.repeat(old['G']
[:,None],4,axis=1);old['U']=np.zeros((365,3,144));old['R']=old['U'].copy()

old['up_cost']=1.5*p*old['U'].sum(1);old['penalty']=.5*p*old['R'].sum(1);old['refund']=n
```

```
p.zeros_like(p)
    old['cash']=old['plan_cost']+old['up_cost']+old['penalty']+old['emergency_cost']
    comparisons['frozen_repriced']=summarize(old,p,a,common_value=value)
    np.savez_compressed(OUT/f'experiments/q{kind}_frozen_repriced.npz',**old)
    # Full clairvoyant ordinary purchase is a relaxation: same formal initial AND final
SOC.
    formal=dict(np.load(OUT/f'experiments/q{kind}_formal.npz'))
    from microgrid_online import plan_net
    oracle=plan_net((a[31:,:,0]-a[31:,:,1]).ravel(),p[31:].ravel(),float(formal['E']
[31,0]),float(formal['E'][-1,-1]))

comparisons['full_oracle_same_boundary']=dict(total_cost_yuan=float(p[31:].ravel()@oracl
e['G']),initial_kwh=float(oracle['E'][0]),terminal_kwh=float(oracle['E'][-1]))
    np.savez_compressed(OUT/f'experiments/q{kind}_full_oracle.npz',**oracle)
    write_json(OUT/f'experiments/q{kind}_comparisons.json',comparisons)
    check_protected()

def main():
    parser=argparse.ArgumentParser();parser.add_argument('--
prepare',action='store_true');parser.add_argument('--precheck',action='store_true')
    parser.add_argument('--compute-
kind',type=int,choices=[42,43]);parser.add_argument('--diagnostics-
kind',type=int,choices=[42,43]);parser.add_argument('--
deliver',action='store_true');args=parser.parse_args()
    if args.prepare:
        from prepare_q4 import prepare;prepare()
    elif args.precheck:
        from price_diagnostics import price_diagnostics;price_diagnostics()

r=subprocess.run([sys.executable,str(OUT/'test_q4.py')],capture_output=True,text=True)
        (OUT/'core_tests.log').write_text(r.stdout+r.stderr)
        if r.returncode:raise RuntimeError('Core tests failed; see Q4/core_tests.log')

write_json(OUT/'experiments/core_tests_passed.json',dict(exit_code=r.returncode))
    elif args.compute_kind:compute_kind(args.compute_kind)
    elif args.diagnostics_kind:diagnostics_kind(args.diagnostics_kind)
    elif args.deliver:
        from report_q4 import deliver;deliver()
    else:
        for flag in ('--prepare','--
precheck'):subprocess.run([sys.executable,__file__,flag],check=True)
        for flag in ('--compute-kind','--diagnostics-kind'):
            processes=[]
            for kind in (42,43):
                log=(OUT/f'q{kind}
_{flag[2:]}.log').open('w');processes.append((subprocess.Popen([sys.executable,__file__,
flag,str(kind)],stdout=log,stderr=subprocess.STDOUT),log))
            for proc,log in processes:
                code=proc.wait();log.close()
                if code:raise RuntimeError(f'{flag} failed')
        subprocess.run([sys.executable,__file__,'--deliver'],check=True)
        subprocess.run([sys.executable,str(OUT/'audit_q4.py')],check=True)

if __name__=='__main__':main()
```

B32 code/Q4/supplement_q4.py

```
"""Fixed commitment execution comparisons and non-additive stage VOI diagnostics."""
from q4_utils import *
from q4_features import Factory
from q4_model import Config,summarize
from q4_execution_mpc import mpc_step
from microgrid_online import feedback_step,terminal_value,validate_execution
from q3_model import simulate_paths
def supplement():
```

```
dates,a,fixed,p,records=read_inputs();f=Factory(a,p,records,fixed);v=terminal_value(p[:3
1].mean(0))
    for kind in (42,43):
        formal=dict(np.load(OUT/f'experiments/q{kind}_formal.npz'))
        comparisons=json.loads((OUT/f'experiments/q{kind}
_comparisons.json').read_text())
        for execution in (0,36):
            s={k:x.copy() for k,x in formal.items()};e=6000.
            for day in range(365):
                s['E'][day,0]=e;conf=Config(**json.loads((OUT/f'experiments/q{kind}
_calendar.json').read_text())[day])
                for j in range(144):
                    k=j//36 if kind==43 else 0;offset=j-k*36
                    if day==0:action=(0.,0.,max(a[day,j,0]-a[day,j,1],0),max(a[day,j,1]-
a[day,j,0],0),e)
                    elif execution:
                        n=min(execution,144-j);pred=formal['predictions']
[day,k,offset:offset+n].copy();pred[0]=a[day,j]

pp=f.prices.execution_curve(day,j,conf.price_model,formal['price_forecasts']
[day,k],k*36,n);pp[0]=p[day,j]
                        # Future G comes from the already issued version, never a later
revision.
                        action=mpc_step(formal['versions'][day,k,j:j+n],pred[:,0]-
pred[:,1],pp,e,terminal_value(formal['price_forecasts'][day,k,:144]))
                    else:action=feedback_step(formal['G'][day,j],*a[day,j],e)
                    s['C'][day,j],s['D'][day,j],s['H'][day,j],s['W']
[day,j],e=action;s['E'][day,j+1]=e
                validate_execution(a[day],{key:value[day] for key,value in s.items()})
            s['emergency_cost']=5*p*s['H'];s['cash']=s['plan_cost']+s['up_cost']
+s['penalty']-s['refund']+s['emergency_cost']

name=f'fixed_commitment_E{execution}';np.savez_compressed(OUT/f'experiments/q{kind}
_{name}.npz',**s)
            comparisons[name]=summarize(s,p,a,common_value=v)
        write_json(OUT/f'experiments/q{kind}_comparisons.json',comparisons)
    s=dict(np.load(OUT/'experiments/q43_formal.npz'));rows=[]
    for day in range(31,365):
        for k in (1,2,3):
            t=k*36;initial=s['E'][day,t];old=s['versions'][day,k-1,t:];new=s['versions']
[day,k,t:]

ho,eo=simulate_paths(old,a[day,None,t:],initial);hn,en=simulate_paths(new,a[day,None,t:]
,initial)
            fee=float((p[day,t:]*(1.5*s['U'][day,k-1,t:]+.5*s['R'][day,k-1,t:])).sum())
            cash=float(5*p[day,t:]@(ho[0]-hn[0]))-fee

rows.append(dict(day=day,date=str(dates[day]),stage=k,realized_E0_proxy_cash_value=cash,
terminal_difference=float(en[0]-eo[0]),adjustment_fee=fee))
    write_csv(OUT/'q43_voi_realized_local.csv',rows)
if __name__=='__main__':supplement()
```

B33 code/Q4/test_q4.py

```
"""Independent counterfactual, synthetic physics, permission and cash tests."""
import unittest
from dataclasses import replace
from datetime import datetime,timedelta
import numpy as np
from q4_utils import *
from q4_features import Factory
from q4_price_forecast import daily_price,PriceFactory,MODELS
from q4_model import Config,run_day,choose,evaluate,CANDIDATES
from q4_risk import cvar,weighted_quantile,joint_paths
from q4_execution_mpc import mpc_step
from microgrid_online import feedback_step,validate_execution
```

```
from microgrid_adjustment import settlement,adjustment_plan

class CoreTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):

cls.dates,cls.a,cls.fixed,cls.p,cls.records=read_inputs();cls.f=Factory(cls.a,cls.p,cls.
records,cls.fixed)

    def test_01_protected(self):self.assertEqual(check_protected(),0)
    def test_02_calendar(self):
        self.assertEqual(self.p.shape,(365,144));self.assertEqual(self.dates[-1]-
self.dates[0],timedelta(days=364))
    def test_03_price_prefix(self):
        for model in MODELS:
            p=self.p.copy();p[31:]*=10
            x=self.f.prices.raw(31,0,model);y=PriceFactory(p).raw(31,0,model)
            np.testing.assert_array_equal(x,y)
    def test_04_reject_uncut_history(self):
        with self.assertRaises(AssertionError):daily_price(self.p,31,'mean7')
    def test_05_constant_price(self):
        for model in MODELS:
            z=daily_price(np.ones((31,144))*.8,31,model)
            np.testing.assert_allclose(z,.8,atol=1e-7)
    def test_06_no_history(self):
        for model in
MODELS:self.assertTrue(np.isfinite(daily_price(np.empty((0,144)),0,model)).all())
    def test_07_joint_sync(self):
        for kind in (42,43):
            f=self.f.feature(kind,60,2,'mean7')
            for j,d in enumerate(f.origins):
                now=d*144+72
                np.testing.assert_allclose(f.residual[j,:,:2],self.f.flat[now:now+288]-
self.f.raw_net(kind,d,2,2))
                np.testing.assert_allclose(f.residual[j,:,2],self.p.ravel()
[now:now+288]-self.f.prices.raw(d,2,'mean7'))
                self.assertLessEqual(now+288,f.now)
    def test_08_scenarios_synchronized_indices(self):

f=self.f.feature(43,60,1,'mean7');a,p,ix=joint_paths(f.prediction,f.price_forecast[:108]
,f.residual)

np.testing.assert_allclose(p,np.maximum(.001,f.price_forecast[None,:108]+f.residual[ix,:
108,2]))
    def test_09_price_future_G0(self):
        p=self.p.copy();p[60:]*=3;ff=Factory(self.a,p,self.records,self.fixed)
        for kind in (42,43):
            for conf in (Config(),Config(risk='weighted'),Config(risk='joint')):
                x,_=run_day(self.f,kind,60,6000,conf);y,_=run_day(ff,kind,60,6000,conf)
                np.testing.assert_array_equal(x['versions'][0],y['versions'][0])
                self.assertGreater(abs(x['cash'].sum()-y['cash'].sum()),1)
    def test_10_stage_price_future(self):
        for t in (72,108):
            p=self.p.copy();p[60,t:]*=2;p[61:]*=2

x,_=run_day(self.f,43,60,6000,Config());y,_=run_day(Factory(self.a,p,self.records,self.f
ixed),43,60,6000,Config())
            np.testing.assert_array_equal(x['versions'][:t//36+1],y['versions'][:t//
36+1])
    def test_11_future_actual(self):
        a=self.a.copy();a[60:,:,0]*=3
        for kind in (42,43):

x,_=run_day(self.f,kind,60,6000,Config());y,_=run_day(Factory(a,self.p,self.records,self
.fixed),kind,60,6000,Config())
            np.testing.assert_array_equal(x['versions'][0],y['versions']
```

```
[0]);self.assertGreater(abs(x['H'].sum()-y['H'].sum()),1)
    def test_12_future_pv_release(self):
        rec=dict(self.records)
        for k in (2,3):
            dt=datetime(2025,1,1)+timedelta(days=60,hours=k*6)
            rec[dt]=replace(rec[dt],power=tuple(np.array(rec[dt].power)*.1))

x,_=run_day(self.f,43,60,6000,Config());y,_=run_day(Factory(self.a,self.p,rec,self.fixed
),43,60,6000,Config())
        np.testing.assert_array_equal(x['versions'][:2],y['versions'][:2])
    def test_13_q42_has_no_pv_permissions(self):
        rec={dt:replace(v,power=tuple(np.zeros(24))) for dt,v in self.records.items()}

x,_=run_day(self.f,42,60,6000,Config());y,_=run_day(Factory(self.a,self.p,rec,self.fixed
),42,60,6000,Config())
        np.testing.assert_array_equal(x['G'],y['G'])
    def test_14_q42_fixed_G(self):
        x,ev=run_day(self.f,42,60,6000,Config(execution=12))
        for k in range(4):np.testing.assert_array_equal(x['versions'][k],x['versions']
[0])
        self.assertEqual(len(ev),0);self.assertEqual(x['U'].sum()+x['R'].sum(),0)
    def test_15_q43_nonretroactivity(self):
        x,_=run_day(self.f,43,60,6000,Config())
        for k in (1,2,3):
            np.testing.assert_array_equal(x['versions'][k,:k*36],x['versions']
[k-1,:k*36])
            np.testing.assert_allclose(x['versions'][k],x['versions'][k-1]+x['U'][k-1]-
x['R'][k-1],atol=1e-8)
    def test_16_dynamic_settlement_hand(self):
        p=np.array([.2,.7,1.4]);g=np.array([100.,80.,50.]);total=p@g;up=down=0.
        for new in
(np.array([120.,90.,60.]),np.array([110.,80.,50.]),np.array([120.,100.,70.])):
            b=settlement(g,new,p);up+=b['up_cost'].sum();down+=b['penalty'].sum();g=new
        # U totals 30,30,30; R 10,10,10; emergency 1,2,3.

self.assertAlmostEqual(up,1.5*30*p.sum());self.assertAlmostEqual(down,.5*10*p.sum())
        self.assertAlmostEqual(total+up+down+5*p@np.array([1.,2.,3.]),146+103.5+11.5+29)
    def test_17_multiple_adjustments(self):
        for rule,expected in [('A',165),('B',125),('B_net',95)]:
            old=np.array([100.]);cost=100
            for x in (80,110,90):

new=np.array([float(x)]);cost+=settlement(old,new,np.ones(1),rule,np.array([100.]))
['charge'].sum();old=new
            self.assertAlmostEqual(cost,expected)
    def test_18_cvar_lp_identity(self):
        from scipy.optimize import linprog
        x=np.array([1.,2.,10.,30.]);a=.9
        sol=linprog(np.r_[1.,np.full(4,1/(4*(1-a)))],A_ub=np.column_stack([-np.ones(4),-
np.eye(4)]),b_ub=-x,bounds=[(None,None)]+[(0,None)]*4)
        self.assertAlmostEqual(cvar(x,a),sol.fun)
    def test_19_cvar_beta_zero(self):
        paths=np.array([[[10.,0.],[20.,0.]],[[40.,0.],
[50.,0.]]]);p=np.ones((2,2));g=np.zeros(2)

score,cash=evaluate(g,paths,p,1200,0,beta=0);self.assertAlmostEqual(score,cash.mean())
    def test_20_price_weighted_tail(self):
        v=np.arange(10.).reshape(-1,1);w=np.ones_like(v);w[-1]=20
        self.assertGreater(weighted_quantile(v,w,.8)
[0],weighted_quantile(v,np.ones_like(v),.8)[0])
    def test_21_mpc_low_now_high_later(self):
        x=mpc_step(np.zeros(2),np.full(2,500.),np.array([.01,2.]),1200+500/.9,0)
        self.assertLess(x[1],1e-6);self.assertAlmostEqual(x[2],500.)
    def test_22_mpc_no_emergency_charging(self):
        x=mpc_step(np.zeros(2),np.array([500.,500.]),np.array([.01,2.]),6000,0)
        self.assertEqual(x[0],0.)
```

```
        y=mpc_step(np.array([100.,0.]),np.array([0.,500.]),np.array([.01,2.]),6000,0)
        self.assertEqual(y[2],0.);self.assertAlmostEqual(y[0],100)
    def test_23_mpc_physics_wrong_forecast(self):

x,_=run_day(self.f,43,60,6000,Config(price_model='last',execution=12));validate_executio
n(self.a[60],x)
    def test_24_mpc_future_prefix(self):
        p=self.p.copy();p[60,72:]*=10

x,_=run_day(self.f,42,60,6000,Config(execution=12));y,_=run_day(Factory(self.a,p,self.re
cords,self.fixed),42,60,6000,Config(execution=12))
        for key in ('G','C','D','H','W'):np.testing.assert_array_equal(x[key]
[:72],y[key][:72])
    def test_25_price_scale(self):

f=self.f.feature(42,60,0,'mean7');scaled=replace(f,price_forecast=f.price_forecast*2,res
idual=f.residual*np.array([1,1,2]))
        x,_=choose(f,6000,Config(),0);y,_=choose(scaled,6000,Config(),0)
        np.testing.assert_allclose(x,y,atol=1e-6)
    def test_26_month_future_independent(self):
        from solve_q4 import select_config
        p=self.p.copy();p[31:]*=10
        configs=(Config(),Config(price_model='last'))
        x,rx=select_config(self.f,42,31,[6000]*32,configs)
        y,ry=select_config(Factory(self.a,p,self.records,self.fixed),42,31,
[6000]*32,configs)
        self.assertEqual(x,y)
        np.testing.assert_allclose([r['score'] for r in rx],[r['score'] for r in
ry],rtol=0,atol=1e-8)
    def test_27_adjustment_lp_milp(self):
        f=self.f.feature(43,60,3,'mean7');net=f.prediction[:36,0]-
f.prediction[:36,1];old=np.full(36,400.)
        x=adjustment_plan(net,f.price_forecast[:36],6000,old,.5)
        y=adjustment_plan(net,f.price_forecast[:36],6000,old,.5,integer=True)
        self.assertAlmostEqual(x['objective'],y['objective'],places=5)

class OutputTests(unittest.TestCase):
    def check(self,kind):
        if not (OUT/f'result4-{kind-40}.xlsx').exists():self.skipTest('Workbook not yet
generated; mandatory final audit checks it')
        from audit_q4 import audit_kind
        dates,a,fixed,p,records=read_inputs();audit_kind(kind,dates,a,p)
    def test_28_q42_full_output(self):self.check(42)
    def test_29_q43_full_output(self):self.check(43)

if __name__=='__main__':

suite=unittest.TestSuite([unittest.defaultTestLoader.loadTestsFromTestCase(CoreTests),un
ittest.defaultTestLoader.loadTestsFromTestCase(OutputTests)])
    result=unittest.TextTestRunner(verbosity=2).run(suite)
    sys.exit(not result.wasSuccessful())
```

B34 code/microgrid_adjustment.py

```
"""Q3/Q4 可复用的增量调整账本及确定性候选优化；不读取真实曲线。"""
import numpy as np
from scipy.optimize import linprog, milp, Bounds, LinearConstraint
from scipy.sparse import hstack,vstack,csr_matrix,coo_matrix
from microgrid_common import Battery
from microgrid_online import balance_matrix,terminal_value

def settlement(old,new,price,rule='A',original=None):
    """A 沉没+额外费用；B 逐事件调减退 p 再收.5p；B_net 原计划净偏差重估。
    B_net 返回相对上次账面净偏差费用的冲销差额，禁止重复收整笔账。
    """
    old,new,p=[np.asarray(v,float) for v in (old,new,price)]
```

```
    u=np.maximum(new-old,0);r=np.maximum(old-new,0)
    up=1.5*p*u;penalty=.5*p*r;refund=np.zeros_like(r)
    if rule=='B':refund=p*r
    elif rule=='B_net':
        assert original is not None
        base=np.asarray(original)
        # Signed changes in cumulative financial components; physical U/R remain
positive.
        up=1.5*p*(np.maximum(new-base,0)-np.maximum(old-base,0))
        penalty=.5*p*(np.maximum(base-new,0)-np.maximum(base-old,0))
        refund=p*(np.maximum(base-new,0)-np.maximum(base-old,0))
    elif rule!='A':raise ValueError(rule)
    return dict(U=u,R=r,up_cost=up,penalty=penalty,refund=refund,charge=up+penalty-
refund)

def
adjustment_plan(net,price,initial,old,value,rule='A',original=None,battery=Battery(),int
eger=False):
    """当前自然日剩余 m 点已有承诺，时域其余点只是正常价格前瞻。
    min adjustment + future purchase - value E_end; LP 后精确消循环。
    """
    net,p,old=[np.asarray(v,float) for v in (net,price,old)]
    n=len(net);m=len(old);b=battery;size=5*n+1+2*m
    assert n>=m>0 and p.shape==net.shape and np.isfinite(net).all() and (p>0).all()
    base=np.asarray(original) if rule=='B_net' else old
    assert base.shape==old.shape
    obj=np.zeros(size);obj[m:n]=p[m:];obj[5*n]=-value
    obj[5*n+1:5*n+1+m]=1.5*p[:m]
    obj[5*n+1+m:]=(.5 if rule=='A' else -.5)*p[:m]
    lo=np.zeros(size);hi=np.full(size,np.inf)
    hi[n:3*n]=b.power*b.dt;lo[4*n:5*n+1]=b.minimum;hi[4*n:5*n+1]=b.maximum
    lo[4*n]=hi[4*n]=np.clip(initial,b.minimum,b.maximum)

a=hstack([balance_matrix(n,b.eta_charge,b.eta_discharge),csr_matrix((2*n,2*m))],format='
csr')
    t=np.arange(m)
    link=coo_matrix((np.r_[np.ones(m),-np.ones(m),np.ones(m)],
        (np.tile(t,3),np.r_[t,5*n+1+t,5*n+1+m+t])),shape=(m,size)).tocsr()
    a=vstack([a,link],format='csr');rhs=np.r_[net,np.zeros(n),base]
    if integer:
        t=np.arange(n);pm=b.power*b.dt
        mutex=coo_matrix((np.r_[np.ones(n),-pm*np.ones(n),np.ones(n),pm*np.ones(n)],

(np.r_[t,t,n+t,n+t],np.r_[n+t,size+t,2*n+t,size+t])),shape=(2*n,size+n)).tocsr()
        aa=vstack([hstack([a,csr_matrix((len(rhs),n))]),mutex],format='csr')
        res=milp(np.r_[obj,np.zeros(n)],integrality=np.r_[np.zeros(size),np.ones(n)],
          bounds=Bounds(np.r_[lo,np.zeros(n)],np.r_[hi,np.ones(n)]),
          constraints=LinearConstraint(aa,np.r_[rhs,np.full(2*n,-
np.inf)],np.r_[rhs,np.zeros(n),np.full(n,pm)]),
          options={'mip_rel_gap':1e-9,'time_limit':120})
    else:res=linprog(obj,A_eq=a,b_eq=rhs,bounds=np.column_stack([lo,hi]),method='highs')
    if not res.success:raise RuntimeError(res.message)

x=res.x;g=x[:n].copy();c=x[n:2*n].copy();d=x[2*n:3*n].copy();w=x[3*n:4*n].copy();e=x[4*n
:5*n+1].copy()
    delta=np.minimum(c,d/(b.eta_charge*b.eta_discharge))
    c-=delta;d-=b.eta_charge*b.eta_discharge*delta;w+=(1-
b.eta_charge*b.eta_discharge)*delta
    assert np.max(np.abs(g+d-c-w-net))<1e-5
    assert np.max(np.abs(np.diff(e)-b.eta_charge*c+d/b.eta_discharge))<1e-5
    return dict(G=g,C=c,D=d,W=w,E=e,objective=float(res.fun))
```

B35 code/microgrid_common.py

```
"""可复用的十分钟数据读取、储能调度与物理校验。所有流量单位为 kWh。"""
from dataclasses import dataclass
```

```
from datetime import time
from pathlib import Path

import numpy as np
from openpyxl import load_workbook
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import lil_matrix, hstack, vstack, csr_matrix


@dataclass(frozen=True)
class Battery:
    capacity: float = 12000.0
    minimum: float = 1200.0
    maximum: float = 10800.0
    power: float = 5000.0
    eta_charge: float = 0.9
    eta_discharge: float = 0.9
    dt: float = 1 / 6


def clock_label(minutes):
    return f"{minutes // 60}:{minutes % 60:02d}"


def end_minutes(value):
    """兼容附件中的 Excel time、字符串时间和次日 0:00。"""
    if isinstance(value, time):
        return value.hour * 60 + value.minute
    text = str(value).strip()
    day = 1440 if '+1' in text else 0
    h, m, *_ = text.replace('+1', '').split(':')
    return day + int(h) * 60 + int(m)


def read_single_day(path: Path):
    wb = load_workbook(path, read_only=True, data_only=True)
    rows = list(wb.active.values)
    wb.close()
    if len(rows) != 145 or list(rows[0]) != ['时间', '电价', '小区负载', '光伏发电预测功率']:
        raise ValueError('附件 1 应为一个表头和 144 条十分钟记录，列名需与原附件一致')
    ends = np.array([end_minutes(r[0]) for r in rows[1:]])
    if not np.array_equal(ends, np.arange(10, 1441, 10)):
        raise ValueError('时间缺失、重复或错位；期望 0:10 至 24:00 的时段结束时间')
    values = np.array([r[1:4] for r in rows[1:]], dtype=float)
    if not np.isfinite(values).all() or (values < 0).any():
        raise ValueError('数据必须是有限非负数')
    return dict(price=values[:, 0], load=values[:, 1] / 6,
                pv=values[:, 2] / 6, end_minutes=ends)


def optimize(price, load, pv, battery=Battery(), initial=6000., terminal=6000.,
integer=False):
    """纯购电费最小化。负载/光伏传入电量；terminal=None 可不固定末状态。

  变量顺序 g,c,d,w,e；g+c 等为交流母线侧电量，e 为电池内部储电量。
    integer=True 增加充放电互斥二进制变量；LP 用作全局成本下界。
  此函数仅是确定性调度内核，Q2 起的预测、场景和费用逻辑需另外实现。
    """
    price, load, pv = [np.asarray(v, dtype=float) for v in (price, load, pv)]
    n = len(price)
    if not n or load.shape != price.shape or pv.shape != price.shape:
        raise ValueError('价格、负载和光伏维度不一致')
    if not all(np.isfinite(v).all() for v in (price, load, pv)):
        raise ValueError('输入包含非有限值')
    if (price < 0).any() or (load < 0).any() or (pv < 0).any():
        raise ValueError('本内核要求非负电价、负载与光伏')
```

```
    b = battery
    if not (0 < b.eta_charge <= 1 and 0 < b.eta_discharge <= 1 and
            0 <= b.minimum <= initial <= b.maximum <= b.capacity and b.dt > 0 and
b.power >= 0):
        raise ValueError('储能参数无效')
    if terminal is not None and not b.minimum <= terminal <= b.maximum:
        raise ValueError('末状态越界')
    size = 5*n+1
    objective = np.zeros(size); objective[:n] = price
    lower = np.zeros(size); upper = np.full(size, np.inf)
    upper[n:3*n] = b.power*b.dt
    upper[3*n:4*n] = pv
    lower[4*n:] = b.minimum; upper[4*n:] = b.maximum
    lower[4*n] = upper[4*n] = initial
    if terminal is not None:
        lower[-1] = upper[-1] = terminal
    a = lil_matrix((2*n, size))
    for t in range(n):
        a[t, t] = 1; a[t, n+t] = -1; a[t, 2*n+t] = 1; a[t, 3*n+t] = -1
        a[n+t, 4*n+t+1] = 1; a[n+t, 4*n+t] = -1
        a[n+t, n+t] = -b.eta_charge; a[n+t, 2*n+t] = 1/b.eta_discharge
    rhs = np.r_[load-pv, np.zeros(n)]
    a = a.tocsr()
    if integer:
        eq = hstack([a, csr_matrix((2*n, n))], format='csr')
        mutex = lil_matrix((2*n, size+n))
        for t in range(n):
            mutex[t,n+t] = 1; mutex[t,size+t] = -b.power*b.dt
            mutex[n+t,2*n+t] = 1; mutex[n+t,size+t] = b.power*b.dt
        constraints = LinearConstraint(vstack([eq,mutex.tocsr()],format='csr'),
            np.r_[rhs,np.full(2*n,-np.inf)],
np.r_[rhs,np.zeros(n),np.full(n,b.power*b.dt)])
        res = milp(np.r_[objective,np.zeros(n)],
integrality=np.r_[np.zeros(size),np.ones(n)],
                   bounds=Bounds(np.r_[lower,np.zeros(n)],np.r_[upper,np.ones(n)]),
                   constraints=constraints,
options={'mip_rel_gap':1e-9,'time_limit':120})
    else:
        res = linprog(objective, A_eq=a, b_eq=rhs, bounds=list(zip(lower,upper)),
method='highs')
    if not res.success:
        raise RuntimeError(f'优化求解失败: {res.message}')
    x = res.x[:size]
    return dict(grid=x[:n], charge=x[n:2*n], discharge=x[2*n:3*n],
                curtail=x[3*n:4*n], energy=x[4*n:], cost=float(price@x[:n]),
                solver_message=res.message, mip_gap=float(getattr(res,'mip_gap',0.0)))


def validate(data, solution, battery=Battery(), initial=6000., terminal=6000.,
tol=1e-6):
    """从导出所用的决策量独立重算逐时段物理约束，不依赖求解器状态。"""
    s, b = solution, battery
    g,c,d,w,e = [s[k] for k in ('grid','charge','discharge','curtail','energy')]
    balance = g + data['pv'] - w + d - data['load'] - c
    reconstructed = np.r_[initial, initial +
np.cumsum(b.eta_charge*c-d/b.eta_discharge)]
    checks = {
        'balance_max_error_kwh':float(np.max(np.abs(balance))),
        'state_max_error_kwh':float(np.max(np.abs(reconstructed-e))),
        'initial_error_kwh':float(abs(e[0]-initial)),
        'terminal_error_kwh':float(abs(e[-1]-terminal)) if terminal is not None else 0.,
        'nonnegative_violation_kwh':float(max(0.,-
min(g.min(),c.min(),d.min(),w.min()))),
        'energy_bound_violation_kwh':float(max(0.,b.minimum-e.min(),e.max()-b.maximum)),
        'power_bound_violation_kw':float(max(0.,max(c.max(),d.max())/b.dt-b.power)),
        'curtail_bound_violation_kwh':float(max(0.,np.max(w-data['pv']))),
```

```
        'simultaneous_charge_discharge_count':int(np.sum((c>tol)&(d>tol))),
        'cost_error_yuan':float(abs(data['price']@g-s['cost'])),
    }
    if any(v > tol for v in checks.values()):
        raise AssertionError(checks)
    return checks
```

B36 code/microgrid_forecasts.py

```
"""Q3/Q4 可复用：显式发行时间的官方光伏预报与因果插值。功率 kW。"""
from dataclasses import dataclass
from datetime import datetime, timedelta
from pathlib import Path
import re
import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator

@dataclass(frozen=True)
class PVForecast:
    issue: datetime
    targets: tuple
    power: tuple

    def __post_init__(self):
        assert self.issue.hour in (0,6,12,18) and self.issue.minute == 0
        assert self.targets == tuple(self.issue+timedelta(hours=h) for h in range(1,25))
        assert len(self.power)==24 and np.isfinite(self.power).all() and
min(self.power)>=0

def read_pv_forecasts(path):
    wb=load_workbook(path,read_only=True,data_only=True)
    rows=list(wb.active.values);wb.close()
    assert rows[0][:2]==('日期','预报时刻')
    leads=[int(re.fullmatch(r'预报(\d+)小时',v).group(1)) for v in rows[0][2:]]
    assert leads==list(range(1,25))
    out={}; date_=None
    for row in rows[1:]:
        if row[0]:
            date_=datetime.strptime(str(row[0]),'%Y-%m-%d') if not
isinstance(row[0],datetime) else row[0]
        assert date_ is not None
        h,m=map(int,str(row[1]).split(':')[:2]);issue=date_.replace(hour=h,minute=m)
        assert issue not in out
        out[issue]=PVForecast(issue,tuple(issue+timedelta(hours=j) for j in
leads),tuple(float(v) for v in row[2:]))
    expected=[datetime(2025,1,1)+timedelta(hours=6*i) for i in range(365*4)]
    assert list(out)==expected
    return out

def interpolate_pv(record,anchor_kw,method='linear'):
    """144 个区间结束点的预报功率；anchor 为发行时已经观测到的最后一个点。

    zero-order：每个整点值向后保持，首小时使用已知锚点；整点严格通过原值。
  以结束点采样功率近似时段均值，与 Q1/Q2 离散化一致；不是精确积分。
    """
    x=np.arange(25,dtype=float); y=np.r_[max(0,anchor_kw),record.power]
    t=np.arange(1,145)/6
    if method=='constant':z=y[np.floor(t+1e-10).astype(int)]
    elif method=='linear':z=np.interp(t,x,y)
    elif method=='pchip':z=PchipInterpolator(x,y)(t)
    else:raise ValueError(method)
    z=np.maximum(z,0)
    assert np.allclose(z[5::6],record.power,rtol=0,atol=1e-8)
    return z
```

```
def released_curve(records,now,anchor_kw,method='linear'):
    """接受已发布子集；传入任何未来预报直接拒绝。"""
    assert records and all(r.issue<=now for r in records)
    record=max(records,key=lambda r:r.issue)
    return record,interpolate_pv(record,anchor_kw,method)
```

B37 code/microgrid_online.py

```
"""Q2–Q4 可复用：全年数据、自然日标签、纯物理计划与实时执行。

不改动 Q1 公共内核；全部流量为交流母线侧 kWh，energy 为内部 kWh。
本模块不读取未来数据，预测/价格信息由调用者显式传入。
"""
from dataclasses import dataclass
from datetime import date, datetime, timedelta
from functools import lru_cache
from pathlib import Path
import time
import numpy as np
from openpyxl import load_workbook
from scipy.optimize import linprog, milp, Bounds, LinearConstraint
from scipy.sparse import coo_matrix, hstack, vstack, csr_matrix
from microgrid_common import Battery, end_minutes, read_single_day

ROOT = Path(__file__).resolve().parent
TOL = 1e-7
PERIODS = [f'{t//6}:{t%6*10:02d}-{(t+1)//6}:{(t+1)%6*10:02d}' for t in range(144)]


def read_year(root=ROOT):
    wb = load_workbook(root/'附件/附件2.xlsx', read_only=True, data_only=True)
    matrices, dates = [], None
    for name in ('小区负载', '光伏发电实际功率'):
        rows = list(wb[name].values)
        assert [end_minutes(v) for v in rows[0][1:]] == list(range(10, 1441, 10))
        ds = [r[0].date() if isinstance(r[0], datetime) else r[0] for r in rows[1:]]
        assert ds == [date(2025, 1, 1)+timedelta(days=i) for i in range(365)]
        if dates is not None:
            assert ds == dates
        dates = ds
        a = np.asarray([r[1:] for r in rows[1:]], float)/6
        assert a.shape == (365, 144) and np.isfinite(a).all() and (a >= 0).all()
        matrices.append(a)
    wb.close()
    return dates, np.stack(matrices, axis=-1), read_single_day(root/'附件/附件1.xlsx')
['price']


@lru_cache(maxsize=12)
def balance_matrix(n, ec, ed):
    t = np.arange(n)
    row = np.concatenate([t]*4+[n+t]*4)
    col = np.concatenate([t, n+t, 2*n+t, 3*n+t, 4*n+t+1, 4*n+t, n+t, 2*n+t])
    val = np.concatenate([np.ones(n), -np.ones(n), np.ones(n), -np.ones(n),
                          np.ones(n), -np.ones(n), -ec*np.ones(n), np.ones(n)/ed])
    return coo_matrix((val, (row, col)), shape=(2*n, 5*n+1)).tocsr()


def terminal_value(price, battery=Battery()):
    return float(np.sort(price)[:min(24, len(price))].mean()/battery.eta_charge)


def plan_net(net, price, initial, terminal='cycle', battery=Battery(), integer=False):
    """净负荷计划 LP，消循环精确恢复互斥；MILP 仅作独立最优性对照。

    terminal='cycle'/'value'/None/数值；net 已为 kWh。允许计划剩余电量 W。
```

```
    """
    tic = time.perf_counter()
    net, price = np.asarray(net, float), np.asarray(price, float)
    n = len(net); b = battery; pmax = b.power*b.dt
    assert price.shape == net.shape and n > 0
    assert np.isfinite(net).all() and np.isfinite(price).all() and (price >= 0).all()
    assert b.minimum-1e-6 <= initial <= b.maximum+1e-6
    initial = float(np.clip(initial, b.minimum, b.maximum))
    objective = np.zeros(5*n+1); objective[:n] = price
    lo = np.zeros(5*n+1); hi = np.full(5*n+1, np.inf)
    hi[n:3*n] = pmax; lo[4*n:] = b.minimum; hi[4*n:] = b.maximum
    lo[4*n] = hi[4*n] = initial
    if terminal == 'cycle':
        lo[-1] = hi[-1] = initial
    elif terminal == 'value':
        objective[-1] = -terminal_value(price[:144], b)
    elif terminal is not None:
        lo[-1] = hi[-1] = float(terminal)
    a = balance_matrix(n, b.eta_charge, b.eta_discharge)
    rhs = np.r_[net, np.zeros(n)]
    if integer:
        eq = hstack([a, csr_matrix((2*n, n))], format='csr')
        t = np.arange(n)
        mutex = coo_matrix((np.r_[np.ones(n), -pmax*np.ones(n), np.ones(n),
pmax*np.ones(n)],
                            (np.r_[t,t,n+t,n+t], np.r_[n+t,5*n+1+t,2*n+t,5*n+1+t])),
                           shape=(2*n,6*n+1)).tocsr()
        res = milp(np.r_[objective,np.zeros(n)],
integrality=np.r_[np.zeros(5*n+1),np.ones(n)],
                   bounds=Bounds(np.r_[lo,np.zeros(n)],np.r_[hi,np.ones(n)]),
                   constraints=LinearConstraint(vstack([eq,mutex],format='csr'),
                     np.r_[rhs,np.full(2*n,-
np.inf)],np.r_[rhs,np.zeros(n),np.full(n,pmax)]),
                   options={'mip_rel_gap':1e-9, 'time_limit':120})
    else:
        res = linprog(objective, A_eq=a, b_eq=rhs, bounds=np.column_stack([lo,hi]),
method='highs')
    if not res.success:
        raise RuntimeError(res.message)
    x = res.x[:5*n+1]
    g,c,d,w,e = [v.copy() for v in (x[:n],x[n:2*n],x[2*n:3*n],x[3*n:4*n],x[4*n:])]
    delta = np.minimum(c, d/(b.eta_charge*b.eta_discharge))
    c -= delta; d -= b.eta_charge*b.eta_discharge*delta
    w += (1-b.eta_charge*b.eta_discharge)*delta
    assert np.max(np.abs(g+d-c-w-net)) < 1e-5
    assert np.max(np.abs(np.diff(e)-b.eta_charge*c+d/b.eta_discharge)) < 1e-5
    assert not np.any((c>1e-6)&(d>1e-6))
    return
dict(G=g,C=c,D=d,W=w,E=e,objective=float(res.fun),seconds=time.perf_counter()-tic,
               variables=6*n+1 if integer else 5*n+1,constraints=4*n if integer else
2*n,

cycles_removed=int((delta>1e-7).sum()),mip_gap=float(getattr(res,'mip_gap',0)))


def feedback_step(g, load, pv, energy, battery=Battery()):
    """只接收当前观测和当前 E；无法读取未来曲线。"""
    b = battery; limit = b.power*b.dt; r = g+pv-load
    c = min(max(r,0.), limit, max(0., (b.maximum-energy)/b.eta_charge))
    d = min(max(-r,0.), limit, max(0., (energy-b.minimum)*b.eta_discharge))
    h = max(0., -r-d); w = max(0., r-c)
    return c,d,h,w,energy+b.eta_charge*c-d/b.eta_discharge


def execute_day(g, actual, initial, battery=Battery(), fixed=None, idle=False):
    """actual 仅供环境逐点揭示；控制器 feedback_step 不能看到数组。
```

```
    fixed=(C,D)为储能完全固定替代口径；idle 用于共同启动/无储能。
    """
    b=battery; n=len(g); e=np.empty(n+1);e[0]=initial
    c=np.zeros(n);d=c.copy();h=c.copy();w=c.copy()
    for t,(load,pv) in enumerate(actual):
        if fixed is None and not idle:
            c[t],d[t],h[t],w[t],e[t+1]=feedback_step(g[t],load,pv,e[t],b)
        else:
            if fixed is not None:
                c[t],d[t]=fixed[0][t],fixed[1][t]
            r=g[t]+pv+d[t]-load-c[t]
            h[t]=max(-r,0.); w[t]=max(r,0.)
            e[t+1]=e[t]+b.eta_charge*c[t]-d[t]/b.eta_discharge
    # source allocation: solar first; only remaining solar is curtailed
    unused_grid=np.minimum(w,g)
    pv_curtail=w-unused_grid
    result=dict(G=np.asarray(g).copy(),C=c,D=d,H=h,W=w,E=e,
                unused_grid=unused_grid,pv_curtail=pv_curtail,
                pv_curtail_upper=np.minimum(w,actual[:,1]))
    validate_execution(actual,result,b)
    return result


def validate_execution(actual,s,battery=Battery()):
    b=battery;g,c,d,h,w,e=[s[k] for k in ('G','C','D','H','W','E')]
    err=dict(balance=float(np.max(np.abs(g+actual[:,1]+d+h-actual[:,0]-c-w))),
             state=float(np.max(np.abs(np.diff(e)-b.eta_charge*c+d/b.eta_discharge))),
             bounds=float(max(0,b.minimum-e.min(),e.max()-b.maximum)),
             power=float(max(0,c.max()-b.power*b.dt,d.max()-b.power*b.dt)),
             nonnegative=float(max(0,-min(g.min(),c.min(),d.min(),h.min(),w.min()))),
             simultaneous=int(np.sum((c>1e-6)&(d>1e-6))))
    assert max(err.values())<1e-5,err
    assert np.max(np.abs(h*w))<1e-5
    return err


def merge_emergency(dates, emergency, tol=TOL):
    out=[]
    for date_,day in zip(dates, emergency):
        start=None
        for t in range(len(day)+1):
            positive=t<len(day) and day[t]>tol
            if positive and start is None:
                start=t
            if not positive and start is not None:
                out.append(dict(date=date_.isoformat(),start=start,stop=t,
                    period=f'{start//6}:{start%6*10:02d}-{t//6}:{t%6*10:02d}',
                    kwh=float(day[start:t].sum())))
                start=None
    return out
```

B38 code/reproduce.py

```
"""Run the complete public-dependency numerical workflow from official attachments."""
from pathlib import Path
import subprocess,sys
ROOT=Path(__file__).resolve().parent
COMMANDS=[['Q1/solve_q1.py'],['Q1/test_q1.py'],['Q2/solve_q2.py','--compute'],['Q2/
report_q2.py'],['Q2/test_q2.py'],
 ['Q3/solve_q3.py','--prototype'],['Q3/solve_q3.py','--compute'],['Q3/report_q3.py'],
['Q3/test_q3.py'],
 ['Q4/solve_q4.py','--prepare'],['Q4/solve_q4.py','--precheck'],
 ['Q4/solve_q4.py','--compute-kind','42'],['Q4/solve_q4.py','--compute-kind','43'],
 ['Q4/solve_q4.py','--diagnostics-kind','42'],['Q4/solve_q4.py','--diagnostics-
kind','43'],
```

```
 ['Q4/solve_q4.py','--deliver'],['Q4/test_q4.py']]
if __name__=='__main__':
 for args in COMMANDS:
  print('RUN',*args,flush=True)
  subprocess.run([sys.executable,*args],cwd=ROOT,check=True)
 print('All solver, report, workbook and output tests passed.')
```

附录 C 波动电价指定时段明细

对应问题 2 指定时段原计划

表 37 2025-03-20 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	535.96	14:00-14:10	0.00
16:00-16:10	477.92	18:00-18:10	0.00	20:00-20:10	0.00
全天计划量	65,597.91	计划费	40,343.85	全天总费	49,979.30

表 38 2025-06-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	170.56	14:00-14:10	0.00
16:00-16:10	422.95	18:00-18:10	591.31	20:00-20:10	0.00
全天计划量	54,650.63	计划费	28,095.98	全天总费	28,095.98

表 39 2025-09-23 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	480.35	14:00-14:10	0.00
16:00-16:10	513.38	18:00-18:10	0.00	20:00-20:10	0.00
全天计划量	66,346.38	计划费	43,001.08	全天总费	48,711.49

表 40 2025-12-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	995.88	14:00-14:10	0.00
16:00-16:10	871.13	18:00-18:10	714.71	20:00-20:10	0.00
全天计划量	93,257.09	计划费	71,476.54	全天总费	71,535.93

附录 C 对应问题 2 储能明细

表 41 2025-03-20 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,207.02	262.19	4:00-8:00	1,023.49	6,114.19
8:00-12:00	5,791.30	2,409.84	12:00-16:00	4,491.10	592.61
16:00-20:00	130.48	6,112.32	20:00-24:00	4,317.97	1,786.80
0:00 储电量	4,455.01		24:00 储电量	5,022.53	

表 42 2025-06-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	2,445.77	0.00	4:00-8:00	2,577.56	1,382.92
8:00-12:00	1,224.11	0.00	12:00-16:00	0.00	0.00
16:00-20:00	0.00	4,755.36	20:00-24:00	3,866.99	1,865.03
0:00 储电量	6,713.89		24:00 储电量	6,924.29	

表 43 2025-09-23 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,781.89	77.69	4:00-8:00	3,983.25	6,677.61
8:00-12:00	5,208.00	1,896.62	12:00-16:00	5,621.03	497.84
16:00-20:00	40.28	6,617.19	20:00-24:00	345.19	1,893.34
0:00 储电量	1,264.82		24:00 储电量	1,423.94	

表 44 2025-12-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	106.31	185.97	4:00-8:00	171.21	860.48
8:00-12:00	5,773.43	7,158.08	12:00-16:00	6,918.58	2,300.79
16:00-20:00	13.42	5,815.95	20:00-24:00	6,845.49	2,859.64
0:00 储电量	10,800.00		24:00 储电量	7,333.47	

附录 C 对应问题 2 紧急购电明细

表 45 指定日期紧急购电区间与电量（对应题面表 3）

03/20	购电量	06/21	购电量	09/23	购电量	12/21	购电量
9:00-10:00	367.53	无	0.00	20:10-20:20	10.23	20:30-20:40	7.61
20:20-20:40	1,200.70	—	—	20:30-20:40	686.18	—	—
—	—	—	—	20:50-21:30	159.53	—	—
—	—	—	—	21:40-22:00	34.54	—	—
—	—	—	—	22:20-22:30	2.44	—	—

注：所有日期均为 2025 年，电量单位为 kWh；相邻的紧急购电十分钟区间合并为连续事件，数值为区间电量之和。“无”表示当日未发生紧急购电，“—”表示该列没有更多事件。

附录 C 对应问题三补充结果

对应问题 3 指定时段原计划

表 46 2025-03-20 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	525.62	14:00-14:10	0.00
16:00-16:10	636.76	18:00-18:10	0.00	20:00-20:10	0.00
全天计划量	67,010.34	计划费	41,544.79	全天总费	46,221.25

表 47 2025-06-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	399.23	18:00-18:10	583.22	20:00-20:10	0.00
全天计划量	42,629.28	计划费	23,273.11	全天总费	23,477.87

表 48 2025-09-23 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	356.46	14:00-14:10	0.00
16:00-16:10	569.79	18:00-18:10	759.11	20:00-20:10	0.00
全天计划量	65,563.60	计划费	42,364.17	全天总费	49,513.71

表 49 2025-12-21 指定时段原计划购电及全天汇总

时间段	购电量	时间段	购电量	时间段或指标	购电量或费用
10:00-10:10	0.00	12:00-12:10	940.86	14:00-14:10	0.00
16:00-16:10	847.42	18:00-18:10	711.63	20:00-20:10	0.00
全天计划量	89,177.56	计划费	68,148.28	全天总费	70,461.69

附录 C 对应问题 3 储能明细

表 50 2025-03-20 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	6,198.60	262.19	4:00-8:00	882.13	6,182.82
8:00-12:00	4,917.76	2,226.71	12:00-16:00	6,063.38	254.72
16:00-20:00	0.00	5,831.55	20:00-24:00	4,317.97	2,865.74
0:00 储电量	4,462.58		24:00 储电量	5,022.53	

表 51 2025-06-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	1,745.36	466.84	4:00-8:00	2,000.22	1,719.95
8:00-12:00	4,269.03	0.00	12:00-16:00	0.00	0.00
16:00-20:00	0.00	4,355.01	20:00-24:00	3,095.45	3,077.96
0:00 储电量	6,016.62		24:00 储电量	5,327.04	

表 52 2025-09-23 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	2,618.69	162.80	4:00-8:00	8,004.45	7,019.38
8:00-12:00	5,206.61	1,476.44	12:00-16:00	5,962.54	411.36
16:00-20:00	58.14	5,833.85	20:00-24:00	308.99	2,938.42
0:00 储电量	1,256.09		24:00 储电量	1,374.85	

表 53 2025-12-21 四小时充放电和首末储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	127.72	258.62	4:00-8:00	187.15	923.76
8:00-12:00	5,560.79	7,460.55	12:00-16:00	7,617.36	2,282.65
16:00-20:00	17.08	5,907.49	20:00-24:00	5,983.86	2,800.82
0:00 储电量	10,795.81		24:00 储电量	6,524.96	

附录 C 对应问题 3 紧急购电明细

表 54 指定日期紧急购电区间与电量（对应题面表 3）

03/20	购电量	06/21	购电量	09/23	购电量	12/21	购电量
9:00-10:00	550.67	无	0.00	2:20-2:40	9.31	7:50-8:00	5.51
20:30-20:40	121.76	—	—	8:40-9:50	420.17	20:30-20:40	96.18
—	—	—	—	20:10-22:00	765.49	—	—
—	—	—	—	22:20-22:30	3.13	—	—

注：所有日期均为 2025 年，电量单位为 kWh；相邻的紧急购电十分钟区间合并为连续事件，数值为区间电量之和。“无”表示当日未发生紧急购电，“—”表示该列没有更多事件。